SPES's SHRI GOPAL GAONKAR MEMORIAL GOA MULTI-FACULTY COLLEGE

Dharbandora, Goa

E-JOURNAL

FULL STACK DEVELOPMENT

Course Code :- CSA 301

Bachelor of Computer Applications (BCA)

Academic Year :- 2026

SUBMITTED BY

Student Name :- Hasmani Bilung

Roll Number :- 2405009

Class/Semester :- TY BCA - Semester 5th

PR Number :- 202402581

SUBMITTED TO

Faculty Name :- Goa Multi-Faculty College

Department of IT and Computer Applications

Submitted in partial fulfilment of the practical requirements of CSA 301 – Full Stack Development.

INDEX

Sr. No.	Contents / Practical Title	Page No.
1	Cover Page	1
2	Table of Contents	2-3
3	Week 1 – Arithmetic Operations and Node REPL Commands	4
4	Week 2 – Node Modules, Dependencies and Functionality	7
5	Exercise 1 – Explore Core Modules (os, path, fs)	7
6	Exercise 2 – Create Local Modules	11
7	Exercise 3 – Multiple Modules	12
8	Exercise 4 – npm and Dependencies (chalk, figlet, validator)	15
9	Exercise 5 – Build Utility Modules	17
10	Exercise 6 – Mini Project: Student Utility System	20
11	Week 3 – Event Loop, Timers, Callbacks and Events	24
12	Week 4 – Promises and Promise Chaining	39
13	Week 5 (Part 1) – Streams and Standard Streams	48
14	Mini Project – Student File Management System	57
15	Week 5 (Part 2) – HTTP Server, Routing, Request Headers and API Gateway	64
16	Mini Project – College API Gateway	72
17	Week 6 (Part 1) – Creating Cron Jobs using Node.js	76
18	Week 6 (Part 2) – Demonstrating Webpack in Node.js	82
19	Week 7 – Node.js with Sequelize ORM and MySQL	89
20	Week 8 – Sequelize Models, Migrations and Basic Model Querying	92
21	Week 9 – Finders, Validation, Constraints and Raw Queries	123
22	Week 10 – Sequelize Associations (1:1, 1:M, M:N)	191

23	Week 11 – React.js Fundamentals and Components	241
24	Exercise 11.1 – Installation and React Application Setup	241
25	Exercise 11.2 – Creating Custom Components	244
26	Exercise 11.3 – Passing Data Using Props	248
27	Exercise 11.4 – Managing State Using useState	249
28	Exercise 11.5 – Form Handling Using State	250
29	Week 12 – React Hooks	255
30	Exercise 12.1 – Dynamic To-Do List using useState()	255
31	Exercise 12.2 – API Data using useEffect()	258
32	Exercise 12.3 – Student Management using useReducer()	262
33	Exercise 12.4 – Theme Management using useContext()	266
34	Exercise 12.5 – DOM Handling using useRef()	270
35	Exercise 12.6 – useMemo() and useCallback()	271
36	Week 13 – React Router and Navigation	275
37	Exercise 13.1 – React Router Setup	275
38	Exercise 13.2 – Dynamic Routing	278
39	Exercise 13.3 – Nested Routes	281
40	Exercise 13.4 – Protected Routes	285
41	Week 14 – React.js and Node.js Integration	291
42	Exercise 14.1 – Creating a Node.js REST API	291
43	Exercise 14.2 – Connecting React to Node.js	293
44	Exercise 14.3 – CORS Configuration	297
45	Exercise 14.4 – React and Node.js POST Request	299
46	Exercise 14.5 – Complete CRUD Application	304
47	Week 15 – Node.js Integration with React Frontend (Student Registration Application)	316

WEEK – 1

Arithmetic Operation

Add

```
Welcome to Node.js v24.18.0.
Type ".help" for more information.
> function add(x,y){
| return x+y;}
undefined
> add (2,5)
7
```

Square , Multiplication , Cube , Subraction

```
Node.js
Welcome to Node.js v24.18.0.
Type ".help" for more information.
> function square (n) {
| return n*n;}
undefined
> square (5)
25
>
> function multiply (x,y){
| return x*y;}
undefined
> multiply (22,18)
396
>
> function cube (n){
| return n*n*n;}
undefined
> cube (3)
27
>
> function subraction (x,y){
| return x-y;}
undefined
> subraction (10,4)
6
>
```

Node Commands

.Save

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\DELL> node -v
v24.14.0
PS C:\Users\DELL> node
Welcome to Node.js v24.14.0.
Type ".help" for more information.
> function square(n) {
| return n*n;}
undefined
> square (5)
25
> function multiply(x,y){
| return x+y;}
undefined
> multiply (4,4)
8
> square(multiply(4,4))
64
> .save arithmetic.txt
Session saved to: arithmetic.txt
> .exit
PS C:\Users\DELL>
```

.Load

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\DELL> node
Welcome to Node.js v24.14.0.
Type ".help" for more information.
> .load arithmetic.txt
function square(n) {
function square(n) {
return n*n;}
square (5)
function multiply(x,y){
function multiply(x,y){
return x+y;}
multiply (4,4)
| square(multiply(4,4))
|
```

.Editor

```
C:\Users\Riddhi>node
Welcome to Node.js v24.18.0.
Type ".help" for more information.
> .editor
// Entering editor mode (Ctrl+D to finish, Ctrl+C to cancel)
function abc() {
return 22+3;}
abc()
abc();
25
>
```

.Exit

```
> .exit
PS C:\Users\DELL>
```

WEEK 2: Node Modules, Dependencies And Functionality

Exercise 1 - Explore Core Modules

Tasks

- Create project
- Use os
- Use path
- Use fs
- Print system information

OS Module

```
const os = require('os');


console.log(os.platform());
console.log(os.arch());
console.log(os.hostname());
console.log(os.cpus().length);
console.log(os.freemem());
console.log(os.availableParallelism());
console.log(os.endianness());
console.log(os.getPriority());
console.log(os.homedir());
console.log(os.loadavg());
console.log(os.machine());
console.log(os.networkInterfaces());
console.log(os.release());
console.log(os.tmpdir());
console.log(os.totalmem());
console.log(os.type());
console.log(os.uptime());
console.log(os.userInfo());
console.log(os.version());
```

Output

C:\Users\TYBCA\Desktop\FSD_2405009>node app.js

```
win32
x64
LAB2-PC4
12
995557376
12
LE
0
C:\Users\TYBCA
[ 0, 0, 0 ]
x86_64
{
  Ethernet: [
    {
      address: 'fe80::9fe2:16b1:f218:8636',
      netmask: 'ffff:ffff:ffff:ffff::',
      family: 'IPv6',
      mac: '64:4e:d7:71:78:20',
      internal: false,
      cidr: 'fe80::9fe2:16b1:f218:8636/64',
      scopeid: 14
    },
    {
      address: '192.168.1.176',
      netmask: '255.255.255.0',
      family: 'IPv4',
      mac: '64:4e:d7:71:78:20',
      internal: false,
      cidr: '192.168.1.176/24'
    }
  ],
  'Ethernet 2': [
    {
      address: 'fe80::8a20:35db:6e46:6db',
      netmask: 'ffff:ffff:ffff:ffff::',
      family: 'IPv6',
      mac: '0a:00:27:00:00:0a',
      internal: false,
      cidr: 'fe80::8a20:35db:6e46:6db/64',
      scopeid: 10
    },
```

```
    {
      address: '192.168.56.1',
      netmask: '255.255.255.0',
      family: 'IPv4',
      mac: '0a:00:27:00:00:0a',
      internal: false,
      cidr: '192.168.56.1/24'
    }
  ],
  'Loopback Pseudo-Interface 1': [
    {
      address: '::1',
      netmask: 'ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff',
      family: 'IPv6',
      mac: '00:00:00:00:00:00',
      internal: true,
      cidr: '::1/128',
      scopeid: 0
    },
    {
      address: '127.0.0.1',
      netmask: '255.0.0.0',
      family: 'IPv4',
      mac: '00:00:00:00:00:00',
      internal: true,
      cidr: '127.0.0.1/8'
    }
  ]
}
10.0.26200
C:\Users\TYBCA\AppData\Local\Temp
8259911680
Windows_NT
11283.984
[Object: null prototype] {
  uid: -1,
  gid: -1,
  username: 'TYBCA',
  homedir: 'C:\\Users\\TYBCA',
  shell: null
}
```

Windows 11 Pro

PATH Module

```
const path = require("path");
console.log("\n=== Path Information ===");

const filePath = path.join(__dirname,'data.js');
console.log("Joined Path:", filePath);
console.log("Directory name: ", path.dirname(filePath));
console.log("File Name:", path.basename(filePath));
console.log("Extension:", path.extname(filePath));
console.log("Is Absolute Path:",
path.isAbsolute(filePath));
console.log("relative path:", path.relative(__dirname,
filePath));
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009>node path.js

=== Path Information ===
Joined Path: C:\Users\TYBCA\Desktop\FSD_2405009\data.js
Directory name:  C:\Users\TYBCA\Desktop\FSD_2405009
File Name: data.js
Extension: .js
Is Absolute Path: true
relative path: data.js
```

FS Module

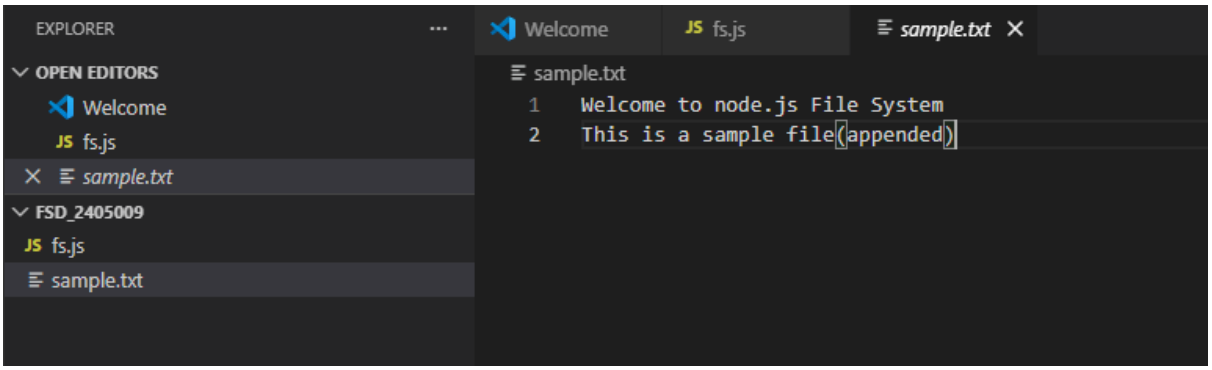
```
const fs = require("fs");

console.log("====File System Module====");

//write
fs.writeFileSync("sample.txt","Welcome to node.js File
System");
```

```
//append
fs.appendFileSync("sample.txt", "\nThis is a sample
file(appended)");

//read
const content = fs.readFileSync("sample.txt","utf8");
console.log("File content: ",content);
```



Output

C:\Users\TYBCA\Desktop\FSD_2405009>node fs.js
====File System Module====
File content: Welcome to node.js File System
This is a sample file(appended)

Exercise 2 -Create Local Modules

Create math.js

- Functions
- add
- subtract
- multiply
- divide

Import into app.js

math.js

```
function add(a,b) {
    return a+b;
}

function subtract(x,y) {
```

```
    return x-y;
}

function multiply(n,m) {
    return n*m;
}

function divide(a,b) {
    return a/b;
}

module.exports = {add,subtract,multiply,divide};
```

app.js

```
const math = require("./math");

console.log("Addition:",math.add(10,20));
console.log("Subtraction:",math.subtract(5,20));
console.log("Multiplication:",math.multiply(5,8));
console.log("Division:",math.divide(20,10));
```

Output

C:\Users\TYBCA\Desktop\FSD_2405009>node app.js
Addition: 30
Subtraction: -15
Multiplication: 40
Division: 2

Exercise 3 -Multiple Modules

Create calculator.js, student.js, employee.js
Each module exports at least three functions.
Use all modules from one file.

calculator.js

```
function add(a,b) {
    return a+b;
```

```
}

function subtract(x,y) {
    return x-y;
}

function multiply(n,m) {
    return n*m;
}

function divide(a,b) {
    return a/b;
}

module.exports = {add,subtract,multiply,divide};
```

student.js

```
function studentName(name) {
    return "Student's Name: "  +name;
}

function studentAge(age) {
    return "Student's Age: " +age;
}

function rollno(rollno) {
    return "Student's Roll no: "+rollno;
}

module.exports ={
    studentName,studentAge,rollno
};
```

employee.js

```
function employeeName(name) {
    return "Employee's Name: "+name;
}

function salary(salary) {
    return "Salary: "+salary;
}

function department(dept) {
    return "Departmnt: "+dept;
}

module.exports = {
    employeeName, salary,department
};
```

final.js

```
const math = require("./calculator");

console.log("===Calculator===");
console.log("Addition:",math.add(10,20));
console.log("Subtraction:",math.subtract(5,20));
console.log("Multiplication:",math.multiply(5,8));
console.log("Division:",math.divide(20,10));

const student = require("./student");

console.log("\n===Student Details===");
console.log(student.studentName("Hasmani Bilung"));
console.log(student.studentAge(20));
console.log(student.rollno(2405009));

const employee = require("./employee");
```

```
console.log("\n===Employee Details===");
console.log(employee.employeeName("Hasmani Bilung"));
console.log(employee.salary(65000));
console.log(employee.department("IT"));
```

Output

C:\Users\TYBCA\Desktop\FSD_2405009>node final.js
===Calculator===
Addition: 30
Subtraction: -15
Multiplication: 40
Division: 2

===Student Details===
Student's Name: Hasmani Bilung
Student's Age: 20
Student's Roll no: Student Roll no.: 2405009

===Employee Details===
Employee's Name: Hasmani Bilung
Salary: 65000
Department: IT

Exercise 4 -npm and Dependencies

Create a new project.
Install chalk
Use different colours for output.

Install figlet
Display WELCOME BCA

Install validator
Validate

- Email
- URL
- Mobile number

```
const chalk = require("chalk");
```

```
const figlet = require("figlet");
const validator = require("validator")

//chalk
console.log(chalk.blue('HELLO EVERYONE!!!!'));
console.log(chalk.magenta('This a the usage of chalk.'))
console.log(chalk.red('This is a red text'));
console.log(chalk.green('This is a green text'));
console.log(chalk.yellow('This is a yellow text\n'));

//Figlet
figlet("WELCOME  TO  BCA", function (err, data) {
  if (err) {
    console.log("Something went wrong...");
    console.log(err.message);
    return;
  }
    console.log(chalk.redBright(data));


//validation
 const email = "asmanibilung08@gmail.com";
    const url = "https://www.google.com";
    const mobile = "8767725306";

    console.log(
        "Email:",
        validator.isEmail(email)
            ? chalk.green("Valid Email")
            : chalk.red("Invalid Email")
    );


    console.log(
        "URL:",
```

```
        validator.isURL(url)
            ? chalk.green("Valid URL")
            : chalk.red("Invalid URL")
    );


    console.log(
        "Mobile Number:",
        validator.isMobilePhone(mobile, "en-IN")
            ? chalk.green("Valid Mobile Number")
            : chalk.red("Invalid Mobile Number")
    );

});
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009>node index.js
HELLO EVERYONE!!!!
This a the usage of chalk.
This is a red text
This is a green text
This is a yellow text

 _       _____ _     ___ ___ __  __ ____   ____ ___   ___  ___   _
 \ \     / / __| |   / __/ _ \  \/  |___|  |_  / _ \  |__ )/ __| /_\
  \ \/\/ /| _|| | | | (_| (_) |\/| |___|   | || (_) | | _ \ |__ / _ \
   \_/\_/ |___|_|_|_|\___\___/|_|  |_|     |_| \___/  |___/\___/_/ \_\

Email: Valid Email
URL: Valid URL
Mobile Number: Valid Mobile Number
```

Exercise 5 -Build Utility Modules

Create
temperature.js
currency.js
area.js

Functions

- Celsius to Fahrenheit
- INR to USD

- Area of Circle
- Rectangle
- Triangle

Export and use them.

temperature.js

```
function celsiusToFahrenheit(celsius) {
    return (celsius * 9 / 5) + 32;
}


module.exports = { celsiusToFahrenheit };
```

currency.js

```
function inrToUsd(inr) {
    const exchangeRate = 0.012;
    return inr * exchangeRate;
}


module.exports = { inrToUsd };
```

area.js

```
// Area of Circle
function areaOfCircle(radius) {
    return Math.PI * radius * radius;
}

// Area of Rectangle
function areaOfRectangle(length, width) {
    return length * width;
}

// Area of Triangle
```

```
function areaOfTriangle(base, height) {
    return 0.5 * base * height;
}

// Export all functions
module.exports = {
    areaOfCircle,
    areaOfRectangle,
    areaOfTriangle
};
```

output.js

```
const temp = require("./temperature");
const currency = require("./currency");
const area = require("./area")

//temperature conversion
console.log("Conversion of Celsius to Fahrenheit")
console.log("25°C =", temp.celsiusToFahrenheit(25), "°F");

//currency
console.log("\nCurrency Conversion");
console.log("₹1000 =", currency.inrToUsd(1000).toFixed(2),
"USD");

//area
console.log("\nArea Calculations");
console.log("Area of Circle =",
area.areaOfCircle(7).toFixed(2));
console.log("Area of Rectangle =",
area.areaOfRectangle(10, 5));
console.log("Area of Triangle =", area.areaOfTriangle(12,
6));
```

Output

C:\Users\TYBCA\Desktop\FSD_2405009>node output.js
Conversion of Celsius to Fahrenheit
25°C = 77 °F

Currency Conversion
₹1000 = 12.00 USD

Area Calculations
Area of Circle = 153.94
Area of Rectangle = 50
Area of Triangle = 36

Exercise 6 -Mini Project – Student Utility System

Create the following modules:

student.js
marks.js
attendance.js
result.js
app.js

Requirements

- Store student details
- Calculate total marks
- Calculate percentage
- Assign Grade
- Calculate attendance percentage

Display report

student_report.js

```
function student() {
    return {
        name: "Hasmani Bilung",
        rollNo: 2405009
    };
}

module.exports = {
    student
```

```
};
```

marks.js

```
function getMarks() {

    const maths = 80;
    const science = 78;
    const english = 85;

    const total = maths + science + english;
    const percentage = total / 3;

    return {
        maths,
        science,
        english,
        total,
        percentage
    };
}

module.exports = {
    getMarks
};
```

attendance.js

```
function attendancePercentage(attendedDays, totalDays) {
    return (attendedDays / totalDays) * 100;
}

module.exports = {
    attendancePercentage
};
```

result.js

```
function getGrade(percentage) {

    if (percentage >= 90)
        return "A+";

    else if (percentage >= 80)
        return "A";

    else if (percentage >= 70)
        return "B";

    else if (percentage >= 60)
        return "C";

    else if (percentage >= 40)
        return "D";

    else
        return "Fail";
}

module.exports = {
    getGrade
};
```

data.js

```
const stud = require("./student_report");
const marks = require("./marks");
const attendance = require("./attendance");
const result = require("./result");

const studentData = stud.student();
const marksData = marks.getMarks();
```

```
const attendancePercent =
attendance.attendancePercentage(46, 50);
const grade = result.getGrade(marksData.percentage);

console.log("========== Student Report ==========\n");
console.log("Name :", studentData.name);
console.log("Roll No :", studentData.rollNo);

console.log("\nMarks Scored");

console.log("Math :", marksData.maths);
console.log("Science :", marksData.science);
console.log("English :", marksData.english);

console.log("\nTotal :", marksData.total);
console.log("Percentage :", marksData.percentage + "%");
console.log("Grade :", grade);
console.log("Attendance :", attendancePercent + "%");
```

Output

C:\Users\TYBCA\Desktop\FSD_2405009>node data.js
========== Student Report ==========

Name : Hasmani Bilung
Roll No : 2405009

Marks Scored
Math : 80
Science : 78
English : 85

Total : 243
Percentage : 81%
Grade : A
Attendance : 92%

WEEK – 3

Lab Exercise 1: Event Loop Execution Order

Objective

Understand how Node.js executes synchronous and asynchronous code using the Event Loop.

Instructions

1. Create a new JavaScript file named eventloop.js.
2. Display a message indicating the start of the program.
3. Schedule a function using setTimeout() with a delay of 0 milliseconds.
4. Schedule another function using setImmediate().
5. Display another message indicating the end of the program.
6. Execute the program and observe the order in which the messages are displayed.
7. Compare the output with your prediction and explain why the order occurs.

```
console.log("1. Program Started");

setTimeout(() => {
    console.log("2. setTimeout executed");
}, 0);

setImmediate(() => {
    console.log("3. setImmediate executed");
});

console.log("4. Program Ended");
```

Output

C:\Users\DELL\OneDrive\Desktop\FSD - 09>node eventloop.js
1. Program Started
4. Program Ended
3. setImmediate executed
2. setTimeout executed

Lab Exercise 2: Multiple Timers

Objective

Understand how multiple timers execute based on different delays.

Instructions

1. Create a program named multitimers.js.
2. Display a message when the program starts.
3. Create three different timers using setTimeout().
4. Assign different delay values (e.g., 1 sec, 2 sec, and 3 sec).
5. Display a different message from each timer.
6. Execute the program and note the order in which the timers are executed.
7. Explain how Node.js schedules timer callbacks.

```
console.log("Program Started");

setTimeout(() => {
    console.log("Task A is complete.");
}, 3000);

setTimeout(() => {
    console.log("Task B is complete.");
}, 1000);

setTimeout(() => {
    console.log("Task C is complete.");
}, 2000);

console.log("Program Ended");
```

Output

```
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\FSD - 09>node multitimer.js
Program Started
Program Ended
Task B is complete.
Task C is complete.
Task A is complete.
```

Lab Exercise 3: Countdown Timer

Objective

Create a countdown timer using setInterval().

Instructions

1. Create a file named countdown.js.
2. Accept a number from the user.
3. Display the countdown every second.
4. Stop the timer when the countdown reaches zero.
5. Display Time Up! after the countdown finishes.
6. Test the program using different countdown values.

```
let count = 1;

let timer = setInterval(() => {

    console.log(count);

    count++;

    if (count>10) {
        clearInterval(timer);
        console.log("Time Up!");
    }
},1000);
```

Output

C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\FSD - 09>node countdown.js
1
2
3
4
5
6
7
8
9

10
Time Up!

Lab Exercise 4: Digital Counter using setInterval()

Objective

Use setInterval() to execute a task repeatedly.

Instructions

1. Create a file named counter.js.
2. Display numbers from 1 to 10.
3. Print one number every second.
4. Stop the counter after displaying 10.
5. Display a completion message after the counter stops.

```
let count = 5;

let timer = setInterval(() => {

    console.log(count);

    count--;

    if (count<0) {
        clearInterval(timer);
        console.log("Time Up!");
    }
},1000);
```

Output

C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\FSD - 09>node counter.js
5
4
3
2
1
0
Time Up!

Lab Exercise 5: Callback Functions

Objective

Understand how callback functions work in Node.js.

Instructions

1. Create a function that accepts another function as a parameter.
2. Display a greeting message inside the first function.
3. Invoke the callback function after displaying the greeting.
4. Create a callback function that prints a thank-you message.
5. Execute the program and observe the order of execution.

```
function greet(name, callback) {

    console.log("Hello " + name);

    callback();

}

function thankYou() {

    console.log("Thank you for visiting");

}

greet("Hasmani", thankYou);
```

Output

```
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\FSD - 09>node pg5.js
Hello Hasmani
Thank you for visiting!
```

Lab Exercise 6: Error-First Callback

Objective

Implement the Node.js Error-First Callback convention.

Instructions

1. Create a function to perform division of two numbers.
2. Accept two numbers as input.
3. If the divisor is zero, generate an error.
4. Otherwise, return the division result.
5. Handle both success and error cases using a callback function.
6. Test the program with valid and invalid inputs.

```
function divide(a, b, callback) {

    if(b==0) {
        callback(new Error("Divide by zero"), null);
    }
    else {
        callback(null,a/b);
    }
}

function abc(err, result) {

    if(err)
        console.log(err.message);
    else
        console.log("Result =",result);

};

divide(8,4,abc);

divide(20,0,abc);
```

Output

```
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\FSD - 09>node pg6.js
Result = 2
Divide by zero
```

Lab Exercise 7: Student Result using Callback

Objective

Use callback functions to evaluate student results.

Instructions

1. Create a function that accepts student marks.
2. Validate that the marks are between 0 and 100.
3. If the marks are invalid, display an appropriate message.
4. If valid, determine whether the student has passed or failed.
5. Display the result using a callback function.
6. Test the program with multiple input values.

```
function result(marks, callback) {
    if (marks < 0 || marks > 100)
        callback(marks + " is Invalid Marks");
    else if (marks >= 40)
        callback("Pass. Marks scored: "+ marks);
    else
        callback("Fail. Marks scored: " + marks);
}

function display(msg) {
    console.log(msg);
}

result(15, display);
result(85, display);
result(150, display);
```

Output

```
C:\Users\DELL\OneDrive\Desktop\FSD - 09>node marks.js
Fail. Marks scored: 15
Pass. Marks scored: 85
150 is Invalid Marks
```

Lab Exercise 8: Read File using fs.readFile()

Objective

Read data from a text file asynchronously.

Instructions

1. Create a text file named student.txt.
2. Add sample student information to the file.
3. Create a Node.js program named readfile.js.
4. Read the contents of the file using fs.readFile().
5. Display the contents on the console.
6. Handle errors if the file is not found.

readfile.js

```
const fs = require("fs");

fs.readFile("student.txt", "utf8", (err, data) => {
    if (err) {
        console.log("File not found!");
    } else {
        console.log(data);
    }
});
```

student.txt

```
1   Name: Hasmani Bilung
2   Roll no: 2405009
3   Class: TYBCA
```

Output

```
C:\Users\DELL\OneDrive\Desktop\FSD - 09>node readfile.js
Name: Hasmani Bilung
Roll no: 2405009
Class: TYBCA
```

Lab Exercise 9: Write File using fs.writeFile()

Objective

Write data to a file asynchronously.

Instructions

1. Create a Node.js program named writefile.js.
2. Store student information in a string.

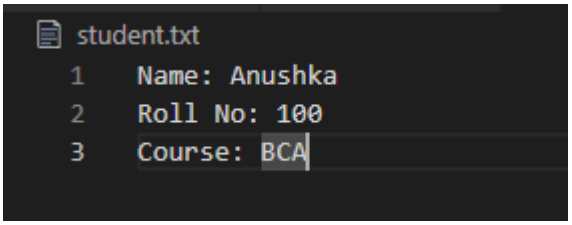
3. Write the data into student.txt using fs.writeFile().
4. Display a success message after writing the file.
5. Verify the contents of the file.

writefile.js

```
const fs = require("fs");

const data = "Name: Anushka\nRoll No: 100\nCourse: BCA";

fs.writeFile("student.txt", data, (err) => {
    if (err) {
        console.log("Error writing file!");
    } else {
        console.log("File written successfully.");
    }
});
```



Output

C:\Users\DELL\OneDrive\Desktop\FSD - 09>node writefile.js
File written successfully.

Lab Exercise 10: Append File

Objective

Append new data to an existing file.

Instructions

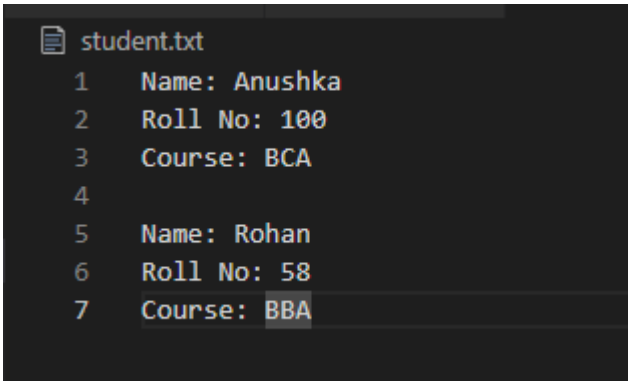
1. Use the existing student.txt file.
2. Create another student record.
3. Append the new record using fs.appendFile().
4. Display a success message after appending.
5. Verify that both records are present in the file.

append.js

```
const fs = require("fs");
```

```
const data = "\n\nName: Rohan\nRoll No: 58\nCourse: BBA";

fs.appendFile("student.txt", data, (err) => {
    if (err)
        console.log("Error appending file!");
    else
        console.log("Data appended successfully.");
});
```



Output

C:\Users\DELL\OneDrive\Desktop\FSD - 09>node append.js
Data appended successfully.

Lab Exercise 11: Basic EventEmitter

Objective

Create and trigger a custom event.

Instructions

1. Create a program named eventemitter.js.
2. Import the events module.
3. Create an EventEmitter object.
4. Register a listener for an event named login.
5. Display an appropriate message when the event occurs.
6. Trigger the event using emit().

```
//create and trigger a custom event

import EventEmitter from "events";
```

```
const event = new EventEmitter();

event.on("login", () => {
    console.log("User logged in successfully");
});

event.emit("login");
```

Output
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\FSD - 09>node pg11.js
User logged in successfully

Lab Exercise 12: Multiple Event Listeners

Objective

Register multiple listeners for the same event.

Instructions

1. Create an EventEmitter object.
2. Register at least three listeners for a single event named payment.
3. Each listener should display a different message.
4. Trigger the event only once.
5. Observe the order in which all listeners are executed.

```
const EventEmitter = require("events");

const paymentEmt = new EventEmitter();

paymentEmt.on("payment", () => {
    console.log("Payment Successful");
});

paymentEmt.on("payment", () => {
    console.log("Receipt Generated");
});

paymentEmt.on("payment", () => {
    console.log("Email Sent");
```

```
});

paymentEmt.emit("payment");
```

Output

C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\FSD - 09>node pg12.js
Payment Successful
Receipt Generated
Email Sent

Lab Exercise 13: once() Method

Objective

Understand the use of once().

Instructions

1. Create an EventEmitter object.
2. Register an event using the once() method.
3. Trigger the same event twice.
4. Observe how many times the listener is executed.
5. Explain the difference between on() and once().

```
const EventEmitter = require("events");

const event = new EventEmitter();

event.once("welcome", () => {
    console.log("Welcome Student");
});

event.emit("welcome");

event.emit("welcome");
```

Output
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\FSD - 09>node pg13.js
Welcome Student

Lab Exercise 14: Student Management Event System

Objective

Develop a simple event-driven student management application.

Instructions

1. Create custom events for:
    - o Student Added
    - o Student Deleted
    - o Student Searched
2. Register appropriate listeners for each event.
3. Pass the student name while triggering the events.
4. Display meaningful messages for each operation.
5. Execute all events and verify the output.

```
const EventEmitter = require("events");

const student = new EventEmitter();

//listener function "addfunc" is added to event
"studentAdd"
const addfunc = (name,rollNo) => {
    console.log(name + ", " + rollNo + "Added
Successfully");
};

//listener function "delfunc" is added to event
"studentDelete"
const delfunc = (name) => {
    console.log("\n" + name + "Deleted Successfully");
};

//listener function "searchfunc" is added to event
"studentSearched"
const searchfunc = (name) => {
    console.log("\nSearching for "+ name);
};
```

```
//event "studentAdd" occurs over here
student.on("studentAdd", addfunc );

//event "studentDelete" occurs over here
student.on("studentDelete", delfunc);

//event "studentSearched" occurs over here
student.on("studentSearched", searchfunc);

student.emit("studentAdd" , "Hasmani ", "2405009 ");
student.emit("studentAdd", "Nayan ", "2405014 ");
student.emit("studentAdd", "Khushi ", "2405011 ");
student.emit("studentAdd", "Anisha ", "2405005 ");

student.emit("studentDelete", "Khushi ");
student.emit("studentSearched", "Hasmani....");

//listener removed
student.removeListener("studentAdd", addfunc);

student.emit("studentAdd", "Priya ", "2405017 ");
student.emit("studentDelete", "Anisha ");
```

Output

C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\FSD - 09>node pg14.js
Hasmani , 2405009 Added Successfully
Nayan , 2405014 Added Successfully
Khushi , 2405011 Added Successfully
Anisha , 2405005 Added Successfully

Khushi Deleted Successfully

Searching for Hasmani....

Anisha Deleted Successfully

Lab Exercise 15: Library / Bank Event-Based Mini Project

Library Management System

Instructions

1. Create an EventEmitter object.
2. Implement the following events:
    - o Book Issued
    - o Book Returned
    - o Book Reserved
3. Display appropriate messages whenever an event is triggered.
4. Execute all events and verify the output.

```
const EventEmitter = require("events");

const library = new EventEmitter();

library.on("bookIssued", (bookName, student) => {
    console.log("Book Issued: " + bookName + " has been issued to " +
student);
});

library.on("bookReturned", (bookName, student) => {
    console.log("Book Returned: "+ bookName + " has been returned by "
+student);
});

library.on("bookReserved", (bookName, student) => {
    console.log("Book Reserved: "+bookName+" has been reserved by "+student);
});

library.emit("bookIssued", "Python", "Kunal");
library.emit("bookReturned", "Python", "Kunal");
library.emit("bookReserved", "Node.js ", "Priya");
```

Output

```
C:\Users\DELL\OneDrive\Desktop\FSD - 09>node library.js
Book Issued: Python has been issued to Kunal
Book Returned: Python has been returned by Kunal
Book Reserved: Node.js  has been reserved by Priya
```

Week 4 Exercises: Promises and Promise Chaining

Exercise 1: Understanding Promises

```
const foodOrder = new Promise((resolve, reject) => {
let restaurantOpen = true;
if (restaurantOpen)
resolve("Pizza Delivered");
else
reject("Restaurant Closed");
});
foodOrder.then(console.log).catch(console.log);
```

Output

C:\Users\TYBCA\Desktop\FAD_005>node promise.js
Pizza Delivered

Activity: Change restaurantOpen to false and observe the output.

```
const foodOrder = new Promise((resolve, reject) => {
let restaurantOpen = false;
if (restaurantOpen)
resolve("Pizza Delivered");
else
reject("Restaurant Closed");
});
foodOrder.then(console.log).catch(console.log);
```

Output

C:\Users\TYBCA\Desktop\FAD_005>node promise.js
Restaurant Closed

Exercise 2: Student Result

```
function checkResult(marks){
return new Promise((resolve,reject)=>{
```

```
if(marks>=40) resolve("Pass! Marks obtained: " + marks);
else reject("Fail. Marks obtained: " + marks);
});
}
checkResult(20).then(console.log).catch(console.log);
checkResult(60).then(console.log).catch(console.log);
checkResult(33).then(console.log).catch(console.log);
checkResult(39).then(console.log).catch(console.log);
checkResult(40).then(console.log).catch(console.log);
checkResult(75).then(console.log).catch(console.log);
```

Output

C:\Users\TYBCA\Desktop\FAD_005>node student.js
Pass! Marks obtained: 60
Pass! Marks obtained: 40
Pass! Marks obtained: 75
Fail. Marks obtained: 20
Fail. Marks obtained: 33
Fail. Marks obtained: 39

Exercise 3: ATM Withdrawal

```
function withdraw(balance,amount){
return new Promise((resolve,reject)=>{
if(balance>=amount) resolve("Cash Withdrawn");
else reject("Insufficient Balance");
});
}
withdraw(10000,5000).then(console.log).catch(console.log);
```

Output

C:\Users\TYBCA\Desktop\FAD_005>node atm.js
Cash Withdrawn

Exercise 4: Simulating Delay

```
function downloadFile(){
return new Promise(resolve=>{
setTimeout(()=>resolve("Download Complete"),3000);
});
}
console.log("Downloading...");
downloadFile().then(console.log);
console.log("Please Wait...");
```

Output

C:\Users\TYBCA\Desktop\FAD_005>node download_file.js
Downloading...
Please Wait...
Download Complete

Exercise 5: Basic Promise Chaining

```
function login(){
return Promise.resolve("User Logged In");
}
login()
.then(msg=>{console.log(msg); return "Profile Loaded";})
.then(msg=>{console.log(msg); return "Products Loaded";})
.then(console.log);
```

Output

C:\Users\TYBCA\Desktop\FAD_005>node chaining.js
User Logged In
Profile Loaded
Products Loaded

Exercise 6: Student Admission Process

Create a Promise chain:
Application Submitted
→ Documents Verified
→ Fees Paid
→ Admission Confirmed

```
function admission(){
return Promise.resolve("Application Submitted");
}
admission()
.then(msg=>{console.log(msg); return "Documents
Verified";})
.then(msg=>{console.log(msg); return "Fees Paid";})
.then(msg=>{console.log(msg); return "Admission
Confirmed";})
.then(console.log);
```

Output

C:\Users\TYBCA\Desktop\FAD_005>node admission.js
Application Submitted
Documents Verified
Fees Paid
Admission Confirmed

Exercise 7: Promise Chaining with Delay

Create login(), getProfile(), and getOrders() using setTimeout(1000).
Chain them using .then().

```
function login() {
    return new Promise((resolve,reject) => {
        setTimeout(() => {
            resolve("Login Successful");
        },1000);
```

```
    });
}

function getProfile() {
    return new Promise((resolve,reject) => {
        setTimeout(() => {
            resolve("Profile Loaded");
        },1000);
    });
}

function getOrders() {
    return new Promise((resolve,reject) => {
        setTimeout(() => {
            resolve("Order Placed");
        },1000);
    });
}

login()
.then((message) => {console.log(message);
    return getProfile();

})

.then((message) => {console.log(message);
    return getOrders();
})

.then((message) => {console.log(message);
})

.catch((error) => console.log(error));
```

Output

C:\Users\TYBCA\Desktop\FSD_2405009>node ex_7.js
Login Successful
Profile Loaded
Order Placed

Exercise 8: Promise Chaining with Error

```
function login() {
```

```
return Promise.resolve("Login Successful");
}
function payment() {
return Promise.reject("Payment Failed");
}
login()
.then(msg => {console.log(msg);
return payment();
})

.then(console.log)
.catch(console.log);
```

Output:

C:\Users\TYBCA\Desktop\FSD_2405009>node error.js
Login Successful
Payment Failed

Challenge Activity

Develop a Library Management System with:
login() → searchBook() → issueBook() → logout()
Each function should return a Promise and use setTimeout().

```
function login() {
    return new Promise((resolve,reject) => {
         setTimeout(() => {
            resolve("Login Successful");
        },1000);
    });
}

function searchBook() {
    return new Promise((resolve,reject) => {
         setTimeout(() => {
            resolve("Book Found");
        },1000);
    });
}
```

```
function issueBook() {
    return new Promise((resolve,reject) => {
         setTimeout(() => {
            resolve("Book issued Successfully");
        },1000);
    });
}


function logout() {
    return new Promise((resolve,reject) => {
         setTimeout(() => {
            resolve("User logged out");
        },1000);
    });
}

login()
.then((message) => {console.log(message);
    return searchBook();
})

.then((message) => {console.log(message);
    return issueBook();
})

.then((message) => {console.log(message);
    return logout();
})

.then((message) => {console.log(message);
});
```

Output

C:\Users\TYBCA\Desktop\FSD_2405009>node library.js
Login Successful
Book Found
Book issued Successfully
User logged out

Viva Questions

1. What is a Promise?

A Promise is a JavaScript object that represents the future result of an asynchronous operation. It can either succeed or fail.

---

2. What are the states of a Promise?

A Promise has three states:

- Pending – The operation is still in progress.
- Fulfilled (Resolved) – The operation completed successfully.
- Rejected – The operation failed.

---

3. What is the difference between `resolve()` and `reject()`?

`resolve()`	`eject()`
Called when the operation is successful.	Called when the operation fails.
Changes the Promise to fulfilled.	Changes the Promise to rejected.
Handled by `.then()`.	Handled by `.catch()`.

---

4. What is the purpose of `.then()`?

`.then()` is used to handle the successful result of a Promise.

---

5. What is the purpose of `.catch()`?

`.catch()` is used to handle errors or rejected Promises.

---

6. What is Promise Chaining?

Promise Chaining means connecting multiple asynchronous operations using multiple `.then()` methods, where each step starts after the previous one finishes.

---

7. What happens after a Promise is rejected?

When a Promise is rejected:

- The remaining `.then()` methods are skipped.
- Control goes directly to the `.catch()` method.

---

8. What are the advantages of Promises over callbacks?

- Easier to read and understand.
- Avoids callback hell (nested callbacks).
- Better error handling using `.catch()`.
- Supports Promise Chaining.
- Makes asynchronous code more organized.

Week 5

PART -1

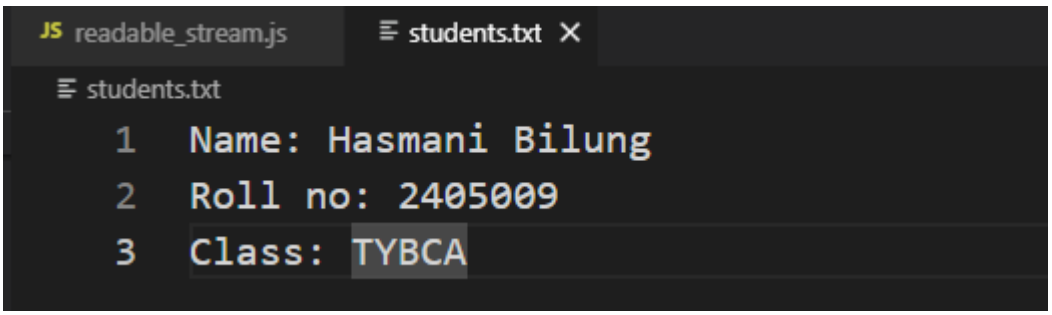
Node.js Lab Manual: Streams and Standard Streams

Exercise 1: Readable Stream

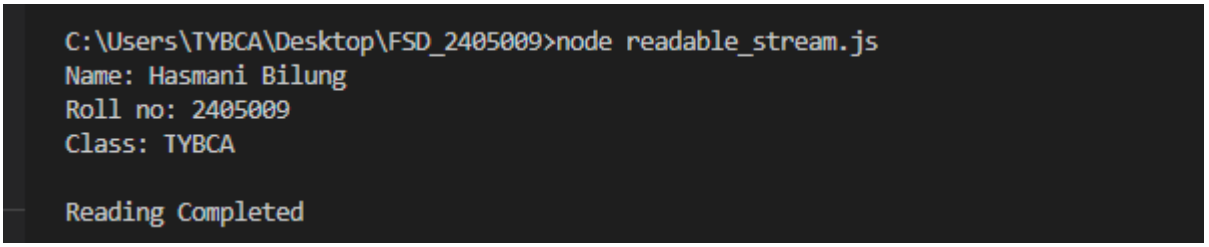
readable_stream.js

```
const fs = require('fs');
const s = fs.createReadStream('students.txt','utf8');
s.on('data', c=> console.log(c));
s.on('end', ()=> console.log('\nReading Completed'));
```

Students.txt



Output



Exercise 2: Read Large File

```
const fs = require('fs');
const s = fs.createReadStream('sample_file.txt','utf8');
s.on('data', c => {console.log('Chunk
Size:',c.length);console.log(c);});
s.on('end',()=>console.log('Completed'));
```

sample_file.txt

```
perfect song lyrics

I found a love for me
Oh, darling, just dive right in and follow my lead
Well, I found a girl, beautiful and sweet
Oh, I never knew you were the someone waitin' for me

'Cause we were just kids when we fell in love, not knowin' what it was
I will not give you up this time
Oh, darling, just kiss me slow, your heart is all I own
And in your eyes, you're holding mine

Baby, I'm dancin' in the dark with you between my arms
Barefoot on the grass while listenin' to our favourite song
When you said you looked a mess, I whispered underneath my breath
But you heard it, "Darling, you look perfect tonight"

Well, I found a woman, stronger than anyone I know
She shares my dreams, I hope that someday, I'll share her home
I found a love to carry more than just my secrets
To carry love, to carry children of our own

We are still kids, but we're so in love, fightin' against all odds
I know we'll be alright this time
Darling, just hold my hand, be my girl, I'll be your man
I see my future in your eyes

Oh, baby, I'm dancin' in the dark with you between my arms
Barefoot on the grass while listenin' to our favourite song
When I saw you in that dress, lookin' so beautiful
I don't deserve this, darling, you look perfect tonight

No, no, no
Mm, oh

Baby, I'm dancin' in the dark with you between my arms
Barefoot on the grass while listenin' to our favourite song
I have faith in what I see, now I know I have met
An angel in person, and she looks perfect
Though I don't deserve this, you look perfect tonight
```

Output

C:\Users\TYBCA\Desktop\FSD_2405009>node large_file.js
Chunk Size: 1567
perfect song lyrics

I found a love for me
Oh, darling, just dive right in and follow my lead
Well, I found a girl, beautiful and sweet
Oh, I never knew you were the someone waitin' for me

'Cause we were just kids when we fell in love, not knowin' what it was

I will not give you up this time
Oh, darling, just kiss me slow, your heart is all I own
And in your eyes, you're holding mine

Baby, I'm dancin' in the dark with you between my arms
Barefoot on the grass while listenin' to our favourite song
When you said you looked a mess, I whispered underneath my breath
But you heard it, "Darling, you look perfect tonight"

Well, I found a woman, stronger than anyone I know
She shares my dreams, I hope that someday, I'll share her home
I found a love to carry more than just my secrets
To carry love, to carry children of our own

We are still kids, but we're so in love, fightin' against all odds
I know we'll be alright this time
Darling, just hold my hand, be my girl, I'll be your man
I see my future in your eyes

Oh, baby, I'm dancin' in the dark with you between my arms
Barefoot on the grass while listenin' to our favourite song
When I saw you in that dress, lookin' so beautiful
I don't deserve this, darling, you look perfect tonight

No, no, no
Mm, oh

Baby, I'm dancin' in the dark with you between my arms
Barefoot on the grass while listenin' to our favourite song
I have faith in what I see, now I know I have met
An angel in person, and she looks perfect
Though I don't deserve this, you look perfect tonight
Completed

Exercise 3: Copy File

```
const fs = require('fs');

const rd = fs.createReadStream('note.txt');
const wr = fs.createWriteStream('backup.txt');
```

```
rd.pipe(wr);
```

note.txt

```
1  Name: Hasmani Bilung
2  Roll no: 2405009
3  Class: TYBCA
4
```

backup.txt

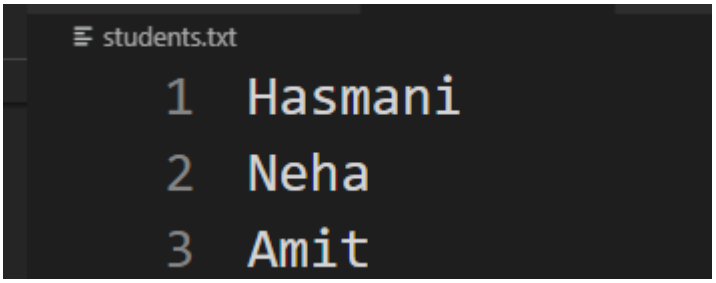
```
1  Name: Hasmani Bilung
2  Roll no: 2405009
3  Class: TYBCA
4
```

Exercise 4: Writable Stream

```
const fs=require('fs');
const w=fs.createWriteStream('students.txt');
w.write('Hasmani\n');
w.write('Neha\n');
w.write('Arjun\n');

w.end();
```

Output



Exercise 5: Attendance

```
const fs = require('fs');
const w = fs.createWriteStream('attendance.txt');

w.write("Roll no 1 : Present\n");
w.write("Roll no 2 : Present\n");
w.write("Roll no 3 : Absent\n");
w.write("Roll no 4 : Present\n");
w.write("Roll no 5 : Present\n");
w.write("Roll no 6 : Present\n");
w.write("Roll no 7 : Absent\n");
w.write("Roll no 8 : Absent\n");
w.write("Roll no 9 : Present\n");
w.write("Roll no 10 : Present\n");

w.end();
```

Output

attendance.txt

```
Roll no 1 : Present
Roll no 2 : Present
Roll no 3 : Absent
Roll no 4 : Present
Roll no 5 : Present
Roll no 6 : Present
Roll no 7 : Absent
Roll no 8 : Absent
Roll no 9 : Present
Roll no 10 : Present
```

Exercise 6: Copy PDF

```
const fs = require('fs');

const rd = fs.createReadStream('resume.pdf');
const wr = fs.createWriteStream('resume_backup.pdf');

rd.pipe(wr);
```

resume.pdf

```
Name: Hasmani Bilung
Roll no: 2405009
Class: TYBCA
```

resume_backup.pdf

```
1 Name: Hasmani Bilung
2 Roll no: 2405009
3 Class: TYBCA
4
```

Exercise 7: stdin

```
process.stdout.write('Enter Name: ');
process.stdin.on('data',
    d => {console.log('Welcome',
        d.toString().trim());
        process.exit();
    });
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009>node stdin.js
Enter Name: Hasmani Bilung
Welcome Hasmani Bilung
```

Exercise 8: Student Info

```
const rl=require('readline').createInterface
({input:process.stdin,output:process.stdout});
rl.question('Name: ',n=>
    {rl.question('Roll no: ',r=>
        {rl.question('Course: ',c=>
            {console.log({name:n,roll:r,course:c});
            rl.close();
        });
    });
});
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009>node student_info.js
Name: Hasmani Bilung
Roll no: 2405009
Course: BCA
{ name: 'Hasmani Bilung', roll: '2405009', course: 'BCA' }
```

Exercise 9: Calculator

```
const rl=require('readline').createInterface
({input:process.stdin,output:process.stdout
});
rl.question('A:',a=>
    {rl.question('B:',b=>{
        c = parseInt(a)+parseInt(b);
                console.log("Addition: " +c);
                console.log("Subtraction: " + (a-b));
                console.log("Multiplication: "+ (a*b));
                console.log("Division: "+ (a/b));
                rl.close();
            });
        });
```

Output

C:\Users\TYBCA\Desktop\FSD_2405009>node calculator.js
A:50
B:30
Addition: 80
Subtraction: 20
Multiplication: 1500
Division: 1.6666666666666667

Exercise 10: stderr

```
process.stdout.write('Age: ');
process.stdin.on('data',d=>
```

```
    {let a=+d;
        if(a<17)process.stderr.write('Invalid Age\n');
        else console.log('Eligible');
        process.exit();
    });
```

Output
C:\Users\TYBCA\Desktop\FSD_2405009>node stderr.js
Age: 20
Eligible

C:\Users\TYBCA\Desktop\FSD_2405009>node stderr.js
Age: 15
Invalid Age

Exercise 11: stdout Redirect

```
console.log('Student Name: Hasmani Bilung');
console.log('Roll No: 2405009');
console.log('Course: BCA');
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009>node redirect.js > output.txt
```

JS redirect.js

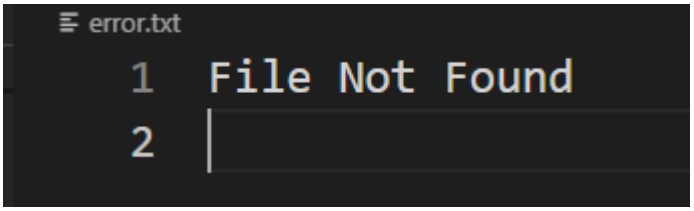
```
1  console.log('Student Name: Hasmani Bilung');
2  console.log('Roll No: 2405009');
3  console.log('Course: BCA');
```

Exercise 12: stderr Redirect

```
process.stderr.write('File Not Found\n');
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009>node stderr_redirect.js 2> error.txt
```



Mini Project

Develop a Student File Management System using Readable Streams, Writable Streams, stdin, stdout, stderr and pipe().

```
const fs = require("fs");

// Function to display menu
function showMenu() {
    console.log("\n===== STUDENT FILE MANAGEMENT SYSTEM
=====");
    console.log("1. Read Student File");
    console.log("2. Copy Student File using pipe()");
    console.log("3. Add New Student");
    console.log("4. Exit");

    process.stdout.write("\nEnter your choice: ");
}

// Take input using stdin
process.stdin.setEncoding("utf8");

showMenu();

process.stdin.on("data", (input) => {
    const choice = input.trim();

    switch (choice) {

        // 1. Read file using Readable Stream
        case "1":
```

```
            console.log("\n--- Student Records ---");

            const readableStream = fs.createReadStream(
                "students.txt",
                "utf8"
            );

            readableStream.on("data", (chunk) => {
                process.stdout.write(chunk);
            });

            readableStream.on("end", () => {
                console.log("\n\nFile reading
completed.");
                showMenu();
            });

            readableStream.on("error", (error) => {
                process.stderr.write(
                    `Error reading file:
${error.message}\n`
                );
                showMenu();
            });

            break;

        // 2. Copy file using pipe()
        case "2":
            const readStream =
fs.createReadStream("students.txt");

            const writeStream =
fs.createWriteStream("output.txt");

            readStream.pipe(writeStream);
```

```
            writeStream.on("finish", () => {
                process.stdout.write(
                    "\nStudent file copied successfully to
output.txt\n"
                );

                showMenu();
            });

            readStream.on("error", (error) => {
                process.stderr.write(
                    `Error: ${error.message}\n`
                );

                showMenu();
            });

            break;

        // 3. Add a new student
        case "3":
            process.stdout.write(
                "\nEnter student details (Example: Roll
No: 05, Name: John, Course: BCA): "
            );

            process.stdin.once("data", (studentData) => {

                const writableStream =
fs.createWriteStream(
                    "students.txt",
                    { flags: "a" }
                );
```

```
                writableStream.write("\n" +
studentData.trim());

                writableStream.end();

                writableStream.on("finish", () => {
                    process.stdout.write(
                        "\nStudent added successfully.\n"
                    );

                    showMenu();
                });

                writableStream.on("error", (error) => {
                    process.stderr.write(
                        `Error writing file:
${error.message}\n`
                    );

                    showMenu();
                });
            });

            break;

        // 4. Exit program
        case "4":
            process.stdout.write(
                "\nExiting Student File Management
System...\n"
            );

            process.exit();

            break;
```

```
        default:
            process.stderr.write(
                "\nInvalid choice! Please enter a number
between 1 and 4.\n"
            );

            showMenu();
    }
});
```

student-file-management > students.txt

```
1  Roll No: 2405009, Name: Hasmani, Course: BCA
2  Roll No: 24050033, Name: Shreya, Course: BCA
3  Roll No: 24050040, Name: Mansi, Course: BCA
4  Roll No: 24050005, Name: Anisha, Course: BCA
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009\student-file-management>node app.js

===== STUDENT FILE MANAGEMENT SYSTEM =====
1. Read Student File
2. Copy Student File using pipe()
3. Add New Student
4. Exit

Enter your choice: 1

--- Student Records ---
Roll No: 2405009, Name: Hasmani, Course: BCA
Roll No: 24050033, Name: Shreya, Course: BCA
Roll No: 24050040, Name: Mansi, Course: BCA
Roll No: 24050005, Name: Anisha, Course: BCA

File reading completed.
```

```
===== STUDENT FILE MANAGEMENT SYSTEM =====
1. Read Student File
2. Copy Student File using pipe()
3. Add New Student
4. Exit

Enter your choice: 2

Student file copied successfully to output.txt
```

```
student-file-management > output.txt
1   Roll No: 2405009, Name: Hasmani, Course: BCA
2   Roll No: 24050033, Name: Shreya, Course: BCA
3   Roll No: 24050040, Name: Mansi, Course: BCA
4   Roll No: 24050005, Name: Anisha, Course: BCA
```

```
===== STUDENT FILE MANAGEMENT SYSTEM =====
1. Read Student File
2. Copy Student File using pipe()
3. Add New Student
4. Exit

Enter your choice:
Student added successfully.

===== STUDENT FILE MANAGEMENT SYSTEM =====
1. Read Student File
2. Copy Student File using pipe()
3. Add New Student
4. Exit

Enter your choice: 4

Exiting Student File Management System...
```

```
student-file-management > students.txt
1   Roll No: 2405009, Name: Hasmani, Course: BCA
2   Roll No: 24050033, Name: Shreya, Course: BCA
3   Roll No: 24050040, Name: Mansi, Course: BCA
4   Roll No: 24050005, Name: Anisha, Course: BCA
5   Roll No: 2405011, Name: Khushi, Course: BCA
```

Viva Questions

1. What is a Stream?

A stream is a mechanism for reading or writing data in small chunks instead of loading the entire data into memory at once.

2. What is `pipe()`?

`pipe()` is a method that transfers data from a Readable Stream directly to a Writable Stream automatically.

3. Difference between Readable and Writable Streams

Readable Stream	Writable Stream
Reads data	Writes data
Source of data	Destination of data
Used for input	Used for output

4. Explain `stdin`, `stdout`, and `stderr`

- **stdin (Standard Input):** Receives input from the user.
- **stdout (Standard Output):** Displays normal output.
- **stderr (Standard Error):** Displays error messages.

5. Why are streams efficient for large files?

Streams process data in **small chunks**, so they use less memory, improve performance, and allow large files to be handled efficiently.

PART - 2

Node.js Lab Exercises
Server, Routing, Request Headers, and API Gateway

Lab Exercise 1: Creating a Basic HTTP Server

```
JS server.js > ...
1   const http = require('http');
2
3   const server = http.createServer((req, res) => {
4     res.write('Hello, World!');
5     res.end();
6   });
7
8   server.listen(3000, () => {
9     console.log('Server running at http://localhost:3000');
10  });
```

Objective: Learn how to create a simple HTTP server using Node.js.

Problem Statement: Create a Node.js server that listens on port 3000. When a client visits the server, display ‘Welcome to Node.js Server’.

```
const http = require('http');
const server = http.createServer((req, res) => {
res.write("Welcome to Node.js Server");
res.end();
});
server.listen(3000, () => {
console.log("Server running on http://localhost:3000");
});
```

Output

```
C:\Users\DELL\OneDrive\Desktop\FSD - 09>node server.js
Server running on http://localhost:3000
```



Lab Exercise 2: Server with HTML Response
Objective: Send HTML content from a Node.js server.

Problem Statement:
Create a server that displays a heading, student name, and course using HTML.

```
const http = require('http');

const server = http.createServer((req,res) => {
    res.writeHead(200, {"Content-Type":
"text/html"});
        res.write('<h1>Node.js
Practical</h1><h2>Student : Hasmani </h2> <h3>Course:
BCA</h3>');
            res.end();
});

server.listen(3000);
```

Output



Lab Exercise 3: Creating Routes
Objective: Understand URL routing.

Routes:
/ -> Home Page

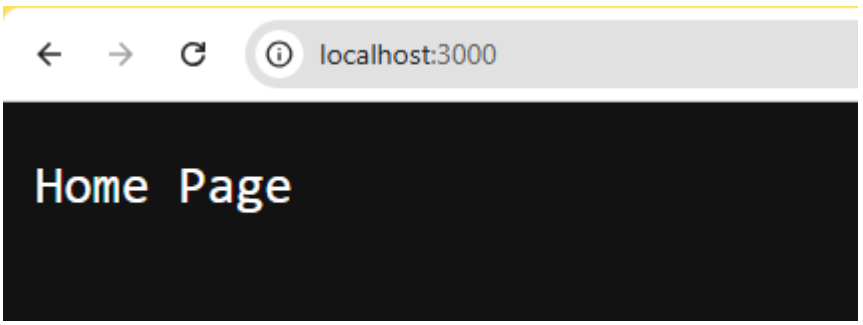
/about -> About Us
/contact -> Contact Page
/services -> Services Page
Others -> 404 Page Not Found

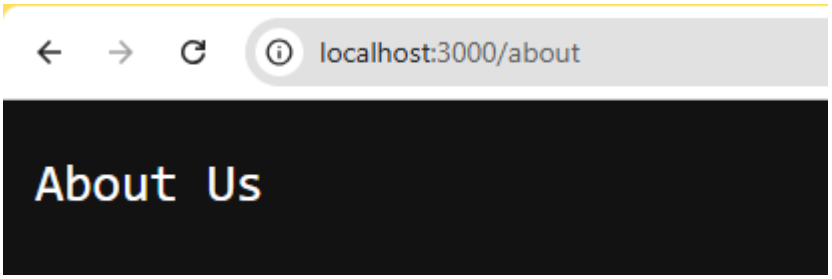
```
const http = require('http');

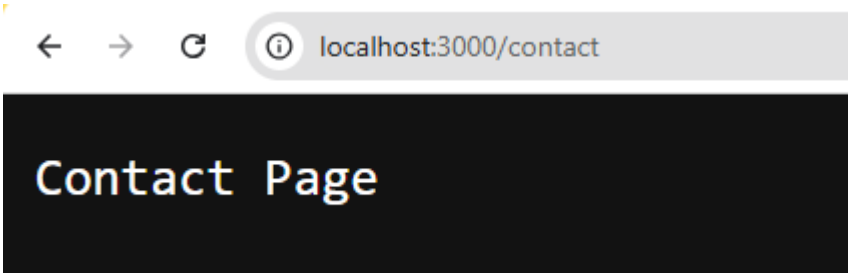
const server = http.createServer((req,res) => {
    if(req.url==="/") {res.end("Home Page");}
    else if(req.url==="/about") {res.end("About
Us");}
    else if(req.url==="/contact") {res.end("Contact
Page");}
    else if(req.url==="/services") {res.end("Services
Page");}
    else{
        res.writeHead(404);
        res.end("404 Page Not Found");
    }
});

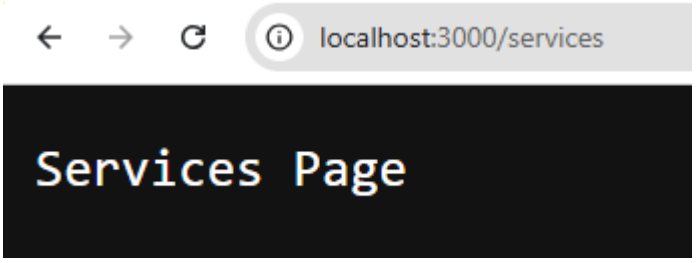
server.listen(3000,() => {
    console.log("Server is running");
});
```

Output







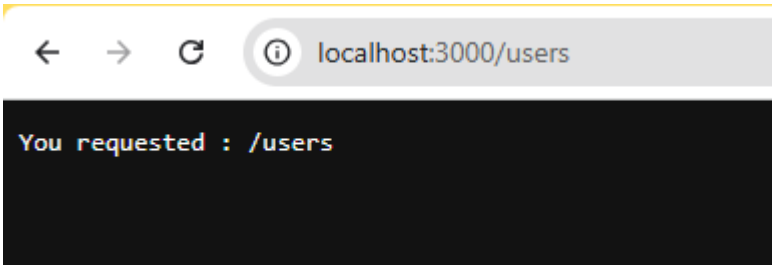


Lab Exercise 4: Dynamic Route
Objective: Display the requested URL.

```
const http = require('http');

http.createServer((req,res) => {
    res.end(`You requested : ${req.url}`);
}).listen(3000);
```

Output

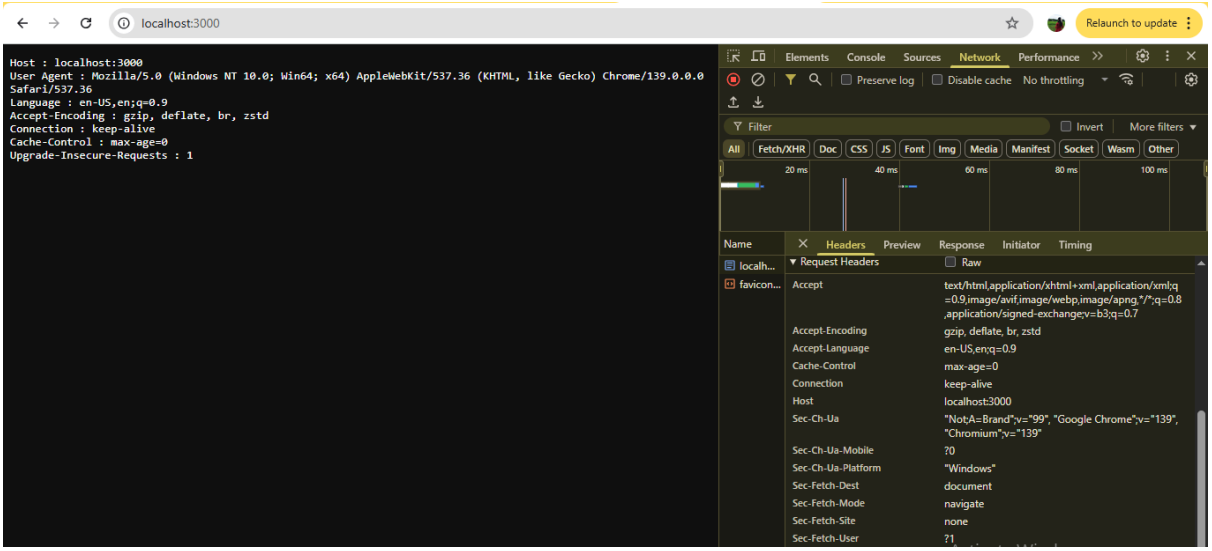


Lab Exercise 5: Accessing Request Headers
Objective: Read HTTP request headers.

```
const http = require('http');

http.createServer((req,res) => {
    res.writeHead(200, {"Content-Type" :
"text/plain"});
    res.write(`Host : ${req.headers.host}\n`);
    res.write(`User Agent :
${req.headers['user-agent']}\n`);
    res.write(`Language :
${req.headers['accept-language']}\n`);
    res.write(`Accept-Encoding :
${req.headers['accept-encoding']}\n`);
    res.write(`Connection :
${req.headers.connection}\n`);
    res.write(`Cache-Control :
${req.headers['cache-control']}\n`);
    res.write(`Upgrade-Insecure-Requests :
${req.headers['upgrade-insecure-requests']}\n`);
    res.end();
}).listen(3000);
```

Output



Lab Exercise 6: Display All Request Headers

Objective: Display all request headers.

```
const http = require('http');

http.createServer((req,res) => {
```

```
    res.writeHead(200, {"Content-Type" : "application/json"});
    res.end(JSON.stringify(req.headers,null,4));
}).listen(3000);
```

Output

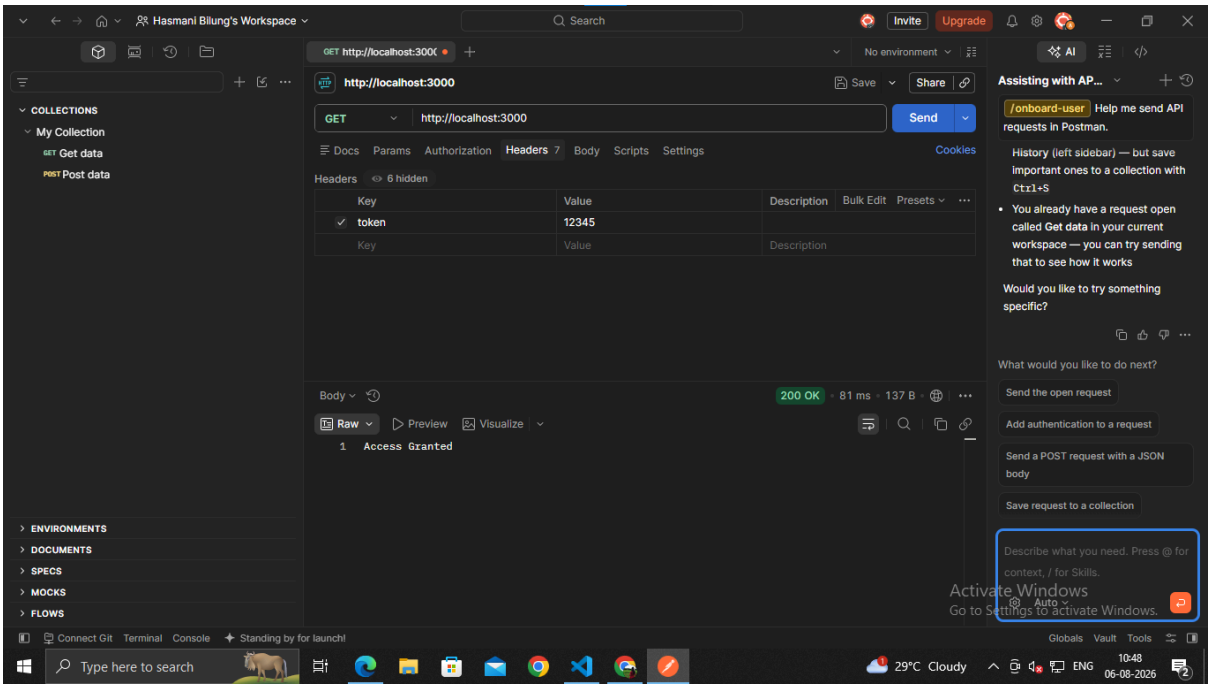


Lab Exercise 7: Header-Based Authentication
Objective: Use a custom request header named token.

```
const http = require('http');

http.createServer((req,res) => {
    const token =  req.headers.token;
    if(token === "12345")
        res.end("Access Granted");
    else res.end("Access Denied");
}).listen(3000);
```

Output



Lab Exercise 8: Simple API Gateway

Objective: Route requests to different services.

```
const http = require('http');

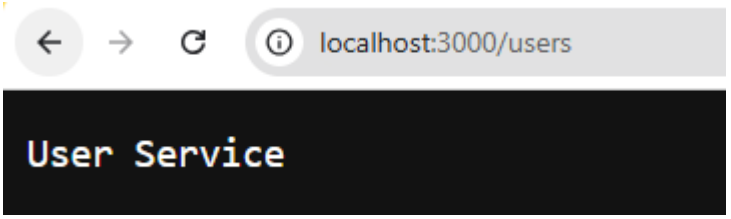
http.createServer((req, res) => {
    switch (req.url) {
        case "/users":
            res.end("User Service");
            break;

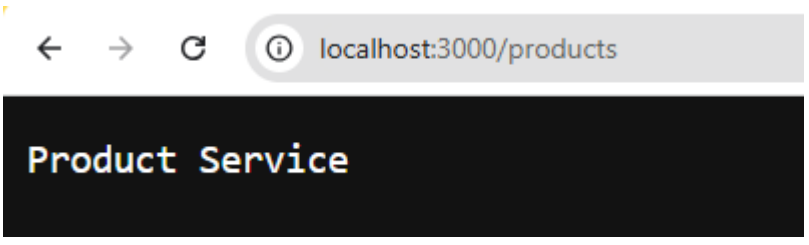
        case "/products":
            res.end("Product Service");
            break;

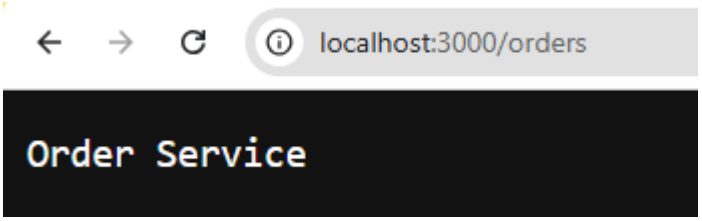
        case "/orders":
            res.end("Order Service");
            break;

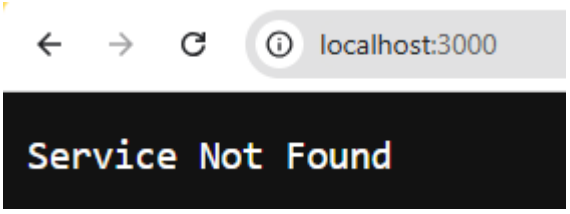
        default:
            res.writeHead(404);
            res.end("Service Not Found");
    }
}).listen(3000);
```

Output









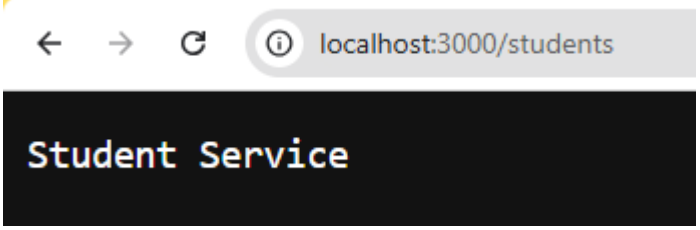
Lab Exercise 9: API Gateway with Logging
Objective: Log request details before routing.

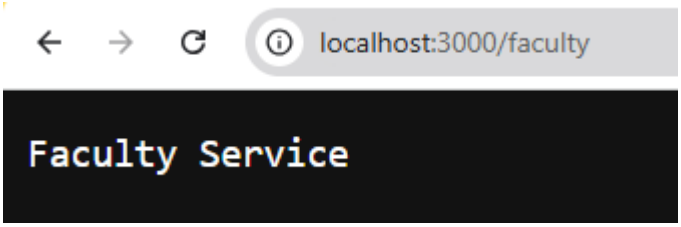
```
const http = require('http');

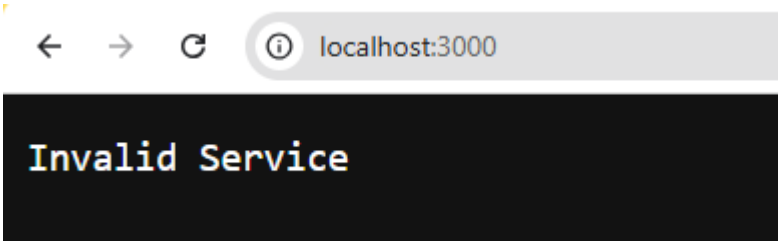
http.createServer((req, res) => {
    console.log(`${new Date()} | ${req.method} |
${req.url}`);

    if (req.url === "/students") {
        res.end("Student Service");
    }
    else if (req.url === "/faculty") {
        res.end("Faculty Service");
    }
    else {
        res.writeHead(404);
        res.end("Invalid Service");
    }
}).listen(3000);
```

Output








```
C:\Users\TYBCA\Desktop\FSD_2405009>node 5Ex9.js
Fri Aug 21 2026 13:28:41 GMT+0530 (India Standard Time) | GET | /
Fri Aug 21 2026 13:29:29 GMT+0530 (India Standard Time) | GET | /students
Fri Aug 21 2026 13:29:50 GMT+0530 (India Standard Time) | GET | /faculty
Fri Aug 21 2026 13:30:21 GMT+0530 (India Standard Time) | GET | /
```

Lab Exercise 10: Mini Project – College API Gateway

Objective: Build a simple API gateway for a college management system.

```
const http = require('http');

http.createServer((req, res) => {

    console.log("Method:", req.method);
    console.log("URL:", req.url);
    console.log("Client:", req.socket.remoteAddress);
    console.log("Headers:", req.headers);

    switch (req.url) {

        case "/students": {
            res.end("Student Management Service");
            break;
        }
```

```
        case "/faculty": {
            res.end("Faculty Management Service");
            break;
        }

        case "/courses": {
            res.end("Course Management Service");
            break;
        }

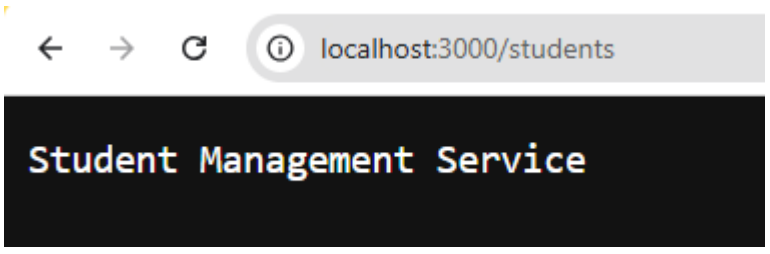
        case "/fees": {
            res.end("Fee Management Service");
            break;
        }

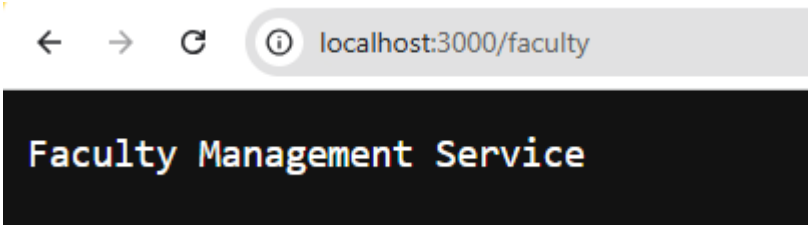
        case "/library": {
            res.end("Library Management Service");
            break;
        }

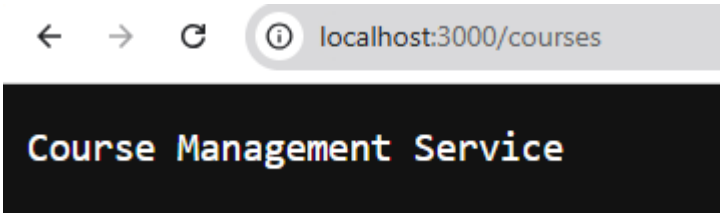
        default: {
            res.writeHead(404);
            res.end("404 - Service Not Found");
        }
    }

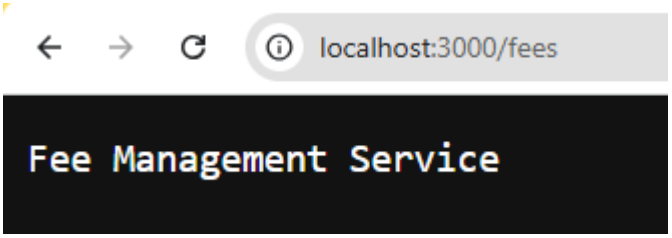
}).listen(3000);
```

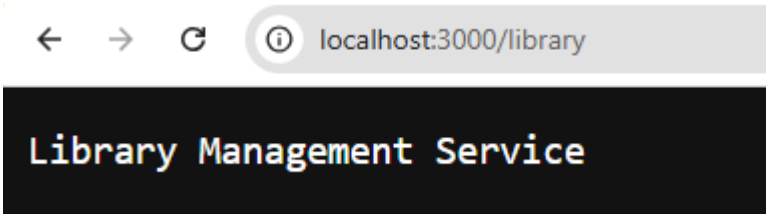
Output

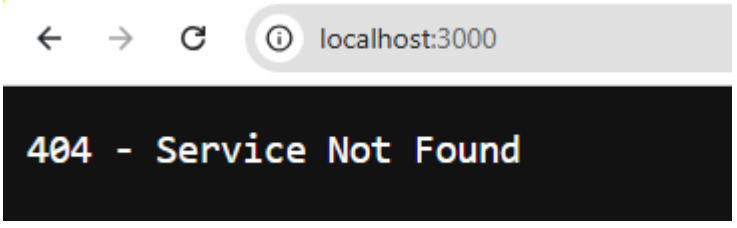



localhost:3000/faculty
Faculty Management Service

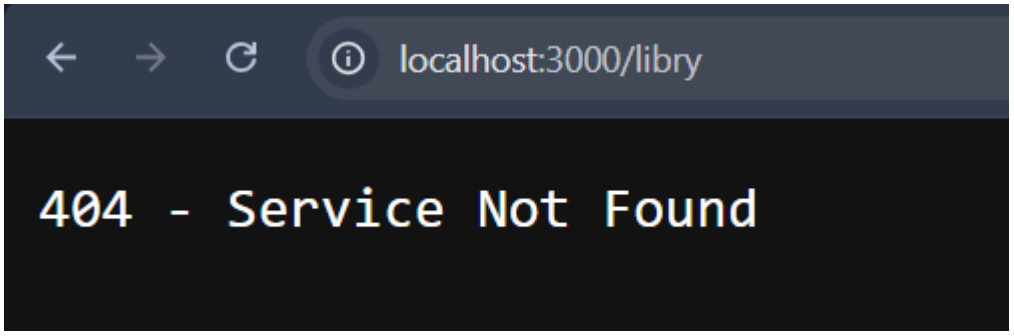

localhost:3000/courses
Course Management Service


localhost:3000/fees
Fee Management Service


localhost:3000/library
Library Management Service


localhost:3000
404 - Service Not Found

```
C:\Users\TYBCA\Desktop\FSD_2405009>node 5Ex10.js
Method: GET
URL: /students
Client: ::1
Headers: {
  host: 'localhost:3000',
  connection: 'keep-alive',
  'sec-ch-ua': '"Not=A?Brand";v="99", "Google Chrome";v="151", "Chromium";v="151"',
  'sec-ch-ua-mobile': '?0',
  'sec-ch-ua-platform': '"Windows"',
  'upgrade-insecure-requests': '1',
  'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36',
  accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7',
  'sec-fetch-site': 'none',
  'sec-fetch-mode': 'navigate',
  'sec-fetch-user': '?1',
  'sec-fetch-dest': 'document',
  'accept-encoding': 'gzip, deflate, br, zstd',
  'accept-language': 'en-US,en;q=0.9'
}
Method: GET
URL: /fauclty
Client: ::1
Headers: {
  host: 'localhost:3000',
  connection: 'keep-alive',
  'sec-ch-ua': '"Not=A?Brand";v="99", "Google Chrome";v="151", "Chromium";v="151"',
  'sec-ch-ua-mobile': '?0',
  'sec-ch-ua-platform': '"Windows"',
  'upgrade-insecure-requests': '1',
  'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36',
  accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7',
  'sec-fetch-site': 'none',
  'sec-fetch-mode': 'navigate',
  'sec-fetch-user': '?1',
  'sec-fetch-dest': 'document',
  'accept-encoding': 'gzip, deflate, br, zstd',
  'accept-language': 'en-US,en;q=0.9'
}
```



WEEK – 6

PART - 1

Creating a Cron Job using Node.js

Create the Node.js Project

Create a new folder and open it in the terminal. Then run:
npm init -y
npm install node-cron

```
C:\Users\TYBCA\Desktop\FSD_2405009>npm init -y
Wrote to C:\Users\TYBCA\Desktop\FSD_2405009\package.json:

{
  "name": "fsd_2405009",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "chalk": "^4.1.2",
    "figlet": "^1.11.0",
    "validator": "^13.15.35"
  },
  "devDependencies": {}
}



C:\Users\TYBCA\Desktop\FSD_2405009>npm install node-cron

added 1 package, removed 1 package, and audited 11 packages in 902ms

2 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Basic Cron Job

every minute

```
const cron = require('node-cron');

cron.schedule('* * * * *', () => {
    console.log('Cron job executed:', new Date());
});
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009\week_6>node cron.js
Cron job executed: 2026-08-10T03:52:00.018Z
Cron job executed: 2026-08-10T03:53:00.014Z
Cron job executed: 2026-08-10T03:54:00.011Z
Cron job executed: 2026-08-10T03:55:00.009Z
Cron job executed: 2026-08-10T03:56:00.005Z
Cron job executed: 2026-08-10T03:57:00.011Z
Cron job executed: 2026-08-10T03:58:00.006Z
```

Every 5 minutes

```
const cron = require("node-cron");

cron.schedule("*/5 * * * *", () => {
  console.log("Cron job executed:", new Date());
});
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009\week_6>node cron1.js
Cron job executed: 2026-08-10T04:10:00.005Z
Cron job executed: 2026-08-10T04:15:00.017Z
Cron job executed: 2026-08-10T04:20:00.006Z
```

Display message at 10:01 AM on August 10

```
const cron = require("node-cron");

cron.schedule("01 10 10 8 *", () => {
  console.log("Cron job executed:", new Date());
});
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009\week_6>node cron2.js
Cron job executed: 2026-08-10T04:31:00.007Z
```

Display message at 9:56 AM on the 10th day of every month

```
const cron = require("node-cron");

cron.schedule("56 9 10 * *", () => {
  console.log("Cron job executed:", new Date());
});
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009\week_6>node cron3.js
Cron job executed: 2026-08-10T04:26:00.016Z
```

Display message at **10:06 AM on August 10, only on Mondays**

```
const cron = require("node-cron");

cron.schedule("06 10 10 8 1", () => {
  console.log("Cron job executed:", new Date());
});
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009\week_6>node cron4.js
Cron job executed: 2026-08-10T04:36:00.014Z
```

Display message at **15 minutes past every hour**

```
const cron = require("node-cron");

cron.schedule("15 */1 * * *", () => {
```

```
  console.log("Cron job executed:", new Date());
});
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009\week_6>node cron5.js
Cron job executed: 2026-08-10T04:45:00.010Z
```

Display message after every 2 minutes during the 10 AM hour

```
const cron = require('node-cron');

cron.schedule('*/2 10 * * *', () => {
    console.log('Cron job executed:', new Date());
});
```

Using a Separate Function

It is good practice to keep the actual task in a separate function:

```
const cron = require('node-cron');

function generateReport() {
    console.log('Generating daily report....');
    console.log('Report generated successfully.');
}

cron.schedule('38 10 * * *', () => {
    generateReport();
});

console.log('Cron scheduler started....');
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009\week_6>node cron7.js
Cron scheduler started....
Generating daily report....
Report generated successfully.
```

Lab Exercise

Exercise: Create a Scheduled Task using Node.js
Create a Node.js program that uses node-cron to execute a function automatically.

Requirements:
1. Install the node-cron module.
2. Create a Node.js program that executes a function every minute.
3. The function should display the current date.
4. The function should display the current time.
5. The function should display the message: “Scheduled task executed successfully”
6. Modify the cron expression so that the task executes every 5 minutes.

Expected Output
Cron scheduler started...
Scheduled task executed successfully
Date: 09/08/2026
Time: 19:40:00

Every minute

```
const cron = require('node-cron');

function scheduledTask() {
    const now = new Date();

    console.log("Current Date:",
now.toLocaleDateString());
    console.log("Current Time:",
now.toLocaleTimeString());
```

```
    console.log("Scheduled task executed
successfully");
    console.log("------------------------------");
}

cron.schedule('* * * * *', scheduledTask);
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009\week_6>node cron9.js
Current Date: 21/8/2026
Current Time: 1:49:00 pm
Scheduled task executed successfully
---------------------------
Current Date: 21/8/2026
Current Time: 1:50:00 pm
Scheduled task executed successfully
---------------------------
Current Date: 21/8/2026
Current Time: 1:51:00 pm
Scheduled task executed successfully
---------------------------
```

Every 5 minutes

```
const cron = require('node-cron');

function scheduledTask() {
    const now = new Date();

    console.log("Current Date:",
now.toLocaleDateString());
    console.log("Current Time:",
now.toLocaleTimeString());
    console.log("Scheduled task executed
successfully");
    console.log("------------------------------");
}

cron.schedule('*/5 * * * *', scheduledTask);
```

Output

```
C:\Users\TYBCA\Desktop\FSD_2405009\week_6>node cron9.js
Current Date: 21/8/2026
Current Time: 1:55:00 pm
Scheduled task executed successfully
---------------------------
Current Date: 21/8/2026
Current Time: 2:00:00 pm
Scheduled task executed successfully
---------------------------
```

PART – 2

Demonstrating Webpack in Node.js

What is Webpack?
Webpack is a module bundler. It starts from an entry point, follows the application's dependencies, and generates one or more output bundles.

module1.js + module2.js + module3.js → Webpack → bundle.js

Webpack is widely used in frontend projects, but it can also bundle Node.js applications when configured with target: 'node'.

Exercise 1: Create a Node.js Project
Objective: Initialize a Node.js project and install Webpack.

1. Create a folder named webpack-demo.
2. Open it in VS Code.
3. Open the terminal.
4. Initialize npm.
5. npm init -y
6. Install Webpack and webpack-cli:
7. npm install --save-dev webpack webpack-cli

Initial structure:
webpack-demo/
├── node_modules/

├── package.json
└── package-lock.json

```
C:\Users\TYBCA\Desktop\webpack-demo_09>npm init -y
Wrote to C:\Users\TYBCA\Desktop\webpack-demo_09\package.json:

{
  "name": "webpack-demo_09",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

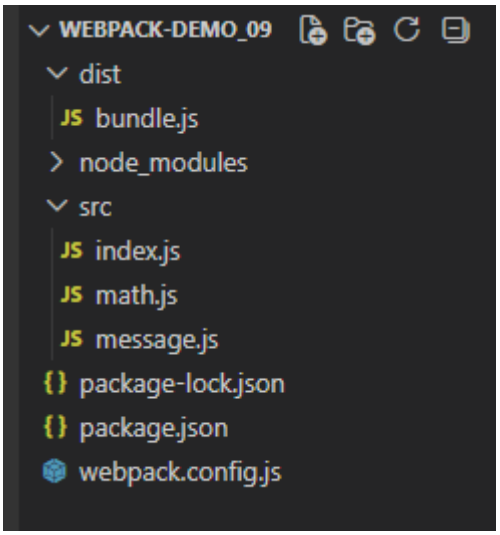


C:\Users\TYBCA\Desktop\webpack-demo_09>npm install --save-dev webpack webpack-cli

added 105 packages, and audited 106 packages in 16s

17 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

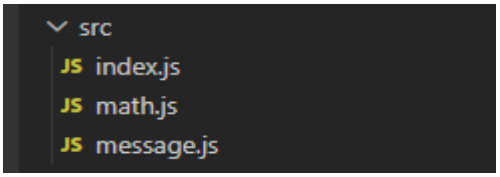


Exercise 2: Create Multiple Modules

Create the following structure:

webpack-demo/
├── src/
│ ├── math.js

```
│   ├── message.js
│   └── index.js
├── package.json
└── package-lock.json
```



math.js

```
function add(a, b) {
return a + b;
}
function multiply(a, b) {
return a * b;
}
module.exports = { add, multiply };
```

message.js

```
function welcome(name) {
return `Welcome ${name} to Node.js Webpack!`;
}
module.exports = welcome;
```

index.js

```
const math = require('./math');
const welcome = require('./message');

console.log(welcome("Hasmani"));
console.log("Addition:", math.add(10, 20));
console.log("Multiplication:", math.multiply(10,
20));
```

Exercise 3: Run Without Webpack

Node.js Lab – Webpack Demonstration

Run the source application directly:
node src/index.js

Expected output:
Welcome Students to Node.js Webpack!
Addition: 30
Multiplication: 200

Output

```
C:\Users\TYBCA\Desktop\webpack-demo_09>node src/index.js
Welcome Hasmani to Node.js Webpack!
Addition: 30
Multiplication: 200
```

Exercise 4: Configure Webpack
Create webpack.config.js in the project root:

```
const path = require('path');

module.exports = {
    target: 'node',

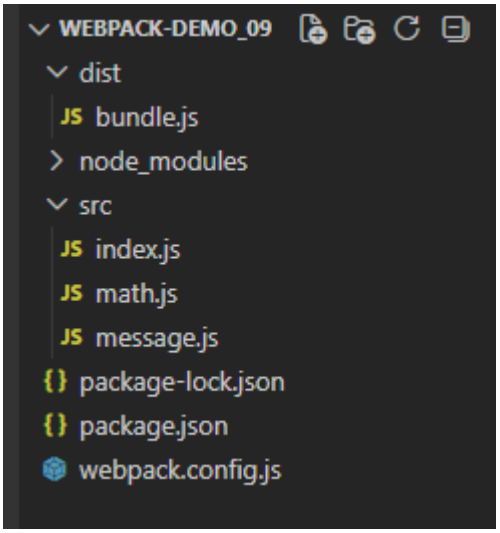
    entry: './src/index.js',

    output: {
        filename: 'bundle.js',
        path: path.resolve(__dirname, 'dist')
    }
};
```

Exercise 5: Build the Application
Run:
npx webpack

A dist folder containing bundle.js should be created:

```
webpack-demo/
├── src/
├── dist/
│   └── bundle.js
├── webpack.config.js
├── package.json
└── package-lock.json
```




```
C:\Users\TYBCA\Desktop\webpack-demo_09>npx webpack
asset bundle.js 444 bytes [compared for emit] [minimized] (name: main)
./src/index.js 207 bytes [built] [code generated]
./src/math.js 120 bytes [built] [code generated]
./src/message.js 101 bytes [built] [code generated]

WARNING in configuration
The 'mode' option has not been set, webpack will fallback to 'production' for this value.
Set 'mode' option to 'development' or 'production' to enable defaults for each environment.
You can also set it to 'none' to disable any default behavior. Learn more: https://webpack.js.org/configuration/mode/

webpack 5.109.2 compiled with 1 warning in 204 ms
```

Exercise 6: Run the Bundle

Node.js Lab – Webpack Demonstration

Execute:
node dist/bundle.js

Expected output:

Welcome Students to Node.js Webpack!
Addition: 30
Multiplication: 200

```
C:\Users\TYBCA\Desktop\webpack-demo_09>node dist/bundle.js
Welcome Hasmani to Node.js Webpack!
Addition: 30
Multiplication: 200
```

Exercise 7: Add an npm Build Script

In package.json, add:
“scripts” {
“build”: “webpack”
}

Now build using:
npm run build
This is a convenient alternative to typing npx webpack.

```
C:\Users\TYBCA\Desktop\webpack-demo_09>npm run build

> webpack-demo_09@1.0.0 build
> webpack

asset bundle.js 444 bytes [compared for emit] [minimized] (name: main)
./src/index.js 207 bytes [built] [code generated]
./src/math.js 120 bytes [built] [code generated]
./src/message.js 101 bytes [built] [code generated]

WARNING in configuration
The 'mode' option has not been set, webpack will fallback to 'production' for this value.
Set 'mode' option to 'development' or 'production' to enable defaults for each environment.
You can also set it to 'none' to disable any default behavior. Learn more: https://webpack.js.org/configuration/mode/

webpack 5.109.2 compiled with 1 warning in 177 ms
```

Exercise 8: Add Another Module
Create src/student.js:

```
function studentDetails(name, course) {
    return `Student: ${name}, Course: ${course}`;
}
```

```
module.exports = studentDetails;
```

Modify src/index.js:

```
const math = require('./math');
const welcome = require('./message');
const studentDetails = require('./students');

console.log(welcome("Students"));
console.log(studentDetails("Hasmani", "BCA"));
console.log("Addition:", math.add(10, 20));
console.log("Multiplication:", math.multiply(10,
20));
```

Build and execute:
npm run build
node dist/bundle.js

Output

```
C:\Users\TYBCA\Desktop\webpack-demo_09>node dist/bundle.js
Welcome Students to Node.js Webpack!
Student: Hasmani, Course: BCA
Addition: 30
Multiplication: 200
```

Exercise: Challenge Activity
Create src/course.js. Add a function that accepts a course code and course name and returns a formatted message. Import it into index.js, use it, rebuild the application, and run bundle.js.

src/course.js

```
function courseDetails(courseCode, courseName) {
    return `Course Code: ${courseCode}, Course Name: ${courseName}`;
}

module.exports = courseDetails;
```

src > JS index.js > ...

```
1  const math = require('./math');
2  const welcome = require('./message');
3  const studentDetails = require('./students');
4  const courseDetails = require('./course');
5
6  console.log(welcome("Students"));
7  console.log(studentDetails("Hasmani", "BCA"));
8  console.log(courseDetails("CSA-301", "Full Stack Development"));
9  console.log("Addition:", math.add(10, 20));
10 console.log("Multiplication:", math.multiply(10, 20));
```

Output

```
C:\Users\TYBCA\Desktop\webpack-demo_09>node dist/bundle.js
Welcome Students to Node.js Webpack!
Student: Hasmani, Course: BCA
Course Code: CSA-301, Course Name: Full Stack Development
Addition: 30
Multiplication: 200
```

Week 7 Exercises: NODE.JS WITH SEQUELIZE ORM AND MYSQL

Creating the Project

Create a new project:

mkdir sequelize-demo
cd sequelize-demo
npm init -y

Install the required packages:

npm install sequelize mysql2

```
C:\Users\TYBCA\Desktop\Week7_09>npm init -y
Wrote to C:\Users\TYBCA\Desktop\Week7_09\package.json:

{
  "name": "week7_09",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}



C:\Users\TYBCA\Desktop\Week7_09>mkdir sequelize-demo

C:\Users\TYBCA\Desktop\Week7_09>cd sequelize-demo

C:\Users\TYBCA\Desktop\Week7_09\sequelize-demo>npm install sequelize mysql2
npm warn deprecated dottie@2.0.7: Package no longer supported. Contact Support at https://www.npmjs.com/support for more info.
npm warn deprecated uuid@8.3.2: uuid@10 and below is no longer supported.  For ESM codebases, update to uuid@latest.  For CommonJS codebases, use uuid@11 (but be aware this vers
ion will likely be deprecated in 2028).

added 31 packages, and audited 32 packages in 4s

4 packages are looking for funding
  run `npm fund` for details

2 moderate severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

Connecting Node.js to MySQL

Create a file named db.js:

```
const { Sequelize } = require('sequelize');


const sequelize = new Sequelize(
    'college',
    'root',
    '',
    {
```

```
        host: 'localhost',
        dialect: 'mysql'
    }
);

module.exports = sequelize;
```

Testing the Connection

Create app.js:

```
const sequelize = require('./db');

async function testConnection() {
    try {
        await sequelize.authenticate();
        console.log('Database connection
successful!');
    } catch (error) {
        console.error('Unable to connect to the
database:', error.message);
    }
}

testConnection();
```

Output

```
C:\Users\TYBCA\Desktop\Week7_09\sequelize-demo>node app.js
Executing (default): SELECT 1+1 AS result
Database connection successful!
```

Week 8 Exercises: Sequelize ORM: Models, Migrations and Basic Model Querying

Project Setup

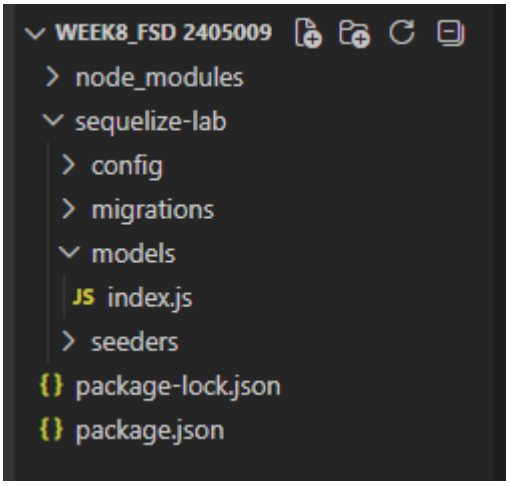
Create a new Node.js project:
mkdir sequelize-lab
cd sequelize-lab
npm init -y

Install Sequelize and the MySQL driver:
npm install sequelize mysql2
npm install --save-dev sequelize-cli

Initialize Sequelize:
npx sequelize-cli init



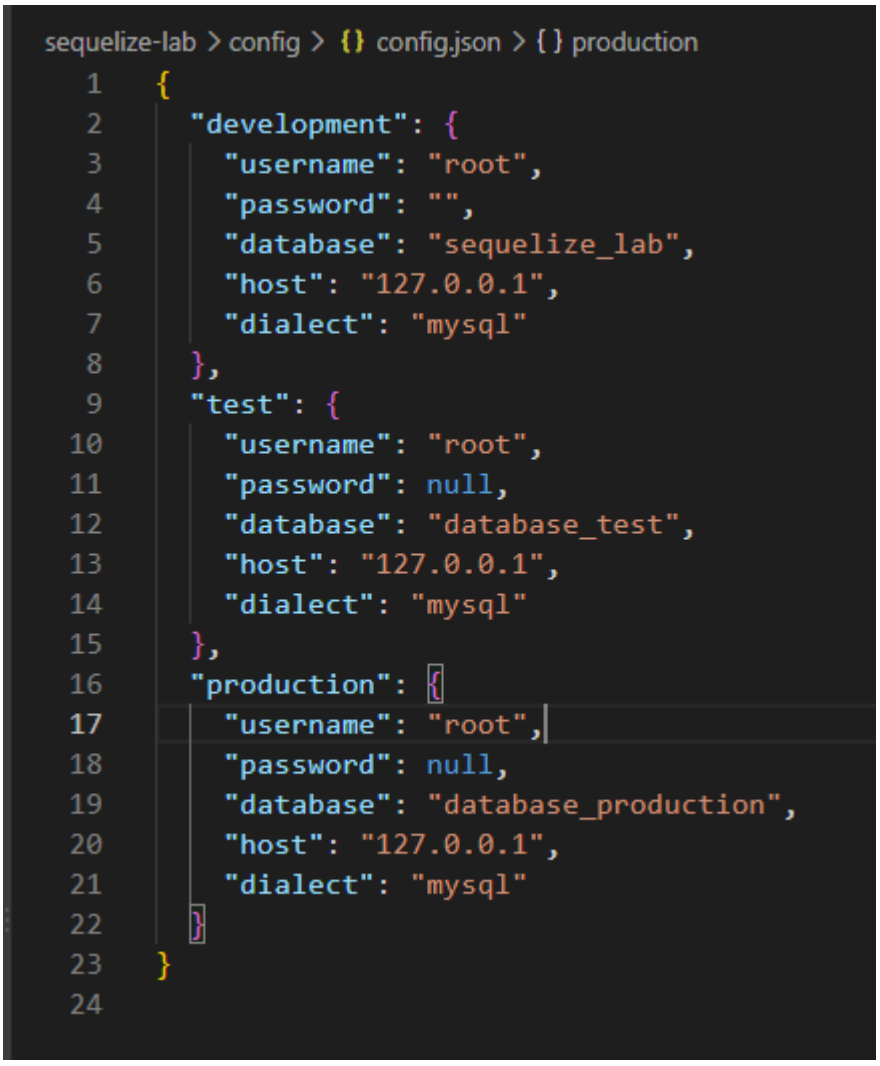
The following folders/files will be created:



Create the MySQL Database

CREATE DATABASE sequelize_lab;

Edit config/config.json and enter your MySQL credentials:



Exercise 1 – Create a Sequelize Model

Generate the model and migration:

npx sequelize-cli model:generate --name Student --attributes name:string,email:string,age:integer,course:string

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>npx sequelize-cli model:generate --name Student \--attributes name:string,email:string,age:integer,course:string

Sequelize CLI [Node: 24.18.0, CLI: 6.6.5, ORM: 6.37.8]

Options:
  --version          Show version number                                                                    [boolean]
  --help             Show help                                                                              [boolean]
  --env              The environment to run the command in                                  [string] [default: "development"]
  --config           The path to the config file                                                            [string]
  --options-path     The path to a JSON file with additional options                                        [string]
  --migrations-path  The path to the migrations folder                                   [string] [default: "migrations"]
  --seeders-path     The path to the seeders folder                                        [string] [default: "seeders"]
  --models-path      The path to the models folder                                         [string] [default: "models"]
  --url              The database connection string to use. Alternative to using --config files             [string]
  --debug            When available show various debug information                        [boolean] [default: false]
  --name             Defines the name of the new model                                          [string] [required]
  --attributes       A list of attributes                                                       [string] [required]
  --force            Forcefully re-creates model with the same name                                         [string]
  --underscored      Use snake case for the timestamp's attribute names                   [boolean] [default: false]

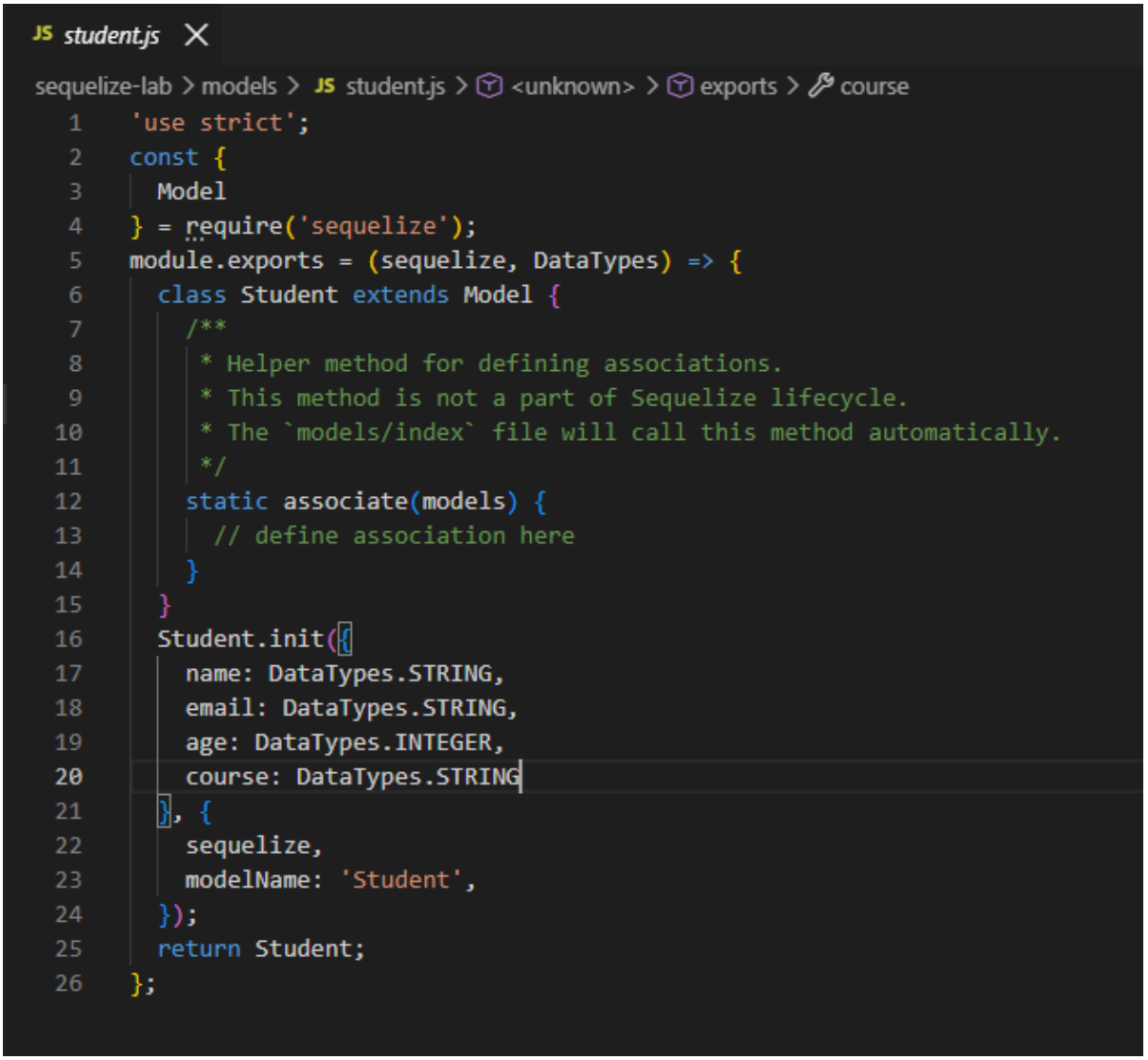
Missing required argument: attributes

C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>npx sequelize-cli model:generate --name Student --attributes name:string,email:string,age:integer,course:string

Sequelize CLI [Node: 24.18.0, CLI: 6.6.5, ORM: 6.37.8]

New model was created at C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab\models\student.js .
New migration was created at C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab\migrations\20260817042444-create-student.js .
```

Model file



Migration file

JS 20260817042444-create-student.js

sequelize-lab > migrations > JS 20260817042444-create-student.js > ...

```
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('Students', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER
      },
      name: {
        type: Sequelize.STRING
      },
      email: {
        type: Sequelize.STRING
      },
      age: {
        type: Sequelize.INTEGER
      },
      course: {
        type: Sequelize.STRING
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE
      }
    });
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable('Students');
  }
};
```

Exercise 2 – Run and Undo a Migration
Objective: Create the Students table using the migration generated in Exercise 1.

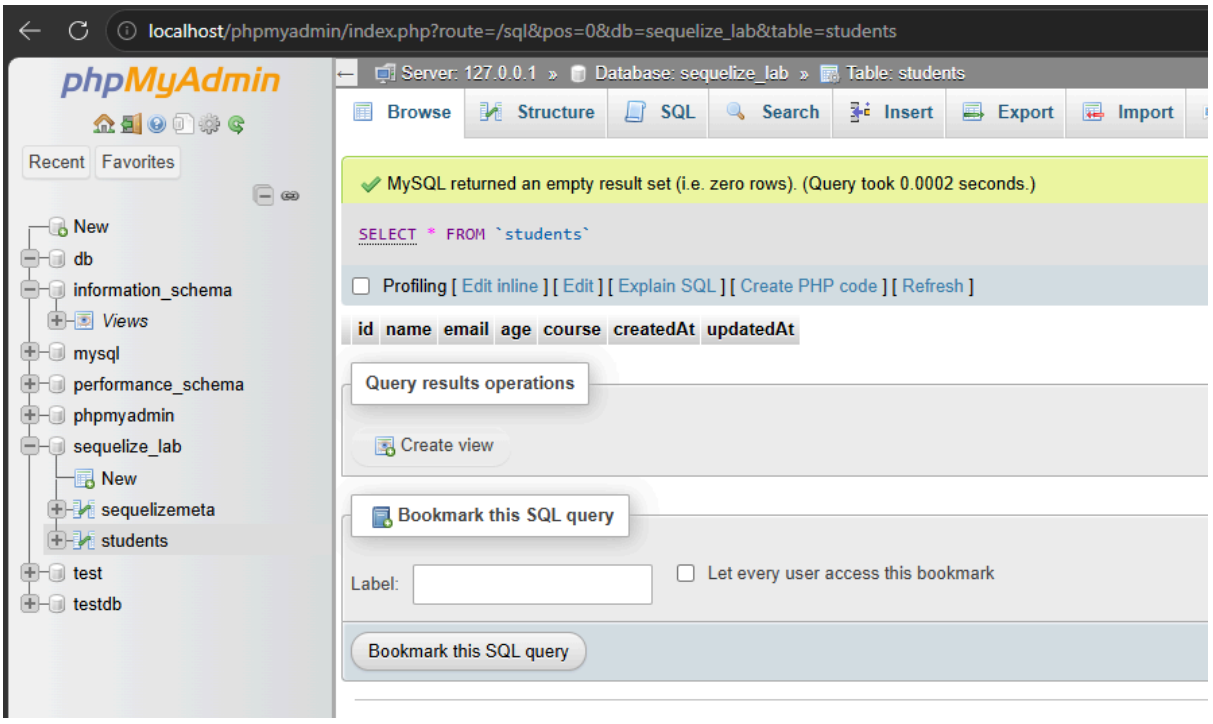
npx sequelize-cli db:migrate

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>npx sequelize-cli db:migrate

Sequelize CLI [Node: 24.18.0, CLI: 6.6.5, ORM: 6.37.8]

Loaded configuration file "config\config.json".
Using environment "development".
== 20260817042444-create-student: migrating =======
== 20260817042444-create-student: migrated (0.030s)
```

Verify the table in MySQL:
USE sequelize_lab;
SHOW TABLES;
DESCRIBE Students;

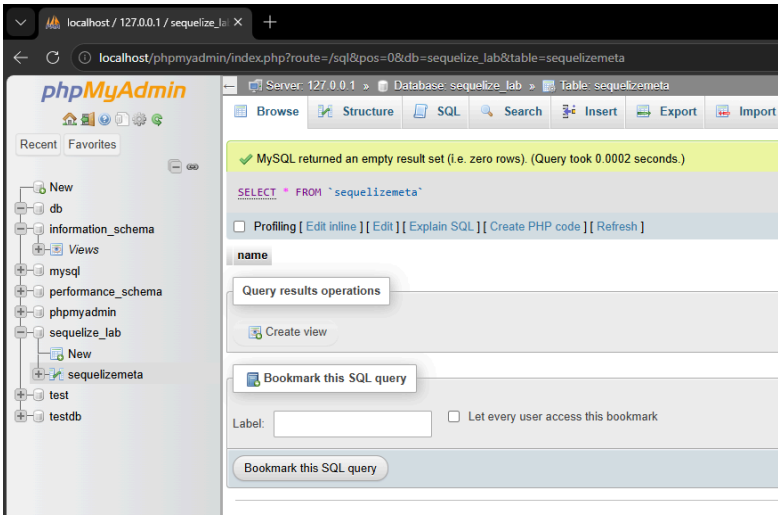


Undo the most recent migration:
npx sequelize-cli db:migrate:undo

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>npx sequelize-cli db:migrate:undo

Sequelize CLI [Node: 24.18.0, CLI: 6.6.5, ORM: 6.37.8]

Loaded configuration file "config\config.json".
Using environment "development".
== 20260817042444-create-student: reverting =======
== 20260817042444-create-student: reverted (0.035s)
```



Exercise 3 – Add a Column Using a Migration

Objective: Add a phone column to the Students table without manually changing the table in MySQL.

npx sequelize-cli migration:generate --name add-phone-to-students

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>npx sequelize-cli migration:generate --name add-phone-to-students

Sequelize CLI [Node: 24.18.0, CLI: 6.6.5, ORM: 6.37.8]

migrations folder at "C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab\migrations" already exists.
New migration was created at C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab\migrations\20260818034328-add-phone-to-students.js .
```

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>npx sequelize-cli db:migrate

Sequelize CLI [Node: 24.18.0, CLI: 6.6.5, ORM: 6.37.8]

Loaded configuration file "config\config.json".
Using environment "development".
== 20260817045521-create-faculty: migrating =======
== 20260817045521-create-faculty: migrated (0.044s)

== 20260818034328-add-phone-to-students: migrating =======
== 20260818034328-add-phone-to-students: migrated (0.020s)
```

Complete the generated migration:

sequelize-lab > migrations > JS 20260818034328-add-phone-to-students.js > ...

```
'use strict';

/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up (queryInterface, Sequelize) {
   await queryInterface.addColumn('Students', 'phone', {
    type: Sequelize.STRING,
    allowNULL: true
   });
  },

  async down (queryInterface, Sequelize) {
   await queryInterface.removeColumn('Students', 'phone');
  }
};

```

Run and verify the migration:
npx sequelize-cli db:migrate
DESCRIBE Students;

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	*NULL*	auto_increment
name	varchar(255)	YES		*NULL*	
email	varchar(255)	YES		*NULL*	
age	int(11)	YES		*NULL*	
course	varchar(255)	YES		*NULL*	
createdAt	datetime	NO		*NULL*	
updatedAt	datetime	NO		*NULL*	
phone	varchar(255)	YES		*NULL*	

Undo the migration and verify that the column is removed:
npx sequelize-cli db:migrate:undo

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>npx sequelize-cli db:migrate:undo

Sequelize CLI [Node: 24.18.0, CLI: 6.6.5, ORM: 6.37.8]

Loaded configuration file "config\config.json".
Using environment "development".
== 20260818034328-add-phone-to-students: reverting =======
== 20260818034328-add-phone-to-students: reverted (0.056s)
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	*NULL*	auto_increment
name	varchar(255)	YES		*NULL*	
email	varchar(255)	YES		*NULL*	
age	int(11)	YES		*NULL*	
course	varchar(255)	YES		*NULL*	
createdAt	datetime	NO		*NULL*	
updatedAt	datetime	NO		*NULL*	

Exercise 4 – Insert Records Using create()
Create studentQueries.js in the project root.

```
const { Student } = require('./models');

async function createStudent() {
  try {
    const student = await Student.create({
      name: 'Hasmani',
      email: 'asmanibilung@example.com',
      age: 20,
      course: 'BCA'
    });

    console.log(student.toJSON());
  } catch (error) {
    console.error(error);
  }
}

createStudent();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
{
  id: 1,
  name: 'Hasmani',
  email: 'asmanibilung@example.com',
  age: 20,
  course: 'BCA',
  updatedAt: 2026-08-24T03:50:15.394Z,
  createdAt: 2026-08-24T03:50:15.394Z
}
```

	id	name	email	age	course	createdAt	updatedAt
Edit Copy Delete	1	Hasmani	asmanibilung@example.com	20	BCA	2026-08-24 03:50:15	2026-08-24 03:50:15

Multiple inputs

```
const {Student}= require('./models');
//console.log(models);



async function
createStudent(name,email,age,course) {
    try {
        const student = await Student.create({
            name: name,
            email: email,
            age: age,
            course: course
        });


        console.log(student.toJSON());
    } catch (error) {
        console.error(error);
    }
}
createStudent('Mansi','mansi08@gmail.com',20,'BCA
');
createStudent('Shreya','shryaa15@gmail.com',19,'B
CA');
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
{
  id: 2,
  name: 'Mansi',
  email: 'mansi08@gmail.com',
  age: 20,
  course: 'BCA',
  updatedAt: 2026-08-24T03:54:42.759Z,
  createdAt: 2026-08-24T03:54:42.759Z
}
{
  id: 3,
  name: 'Shreya',
  email: 'shryaa15@gmail.com',
  age: 19,
  course: 'BCA',
  updatedAt: 2026-08-24T03:54:42.759Z,
  createdAt: 2026-08-24T03:54:42.759Z
}
```

	id	name	email	age	course	createdAt	updatedAt
Edit Copy Delete	1	Hasmani	asmanibilung@example.com	20	BCA	2026-08-24 03:50:15	2026-08-24 03:50:15
Edit Copy Delete	2	Mansi	mansi08@gmail.com	20	BCA	2026-08-24 03:54:42	2026-08-24 03:54:42
Edit Copy Delete	3	Shreya	shryaa15@gmail.com	19	BCA	2026-08-24 03:54:42	2026-08-24 03:54:42

Exercise 5 – Retrieve Records

Retrieve all students:

```
const { Student } = require('./models');


async function retrieveStudent() {
  try {
      const students = await Student.findAll();
        console.log(students);
    } catch (error) {
    console.error(error);
  }
}

retrieveStudent();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node retriveStudent.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student`;
[
  Student {
    dataValues: {
      id: 1,
      name: 'Hasmani',
      email: 'asmanibilung@example.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:50:15.000Z,
      updatedAt: 2026-08-24T03:50:15.000Z
    },
    _previousDataValues: {
      id: 1,
      name: 'Hasmani',
      email: 'asmanibilung@example.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:50:15.000Z,
      updatedAt: 2026-08-24T03:50:15.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 2,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    _previousDataValues: {
      id: 2,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    uniqno: 1,
```

```
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 3,
      name: 'Shreya',
      email: 'shryaa15@gmail.com',
      age: 19,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    _previousDataValues: {
      id: 3,
      name: 'Shreya',
      email: 'shryaa15@gmail.com',
      age: 19,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

Retrieve one student using email:

```
const { Student } = require('./models');

async function retrieve() {
  try {
    const student = await Student.findOne({
      where: {
        email: 'mansi08@gmail.com'
      }
```

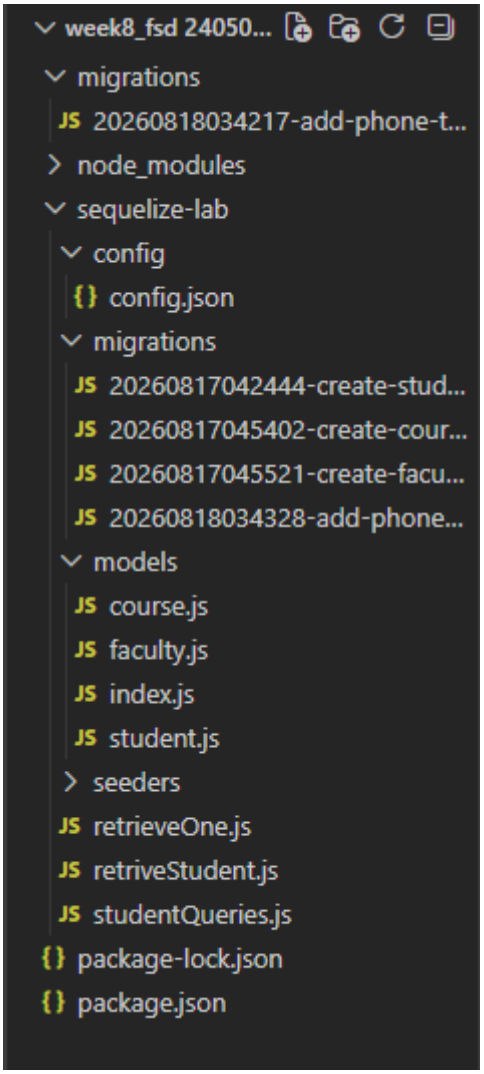
```
    });

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

retrieve();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node retrieveOne.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`email` = 'mansi08@gmail.com' LIMIT 1;
Student {
  dataValues: {
    id: 2,
    name: 'Mansi',
    email: 'mansi08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-08-24T03:54:42.000Z
  },
  _previousDataValues: {
    id: 2,
    name: 'Mansi',
    email: 'mansi08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-08-24T03:54:42.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [
      'id',
      'name',
      'email',
      'age',
      'course',
      'createdAt',
      'updatedAt'
    ]
  },
  isNewRecord: false
}
```



Retrieve a student using primary key:

```
const { Student } = require('./models');

async function retrieveUsingPrimaryKey() {
  try {
    const student = await Student.findByPk(1);

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

retrieveUsingPrimaryKey();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node primaryKey.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`id` = 1;
Student {
  dataValues: {
    id: 1,
    name: 'Hasmani',
    email: 'asmanibilung@example.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T03:50:15.000Z,
    updatedAt: 2026-08-24T03:50:15.000Z
  },
  _previousDataValues: {
    id: 1,
    name: 'Hasmani',
    email: 'asmanibilung@example.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T03:50:15.000Z,
    updatedAt: 2026-08-24T03:50:15.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [
      'id',
      'name',
      'email',
      'age',
      'course',
      'createdAt',
      'updatedAt'
    ]
  },
  isNewRecord: false
}
```

Retrieve only selected attributes:

```
const { Student } = require('./models');

async function retrieveAttribute() {
  try {
    const student = await Student.findAll({
        attributes: ['name','course']
    });

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

retrieveAttribute();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node retrieveAttribute.js
Executing (default): SELECT `name`, `course` FROM `Students` AS `Student`;
[
  Student {
    dataValues: { name: 'Hasmani', course: 'BCA' },
    _previousDataValues: { name: 'Hasmani', course: 'BCA' },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: { name: 'Mansi', course: 'BCA' },
    _previousDataValues: { name: 'Mansi', course: 'BCA' },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: { name: 'Shreya', course: 'BCA' },
    _previousDataValues: { name: 'Shreya', course: 'BCA' },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
```

```
  Student {
    dataValues: { name: 'Mansi', course: 'BCA' },
    _previousDataValues: { name: 'Mansi', course: 'BCA' },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: { name: 'Shreya', course: 'BCA' },
    _previousDataValues: { name: 'Shreya', course: 'BCA' },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

Display one student using ID.

```
const { Student } = require('./models');

async function retrieve() {
  try {
    const student = await Student.findOne({
      where: {
        id: '2'
      }
    });

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}
```

```
retrieve();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node retrieveOne.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`id` = '2';
Student {
  dataValues: {
    id: 2,
    name: 'Mansi',
    email: 'mansi08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-08-24T03:54:42.000Z
  },
  _previousDataValues: {
    id: 2,
    name: 'Mansi',
    email: 'mansi08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-08-24T03:54:42.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [
      'id',
      'name',
      'email',
      'age',
      'course',
      'createdAt',
      'updatedAt'
    ]
  },
  isNewRecord: false
}
```

	id	name	email	age	course	createdAt	updatedAt
Edit Copy Delete	2	Mansi	mansi08@gmail.com	20	BCA	2026-08-24 03:54:42	2026-08-24 03:54:42

Exercise 6 – Filtering with where

Find all BCA students:

```
const { Student } = require('./models');

async function retrieveAttribute() {
  try {
    const student = await Student.findAll({
        where: {
         course: 'BCA'
        }
    });

    console.log(student);
  } catch (error) {
```

```
    console.error(error);
  }
}

retrieveAttribute();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node filtering.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`course` = 'BCA';
[
  Student {
    dataValues: {
      id: 1,
      name: 'Hasmani',
      email: 'asmanibilung@example.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:50:15.000Z,
      updatedAt: 2026-08-24T03:50:15.000Z
    },
    _previousDataValues: {
      id: 1,
      name: 'Hasmani',
      email: 'asmanibilung@example.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:50:15.000Z,
      updatedAt: 2026-08-24T03:50:15.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 2,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    _previousDataValues: {
      id: 2,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
```

```
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 3,
      name: 'Shreya',
      email: 'shryaa15@gmail.com',
      age: 19,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    _previousDataValues: {
      id: 3,
      name: 'Shreya',
      email: 'shryaa15@gmail.com',
      age: 19,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 4,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T04:48:38.000Z,
      updatedAt: 2026-08-24T04:48:38.000Z
    },
    _previousDataValues: {
      id: 4,
```

```
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T04:48:38.000Z,
      updatedAt: 2026-08-24T04:48:38.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 5,
      name: 'Shreya',
      email: 'shryaa15@gmail.com',
      age: 19,
      course: 'BCA',
      createdAt: 2026-08-24T04:48:38.000Z,
      updatedAt: 2026-08-24T04:48:38.000Z
    },
    _previousDataValues: {
      id: 5,
      name: 'Shreya',
      email: 'shryaa15@gmail.com',
      age: 19,
      course: 'BCA',
      createdAt: 2026-08-24T04:48:38.000Z,
      updatedAt: 2026-08-24T04:48:38.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

Use Sequelize operators for comparisons:

```
const { Student } = require('./models');
const { Op } = require('sequelize');
```

```
async function comparison() {
  try {
    const student = await Student.findAll({
       where: {
        age: {
            [Op.gt]: 19
        }
       }
    });

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

comparison();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node sequelizeOperator.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`age` > 19;
[
  Student {
    dataValues: {
      id: 1,
      name: 'Hasmani',
      email: 'asmanibilung@example.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:50:15.000Z,
      updatedAt: 2026-08-24T03:50:15.000Z
    },
    _previousDataValues: {
      id: 1,
      name: 'Hasmani',
      email: 'asmanibilung@example.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:50:15.000Z,
      updatedAt: 2026-08-24T03:50:15.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 2,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    _previousDataValues: {
      id: 2,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
```

```
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 4,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T04:48:38.000Z,
      updatedAt: 2026-08-24T04:48:38.000Z
    },
    _previousDataValues: {
      id: 4,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T04:48:38.000Z,
      updatedAt: 2026-08-24T04:48:38.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

Find students with a particular email

```
const { Student } = require('./models');

async function filtering() {
  try {
    const student = await Student.findAll({
       where: {
        email: 'shryaa15@gmail.com'
       }
    });
```

```
    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

filtering();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node filtering.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`email` = 'shryaa15@gmail.com';
[
  Student {
    dataValues: {
      id: 2,
      name: 'Shreya',
      email: 'shryaa15@gmail.com',
      age: 19,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    _previousDataValues: {
      id: 2,
      name: 'Shreya',
      email: 'shryaa15@gmail.com',
      age: 19,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

Find students belonging to a particular course.

```
const { Student } = require('./models');

async function filtering() {
  try {
    const student = await Student.findAll({
       where: {
        course: 'BBA'
       }
    });

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

filtering();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node filtering.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`course` = 'BBA';
[
  Student {
    dataValues: {
      id: 10,
      name: 'Priya',
      email: 'priya@gmail.com',
      age: 20,
      course: 'BBA',
      createdAt: 2026-09-07T03:59:16.000Z,
      updatedAt: 2026-09-07T03:59:16.000Z
    },
    _previousDataValues: {
      id: 10,
      name: 'Priya',
      email: 'priya@gmail.com',
      age: 20,
      course: 'BBA',
      createdAt: 2026-09-07T03:59:16.000Z,
      updatedAt: 2026-09-07T03:59:16.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

Exercise 7 – Update Records

```
const { Student } = require('./models');

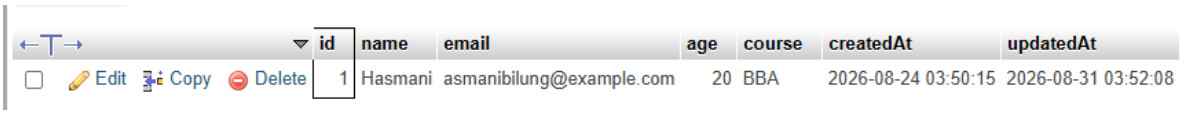
async function update() {
  try {
    const student = await Student.update({
        course: 'BBA'
    },
    {
         where: {
              id:1
         }
    }
);

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}
```

```
update();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node records.js
Executing (default): UPDATE `Students` SET `course`=?,`updatedAt`=? WHERE `id` = ?
[ 1 ]
```



	id	name	email	age	course	createdAt	updatedAt
Edit Copy Delete	1	Hasmani	asmanibilung@example.com	20	BBA	2026-08-24 03:50:15	2026-08-31 03:52:08

Change a student's course.

```
const { Student } = require('./models');

async function update() {
  try {
    const student = await Student.update(
      {
        age: 21
    },
    {
        where: {
            id: 1
        }
    }
);

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

update();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node records.js
Executing (default): UPDATE `Students` SET `age`=?,`updatedAt`=? WHERE `id` = ?
[ 1 ]
```

Update all students belonging to a particular course.

```
const { Student } = require('./models');
```

```
async function update() {
  try {
    const student = await Student.update(
      {
        course: 'BCA'
    },
    {
        where: {
            course: 'BBA'
        }
    }
);

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

update();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node records.js
Executing (default): UPDATE `Students` SET `course`=?,`updatedAt`=? WHERE `course` = ?
[ 1 ]
```

Exercise 8 – Delete Records

Delete a student by ID:

```
const { Student } = require('./models');

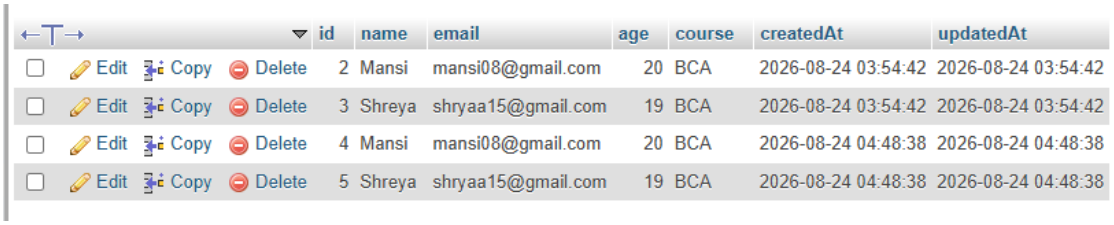
async function deleteRecord() {
  try {
    const student = await Student.destroy({
        where: {
            id:1
        }
    }
);
```

```
    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

deleteRecord();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node delete.js
Executing (default): DELETE FROM `Students` WHERE `id` = 1
1
```



	id	name	email	age	course	createdAt	updatedAt
Edit Copy Delete	2	Mansi	mansi08@gmail.com	20	BCA	2026-08-24 03:54:42	2026-08-24 03:54:42
Edit Copy Delete	3	Shreya	shryaa15@gmail.com	19	BCA	2026-08-24 03:54:42	2026-08-24 03:54:42
Edit Copy Delete	4	Mansi	mansi08@gmail.com	20	BCA	2026-08-24 04:48:38	2026-08-24 04:48:38
Edit Copy Delete	5	Shreya	shryaa15@gmail.com	19	BCA	2026-08-24 04:48:38	2026-08-24 04:48:38

Delete a student using email.

```
const { Student } = require('./models');

async function deleteRecord() {
  try {
    const student = await Student.destroy({
        where: {
        email: 'priya@gmail.com'
    }
    }
);

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

deleteRecord();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node delete.js
Executing (default): DELETE FROM `Students` WHERE `email` = 'priya@gmail.com'
1
```

Delete all students belonging to a particular course.

```
const { Student } = require('./models');

async function deleteRecord() {
  try {
    const student = await Student.destroy({
        where: {
        course: 'BBA'
    }
    }
);

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

deleteRecord();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node delete.js
Executing (default): DELETE FROM `Students` WHERE `course` = 'BBA'
1
```

Week 9 Exercises: Sequelize: Model Querying – Finders, Validation & Constraints, Raw Queries

Exercise 1 – findAll()

Objective: Retrieve multiple records.

```
const { Student } = require('./models');


async function retrieveStudent() {
  try {
      const students = await Student.findAll();
        console.log(students);
    } catch (error) {
    console.error(error);
  }
}

retrieveStudent();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node retriveStudent.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student`;
[
  Student {
    dataValues: {
      id: 1,
      name: 'Hasmani',
      email: 'asmanibilung@example.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:50:15.000Z,
      updatedAt: 2026-08-24T03:50:15.000Z
    },
    _previousDataValues: {
      id: 1,
      name: 'Hasmani',
      email: 'asmanibilung@example.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:50:15.000Z,
      updatedAt: 2026-08-24T03:50:15.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 2,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    _previousDataValues: {
      id: 2,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-08-24T03:54:42.000Z
    },
    uniqno: 1,
```

Display only name, email and course:

```
const { Student } = require('./models');

async function retrieveAttribute() {
  try {
    const student = await Student.findAll({
        attributes: ['name','email','course']
    });

    console.log(student);
  } catch (error) {
    console.error(error);
  }
```

```
}

retrieveAttribute();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node studentQueries.js
Executing (default): SELECT `name`, `email`, `course` FROM `Students` AS `Student`;
[
  Student {
    dataValues: { name: 'Riya', email: 'MCA@gmail.com', course: 'MCA' },
    _previousDataValues: { name: 'Riya', email: 'MCA@gmail.com', course: 'MCA' },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      name: 'Hasmani Bilung',
      email: 'asmanibilung08@gmail.com',
      course: 'BCA'
    },
    _previousDataValues: {
      name: 'Hasmani Bilung',
      email: 'asmanibilung08@gmail.com',
      course: 'BCA'
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
```

```
  Student {
    dataValues: { name: 'Mansi Bhandari', email: 'mansi@gmail.com', course: 'BCA' },
    _previousDataValues: { name: 'Mansi Bhandari', email: 'mansi@gmail.com', course: 'BCA' },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: { name: 'Pramila', email: 'pramila@gmail.com', course: 'BBA' },
    _previousDataValues: { name: 'Pramila', email: 'pramila@gmail.com', course: 'BBA' },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

Exercise 2 – findOne()

Find a student by email:

```
const { Student } = require('./models');

async function run() {

    const student = await Student.findOne({
        where: { email: 'shryaa15@gmail.com' }
    });

    if (student) {
        console.log(student.toJSON());
    } else {
        console.log('Student not found');
    }
}

run();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node retrieve.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt
`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`email` = 'shryaa1
5@gmail.com' LIMIT 1;
{
  id: 2,
  name: 'Shreya',
  email: 'shryaa15@gmail.com',
  age: 19,
  course: 'BCA',
  createdAt: 2026-08-24T03:54:42.000Z,
  updatedAt: 2026-08-24T03:54:42.000Z
}
```

Find the first student belonging to BCA:

```
const { Student } = require('./models');

async function run() {

    const student = await Student.findOne({
        where: { course: 'BCA' }
    });

    if (student) {
        console.log(student.toJSON());
    } else {
        console.log('BCA student not found');
    }
}

run();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node retrieve.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt
`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`course` = 'BCA' L
IMIT 1;
{
  id: 1,
  name: 'Mansi',
  email: 'mansi08@gmail.com',
  age: 21,
  course: 'BCA',
  createdAt: 2026-08-24T03:54:42.000Z,
  updatedAt: 2026-09-21T04:23:04.000Z
}
```

Exercise 3 – findByPk()

Find a student by ID

```
const { Student } = require('./models');

async function run() {

    const student = await Student.findByPk(3);

    if (student) {
        console.log(student.toJSON());
    } else {
        console.log('Student not found');
    }
}

run();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node primaryKey.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt
`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`id` = 3;
{
  id: 3,
  name: 'Anisha',
  email: 'anisha@gmail.com',
  age: 20,
  course: 'BCA',
  createdAt: 2026-08-24T04:48:38.000Z,
  updatedAt: 2026-08-24T04:48:38.000Z
}
```

Test a non-existing ID

```
const { Student } = require('./models');

async function run() {

    const student = await Student.findByPk(20);

    if (student) {
        console.log(student.toJSON());
    } else {
        console.log('Student not found');
    }
}

run();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node primaryKey.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt
`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`id` = 20;
Student not found
```

Accept ID using readline:

```
const { Student } = require('./models');
const readline = require('readline');

const rl = readline.createInterface({
```

```
    input: process.stdin,
    output: process.stdout
});

rl.question('Enter student ID: ', async (id) => {

    const student = await Student.findByPk(id);

    if (student) {
        console.log(student.toJSON());
    } else {
        console.log('Student not found');
    }

    rl.close();
});
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node primaryKey.js
Enter student ID: 3
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt
`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`id` = '3';
{
  id: 3,
  name: 'Anisha',
  email: 'anisha@gmail.com',
  age: 20,
  course: 'BCA',
  createdAt: 2026-08-24T04:48:38.000Z,
  updatedAt: 2026-08-24T04:48:38.000Z
}
```

Exercise 4 – Finder with where and Operators

Find BCA students

```
const { Student } = require('./models');

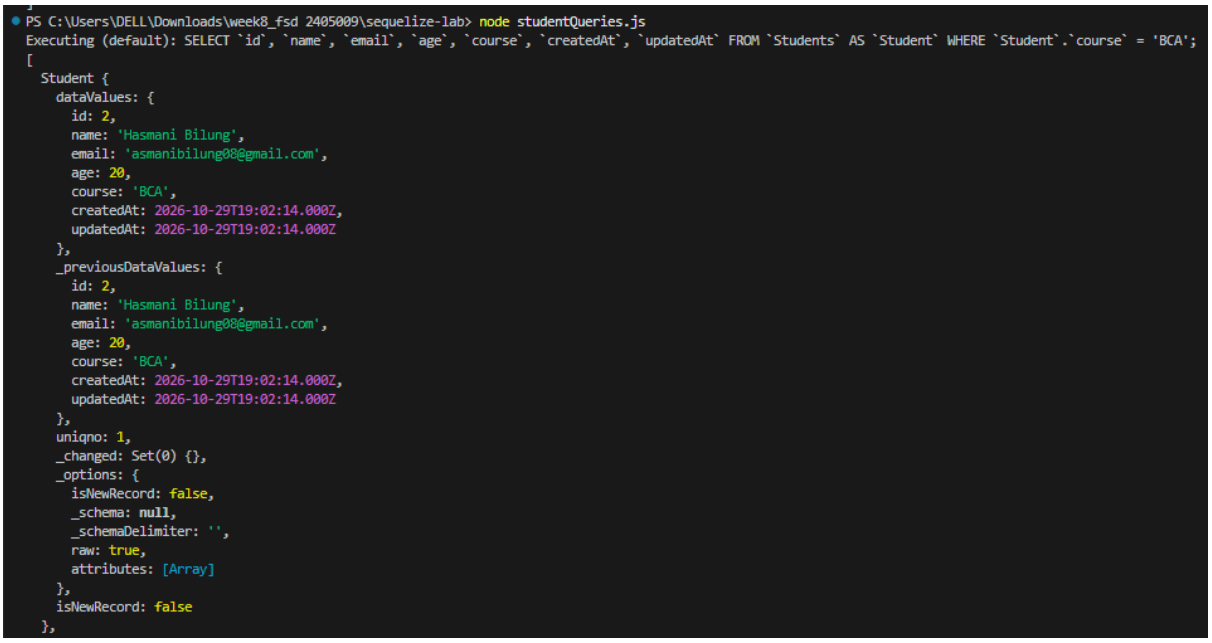
async function run() {
```

```
    const students = await Student.findAll({
        where: { course: 'BCA' }
    });

    console.log(students);
}

run();
```

Output



```
  Student {
    dataValues: {
      id: 3,
      name: 'Mansi Bhandari',
      email: 'mansi@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T20:08:27.000Z,
      updatedAt: 2026-10-29T20:08:27.000Z
    },
    _previousDataValues: {
      id: 3,
      name: 'Mansi Bhandari',
      email: 'mansi@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T20:08:27.000Z,
      updatedAt: 2026-10-29T20:08:27.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

Find students aged 20:

```
const { Student } = require('./models');

async function filtering() {
  try {
    const student = await Student.findAll({
        where: {
         age: '20'
        }
    });

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}
```

```
filtering();
```

Output



```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node studentQueries.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`age` = '20';
[
  Student {
    dataValues: {
      id: 2,
      name: 'Hasmani Bilung',
      email: 'asmanibilung08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T19:02:14.000Z,
      updatedAt: 2026-10-29T19:02:14.000Z
    },
    _previousDataValues: {
      id: 2,
      name: 'Hasmani Bilung',
      email: 'asmanibilung08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T19:02:14.000Z,
      updatedAt: 2026-10-29T19:02:14.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 3,
      name: 'Mansi Bhandari',
      email: 'mansi@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T20:08:27.000Z,
      updatedAt: 2026-10-29T20:08:27.000Z
    },
    _previousDataValues: {
      id: 3,
      name: 'Mansi Bhandari',
      email: 'mansi@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T20:08:27.000Z,
      updatedAt: 2026-10-29T20:08:27.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

Exercise 5 – order, limit and offset

Sort by name ascending

```
const { Student } = require('./models');

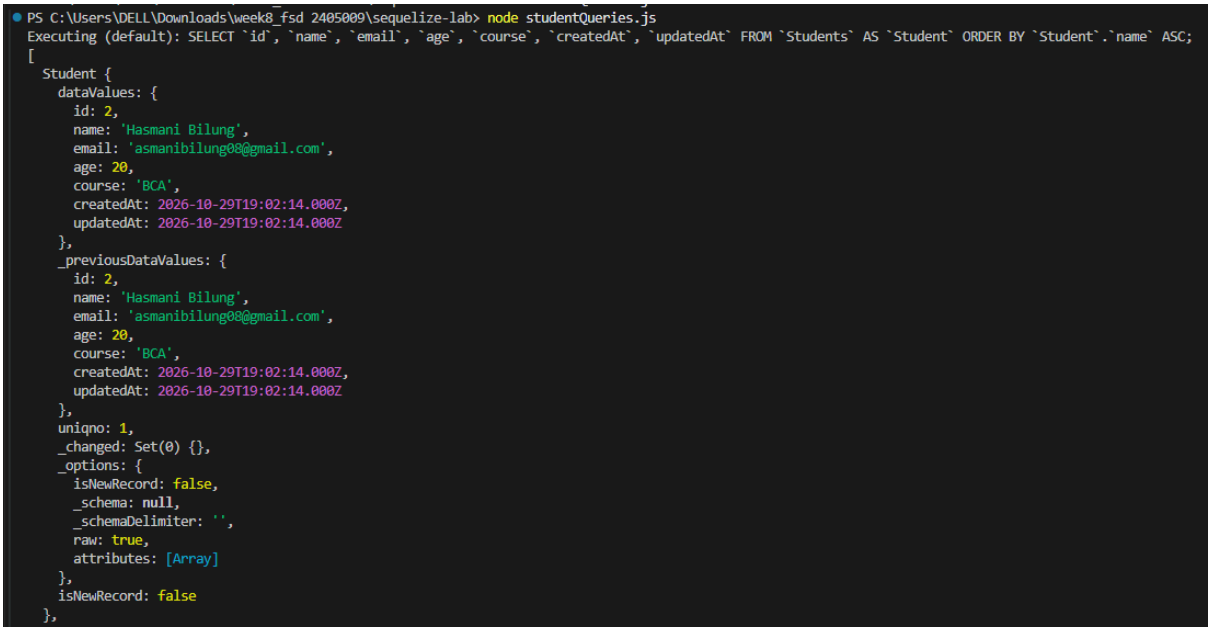
async function retrieveAttribute() {
  try {
    const student = await Student.findAll({
        order: [['name','ASC']]
    });

    console.log(student);
```

```
  } catch (error) {
    console.error(error);
  }
}

retrieveAttribute();
```

Output



```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node studentQueries.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` ORDER BY `Student`.`name` ASC;
[
  Student {
    dataValues: {
      id: 2,
      name: 'Hasmani Bilung',
      email: 'asmanibilung08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T19:02:14.000Z,
      updatedAt: 2026-10-29T19:02:14.000Z
    },
    _previousDataValues: {
      id: 2,
      name: 'Hasmani Bilung',
      email: 'asmanibilung08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T19:02:14.000Z,
      updatedAt: 2026-10-29T19:02:14.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
```


Student {
dataValues: {
id: 3,
name: 'Mansi Bhandari',
email: 'mansi@gmail.com',
age: 20,
course: 'BCA',
createdAt: 2026-10-29T20:08:27.000Z,
updatedAt: 2026-10-29T20:08:27.000Z
},
_previousDataValues: {
id: 3,
name: 'Mansi Bhandari',
email: 'mansi@gmail.com',
age: 20,
course: 'BCA',
createdAt: 2026-10-29T20:08:27.000Z,
updatedAt: 2026-10-29T20:08:27.000Z
},
uniqno: 1,
_changed: Set(0) {},
_options: {
isNewRecord: false,
_schema: null,
_schemaDelimiter: '',
raw: true,
attributes: [Array]
},
isNewRecord: false
},
Student {
dataValues: {
id: 4,
name: 'Pramila',
email: 'pramila@gmail.com',
age: 23,
course: 'BBA',
createdAt: 2026-10-29T20:08:59.000Z,
updatedAt: 2026-10-29T20:08:59.000Z
},
_previousDataValues: {
id: 4,
name: 'Pramila',
email: 'pramila@gmail.com',
age: 23,
course: 'BBA',
createdAt: 2026-10-29T20:08:59.000Z,
updatedAt: 2026-10-29T20:08:59.000Z
},
uniqno: 1,
_changed: Set(0) {},
_options: {
isNewRecord: false,
_schema: null,
_schemaDelimiter: '',
raw: true,
attributes: [Array]
},
isNewRecord: false
},

```
  Student {
    dataValues: {
      id: 1,
      name: 'Riya',
      email: 'MCA@gmail.com',
      age: 19,
      course: 'MCA',
      createdAt: 2026-10-27T06:05:49.000Z,
      updatedAt: 2026-10-29T19:02:41.000Z
    },
    _previousDataValues: {
      id: 1,
      name: 'Riya',
      email: 'MCA@gmail.com',
      age: 19,
      course: 'MCA',
      createdAt: 2026-10-27T06:05:49.000Z,
      updatedAt: 2026-10-29T19:02:41.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

Sort by age descending

```
const { Student } = require('./models');

async function retrieveAttribute() {
  try {
    const student = await Student.findAll({
        order: [['age','DESC']]
    });

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

retrieveAttribute();
```

Output

C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node attribute.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` ORDER BY `Student`.`age` DESC;
[
  Student {

```
  dataValues: {
    id: 9,
    name: 'Prabal',
    email: 'prabal@gmail.com',
    age: 26,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:06.000Z,
    updatedAt: 2026-09-21T05:18:06.000Z
  },
  _previousDataValues: {
    id: 9,
    name: 'Prabal',
    email: 'prabal@gmail.com',
    age: 26,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:06.000Z,
    updatedAt: 2026-09-21T05:18:06.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 8,
    name: 'Pramila',
    email: 'pramila@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:53.000Z,
    updatedAt: 2026-09-21T05:17:53.000Z
  },
  _previousDataValues: {
    id: 8,
    name: 'Pramila',
    email: 'pramila@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:53.000Z,
    updatedAt: 2026-09-21T05:17:53.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
```

```
  isNewRecord: false
},
Student {
  dataValues: {
    id: 10,
    name: 'Kiran',
    email: 'kiran@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:51.000Z,
    updatedAt: 2026-09-21T05:18:51.000Z
  },
  _previousDataValues: {
    id: 10,
    name: 'Kiran',
    email: 'kiran@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:51.000Z,
    updatedAt: 2026-09-21T05:18:51.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 1,
    name: 'Mansi',
    email: 'mansi08@gmail.com',
    age: 21,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-09-21T04:23:04.000Z
  },
  _previousDataValues: {
    id: 1,
    name: 'Mansi',
    email: 'mansi08@gmail.com',
    age: 21,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-09-21T04:23:04.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
```

```
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 3,
    name: 'Anisha',
    email: 'anisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T04:48:38.000Z,
    updatedAt: 2026-08-24T04:48:38.000Z
  },
  _previousDataValues: {
    id: 3,
    name: 'Anisha',
    email: 'anisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T04:48:38.000Z,
    updatedAt: 2026-08-24T04:48:38.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 4,
    name: 'Khushi',
    email: 'khushi15@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  _previousDataValues: {
    id: 4,
    name: 'Khushi',
    email: 'khushi15@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
```

```
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 5,
    name: 'Hasmani',
    email: 'asmani08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  _previousDataValues: {
    id: 5,
    name: 'Hasmani',
    email: 'asmani08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 6,
    name: 'Riddhi',
    email: 'riddhi@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:16:02.000Z,
    updatedAt: 2026-09-21T05:16:02.000Z
  },
  _previousDataValues: {
    id: 6,
    name: 'Riddhi',
    email: 'riddhi@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:16:02.000Z,
    updatedAt: 2026-09-21T05:16:02.000Z
  },
```

```
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 7,
    name: 'Ravisha',
    email: 'ravisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:35.000Z,
    updatedAt: 2026-09-21T05:17:35.000Z
  },
  _previousDataValues: {
    id: 7,
    name: 'Ravisha',
    email: 'ravisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:35.000Z,
    updatedAt: 2026-09-21T05:17:35.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 2,
    name: 'Shreya',
    email: 'shryaa15@gmail.com',
    age: 19,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-08-24T03:54:42.000Z
  },
  _previousDataValues: {
    id: 2,
    name: 'Shreya',
    email: 'shryaa15@gmail.com',
    age: 19,
    course: 'BCA',
```

```
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-08-24T03:54:42.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
 }
]
```

Display the first five students

```
const { Student } = require('./models');

async function retrieveAttribute() {
  try {
    const student = await Student.findAll({
        limit:5
    });

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

retrieveAttribute();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node attribute.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM
`Students` AS `Student` LIMIT 5;
[
 Student {
  dataValues: {
    id: 1,
    name: 'Mansi',
    email: 'mansi08@gmail.com',
    age: 21,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-09-21T04:23:04.000Z
  },
  _previousDataValues: {
```

```
    id: 1,
    name: 'Mansi',
    email: 'mansi08@gmail.com',
    age: 21,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-09-21T04:23:04.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 2,
    name: 'Shreya',
    email: 'shryaa15@gmail.com',
    age: 19,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-08-24T03:54:42.000Z
  },
  _previousDataValues: {
    id: 2,
    name: 'Shreya',
    email: 'shryaa15@gmail.com',
    age: 19,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-08-24T03:54:42.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 3,
    name: 'Anisha',
    email: 'anisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T04:48:38.000Z,
```

```
    updatedAt: 2026-08-24T04:48:38.000Z
  },
  _previousDataValues: {
    id: 3,
    name: 'Anisha',
    email: 'anisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T04:48:38.000Z,
    updatedAt: 2026-08-24T04:48:38.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 4,
    name: 'Khushi',
    email: 'khushi15@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  _previousDataValues: {
    id: 4,
    name: 'Khushi',
    email: 'khushi15@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 5,
    name: 'Hasmani',
    email: 'asmani08@gmail.com',
```

```
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
   },
   _previousDataValues: {
    id: 5,
    name: 'Hasmani',
    email: 'asmani08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
   },
   uniqno: 1,
   _changed: Set(0) {},
   _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
   },
   isNewRecord: false
  }
]
```

Skip five and display the next five

```
const { Student } = require('./models');

async function retrieveAttribute() {
  try {
    const student = await Student.findAll({limit:5,
        offset:5
    });

    console.log(student);
  } catch (error) {
    console.error(error);
  }
}

retrieveAttribute();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node attribute.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM
`Students` AS `Student` LIMIT 5, 5;
[
  Student {
    dataValues: {
```

```
    id: 6,
    name: 'Riddhi',
    email: 'riddhi@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:16:02.000Z,
    updatedAt: 2026-09-21T05:16:02.000Z
  },
  _previousDataValues: {
    id: 6,
    name: 'Riddhi',
    email: 'riddhi@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:16:02.000Z,
    updatedAt: 2026-09-21T05:16:02.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 7,
    name: 'Ravisha',
    email: 'ravisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:35.000Z,
    updatedAt: 2026-09-21T05:17:35.000Z
  },
  _previousDataValues: {
    id: 7,
    name: 'Ravisha',
    email: 'ravisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:35.000Z,
    updatedAt: 2026-09-21T05:17:35.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
```

```
},
Student {
  dataValues: {
    id: 8,
    name: 'Pramila',
    email: 'pramila@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:53.000Z,
    updatedAt: 2026-09-21T05:17:53.000Z
  },
  _previousDataValues: {
    id: 8,
    name: 'Pramila',
    email: 'pramila@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:53.000Z,
    updatedAt: 2026-09-21T05:17:53.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 9,
    name: 'Prabal',
    email: 'prabal@gmail.com',
    age: 26,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:06.000Z,
    updatedAt: 2026-09-21T05:18:06.000Z
  },
  _previousDataValues: {
    id: 9,
    name: 'Prabal',
    email: 'prabal@gmail.com',
    age: 26,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:06.000Z,
    updatedAt: 2026-09-21T05:18:06.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
```

```
    attributes: [Array]
   },
   isNewRecord: false
  },
  Student {
   dataValues: {
    id: 10,
    name: 'Kiran',
    email: 'kiran@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:51.000Z,
    updatedAt: 2026-09-21T05:18:51.000Z
   },
   _previousDataValues: {
    id: 10,
    name: 'Kiran',
    email: 'kiran@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:51.000Z,
    updatedAt: 2026-09-21T05:18:51.000Z
   },
   uniqno: 1,
   _changed: Set(0) {},
   _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
   },
   isNewRecord: false
  }
]
```

Exercise 6 – findOrCreate()

```
const { Student } = require('./models');

async function run() {

    const [student, created] = await Student.findOrCreate({
        where: {
            email: 'student@gmail.com'
        },
        defaults: {
            name: 'John',
            age: 20,
            course: 'BCA'
        }
    });
```

```
    console.log(created ? 'New student created' : 'Student already
exists');
    console.log(student.toJSON());
}

run();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node findOrCreate.js
Executing (270e508b-a0b9-4da6-bbe3-d10b17363044): START TRANSACTION;
Executing (270e508b-a0b9-4da6-bbe3-d10b17363044): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`
, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`email` = 'student@gmail.com' LIMIT 1;
Executing (270e508b-a0b9-4da6-bbe3-d10b17363044): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`
,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
Executing (270e508b-a0b9-4da6-bbe3-d10b17363044): COMMIT;
New student created
{
  id: 11,
  name: 'John',
  age: 20,
  course: 'BCA',
  email: 'student@gmail.com',
  updatedAt: 2026-09-22T04:00:49.776Z,
  createdAt: 2026-09-22T04:00:49.776Z
}
```

Run twice and observe the difference

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node findOrCreate.js
Executing (88096ba0-6eb3-4695-b679-a531881260ab): START TRANSACTION;
Executing (88096ba0-6eb3-4695-b679-a531881260ab): SELECT `id`, `name`, `email`, `age`, `course`, `createdA
t`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`email` = 'student@gmail.com' LIMIT 1;
Executing (88096ba0-6eb3-4695-b679-a531881260ab): COMMIT;
Student already exists
```

Explain where, defaults and the returned values

- where tells Sequelize which record to search for.
- defaults contains the values used only when a new record needs to be created. If the student already exists, these default values are not used to update the existing student.
- findOrCreate() returns two values:
    - **student**
    - Contains the student record.
    - It can be either an existing student or a newly created student.
    - **created**
    - A Boolean value.
    - true → a new student was created.
    - false → the student already existed.

Exercise 7 – count() and findAndCountAll()

Count all students

```
const { Student } = require('./models');

async function retrieveAttribute() {
    const count = await Student.count();

    console.log('Total: ',count);
  }
retrieveAttribute();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node count.js
Executing (default): SELECT count(*) AS `count` FROM `Students` AS `Student`;
Total:  6
```

Count BCA students

```
const { Student } = require('./models');

async function run() {
    const count = await Student.count({
        where: { course: 'BCA' }
    });

    console.log('BCA Students:', count);
}

run();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node counts.js
Executing (default): SELECT count(*) AS `count` FROM `Students` AS `Student` WHERE `Student`.`course` = 'B
CA';
BCA Students: 11
```

Display BCA students and their count

```
const { Student } = require('./models');

async function run() {
    const result = await Student.findAndCountAll({
```

```
        where: { course: 'BCA' }
    });

    console.log('Count:', result.count);
    console.log('Rows:', result.rows);
}

run();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node counts.js
Executing (default): SELECT count(*) AS `count` FROM `Students` AS `Student` WHERE
`Student`.`course` = 'BCA';
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM
`Students` AS `Student` WHERE `Student`.`course` = 'BCA';
Count: 11
Rows: [
  Student {
    dataValues: {
      id: 1,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 21,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-09-21T04:23:04.000Z
    },
    _previousDataValues: {
      id: 1,
      name: 'Mansi',
      email: 'mansi08@gmail.com',
      age: 21,
      course: 'BCA',
      createdAt: 2026-08-24T03:54:42.000Z,
      updatedAt: 2026-09-21T04:23:04.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  Student {
    dataValues: {
      id: 2,
      name: 'Shreya',
      email: 'shryaa15@gmail.com',
      age: 19,
      course: 'BCA',
```

```
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-08-24T03:54:42.000Z
  },
  _previousDataValues: {
    id: 2,
    name: 'Shreya',
    email: 'shryaa15@gmail.com',
    age: 19,
    course: 'BCA',
    createdAt: 2026-08-24T03:54:42.000Z,
    updatedAt: 2026-08-24T03:54:42.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 3,
    name: 'Anisha',
    email: 'anisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T04:48:38.000Z,
    updatedAt: 2026-08-24T04:48:38.000Z
  },
  _previousDataValues: {
    id: 3,
    name: 'Anisha',
    email: 'anisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-08-24T04:48:38.000Z,
    updatedAt: 2026-08-24T04:48:38.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 4,
    name: 'Khushi',
```

```
    email: 'khushi15@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  _previousDataValues: {
    id: 4,
    name: 'Khushi',
    email: 'khushi15@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 5,
    name: 'Hasmani',
    email: 'asmani08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  _previousDataValues: {
    id: 5,
    name: 'Hasmani',
    email: 'asmani08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
```

```
  dataValues: {
    id: 6,
    name: 'Riddhi',
    email: 'riddhi@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:16:02.000Z,
    updatedAt: 2026-09-21T05:16:02.000Z
  },
  _previousDataValues: {
    id: 6,
    name: 'Riddhi',
    email: 'riddhi@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:16:02.000Z,
    updatedAt: 2026-09-21T05:16:02.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 7,
    name: 'Ravisha',
    email: 'ravisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:35.000Z,
    updatedAt: 2026-09-21T05:17:35.000Z
  },
  _previousDataValues: {
    id: 7,
    name: 'Ravisha',
    email: 'ravisha@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:35.000Z,
    updatedAt: 2026-09-21T05:17:35.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
```

```
  isNewRecord: false
},
Student {
  dataValues: {
    id: 8,
    name: 'Pramila',
    email: 'pramila@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:53.000Z,
    updatedAt: 2026-09-21T05:17:53.000Z
  },
  _previousDataValues: {
    id: 8,
    name: 'Pramila',
    email: 'pramila@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:17:53.000Z,
    updatedAt: 2026-09-21T05:17:53.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 9,
    name: 'Prabal',
    email: 'prabal@gmail.com',
    age: 26,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:06.000Z,
    updatedAt: 2026-09-21T05:18:06.000Z
  },
  _previousDataValues: {
    id: 9,
    name: 'Prabal',
    email: 'prabal@gmail.com',
    age: 26,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:06.000Z,
    updatedAt: 2026-09-21T05:18:06.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
```

```
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 10,
    name: 'Kiran',
    email: 'kiran@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:51.000Z,
    updatedAt: 2026-09-21T05:18:51.000Z
  },
  _previousDataValues: {
    id: 10,
    name: 'Kiran',
    email: 'kiran@gmail.com',
    age: 23,
    course: 'BCA',
    createdAt: 2026-09-21T05:18:51.000Z,
    updatedAt: 2026-09-21T05:18:51.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 11,
    name: 'John',
    email: 'student@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-22T04:00:49.000Z,
    updatedAt: 2026-09-22T04:00:49.000Z
  },
  _previousDataValues: {
    id: 11,
    name: 'John',
    email: 'student@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-22T04:00:49.000Z,
    updatedAt: 2026-09-22T04:00:49.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
```

```
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
 }
]
```

Combine findAndCountAll with limit, offset and order

```
const { Student } = require('./models');

async function run() {
    const result = await Student.findAndCountAll({
        where: { course: 'BBA' },
        order: [['name', 'ASC']],
        limit: 5,
        offset: 0
    });

    console.log('Total BCA Students:', result.count);
    console.log('Students:', result.rows);
}

run();
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node counts.js
Executing (default): SELECT count(*) AS `count` FROM `Students` AS `Student` WHERE
`Student`.`course` = 'BBA';
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM
`Students` AS `Student` WHERE `Student`.`course` = 'BBA' ORDER BY `Student`.`name` ASC LIMIT
0, 5;
Total BCA Students: 3
Students: [
  Student {
    dataValues: {
      id: 5,
      name: 'Hasmani',
      email: 'asmani08@gmail.com',
      age: 20,
      course: 'BBA',
      createdAt: 2026-09-01T04:13:50.000Z,
      updatedAt: 2026-09-01T04:13:50.000Z
    },
    _previousDataValues: {
      id: 5,
      name: 'Hasmani',
      email: 'asmani08@gmail.com',
```

```
    age: 20,
    course: 'BBA',
    createdAt: 2026-09-01T04:13:50.000Z,
    updatedAt: 2026-09-01T04:13:50.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 11,
    name: 'John',
    email: 'student@gmail.com',
    age: 20,
    course: 'BBA',
    createdAt: 2026-09-22T04:00:49.000Z,
    updatedAt: 2026-09-22T04:00:49.000Z
  },
  _previousDataValues: {
    id: 11,
    name: 'John',
    email: 'student@gmail.com',
    age: 20,
    course: 'BBA',
    createdAt: 2026-09-22T04:00:49.000Z,
    updatedAt: 2026-09-22T04:00:49.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
},
Student {
  dataValues: {
    id: 6,
    name: 'Riddhi',
    email: 'riddhi@gmail.com',
    age: 20,
    course: 'BBA',
    createdAt: 2026-09-21T05:16:02.000Z,
    updatedAt: 2026-09-21T05:16:02.000Z
  },
  _previousDataValues: {
```

```
    id: 6,
    name: 'Riddhi',
    email: 'riddhi@gmail.com',
    age: 20,
    course: 'BBA',
    createdAt: 2026-09-21T05:16:02.000Z,
    updatedAt: 2026-09-21T05:16:02.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [Array]
  },
  isNewRecord: false
 }
]
```

Exercise 8 – Required Fields

Attempt insertion without name:

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class Student extends Model {
    /**
     * Helper method for defining associations.
     * This method is not a part of Sequelize
lifecycle.
     * The `models/index` file will call this method
automatically.
     */
    static associate(models) {
      // define association here
    }
  }
  Student.init({
    name: {
```

```
            type:DataTypes.STRING,
            allowNull: false
          },
    email: DataTypes.STRING,
    age: DataTypes.INTEGER,
    course: DataTypes.STRING
  }, {
    sequelize,
    modelName: 'Student',
  });
  return Student;
};
```

```
const { Student } = require('./models');

async function run() {
    try {
        await Student.create({
            email: 'max@gmail.com',
            age: 20,
            course: 'BCA'
        });

        console.log('Student created');
    } catch (error) {
        console.log(error.message);
    }
}

run();
```

Error message

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node studentQueries.js
notNull Violation: Student.name cannot be null
```

Exercise 9 – Unique Constraint

Insert an email once
Attempt the same email again
student.js

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class Student extends Model {
    /**
     * Helper method for defining associations.
     * This method is not a part of Sequelize lifecycle.
     * The `models/index` file will call this method automatically.
     */
    static associate(models) {
      // define association here
      Student.hasMany(models.StudentProfile, {
        foreignKey: 'studentId'
      });
    }
  }
  Student.init({
    name: {
            type:DataTypes.STRING,
            allowNull: false,
            unique: false
          },
    email: {
            type:DataTypes.STRING,
            allowNull: false,
            unique: true
            },
    age: DataTypes.INTEGER,
    course: DataTypes.STRING
  }, {
    sequelize,
    modelName: 'Student',
  });
  return Student;
};
```

studentQueries.js

```
const { Student } = require('./models');

async function run() {
```

```
    try {
        await Student.create({
            name:'Davin',
            email: 'john@gmail.com',
            age: 20,
            course: 'BCA'
        });

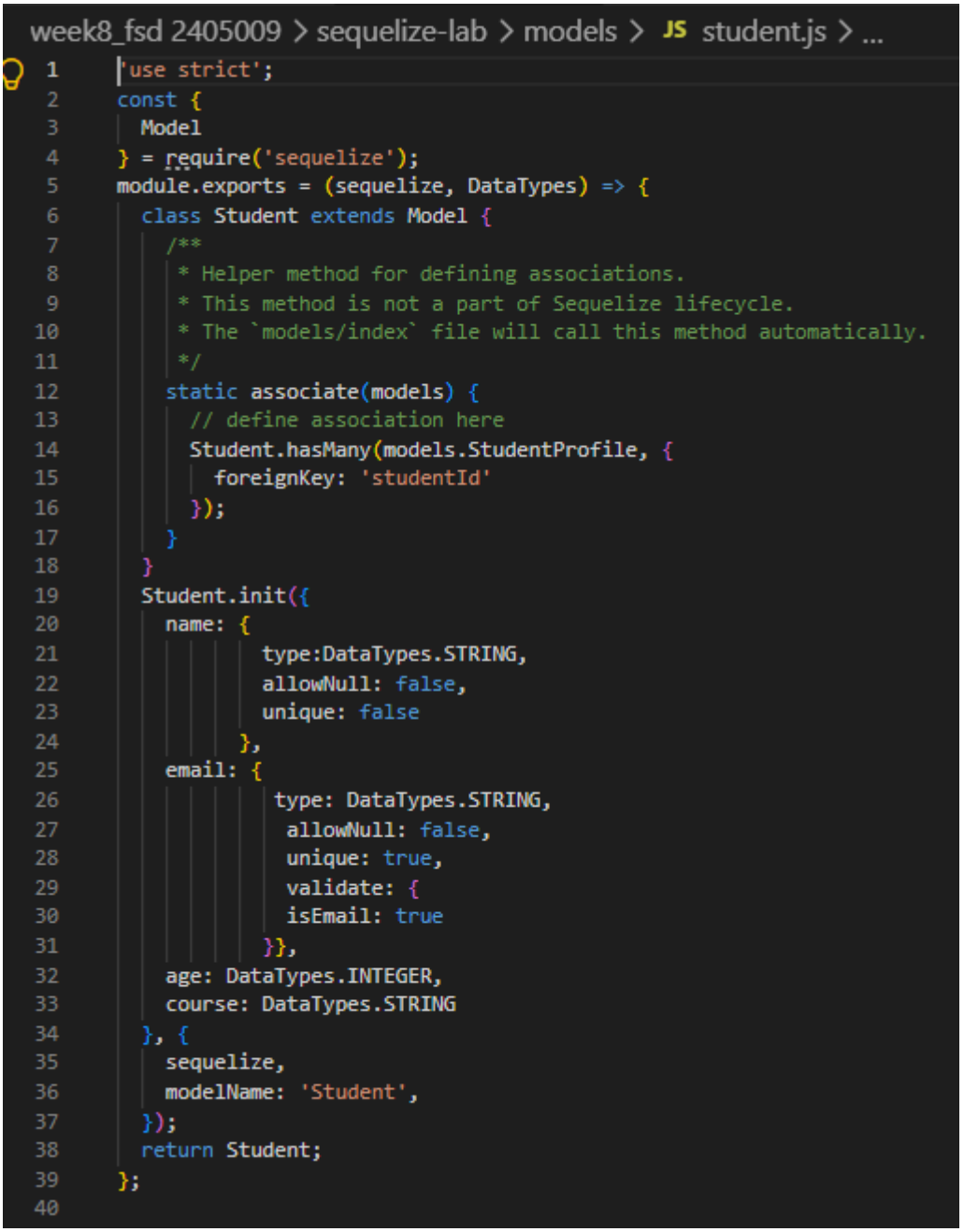
        console.log('Email address already exists.');
    } catch (error) {
        console.log(error.message);
    }
}

run();
```

Error message

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) V
ALUES (DEFAULT,?,?,?,?,?,?);
Email address already exists.
```

Exercise 10 – Email Validation



week8_fsd 2405009 > sequelize-lab > JS studentQueries.js > ...

```
const { Student } = require('./models');

async function run() {
    try {
        await Student.create({
            name: 'Max',
            email: 'student@gmail.com',
            age: 20,
            course: 'BCA'
        });

        console.log('Valid email accepted.');

        await Student.create({
            name: 'Max',
            email: 'student@abc',
            age: 20,
            course: 'BCA'
        });

        console.log('Valid email accepted.');

    } catch (error) {
        console.log('Invalid email address.');
    }
}

run();
```

Output

```
C:\Users\TYBCA\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab>node studentQu
eries.js
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES
(DEFAULT,?,?,?,?,?,?);
Valid email accepted.
Invalid email address.
```

Exercise 11 – Age Validation

week8_fsd 2405009 > sequelize-lab > JS studentQueries.js > ...

```
1
2   const { Student } = require('./models');
3
4   async function run() {
5       for (let age of [15, 17, 25, 30, 35]) {
6           try {
7               await Student.create({
8                   name: 'Max',
9                   email: `max${age}@gmail.com`,
10                  age: age,
11                  course: 'BCA'
12              });
13
14              console.log(age + ' - Accepted');
15          } catch (error) {
16              console.log(age + ' - Rejected');
17          }
18      }
19  }
20
21  run();
```

```
    age: {
        type: DataTypes.INTEGER,
        allowNull: false,
        validate: {
            min: 17,
            max: 30
        }
    },
```

Output

```
C:\Users\TYBCA\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab>node studentQueries.js
15 - Rejected
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
17 - Accepted
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
25 - Accepted
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
30 - Accepted
35 - Rejected
```

☐	Edit	Copy	Delete	15	Max	max17@gmail.com	17	BCA	2026-10-27 05:54:55	2026-10-27 05:54:55	NULL
☐	Edit	Copy	Delete	16	Max	max25@gmail.com	25	BCA	2026-10-27 05:54:55	2026-10-27 05:54:55	NULL
☐	Edit	Copy	Delete	17	Max	max30@gmail.com	30	BCA	2026-10-27 05:54:55	2026-10-27 05:54:55	NULL

Exercise 12 – Name Validation

```
name: {
        type: DataTypes.STRING,
        allowNull: false,
        validate: {
       len: [3, 50],
        is: /^[a-zA-Z ]+$/
         }
       },
```

week8_fsd 2405009 > sequelize-lab > JS studentQueries.js > ...

```
1   const { Student } = require('./models');
2
3   async function run() {
4       for (let name of ['Shivani', 'Shivam', 'Shivani123']) {
5           try {
6               await Student.create({
7                   name: name,
8                   email: `${name}@gmail.com`,
9                   age: 20,
10                  course: 'BCA'
11              });
12
13              console.log(name, 'Pass');
14          } catch (error) {
15              console.log(name, 'Fail');
16          }
17      }
18  }
19
20  run();
```

Output

```
C:\Users\TYBCA\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
Shivani Pass
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
Shivam Pass
Shivani123 Fail
```

☐	Edit	Copy	Delete	18	Shivani	Shivani@gmail.com	20	BCA	2026-10-27 06:01:17	2026-10-27 06:01:17	NULL
☐	Edit	Copy	Delete	19	Shivam	Shivam@gmail.com	20	BCA	2026-10-27 06:01:17	2026-10-27 06:01:17	NULL

Exercise 13 – Course Validation

week8_fsd 2405009 > sequelize-lab > JS studentQueries.js > ...

```
const { Student } = require('./models');

async function run() {
    for (let course of ['BCA', 'MCA', 'BCom', 'BA']) {
        try {
            await Student.create({
                name: 'Riya',
                email: `${course}@gmail.com`,
                age: 20,
                course: course
            });

            console.log(course, 'Pass');
        } catch (error) {
            console.log(course, 'Fail');
        }
    }
}

run();
```

```
course: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
    isIn: [['BCA', 'BSc CS', 'MCA', 'MSc CS']]
      }
  }
```

Output

```
C:\Users\TYBCA\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
BCA Pass
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
MCA Pass
BCom Fail
BA Fail
```

☐	Edit	Copy	Delete	20	Riya	BCA@gmail.com	20	BCA	2026-10-27 06:05:49	2026-10-27 06:05:49	NULL
☐	Edit	Copy	Delete	21	Riya	MCA@gmail.com	20	MCA	2026-10-27 06:05:49	2026-10-27 06:05:49	NULL

Exercise 14 – Combined Validation

week8_fsd 2405009 > sequelize-lab > models > JS student.js > ...

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class Student extends Model {
    /**
     * Helper method for defining associations.
     * This method is not a part of Sequelize lifecycle.
     * The `models/index` file will call this method automatically.
     */
    static associate(models) {
      // define association here
      Student.hasMany(models.StudentProfile, {
        foreignKey: 'studentId'
      });
    }
  }
  Student.init({
    name: {
          type: DataTypes.STRING,
          allowNull: false,
          validate: {
         len: [3, 50],
          is: /^[a-zA-Z ]+$/
           }
         },
    email: {
           type: DataTypes.STRING,
            allowNull: false,
            unique: true,
            validate: { isEmail: true }
         },
    age: {
          type: DataTypes.INTEGER,
          allowNull: false,
          validate: { min: 17, max: 30 }
        },
    course: {
        type: DataTypes.STRING,
        allowNull: false,
        validate: {
        isIn: [['BCA', 'BSc CS', 'MCA', 'MSc CS']]
          }
      }
  },
  {
    sequelize,
    modelName: 'Student',
  });
  return Student;
};

```

week8_fsd 2405009 > sequelize-lab > JS studentQueries.js > ...

```
const { Student } = require('./models');

async function test(student, message) {
    await Student.create(student)
        .then(() => console.log(message + ' Pass'))
        .catch(() => console.log(message + ' Fail'));
}

async function run() {
    await test({
        name: 'Shivani',
        email: 'shivani@gmail.com',
        age: 20,
        course: 'BCA'
    }, 'Valid record');

    await test({
        name: '',
        email: 'missing@gmail.com',
        age: 20,
        course: 'BCA'
    }, 'Missing name');

    await test({
        name: 'Shivam',
        email: 'shivam@abc',
        age: 20,
        course: 'BCA'
    }, 'Invalid email');

    await test({
        name: 'Rahul',
        email: 'shivani@gmail.com',
        age: 20,
        course: 'BCA'
    }, 'Duplicate email');

    await test({
        name: 'Priya',
        email: 'priya@gmail.com',
        age: 15,
        course: 'BCA'
    }, 'Age 15');

    await test({
        name: 'Aman',
        email: 'aman@gmail.com',
        age: 40,
        course: 'BCA'
    }, 'Age 40');

    await test({
        name: 'Riya',
        email: 'riya@gmail.com',
        age: 20,
        course: 'BCom'
    }, 'Invalid course');
}

run();
```

Output

```
C:\Users\TYBCA\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
Valid record Pass
Missing name Fail
Invalid email Fail
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
Duplicate email Pass
Age 15 Fail
Age 40 Fail
Invalid course Fail
```

☐	Edit	Copy	Delete	22	Shivani	shivani@gmail.com	20	BCA	2026-10-27 06:19:22	2026-10-27 06:19:22	NULL
☐	Edit	Copy	Delete	23	Rahul	shivani@gmail.com	20	BCA	2026-10-27 06:19:22	2026-10-27 06:19:22	NULL

Exercise 15 – Raw SELECT

week8_fsd 2405009 > sequelize-lab > JS studentQueries.js > run

```
1   const { sequelize } = require('./models');
2
3   async function run() {
4       // Display all students
5       const [all] = await sequelize.query('SELECT * FROM Students');
6       console.log('All Students:', all);
7
8       // Display name and email
9       const [students] = await sequelize.query(
10          'SELECT name, email FROM Students'
11      );
12      console.log('Name and Email:', students);
13
14      // Display BCA students
15      const [bca] = await sequelize.query(
16          "SELECT * FROM Students WHERE course = 'BCA'"
17      );
18      console.log('BCA Students:', bca);
19
20      // Display students older than 20
21      const [older] = await sequelize.query(
22          'SELECT * FROM Students WHERE age > 20'
23      );
24      console.log('Students older than 20:', older);
25  }
26
27  run().catch(error => console.log(error.message));
```

Output

```
C:\Users\TYBCA\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): SELECT * FROM Students
All Students: [
  {
    id: 12,
    name: 'Davin',
    email: 'john@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-22T04:36:37.000Z,
    updatedAt: 2026-09-22T04:36:37.000Z,
    phone: null
  },
  {
    id: 13,
    name: 'Davin',
    email: 'john@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-22T04:37:02.000Z,
    updatedAt: 2026-09-22T04:37:02.000Z,
    phone: null
  },
  {
    id: 14,
    name: 'Max',
    email: 'student@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T05:46:27.000Z,
    updatedAt: 2026-10-27T05:46:27.000Z,
    phone: null
  },
  {
    id: 15,
    name: 'Max',
    email: 'max17@gmail.com',
    age: 17,
    course: 'BCA',
    createdAt: 2026-10-27T05:54:55.000Z,
    updatedAt: 2026-10-27T05:54:55.000Z,
    phone: null
  },
```

```
  id: 16,
  name: 'Max',
  email: 'max25@gmail.com',
  age: 25,
  course: 'BCA',
  createdAt: 2026-10-27T05:54:55.000Z,
  updatedAt: 2026-10-27T05:54:55.000Z,
  phone: null
},
{
  id: 17,
  name: 'Max',
  email: 'max30@gmail.com',
  age: 30,
  course: 'BCA',
  createdAt: 2026-10-27T05:54:55.000Z,
  updatedAt: 2026-10-27T05:54:55.000Z,
  phone: null
},
{
  id: 18,
  name: 'Shivani',
  email: 'Shivani@gmail.com',
  age: 20,
  course: 'BCA',
  createdAt: 2026-10-27T06:01:17.000Z,
  updatedAt: 2026-10-27T06:01:17.000Z,
  phone: null
},
{
  id: 19,
  name: 'Shivam',
  email: 'Shivam@gmail.com',
  age: 20,
  course: 'BCA',
  createdAt: 2026-10-27T06:01:17.000Z,
  updatedAt: 2026-10-27T06:01:17.000Z,
  phone: null
},
{
  id: 20,
  name: 'Riya',
  email: 'BCA@gmail.com',
  age: 20,
  course: 'BCA',
  createdAt: 2026-10-27T06:05:49.000Z,
  updatedAt: 2026-10-27T06:05:49.000Z,
  phone: null
},
```

```
  {
    id: 21,
    name: 'Riya',
    email: 'MCA@gmail.com',
    age: 20,
    course: 'MCA',
    createdAt: 2026-10-27T06:05:49.000Z,
    updatedAt: 2026-10-27T06:05:49.000Z,
    phone: null
  },
  {
    id: 22,
    name: 'Shivani',
    email: 'shivani@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:19:22.000Z,
    updatedAt: 2026-10-27T06:19:22.000Z,
    phone: null
  },
  {
    id: 23,
    name: 'Rahul',
    email: 'shivani@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:19:22.000Z,
    updatedAt: 2026-10-27T06:19:22.000Z,
    phone: null
  }
]
Executing (default): SELECT name, email FROM Students
Name and Email: [
  { name: 'Davin', email: 'john@gmail.com' },
  { name: 'Davin', email: 'john@gmail.com' },
  { name: 'Max', email: 'student@gmail.com' },
  { name: 'Max', email: 'max17@gmail.com' },
  { name: 'Max', email: 'max25@gmail.com' },
  { name: 'Max', email: 'max30@gmail.com' },
  { name: 'Shivani', email: 'Shivani@gmail.com' },
  { name: 'Shivam', email: 'Shivam@gmail.com' },
  { name: 'Riya', email: 'BCA@gmail.com' },
  { name: 'Riya', email: 'MCA@gmail.com' },
  { name: 'Shivani', email: 'shivani@gmail.com' },
  { name: 'Rahul', email: 'shivani@gmail.com' }
]
```

```
]
Executing (default): SELECT * FROM Students WHERE course = 'BCA'
BCA Students: [
  {
    id: 12,
    name: 'Davin',
    email: 'john@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-22T04:36:37.000Z,
    updatedAt: 2026-09-22T04:36:37.000Z,
    phone: null
  },
  {
    id: 13,
    name: 'Davin',
    email: 'john@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-22T04:37:02.000Z,
    updatedAt: 2026-09-22T04:37:02.000Z,
    phone: null
  },
  {
    id: 14,
    name: 'Max',
    email: 'student@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T05:46:27.000Z,
    updatedAt: 2026-10-27T05:46:27.000Z,
    phone: null
  },
  {
    id: 15,
    name: 'Max',
    email: 'max17@gmail.com',
    age: 17,
    course: 'BCA',
    createdAt: 2026-10-27T05:54:55.000Z,
    updatedAt: 2026-10-27T05:54:55.000Z,
    phone: null
  },
```

```
  id: 16,
  name: 'Max',
  email: 'max25@gmail.com',
  age: 25,
  course: 'BCA',
  createdAt: 2026-10-27T05:54:55.000Z,
  updatedAt: 2026-10-27T05:54:55.000Z,
  phone: null
},
{
  id: 17,
  name: 'Max',
  email: 'max30@gmail.com',
  age: 30,
  course: 'BCA',
  createdAt: 2026-10-27T05:54:55.000Z,
  updatedAt: 2026-10-27T05:54:55.000Z,
  phone: null
},
{
  id: 18,
  name: 'Shivani',
  email: 'Shivani@gmail.com',
  age: 20,
  course: 'BCA',
  createdAt: 2026-10-27T06:01:17.000Z,
  updatedAt: 2026-10-27T06:01:17.000Z,
  phone: null
},
{
  id: 19,
  name: 'Shivam',
  email: 'Shivam@gmail.com',
  age: 20,
  course: 'BCA',
  createdAt: 2026-10-27T06:01:17.000Z,
  updatedAt: 2026-10-27T06:01:17.000Z,
  phone: null
},
{
  id: 20,
  name: 'Riya',
  email: 'BCA@gmail.com',
  age: 20,
  course: 'BCA',
  createdAt: 2026-10-27T06:05:49.000Z,
  updatedAt: 2026-10-27T06:05:49.000Z,
  phone: null
},
{
```

```
  {
    id: 22,
    name: 'Shivani',
    email: 'shivani@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:19:22.000Z,
    updatedAt: 2026-10-27T06:19:22.000Z,
    phone: null
  },
  {
    id: 23,
    name: 'Rahul',
    email: 'shivani@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:19:22.000Z,
    updatedAt: 2026-10-27T06:19:22.000Z,
    phone: null
  }
]
Executing (default): SELECT * FROM Students WHERE age > 20
Students older than 20: [
  {
    id: 16,
    name: 'Max',
    email: 'max25@gmail.com',
    age: 25,
    course: 'BCA',
    createdAt: 2026-10-27T05:54:55.000Z,
    updatedAt: 2026-10-27T05:54:55.000Z,
    phone: null
  },
  {
    id: 17,
    name: 'Max',
    email: 'max30@gmail.com',
    age: 30,
    course: 'BCA',
    createdAt: 2026-10-27T05:54:55.000Z,
    updatedAt: 2026-10-27T05:54:55.000Z,
    phone: null
  }
]
```

	id	name	email	age	course	createdAt	updatedAt	phone
Edit Copy Delete	12	Davin	john@gmail.com	20	BCA	2026-09-22 04:36:37	2026-09-22 04:36:37	NULL
Edit Copy Delete	13	Davin	john@gmail.com	20	BCA	2026-09-22 04:37:02	2026-09-22 04:37:02	NULL
Edit Copy Delete	14	Max	student@gmail.com	20	BCA	2026-10-27 05:46:27	2026-10-27 05:46:27	NULL
Edit Copy Delete	15	Max	max17@gmail.com	17	BCA	2026-10-27 05:54:55	2026-10-27 05:54:55	NULL
Edit Copy Delete	16	Max	max25@gmail.com	25	BCA	2026-10-27 05:54:55	2026-10-27 05:54:55	NULL
Edit Copy Delete	17	Max	max30@gmail.com	30	BCA	2026-10-27 05:54:55	2026-10-27 05:54:55	NULL
Edit Copy Delete	18	Shivani	Shivani@gmail.com	20	BCA	2026-10-27 06:01:17	2026-10-27 06:01:17	NULL
Edit Copy Delete	19	Shivam	Shivam@gmail.com	20	BCA	2026-10-27 06:01:17	2026-10-27 06:01:17	NULL
Edit Copy Delete	20	Riya	BCA@gmail.com	20	BCA	2026-10-27 06:05:49	2026-10-27 06:05:49	NULL
Edit Copy Delete	21	Riya	MCA@gmail.com	20	MCA	2026-10-27 06:05:49	2026-10-27 06:05:49	NULL
Edit Copy Delete	22	Shivani	shivani@gmail.com	20	BCA	2026-10-27 06:19:22	2026-10-27 06:19:22	NULL
Edit Copy Delete	23	Rahul	shivani@gmail.com	20	BCA	2026-10-27 06:19:22	2026-10-27 06:19:22	NULL

Exercise 16 – Parameterized Raw Queries

1. Search by course

week8_fsd 2405009 > sequelize-lab > JS studentQueries.js > ...

```
1  const { sequelize } = require('./models');
2
3  async function run() {
4      const [students] = await sequelize.query(
5          'SELECT * FROM Students WHERE course = :course',
6          { replacements: { course: 'BCA' } }
7      );
8
9      console.log(students);
10 }
11
12 run();
```

Output

```
C:\Users\TYBCA\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): SELECT * FROM Students WHERE course = 'BCA'
[
  {
    id: 12,
    name: 'Davin',
    email: 'john@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-22T04:36:37.000Z,
    updatedAt: 2026-09-22T04:36:37.000Z,
    phone: null
  },
  {
    id: 13,
    name: 'Davin',
    email: 'john@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-09-22T04:37:02.000Z,
    updatedAt: 2026-09-22T04:37:02.000Z,
    phone: null
  },
  {
    id: 14,
    name: 'Max',
    email: 'student@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T05:46:27.000Z,
    updatedAt: 2026-10-27T05:46:27.000Z,
    phone: null
  },
  {
    id: 15,
    name: 'Max',
    email: 'max17@gmail.com',
    age: 17,
    course: 'BCA',
    createdAt: 2026-10-27T05:54:55.000Z,
    updatedAt: 2026-10-27T05:54:55.000Z,
    phone: null
  },
```

```
[
  {
    id: 16,
    name: 'Max',
    email: 'max25@gmail.com',
    age: 25,
    course: 'BCA',
    createdAt: 2026-10-27T05:54:55.000Z,
    updatedAt: 2026-10-27T05:54:55.000Z,
    phone: null
  },
  {
    id: 17,
    name: 'Max',
    email: 'max30@gmail.com',
    age: 30,
    course: 'BCA',
    createdAt: 2026-10-27T05:54:55.000Z,
    updatedAt: 2026-10-27T05:54:55.000Z,
    phone: null
  },
  {
    id: 18,
    name: 'Shivani',
    email: 'Shivani@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:01:17.000Z,
    updatedAt: 2026-10-27T06:01:17.000Z,
    phone: null
  },
  {
    id: 19,
    name: 'Shivam',
    email: 'Shivam@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:01:17.000Z,
    updatedAt: 2026-10-27T06:01:17.000Z,
    phone: null
  },
  {
    id: 20,
    name: 'Riya',
    email: 'BCA@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:05:49.000Z,
    updatedAt: 2026-10-27T06:05:49.000Z,
    phone: null
  },
  {
    id: 22,
    name: 'Shivani',
    email: 'shivani@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:19:22.000Z,
    updatedAt: 2026-10-27T06:19:22.000Z,
    phone: null
  },
  {
    id: 23,
    name: 'Rahul',
    email: 'shivani@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:19:22.000Z,
    updatedAt: 2026-10-27T06:19:22.000Z,
    phone: null
  }
]
```


2. Search by email

week8_fsd 2405009 > sequelize-lab > JS studentQueries.js > ...

```
const { sequelize } = require('./models');

async function run() {
    const [students] = await sequelize.query(
        'SELECT * FROM Students WHERE email = :email',
        { replacements: { email: 'shivani@gmail.com' } }
    );

    console.log(students);
}

run();
```

Output

```
C:\Users\TYBCA\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): SELECT * FROM Students WHERE email = 'shivani@gmail.com'
[
  {
    id: 18,
    name: 'Shivani',
    email: 'Shivani@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:01:17.000Z,
    updatedAt: 2026-10-27T06:01:17.000Z,
    phone: null
  },
  {
    id: 22,
    name: 'Shivani',
    email: 'shivani@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:19:22.000Z,
    updatedAt: 2026-10-27T06:19:22.000Z,
    phone: null
  },
```

3. Search by age

week8_fsd 2405009 > sequelize-lab > JS studentQueries.js > run

```
const { sequelize } = require('./models');

async function run() {
    const [students] = await sequelize.query(
        'SELECT * FROM Students WHERE age = :age',
        { replacements: { age: 17 } }
    );

    console.log(students);
}

run();
```

Output

```
C:\Users\TYBCA\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): SELECT * FROM Students WHERE age = 17
[
  {
    id: 15,
    name: 'Max',
    email: 'max17@gmail.com',
    age: 17,
    course: 'BCA',
    createdAt: 2026-10-27T05:54:55.000Z,
    updatedAt: 2026-10-27T05:54:55.000Z,
    phone: null
  }
]
```

4. Search by name

week8_fsd 2405009 > sequelize-lab > JS studentQueries.js > ...

```
1  const { sequelize } = require('./models');
2
3  async function run() {
4      const [students] = await sequelize.query(
5          'SELECT * FROM Students WHERE name = :name',
6          { replacements: { name: 'Rahul' } }
7      );
8
9      console.log(students);
10 }
11
12 run();
```

Output

```
C:\Users\TYBCA\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab>node studentQueries.js
Executing (default): SELECT * FROM Students WHERE name = 'Rahul'
[
  {
    id: 23,
    name: 'Rahul',
    email: 'shivani@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-27T06:19:22.000Z,
    updatedAt: 2026-10-27T06:19:22.000Z,
    phone: null
  }
]
```

Exercise 17 – Raw INSERT, UPDATE and DELETE

```
const { sequelize } = require('./models');
const readline = require('readline');

const rl = readline.createInterface({
```

```
    input: process.stdin,
    output: process.stdout
});

function ask(question) {
    return new Promise(resolve => rl.question(question, resolve));
}

async function run() {
    // INSERT
    const name = await ask('Enter name: ');
    const email = await ask('Enter email: ');
    const age = await ask('Enter age: ');
    const course = await ask('Enter course: ');

    await sequelize.query(`
        INSERT INTO Students
        (name, email, age, course, createdAt, updatedAt)
        VALUES (:name, :email, :age, :course, NOW(), NOW())
    `, {
        replacements: { name, email, age, course }
    });

    console.log('Student inserted.');

    // UPDATE
    const id1 = await ask('Enter student ID to update: ');
    const newAge = await ask('Enter new age: ');

    await sequelize.query(
        'UPDATE Students SET age = :age, updatedAt = NOW() WHERE id =
:id',
        { replacements: { age: newAge, id: id1 } }
    );

    console.log('Student updated.');

    // DELETE
    const id2 = await ask('Enter student ID to delete: ');

    await sequelize.query(
        'DELETE FROM Students WHERE id = :id',
        { replacements: { id: id2 } }
    );

    console.log('Student deleted.');
```

```
    rl.close();
}

run()
    .catch(error => console.log(error.message))
    .finally(() => rl.close());
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009-20261027T044130Z-1-001\week8_fsd 2405009\sequelize-lab> node studentQueries.js
Enter name: Hasmani Bilung
Enter email: asmanibilung08@gmail.com
Enter age: 20
Enter course: BCA
Executing (default): INSERT INTO Students
       (name, email, age, course, createdAt, updatedAt)
       VALUES ('Hasmani Bilung', 'asmanibilung08@gmail.com', '20', 'BCA', NOW(), NOW())
Student inserted.
Enter student ID to update: 1
Enter new age: 19
Executing (default): UPDATE Students SET age = '19', updatedAt = NOW() WHERE id = '1'
Student updated.
Enter student ID to delete: 3
Executing (default): DELETE FROM Students WHERE id = '3'
Student deleted.
```

Exercise 18 – Raw Aggregate Queries

1. Count total students

sequelize-lab > JS rawquery.js > ...

```
1   const { sequelize } = require('./models');
2
3   async function run() {
4       const [result] = await sequelize.query(
5           'SELECT COUNT(*) AS totalStudents FROM Students'
6       );
7
8       console.log(result);
9   }
10
11  run();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node rawquery.js
Executing (default): SELECT COUNT(*) AS totalStudents FROM Students
[ { totalStudents: 4 } ]
```

←T→	id	name	email	age	course	createdAt	updatedAt	phone
Edit Copy Delete	1	Riya	MCA@gmail.com	19	MCA	2026-10-27 06:05:49	2026-10-29 19:02:41	NULL
Edit Copy Delete	2	Hasmani Bilung	asmanibilung08@gmail.com	20	BCA	2026-10-29 19:02:14	2026-10-29 19:02:14	NULL
Edit Copy Delete	3	Mansi Bhandari	mansi@gmail.com	20	BCA	2026-10-29 20:08:27	2026-10-29 20:08:27	NULL
Edit Copy Delete	4	Pramila	pramila@gmail.com	23	BBA	2026-10-29 20:08:59	2026-10-29 20:08:59	NULL

2. Find average age

```
sequelize-lab > JS rawquery.js > ...
1   const { sequelize } = require('./models');
2
3   async function run() {
4       const [result] = await sequelize.query(
5           'SELECT AVG(age) AS averageAge FROM Students'
6       );
7
8       console.log(result);
9   }
10
11  run();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node rawquery.js
Executing (default): SELECT AVG(age) AS averageAge FROM Students
[ { averageAge: '20.5000' } ]
```

3. Find maximum age

```
sequelize-lab > JS rawquery.js > ...
1   const { sequelize } = require('./models');
2
3   async function run() {
4       const [result] = await sequelize.query(
5           'SELECT MAX(age) AS maximumAge FROM Students'
6       );
7
8       console.log(result);
9   }
10
11  run();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node rawquery.js
Executing (default): SELECT MAX(age) AS maximumAge FROM Students
[ { maximumAge: 23 } ]
```

4. Count students in each course

```
sequelize-lab > JS rawquery.js > ...
1   const { sequelize } = require('./models');
2
3   async function run() {
4       const [result] = await sequelize.query(
5           'SELECT course, COUNT(*) AS total FROM Students GROUP BY course'
6       );
7
8       console.log(result);
9   }
10
11  run();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node rawquery.js
Executing (default): SELECT course, COUNT(*) AS total FROM Students GROUP BY course
[
  { course: 'BBA', total: 1 },
  { course: 'BCA', total: 2 },
  { course: 'MCA', total: 1 }
]
```

5. Find the course with the highest number of students

```
sequelize-lab > JS rawquery.js > ...
1   const { sequelize } = require('./models');
2
3   async function run() {
4       const [result] = await sequelize.query(`
5           SELECT course, COUNT(*) AS total
6           FROM Students
7           GROUP BY course
8           ORDER BY total DESC
9           LIMIT 1
10      `);
11
12      console.log(result);
13  }
14
15  run();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node rawquery.js
Executing (default): SELECT course, COUNT(*) AS total
        FROM Students
        GROUP BY course
        ORDER BY total DESC
        LIMIT 1
[ { course: 'BCA', total: 2 } ]
```

Exercise 19 – Finder vs Raw SQL

1. Using Finder (findAll)

```
sequelize-lab > JS finder.js > ...
1   const { Student } = require('./models');
2
3   async function run() {
4       const students = await Student.findAll({
5           where: { course: 'BCA' }
6       });
7
8       console.log(students);
9   }
10
11  run();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node finder.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`course` = 'BCA';
[
  Student {
    dataValues: {
      id: 2,
      name: 'Hasmani Bilung',
      email: 'asmanibilung08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T19:02:14.000Z,
      updatedAt: 2026-10-29T19:02:14.000Z
    },
    _previousDataValues: {
      id: 2,
      name: 'Hasmani Bilung',
      email: 'asmanibilung08@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T19:02:14.000Z,
      updatedAt: 2026-10-29T19:02:14.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
```

```
  Student {
    dataValues: {
      id: 3,
      name: 'Mansi Bhandari',
      email: 'mansi@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T20:08:27.000Z,
      updatedAt: 2026-10-29T20:08:27.000Z
    },
    _previousDataValues: {
      id: 3,
      name: 'Mansi Bhandari',
      email: 'mansi@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-10-29T20:08:27.000Z,
      updatedAt: 2026-10-29T20:08:27.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

2. Using Raw SQL

sequelize-lab > JS rawsql.js > ...

```
1   const { sequelize } = require('./models');
2
3   async function run() {
4       const [students] = await sequelize.query(
5           'SELECT * FROM Students WHERE course = :course',
6           { replacements: { course: 'BCA' } }
7       );
8
9       console.log(students);
10  }
11
12  run();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node rawsql.js
Executing (default): SELECT * FROM Students WHERE course = 'BCA'
[
  {
    id: 2,
    name: 'Hasmani Bilung',
    email: 'asmanibilung08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-29T19:02:14.000Z,
    updatedAt: 2026-10-29T19:02:14.000Z,
    phone: null
  },
  {
    id: 3,
    name: 'Mansi Bhandari',
    email: 'mansi@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-29T20:08:27.000Z,
    updatedAt: 2026-10-29T20:08:27.000Z,
    phone: null
  }
]
```

Exercise 20 – Final Challenge: Student Search System

===== STUDENT SEARCH SYSTEM =====

1. Search by ID
2. Search by Name
3. Search by Email
4. Search by Course
5. Search by Age Range
6. Display All Students
7. Count Students
8. Exit

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node studentQueries.js
2. Search by Name
3. Search by Email
4. Search by Course
5. Search by Age Range
6. Display All Students
7. Count Students
8. Exit
Enter choice: 1
Enter ID: 2
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`id` = '2';
Student {
  dataValues: {
    id: 2,
    name: 'Hasmani Bilung',
    email: 'asmanibilung08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-29T19:02:14.000Z,
    updatedAt: 2026-10-29T19:02:14.000Z
  },
  _previousDataValues: {
    id: 2,
    name: 'Hasmani Bilung',
    email: 'asmanibilung08@gmail.com',
    age: 20,
    course: 'BCA',
    createdAt: 2026-10-29T19:02:14.000Z,
    updatedAt: 2026-10-29T19:02:14.000Z
  },
  uniqno: 1,
  _changed: Set(0) {},
  _options: {
    isNewRecord: false,
    _schema: null,
    _schemaDelimiter: '',
    raw: true,
    attributes: [
      'id',
      'name',
      'email',
      'age',
      'course',
      'createdAt',
      'updatedAt'
    ]
  },
  isNewRecord: false
}
```

```
===== STUDENT SEARCH SYSTEM =====
1. Search by ID
2. Search by Name
3. Search by Email
4. Search by Course
5. Search by Age Range
6. Display All Students
7. Count Students
8. Exit
Enter choice: 2
Enter name: Riya
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`name` = 'Riya';
[
  Student {
    dataValues: {
      id: 1,
      name: 'Riya',
      email: 'MCA@gmail.com',
      age: 19,
      course: 'MCA',
      createdAt: 2026-10-27T06:05:49.000Z,
      updatedAt: 2026-10-29T19:02:41.000Z
    },
    _previousDataValues: {
      id: 1,
      name: 'Riya',
      email: 'MCA@gmail.com',
      age: 19,
      course: 'MCA',
      createdAt: 2026-10-27T06:05:49.000Z,
      updatedAt: 2026-10-29T19:02:41.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]
```

```
===== STUDENT SEARCH SYSTEM =====
1. Search by ID
2. Search by Name
3. Search by Email
4. Search by Course
5. Search by Age Range
6. Display All Students
7. Count Students
8. Exit
Enter choice: 7
Executing (default): SELECT count(*) AS `count` FROM `Students` AS `Student`;
Total students: 4

===== STUDENT SEARCH SYSTEM =====
1. Search by ID
2. Search by Name
3. Search by Email
4. Search by Course
5. Search by Age Range
6. Display All Students
7. Count Students
8. Exit
Enter choice: 8
Exiting...
```

Week 10 Exercises: Sequelize Associations

Association Concepts

Association	Meaning	Example
1:1	One record is associated with one record	Student → StudentProfile
1:M	One record is associated with many records	Department → Employees
M:N	Many records are associated with many records	Students ↔ Courses

Exercise 1 – One-to-One Association

Objective: Create a 1:1 relationship between Student and StudentProfile.

Generate the model:
npx sequelize-cli model:generate --name StudentProfile --attributes studentId:integer,address:string,phone:string

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>npx sequelize-cli model:generate --name
StudentProfile --attributes studentId:integer,address:string,phone:string

Sequelize CLI [Node: 24.18.0, CLI: 6.6.5, ORM: 6.37.8]

New model was created at C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab\models\studentp
rofile.js .
New migration was created at C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab\migrations\
20260907035221-create-student-profile.js .
```

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>npx sequelize-cli db:migrate

Sequelize CLI [Node: 24.18.0, CLI: 6.6.5, ORM: 6.37.8]

Loaded configuration file "config\config.json".
Using environment "development".
== 20260818034328-add-phone-to-students: migrating =======
== 20260818034328-add-phone-to-students: migrated (0.022s)

== 20260907035221-create-student-profile: migrating =======
== 20260907035221-create-student-profile: migrated (0.027s)
```

Exercise 2 – Define the 1:1 Association

In studentProfile.js

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
```

```
  class StudentProfile extends Model {
    /**
     * Helper method for defining associations.
     * This method is not a part of Sequelize
lifecycle.
     * The `models/index` file will call this method
automatically.
     */
    static associate(models) {
      // define association here
       StudentProfile.belongsTo(models.Student, {
       foreignKey: 'studentId'
      });
    }
  }
  StudentProfile.init({
    studentId: DataTypes.INTEGER,
    address: DataTypes.STRING,
    phone: DataTypes.STRING
  }, {
    sequelize,
    modelName: 'StudentProfile',
  });
  return StudentProfile;
};
```

In [student.js]

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class Student extends Model {
    /**
```

```
     * Helper method for defining associations.
     * This method is not a part of Sequelize
lifecycle.
     * The `models/index` file will call this method
automatically.
     */
    static associate(models) {
      // define association here
      Student.hasOne(models.StudentProfile, {
        foreignKey: 'studentId'
      });
    }
  }
  Student.init({
    name: DataTypes.STRING,
    email: {
              type: DataTypes.STRING,
              allowNull: false
            },
    age: DataTypes.INTEGER,
    course: DataTypes.STRING
  }, {
    sequelize,
    modelName: 'Student',
  });
  return Student;
};
```

Exercise 3 – Create Student and Profile

studentq.js (1:1) creates one student and one profile

```
const models= require('./models');
//console.log(models);


async function addData() {
```

```
        const st = await models.Student.create({
            name: 'Riya',
            email: 'riya@gmail.com',
            age: 19,
            course: 'MCA'
        });

        await models.StudentProfile.create({
            studentId: st.id,
            address: 'Panaji',
            phone: 123456788
        });

        //console.log(student);
    }

addData();
```

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node studentq.js
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`up
datedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
Executing (default): INSERT INTO `StudentProfiles` (`id`,`studentId`,`address`,`phone`,`created
At`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?);
```

	id	name	email	age	course	createdAt	updatedAt	phone
Edit Copy Delete	1	Riya	riya@gmail.com	19	MCA	2026-10-27 06:05:49	2026-10-29 19:02:41	NULL

	id	studentId	address	phone	createdAt	updatedAt
Edit Copy Delete	1	1	Panaji	123456788	2026-09-07 03:59:16	2026-09-07 03:59:16

	id	studentId	address	phone	createdAt	updatedAt
Edit Copy Delete	1	1	Panaji	123456788	2026-09-07 03:59:16	2026-09-07 03:59:16
Edit Copy Delete	2	4	Usgao	9876543211	2026-09-07 04:58:19	2026-09-07 04:58:19
Edit Copy Delete	3	2	Ponda	9000000002	2026-10-30 02:27:21	2026-10-30 02:27:21
Edit Copy Delete	4	3	Vasco	9000000003	2026-10-30 02:27:21	2026-10-30 02:27:21

	id	name	email	age	course	createdAt	updatedAt	phone
Edit Copy Delete	1	Riya	riya@gmail.com	19	MCA	2026-10-27 06:05:49	2026-10-29 19:02:41	NULL
Edit Copy Delete	2	Hasmani Bilung	asmanibilung08@gmail.com	20	BCA	2026-10-29 19:02:14	2026-10-29 19:02:14	NULL
Edit Copy Delete	3	Mansi Bhandari	mansi@gmail.com	20	BCA	2026-10-29 20:08:27	2026-10-29 20:08:27	NULL
Edit Copy Delete	4	Pramila	pramila@gmail.com	23	BBA	2026-10-29 20:08:59	2026-10-29 20:08:59	NULL

Exercise 4 – Retrieve 1:1 Associated Data

1: Display one student with profile information

sequelize-lab > JS student11.js > ...

```
1   const models = require('./models');
2
3   async function getData() {
4       const student = await models.Student.findOne({
5           where: { id: 1 },
6           include: models.StudentProfile
7       });
8
9       console.log(student.toJSON());
10  }
11
12  getData();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node student11.js
Executing (default): SELECT `Student`.`id`, `Student`.`name`, `Student`.`email`, `Student`.`age`, `Student`.`course`, `Student`.`createdAt`, `Student
`.`updatedAt`, `StudentProfile`.`id` AS `StudentProfile.id`, `StudentProfile`.`studentId` AS `StudentProfile.studentId`, `StudentProfile`.`address` A
S `StudentProfile.address`, `StudentProfile`.`phone` AS `StudentProfile.phone`, `StudentProfile`.`createdAt` AS `StudentProfile.createdAt`, `StudentP
rofile`.`updatedAt` AS `StudentProfile.updatedAt` FROM `Students` AS `Student` LEFT OUTER JOIN `StudentProfiles` AS `StudentProfile` ON `Student`.`id
` = `StudentProfile`.`studentId` WHERE `Student`.`id` = 1;
{
  id: 1,
  name: 'Riya',
  email: 'riya@gmail.com',
  age: 19,
  course: 'MCA',
  createdAt: 2026-10-27T06:05:49.000Z,
  updatedAt: 2026-10-29T19:02:41.000Z,
  StudentProfile: {
    id: 1,
    studentId: 1,
    address: 'Panaji',
    phone: '123456788',
    createdAt: 2026-09-07T03:59:16.000Z,
    updatedAt: 2026-09-07T03:59:16.000Z
  }
}
```

2: Display all students with profiles

sequelize-lab > JS student11.js > ...

```
1   const models = require('./models');
2
3   async function getData() {
4       const students = await models.Student.findAll({
5        include: models.StudentProfile
6   });
7
8   console.log(JSON.stringify(students, null, 2));
9   }
10
11  getData();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node student11.js
Executing (default): SELECT `Student`.`id`, `Student`.`name`, `Student`.`email`, `Student`.`age`, `Student`.`course`, `Student`.`createdAt`, `Student
`.`updatedAt`, `StudentProfile`.`id` AS `StudentProfile.id`, `StudentProfile`.`studentId` AS `StudentProfile.studentId`, `StudentProfile`.`address` A
S `StudentProfile.address`, `StudentProfile`.`phone` AS `StudentProfile.phone`, `StudentProfile`.`createdAt` AS `StudentProfile.createdAt`, `StudentP
rofile`.`updatedAt` AS `StudentProfile.updatedAt` FROM `Students` AS `Student` LEFT OUTER JOIN `StudentProfiles` AS `StudentProfile` ON `Student`.`id
` = `StudentProfile`.`studentId`;
[
  {
    "id": 1,
    "name": "Riya",
    "email": "riya@gmail.com",
    "age": 19,
    "course": "MCA",
    "createdAt": "2026-10-27T06:05:49.000Z",
    "updatedAt": "2026-10-29T19:02:41.000Z",
    "StudentProfile": {
      "id": 1,
      "studentId": 1,
      "address": "Panaji",
      "phone": "123456788",
      "createdAt": "2026-09-07T03:59:16.000Z",
      "updatedAt": "2026-09-07T03:59:16.000Z"
    }
  },
  {
    "id": 4,
    "name": "Pramila",
    "email": "pramila@gmail.com",
    "age": 23,
    "course": "BBA",
    "createdAt": "2026-10-29T20:08:59.000Z",
    "updatedAt": "2026-10-29T20:08:59.000Z",
    "StudentProfile": {
      "id": 2,
      "studentId": 4,
      "address": "Usgao",
      "phone": "9876543211",
      "createdAt": "2026-09-07T04:58:19.000Z",
      "updatedAt": "2026-09-07T04:58:19.000Z"
    }
  },
```

Activate Windows
Go to Settings to activate Windows.

```
  {
    "id": 2,
    "name": "Hasmani Bilung",
    "email": "asmanibilung08@gmail.com",
    "age": 20,
    "course": "BCA",
    "createdAt": "2026-10-29T19:02:14.000Z",
    "updatedAt": "2026-10-29T19:02:14.000Z",
    "StudentProfile": {
      "id": 3,
      "studentId": 2,
      "address": "Ponda",
      "phone": "9000000002",
      "createdAt": "2026-10-30T02:27:21.000Z",
      "updatedAt": "2026-10-30T02:27:21.000Z"
    }
  },
  {
    "id": 3,
    "name": "Mansi Bhandari",
    "email": "mansi@gmail.com",
    "age": 20,
    "course": "BCA",
    "createdAt": "2026-10-29T20:08:27.000Z",
    "updatedAt": "2026-10-29T20:08:27.000Z",
    "StudentProfile": {
      "id": 4,
      "studentId": 3,
      "address": "Vasco",
      "phone": "9000000003",
      "createdAt": "2026-10-30T02:27:21.000Z",
      "updatedAt": "2026-10-30T02:27:21.000Z"
    }
  }
]
```


3: Display only student name and profile phone

sequelize-lab > JS student11.js > ...

```
1  const models = require('./models');
2
3  async function getData() {
4     const data = await models.Student.findAll({
5      attributes: ['name'],
6      include: [{
7          model: models.StudentProfile,
8          attributes: ['phone']
9      }]
10 });
11
12 console.log(JSON.stringify(data, null, 2));
13 }
14
15 getData();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node student11.js
Executing (default): SELECT `Student`.`id`, `Student`.`name`, `StudentProfile`.`id` AS `StudentProfile.id`, `StudentProfile`.`phone` AS `StudentProfi
le.phone` FROM `Students` AS `Student` LEFT OUTER JOIN `StudentProfiles` AS `StudentProfile` ON `Student`.`id` = `StudentProfile`.`studentId`;
[
  {
    "name": "Riya",
    "StudentProfile": {
      "phone": "123456788"
    }
  },
  {
    "name": "Pramila",
    "StudentProfile": {
      "phone": "9876543211"
    }
  },
  {
    "name": "Hasmani Bilung",
    "StudentProfile": {
      "phone": "9000000002"
    }
  },
  {
    "name": "Mansi Bhandari",
    "StudentProfile": {
      "phone": "9000000003"
    }
  }
]
```

4: Test what happens when a student has no profile

sequelize-lab > JS student11.js > ...

```
1   const models = require('./models');
2
3   async function getData() {
4       const student = await models.Student.create({
5        name: 'Amit',
6        email: 'amit@gmail.com',
7        age: 21,
8        course: 'BCA'
9   });
10
11  const result = await models.Student.findOne({
12       where: { id: student.id },
13       include: models.StudentProfile
14  });
15
16  console.log(result.toJSON());
17  }
18
19  getData();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node student11.js
Executing (default): INSERT INTO `Students` (`id`,`name`,`email`,`age`,`course`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
Executing (default): SELECT `Student`.`id`, `Student`.`name`, `Student`.`email`, `Student`.`age`, `Student`.`course`, `Student`.`createdAt`, `Student
`.`updatedAt`, `StudentProfile`.`id` AS `StudentProfile.id`, `StudentProfile`.`studentId` AS `StudentProfile.studentId`, `StudentProfile`.`address` A
S `StudentProfile.address`, `StudentProfile`.`phone` AS `StudentProfile.phone`, `StudentProfile`.`createdAt` AS `StudentProfile.createdAt`, `StudentP
rofile`.`updatedAt` AS `StudentProfile.updatedAt` FROM `Students` AS `Student` LEFT OUTER JOIN `StudentProfiles` AS `StudentProfile` ON `Student`.`id
` = `StudentProfile`.`studentId` WHERE `Student`.`id` = 5;
{
  id: 5,
  name: 'Amit',
  email: 'amit@gmail.com',
  age: 21,
  course: 'BCA',
  createdAt: 2026-10-30T09:07:24.000Z,
  updatedAt: 2026-10-30T09:07:24.000Z,
  StudentProfile: null
}
```

	id	name	email	age	course	createdAt	updatedAt	phone
Edit Copy Delete	1	Riya	riya@gmail.com	19	MCA	2026-10-27 06:05:49	2026-10-29 19:02:41	NULL
Edit Copy Delete	2	Hasmani Bilung	asmanibilung08@gmail.com	20	BCA	2026-10-29 19:02:14	2026-10-29 19:02:14	NULL
Edit Copy Delete	3	Mansi Bhandari	mansi@gmail.com	20	BCA	2026-10-29 20:08:27	2026-10-29 20:08:27	NULL
Edit Copy Delete	4	Pramila	pramila@gmail.com	23	BBA	2026-10-29 20:08:59	2026-10-29 20:08:59	NULL
Edit Copy Delete	5	Amit	amit@gmail.com	21	BCA	2026-10-30 09:07:24	2026-10-30 09:07:24	NULL

Exercise 5 – Association Methods for 1:1

sequelize-lab > JS student11.js > ...

```
const models = require('./models');

async function getData() {

const models = require('./models');

async function addData() {
    const student = await models.Student.findByPk(5);

    // Create profile
    const profile = await student.createStudentProfile({
        address: 'Margao',
        phone: '9999999999'
    });

    console.log('Profile created successfully');

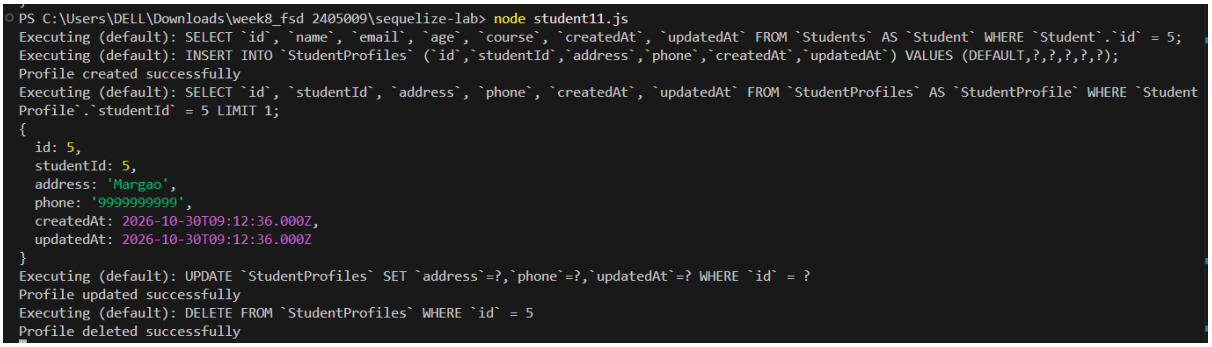
    // Retrieve profile
    const data = await student.getStudentProfile();
    console.log(data.toJSON());

    // Update profile
    await profile.update({
        address: 'Ponda',
        phone: '8888888888'
    });

    console.log('Profile updated successfully');

    // Delete profile
    await profile.destroy();
    console.log('Profile deleted successfully');
}
addData();
}
getData();
```

Output



```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node student11.js
Executing (default): SELECT `id`, `name`, `email`, `age`, `course`, `createdAt`, `updatedAt` FROM `Students` AS `Student` WHERE `Student`.`id` = 5;
Executing (default): INSERT INTO `StudentProfiles` (`id`,`studentId`,`address`,`phone`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?);
Profile created successfully
Executing (default): SELECT `id`, `studentId`, `address`, `phone`, `createdAt`, `updatedAt` FROM `StudentProfiles` AS `StudentProfile` WHERE `Student
Profile`.`studentId` = 5 LIMIT 1;
{
  id: 5,
  studentId: 5,
  address: 'Margao',
  phone: '9999999999',
  createdAt: 2026-10-30T09:12:36.000Z,
  updatedAt: 2026-10-30T09:12:36.000Z
}
Executing (default): UPDATE `StudentProfiles` SET `address`=?,`phone`=?,`updatedAt`=? WHERE `id` = ?
Profile updated successfully
Executing (default): DELETE FROM `StudentProfiles` WHERE `id` = 5
Profile deleted successfully
```

student.js - hasMany

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class Student extends Model {
    /**
     * Helper method for defining associations.
     * This method is not a part of Sequelize
lifecycle.
     * The `models/index` file will call this method
automatically.
     */
    static associate(models) {
      // define association here
      Student.hasMany(models.StudentProfile, {
        foreignKey: 'studentId'
      });
    }
  }
  Student.init({
    name: {
            type:DataTypes.STRING,
            allowNull: false,
            unique: false
          },
    email: DataTypes.STRING,
```

```
    age: DataTypes.INTEGER,
    course: DataTypes.STRING
  }, {
    sequelize,
    modelName: 'Student',
  });
  return Student;
};
```

Output

```
C:\Users\TYBCA\Desktop\week8_fsd 2405009\sequelize-lab>node studentq.js
Executing (default): SELECT `Student`.`id`, `Student`.`name`, `Student`.`email`, `Student`.`age`, `Student`.`course`, `Student`.`createdAt`
, `Student`.`updatedAt`, `StudentProfiles`.`id` AS `StudentProfiles.id`, `StudentProfiles`.`studentId` AS `StudentProfiles.studentId`, `Stu
dentProfiles`.`address` AS `StudentProfiles.address`, `StudentProfiles`.`phone` AS `StudentProfiles.phone`, `StudentProfiles`.`createdAt` A
S `StudentProfiles.createdAt`, `StudentProfiles`.`updatedAt` AS `StudentProfiles.updatedAt` FROM `Students` AS `Student` LEFT OUTER JOIN `S
tudentProfiles` AS `StudentProfiles` ON `Student`.`id` = `StudentProfiles`.`studentId` WHERE `Student`.`id` = 10;
[
  Student {
    dataValues: {
      id: 10,
      name: 'Priya',
      email: 'priya@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-09-07T03:59:16.000Z,
      updatedAt: 2026-09-07T03:59:16.000Z,
      StudentProfiles: [Array]
    },
    _previousDataValues: {
      id: 10,
      name: 'Priya',
      email: 'priya@gmail.com',
      age: 20,
      course: 'BCA',
      createdAt: 2026-09-07T03:59:16.000Z,
      updatedAt: 2026-09-07T03:59:16.000Z,
      StudentProfiles: [Array]
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
```

```
      _schemaDelimiter: '',
      include: [Array],
      includeNames: [Array],
      includeMap: [Object],
      includeValidated: true,
      attributes: [Array],
      raw: true
    },
    isNewRecord: false,
    StudentProfiles: [ [StudentProfile], [StudentProfile] ]
  }
]
```

Exercise 6 – One-to-Many Association

Objective: Create a Department–Employee relationship.

Generate the models:
npx sequelize-cli model:generate --name Department --attributes name:string,location:string

npx sequelize-cli model:generate --name Employee --attributes name:string,email:string,salary:integer,departmentId:integer

```
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab>npx sequelize-cli model:generate
,location:string

Sequelize CLI [Node: 23.6.0, CLI: 6.6.5, ORM: 6.37.8]

New model was created at C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab\models\d
New migration was created at C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab\migr
 .

C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab>npx sequelize-cli db:migrate

Sequelize CLI [Node: 23.6.0, CLI: 6.6.5, ORM: 6.37.8]

Loaded configuration file "config\config.json".
Using environment "development".
== 20260908062736-create-department: migrating =======
== 20260908062736-create-department: migrated (0.064s)
```

```
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab>npx sequelize-cli model:generate
ployee --attributes depart_Id:integer,name:string,email:string,salary:string

Sequelize CLI [Node: 23.6.0, CLI: 6.6.5, ORM: 6.37.8]

New model was created at C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab\models\e
s .
New migration was created at C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab\migr
260908063227-create-employee.js .

C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab>npx sequelize-cli db:migrate

Sequelize CLI [Node: 23.6.0, CLI: 6.6.5, ORM: 6.37.8]

Loaded configuration file "config\config.json".
Using environment "development".
== 20260908063227-create-employee: migrating =======
== 20260908063227-create-employee: migrated (0.058s)
```

Exercise 7 – Define the 1:M Association
department.js

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class department extends Model {
```

```
    /**
     * Helper method for defining associations.
     * This method is not a part of Sequelize
lifecycle.
     * The `models/index` file will call this method
automatically.
     */
    static associate(models) {
      // define association here
      department.hasMany(models.employee, {
        foreignKey: 'depart_Id'
      });
    }
  }
  department.init({
    name: DataTypes.STRING,
    location: DataTypes.STRING
  }, {
    sequelize,
    modelName: 'department',
  });
  return department;
};
```

employee.js

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class employee extends Model {
    /**
     * Helper method for defining associations.
     * This method is not a part of Sequelize
lifecycle.
```

```
     * The `models/index` file will call this method
automatically.
     */
    static associate(models) {
      // define association here
      employee.belongsTo(models.department, {
       foreignKey: 'depart_Id'
      });
    }
  }
  employee.init({
    depart_Id: DataTypes.INTEGER,
    name: DataTypes.STRING,
    email: DataTypes.STRING,
    salary: DataTypes.STRING
  }, {
    sequelize,
    modelName: 'employee',
  });
  return employee;
};
```

Exercise 8 – Create Departments and Employees

queries.js Create a department and an employee

```
const models = require('./models');

async function addData() {
    const st = await models.department.create({
        name: 'IT',
        location: 'Ponda'
    });

    const em = await models.employee.create({
        depart_Id : st.id,
```

```
        name: 'Max',
        email: 'max@gmail.com',
        salary: 50000
    });

    console.log(st);
     console.log(em);
}

addData();
```

Output

```
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab>node queries.js
Executing (default): INSERT INTO `departments` (`id`,`name`,`location`,`createdAt`,`updatedAt`) V
FAULT,?,?,?,?);
Executing (default): INSERT INTO `employees` (`id`,`depart_Id`,`name`,`email`,`salary`,`createdAt
dAt`) VALUES (DEFAULT,?,?,?,?,?,?);
```

	id	name	location	createdAt	updatedAt
Edit Copy Delete	1	IT	Ponda	2026-09-08 06:57:23	2026-09-08 06:57:23

	id	depart_Id	name	email	salary	createdAt	updatedAt
Edit Copy Delete	1	1	Max	max@gmail.com	50000	2026-09-08 06:57:23	2026-09-08 06:57:23

queries.js Add another employee to the same department

```
const models = require('./models');

async function addData() {
    const dept = await models.department.findByPk(1);

const em = await models.employee.create({
    depart_Id: dept.id,
    name:'John',
    email:'john@gmail.com',
    salary: 100000
});
console.log(dept);
```

```
console.log(em);
}

addData();
```

Output

```
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab>node 1istoM.js
Executing (default): SELECT `id`, `name`, `location`, `createdAt`, `updatedAt` FROM `departments`
S `department` WHERE `department`.`id` = 1;
Executing (default): INSERT INTO `employees` (`id`,`depart_Id`,`name`,`email`,`salary`,`createdAt
`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
```

	id	depart_Id	name	email	salary	createdAt	updatedAt
Edit Copy Delete	1	1	Max	max@gmail.com	50000	2026-09-08 06:57:23	2026-09-08 06:57:23
Edit Copy Delete	2	1	John	john@gmail.com	100000	2026-09-08 06:57:41	2026-09-08 06:57:41

Add more departments and employees

```
const models = require('./models');

async function addData() {

    // Create HR and Finance departments
    const hr = await models.department.create({
        name: 'HR',
        location: 'Ponda'
    });

    const finance = await models.department.create({
        name: 'Finance',
        location: 'Panaji'
    });

    // Create employees
    await models.employee.bulkCreate([
        {
            name: 'Rahul',
            email: 'rahul@college.com',
            salary: 35000,
            depart_Id: 1
        },
        {
            name: 'Priya',
            email: 'priya@college.com',
            salary: 40000,
            depart_Id: hr.id
        },
        {
            name: 'Amit',
```

```
            email: 'amit@college.com',
            salary: 30000,
            depart_Id: finance.id
        },
        {
            name: 'Sneha',
            email: 'sneha@college.com',
            salary: 38000,
            depart_Id: 1
        },
        {
            name: 'Karan',
            email: 'karan@college.com',
            salary: 32000,
            depart_Id: hr.id
        },
        {
            name: 'Neha',
            email: 'neha@college.com',
            salary: 45000,
            depart_Id: finance.id
        }
    ]);

    console.log('Departments and employees created successfully');
}

addData();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node queries.js
Executing (default): INSERT INTO `departments` (`id`,`name`,`location`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?);
Executing (default): INSERT INTO `departments` (`id`,`name`,`location`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?);
Executing (default): INSERT INTO `employees` (`id`,`depart_Id`,`name`,`email`,`salary`,`createdAt`,`updatedAt`) VALUES (NULL,1,'Rahul','rahul@college.c
,'2026-10-30 09:32:51','2026-10-30 09:32:51'),(NULL,2,'Priya','priya@college.com',40000,'2026-10-30 09:32:51','2026-10-30 09:32:51'),(NULL,3,'Amit','am
e.com',30000,'2026-10-30 09:32:51','2026-10-30 09:32:51'),(NULL,1,'Sneha','sneha@college.com',38000,'2026-10-30 09:32:51','2026-10-30 09:32:51'),(NULL,
,'karan@college.com',32000,'2026-10-30 09:32:51','2026-10-30 09:32:51'),(NULL,3,'Neha','neha@college.com',45000,'2026-10-30 09:32:51','2026-10-30 09:32
Departments and employees created successfully
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab>
```

	id	depart_Id	name	email	salary	createdAt	updatedAt
Edit Copy Delete	1	1	Max	max@gmail.com	50000	2026-09-08 04:58:05	2026-09-08 04:58:05
Edit Copy Delete	2	1	John	john@gmail.com	100000	2026-09-08 05:14:40	2026-09-08 05:14:40
Edit Copy Delete	3	1	Rahul	rahul@college.com	35000	2026-10-30 09:32:51	2026-10-30 09:32:51
Edit Copy Delete	4	2	Priya	priya@college.com	40000	2026-10-30 09:32:51	2026-10-30 09:32:51
Edit Copy Delete	5	3	Amit	amit@college.com	30000	2026-10-30 09:32:51	2026-10-30 09:32:51
Edit Copy Delete	6	1	Sneha	sneha@college.com	38000	2026-10-30 09:32:51	2026-10-30 09:32:51
Edit Copy Delete	7	2	Karan	karan@college.com	32000	2026-10-30 09:32:51	2026-10-30 09:32:51
Edit Copy Delete	8	3	Neha	neha@college.com	45000	2026-10-30 09:32:51	2026-10-30 09:32:51

	id	name	location	createdAt	updatedAt
Edit Copy Delete	1	IT	Usago	2026-09-08 04:58:04	2026-09-08 04:58:04
Edit Copy Delete	2	HR	Ponda	2026-10-30 09:32:51	2026-10-30 09:32:51
Edit Copy Delete	3	Finance	Panaji	2026-10-30 09:32:51	2026-10-30 09:32:51

Exercise 9 – Retrieve Department with Employees.

1. Display one department with all employees

```
sequelize-lab > JS exercise9.js > ...
1   const models = require('./models');
2
3   async function getData() {
4
5       const department = await models.department.findOne({
6           where: { id: 1 },
7           include: models.employee
8       });
9
10      console.log(department.toJSON());
11  }
12
13  getData();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node exercise9.js
Executing (default): SELECT `department`.`id`, `department`.`name`, `department`.`location`, `department`.`createdAt`, `department`.`updatedAt`, `employees`.`id` AS `employees.id`, `employees`.`depart_Id` AS `employees.depart_Id`, `employees`.`name` AS `employees.name`, `employees`.`email` AS `employees.email`, `employees`.`salary` AS `employees.salary`, `employees`.`createdAt` AS `employees.createdAt`, `employees`.`updatedAt` AS `employees.updatedAt` FROM `departments` AS `department` LEFT OUTER JOIN `employees` AS `employees` ON `department`.`id` = `employees`.`depart_Id` WHERE `department`.`id` = 1;
{
  id: 1,
  name: 'IT',
  location: 'Usago',
  createdAt: 2026-09-08T04:58:04.000Z,
  updatedAt: 2026-09-08T04:58:04.000Z,
  employees: [
    {
      id: 1,
      depart_Id: 1,
      name: 'Max',
      email: 'max@gmail.com',
      salary: 50000,
      createdAt: 2026-09-08T04:58:05.000Z,
      updatedAt: 2026-09-08T04:58:05.000Z
    },
    {
      id: 2,
      depart_Id: 1,
      name: 'John',
      email: 'john@gmail.com',
      salary: 100000,
      createdAt: 2026-09-08T05:14:40.000Z,
      updatedAt: 2026-09-08T05:14:40.000Z
    },
    {
      id: 3,
      depart_Id: 1,
      name: 'Rahul',
      email: 'rahul@college.com',
      salary: 35000,
      createdAt: 2026-10-30T09:32:51.000Z,
      updatedAt: 2026-10-30T09:32:51.000Z
    },
```

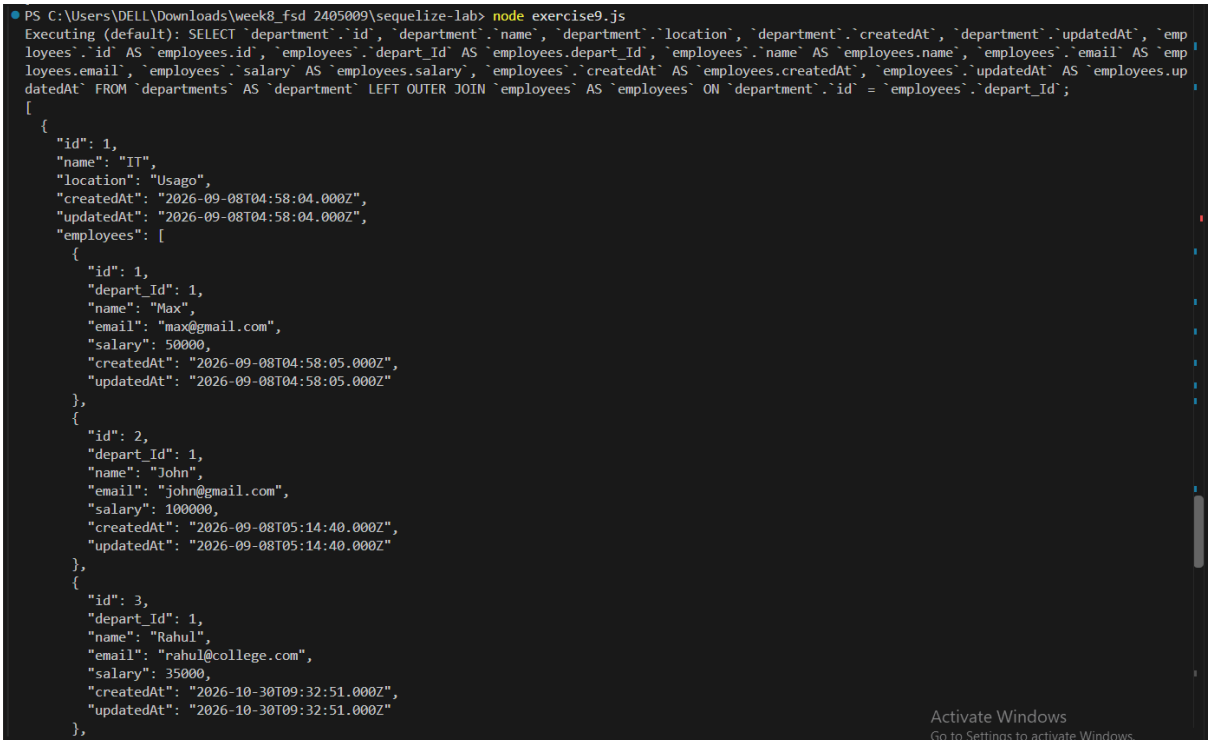
Activate Windows
Go to Settings to activate Windows.

```
      id: 6,
      depart_Id: 1,
      name: 'Sneha',
      email: 'sneha@college.com',
      salary: 38000,
      createdAt: 2026-10-30T09:32:51.000Z,
      updatedAt: 2026-10-30T09:32:51.000Z
    }
  ]
}
```

2. Display all departments with employees

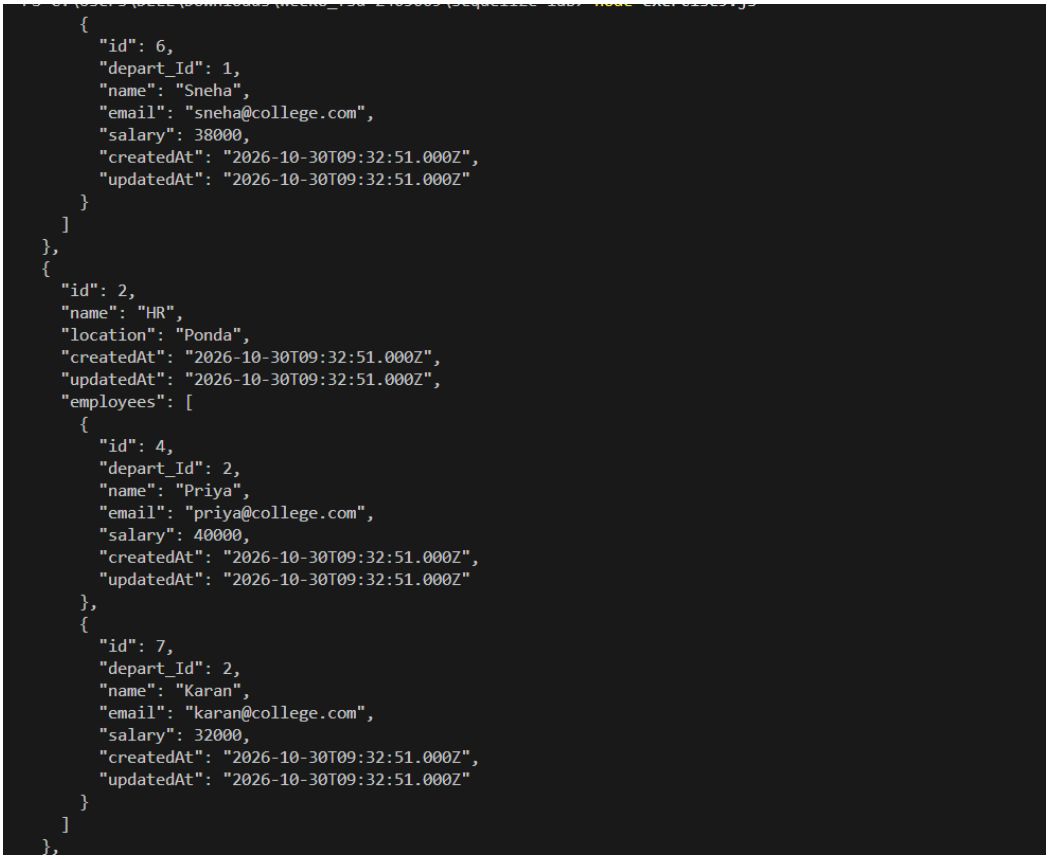
```
sequelize-lab > JS exercise9.js > ...
1   const models = require('./models');
2
3   async function getData() {
4
5       const departments = await models.department.findAll({
6       include: models.employee
7   });
8
9   console.log(JSON.stringify(departments, null, 2));
10  }
11
12  getData();
```

Output



```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node exercise9.js
Executing (default): SELECT `department`.`id`, `department`.`name`, `department`.`location`, `department`.`createdAt`, `department`.`updatedAt`, `employees`.`id` AS `employees.id`, `employees`.`depart_Id` AS `employees.depart_Id`, `employees`.`name` AS `employees.name`, `employees`.`email` AS `employees.email`, `employees`.`salary` AS `employees.salary`, `employees`.`createdAt` AS `employees.createdAt`, `employees`.`updatedAt` AS `employees.updatedAt` FROM `departments` AS `department` LEFT OUTER JOIN `employees` AS `employees` ON `department`.`id` = `employees`.`depart_Id`;
[
  {
    "id": 1,
    "name": "IT",
    "location": "Usago",
    "createdAt": "2026-09-08T04:58:04.000Z",
    "updatedAt": "2026-09-08T04:58:04.000Z",
    "employees": [
      {
        "id": 1,
        "depart_Id": 1,
        "name": "Max",
        "email": "max@gmail.com",
        "salary": 50000,
        "createdAt": "2026-09-08T04:58:05.000Z",
        "updatedAt": "2026-09-08T04:58:05.000Z"
      },
      {
        "id": 2,
        "depart_Id": 1,
        "name": "John",
        "email": "john@gmail.com",
        "salary": 100000,
        "createdAt": "2026-09-08T05:14:40.000Z",
        "updatedAt": "2026-09-08T05:14:40.000Z"
      },
      {
        "id": 3,
        "depart_Id": 1,
        "name": "Rahul",
        "email": "rahul@college.com",
        "salary": 35000,
        "createdAt": "2026-10-30T09:32:51.000Z",
        "updatedAt": "2026-10-30T09:32:51.000Z"
      },
```

Activate Windows
Go to Settings to activate Windows.



```
    {
      "id": 6,
      "depart_Id": 1,
      "name": "Sneha",
      "email": "sneha@college.com",
      "salary": 38000,
      "createdAt": "2026-10-30T09:32:51.000Z",
      "updatedAt": "2026-10-30T09:32:51.000Z"
    }
  ]
},
{
  "id": 2,
  "name": "HR",
  "location": "Ponda",
  "createdAt": "2026-10-30T09:32:51.000Z",
  "updatedAt": "2026-10-30T09:32:51.000Z",
  "employees": [
    {
      "id": 4,
      "depart_Id": 2,
      "name": "Priya",
      "email": "priya@college.com",
      "salary": 40000,
      "createdAt": "2026-10-30T09:32:51.000Z",
      "updatedAt": "2026-10-30T09:32:51.000Z"
    },
    {
      "id": 7,
      "depart_Id": 2,
      "name": "Karan",
      "email": "karan@college.com",
      "salary": 32000,
      "createdAt": "2026-10-30T09:32:51.000Z",
      "updatedAt": "2026-10-30T09:32:51.000Z"
    }
  ]
},
```




```
{
  "id": 3,
  "name": "Finance",
  "location": "Panaji",
  "createdAt": "2026-10-30T09:32:51.000Z",
  "updatedAt": "2026-10-30T09:32:51.000Z",
  "employees": [
    {
      "id": 5,
      "depart_Id": 3,
      "name": "Amit",
      "email": "amit@college.com",
      "salary": 30000,
      "createdAt": "2026-10-30T09:32:51.000Z",
      "updatedAt": "2026-10-30T09:32:51.000Z"
    },
    {
      "id": 8,
      "depart_Id": 3,
      "name": "Neha",
      "email": "neha@college.com",
      "salary": 45000,
      "createdAt": "2026-10-30T09:32:51.000Z",
      "updatedAt": "2026-10-30T09:32:51.000Z"
    }
  ]
}
]
```


3. Display only employee names

```
sequelize-lab > JS exercise9.js > ...
1   const models = require('./models');
2
3   async function getData() {
4
5       const departments = await models.department.findAll({
6       include: [{
7           model: models.employee,
8           attributes: ['name']
9       }]
10  });
11
12  console.log(JSON.stringify(departments, null, 2));
13  }
14
15  getData();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node exercise9.js
Executing (default): SELECT `department`.`id`, `department`.`name`, `department`.`location`, `department`.`createdAt`, `department`.`updatedAt`, `employees`.`id` AS `employees.id`, `employees`.`name` AS `employees.name` FROM `departments` AS `department` LEFT OUTER JOIN `employees` AS `employees` ON `department`.`id` = `employees`.`depart_Id`;
[
  {
    "id": 1,
    "name": "IT",
    "location": "Usago",
    "createdAt": "2026-09-08T04:58:04.000Z",
    "updatedAt": "2026-09-08T04:58:04.000Z",
    "employees": [
      {
        "name": "Max"
      },
      {
        "name": "John"
      },
      {
        "name": "Rahul"
      },
      {
        "name": "Sneha"
      }
    ]
  },
  {
    "id": 2,
    "name": "HR",
    "location": "Ponda",
    "createdAt": "2026-10-30T09:32:51.000Z",
    "updatedAt": "2026-10-30T09:32:51.000Z",
    "employees": [
      {
        "name": "Priya"
      },
      {
        "name": "Karan"
      }
    ]
  },
```

Activate Windows

```
  {
    "id": 3,
    "name": "Finance",
    "location": "Panaji",
    "createdAt": "2026-10-30T09:32:51.000Z",
    "updatedAt": "2026-10-30T09:32:51.000Z",
    "employees": [
      {
        "name": "Amit"
      },
      {
        "name": "Neha"
      }
    ]
  }
]
```

4. Sort employees by salary

sequelize-lab > JS exercise9.js > ...

```
1   const models = require('./models');
2
3   async function getData() {
4
5       const employees = await models.employee.findAll({
6       order: [['salary', 'ASC']]
7   });
8
9   console.log(JSON.stringify(employees, null, 2));
10  }
11
12  getData();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node exercise9.js
Executing (default): SELECT `id`, `depart_Id`, `name`, `email`, `salary`, `createdAt`, `updatedAt` FROM `employees` AS `employee` ORDER BY `employee`
.`salary` ASC;
[
  {
    "id": 5,
    "depart_Id": 3,
    "name": "Amit",
    "email": "amit@college.com",
    "salary": 30000,
    "createdAt": "2026-10-30T09:32:51.000Z",
    "updatedAt": "2026-10-30T09:32:51.000Z"
  },
  {
    "id": 7,
    "depart_Id": 2,
    "name": "Karan",
    "email": "karan@college.com",
    "salary": 32000,
    "createdAt": "2026-10-30T09:32:51.000Z",
    "updatedAt": "2026-10-30T09:32:51.000Z"
  },
  {
    "id": 3,
    "depart_Id": 1,
    "name": "Rahul",
    "email": "rahul@college.com",
    "salary": 35000,
    "createdAt": "2026-10-30T09:32:51.000Z",
    "updatedAt": "2026-10-30T09:32:51.000Z"
  },
  {
    "id": 6,
    "depart_Id": 1,
    "name": "Sneha",
    "email": "sneha@college.com",
    "salary": 38000,
    "createdAt": "2026-10-30T09:32:51.000Z",
    "updatedAt": "2026-10-30T09:32:51.000Z"
  },
```

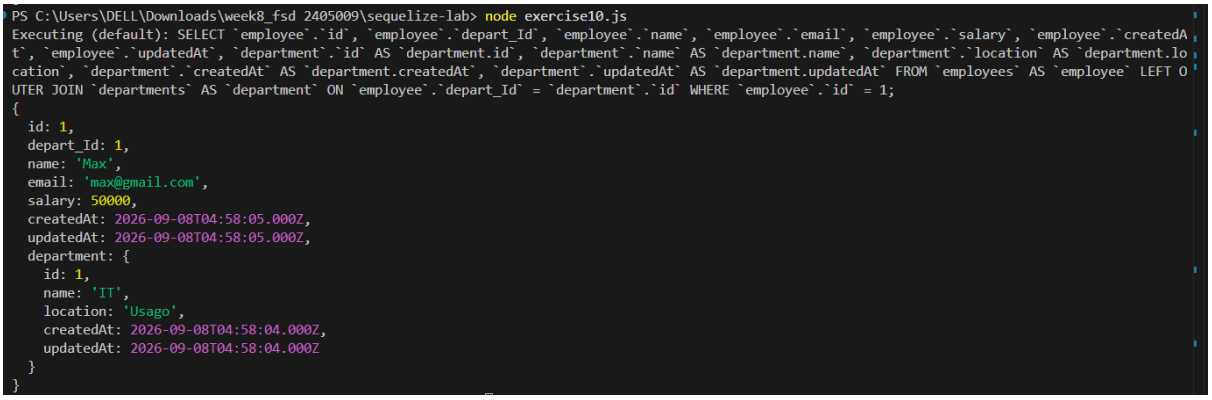
```
  {
    "id": 4,
    "depart_Id": 2,
    "name": "Priya",
    "email": "priya@college.com",
    "salary": 40000,
    "createdAt": "2026-10-30T09:32:51.000Z",
    "updatedAt": "2026-10-30T09:32:51.000Z"
  },
  {
    "id": 8,
    "depart_Id": 3,
    "name": "Neha",
    "email": "neha@college.com",
    "salary": 45000,
    "createdAt": "2026-10-30T09:32:51.000Z",
    "updatedAt": "2026-10-30T09:32:51.000Z"
  },
  {
    "id": 1,
    "depart_Id": 1,
    "name": "Max",
    "email": "max@gmail.com",
    "salary": 50000,
    "createdAt": "2026-09-08T04:58:05.000Z",
    "updatedAt": "2026-09-08T04:58:05.000Z"
  },
  {
    "id": 2,
    "depart_Id": 1,
    "name": "John",
    "email": "john@gmail.com",
    "salary": 100000,
    "createdAt": "2026-09-08T05:14:40.000Z",
    "updatedAt": "2026-09-08T05:14:40.000Z"
  }
]
```


Exercise 10 – Retrieve Employee with Department

sequelize-lab > JS exercise10.js > ...

```
1   const models = require('./models');
2
3   async function getData() {
4
5       const employee = await models.employee.findOne({
6           where: { id: 1 },
7           include: models.department
8       });
9
10      console.log(employee.toJSON());
11  }
12
13  getData();
```

Output



Exercise 11 – 1:M Association Methods
Demonstrate the following:

getEmployees()
createEmployee()
countEmployees()
hasEmployee()
addEmployee()
removeEmployee()
Record the result of each method.

```
const models = require('./models');

async function associationMethods() {

    const department = await
models.department.findByPk(1);

    // 1. getEmployees()
    console.log('\n1. GET EMPLOYEES');

    const employees = await
department.getEmployees();
    console.log(employees);



    // 2. countEmployees()
    console.log('\n2. COUNT EMPLOYEES');
```

```
    const count = await department.countEmployees();
    console.log('Total Employees:', count);


    // 3. createEmployee()
    console.log('\n3. CREATE EMPLOYEE');

    const employee = await
department.createEmployee({
        name: 'Temporary Employee',
        email: 'temporary@college.com',
        salary: 25000
    });

    console.log('Employee created:',
employee.toJSON());


    // 4. hasEmployee()
    console.log('\n4. HAS EMPLOYEE');

    const exists = await
department.hasEmployee(employee);
    console.log('Employee belongs to department:',
exists);


    // 5. removeEmployee()
    console.log('\n5. REMOVE EMPLOYEE');

    await department.removeEmployee(employee);
```

```
    const removed = await
department.hasEmployee(employee);
    console.log('Employee belongs to department:',
removed);


    // 6. addEmployee()
    console.log('\n6. ADD EMPLOYEE');

    await department.addEmployee(employee);

    const added = await
department.hasEmployee(employee);
    console.log('Employee belongs to department:',
added);


    // Clean up the temporary record
    await employee.destroy();

    console.log('\nTemporary employee deleted.');

}

associationMethods();
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node exercise11.js
Executing (default): SELECT `id`, `name`, `location`, `createdAt`, `updatedAt` FROM `departments` AS `department` WHERE `department`.`id` = 1;

1. GET EMPLOYEES
Executing (default): SELECT `id`, `depart_Id`, `name`, `email`, `salary`, `createdAt`, `updatedAt` FROM `employees` AS `employee` WHERE `employee`.`d
epart_Id` = 1;
[
  employee {
    dataValues: {
      id: 1,
      depart_Id: 1,
      name: 'Max',
      email: 'max@gmail.com',
      salary: 50000,
      createdAt: 2026-09-08T04:58:05.000Z,
      updatedAt: 2026-09-08T04:58:05.000Z
    },
    _previousDataValues: {
      id: 1,
      depart_Id: 1,
      name: 'Max',
      email: 'max@gmail.com',
      salary: 50000,
      createdAt: 2026-09-08T04:58:05.000Z,
      updatedAt: 2026-09-08T04:58:05.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
```

```
  employee {
    dataValues: {
      id: 2,
      depart_Id: 1,
      name: 'John',
      email: 'john@gmail.com',
      salary: 100000,
      createdAt: 2026-09-08T05:14:40.000Z,
      updatedAt: 2026-09-08T05:14:40.000Z
    },
    _previousDataValues: {
      id: 2,
      depart_Id: 1,
      name: 'John',
      email: 'john@gmail.com',
      salary: 100000,
      createdAt: 2026-09-08T05:14:40.000Z,
      updatedAt: 2026-09-08T05:14:40.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  employee {
    dataValues: {
      id: 3,
      depart_Id: 1,
      name: 'Rahul',
      email: 'rahul@college.com',
      salary: 35000,
      createdAt: 2026-10-30T09:32:51.000Z,
      updatedAt: 2026-10-30T09:32:51.000Z
    },
```

```
    _previousDataValues: {
      id: 3,
      depart_Id: 1,
      name: 'Rahul',
      email: 'rahul@college.com',
      salary: 35000,
      createdAt: 2026-10-30T09:32:51.000Z,
      updatedAt: 2026-10-30T09:32:51.000Z
    },
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  },
  employee {
    dataValues: {
      id: 6,
      depart_Id: 1,
      name: 'Sneha',
      email: 'sneha@college.com',
      salary: 38000,
      createdAt: 2026-10-30T09:32:51.000Z,
      updatedAt: 2026-10-30T09:32:51.000Z
    },
    _previousDataValues: {
      id: 6,
      depart_Id: 1,
      name: 'Sneha',
      email: 'sneha@college.com',
      salary: 38000,
      createdAt: 2026-10-30T09:32:51.000Z,
      updatedAt: 2026-10-30T09:32:51.000Z
    },
```

```
    uniqno: 1,
    _changed: Set(0) {},
    _options: {
      isNewRecord: false,
      _schema: null,
      _schemaDelimiter: '',
      raw: true,
      attributes: [Array]
    },
    isNewRecord: false
  }
]

2. COUNT EMPLOYEES
Executing (default): SELECT COUNT(`employee`.`id`) AS `count` FROM `employees` AS `employee` WHERE `employee`.`depart_Id` = 1;
Total Employees: 4

3. CREATE EMPLOYEE
Executing (default): INSERT INTO `employees` (`id`,`depart_Id`,`name`,`email`,`salary`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?);
Employee created: {
  id: 9,
  name: 'Temporary Employee',
  email: 'temporary@college.com',
  salary: 25000,
  depart_Id: 1,
  updatedAt: 2026-10-30T10:04:16.223Z,
  createdAt: 2026-10-30T10:04:16.223Z
}

4. HAS EMPLOYEE
Executing (default): SELECT `id` FROM `employees` AS `employee` WHERE (`employee`.`depart_Id` = 1 AND ((`employee`.`id` = 9)));
Employee belongs to department: true

5. REMOVE EMPLOYEE
Executing (default): UPDATE `employees` SET `depart_Id`=?,`updatedAt`=? WHERE `depart_Id` = ? AND `id` IN (9)
Executing (default): SELECT `id` FROM `employees` AS `employee` WHERE (`employee`.`depart_Id` = 1 AND ((`employee`.`id` = 9)));
Employee belongs to department: false
```

```
6. ADD EMPLOYEE
Executing (default): UPDATE `employees` SET `depart_Id`=?,`updatedAt`=? WHERE `id` IN (9)
Executing (default): SELECT `id` FROM `employees` AS `employee` WHERE (`employee`.`depart_Id` = 1 AND ((`employee`.`id` = 9)));
Employee belongs to department: true
Executing (default): DELETE FROM `employees` WHERE `id` = 9

Temporary employee deleted.
```

Exercise 12 – Employee Department Transfer

```
const models = require('./models');

async function transferEmployee() {

    // 1. Retrieve Rahul before transfer
    console.log('\n1. BEFORE TRANSFER');

    const employee = await models.employee.findOne({
        where: { name: 'Rahul' },
        include: models.department
    });

    console.log(employee.toJSON());


    // 2. Find HR department
```

```
    const hrDepartment = await
models.department.findOne({
        where: { name: 'HR' }
    });


    // 3. Change Rahul's department
    await employee.update({
        depart_Id: hrDepartment.id
    });

    console.log('\nRahul transferred to HR
successfully');


    // 4. Retrieve Rahul after transfer
    console.log('\n2. AFTER TRANSFER');

    const updatedEmployee = await
models.employee.findOne({
        where: { name: 'Rahul' },
        include: models.department
    });

    console.log(updatedEmployee.toJSON());

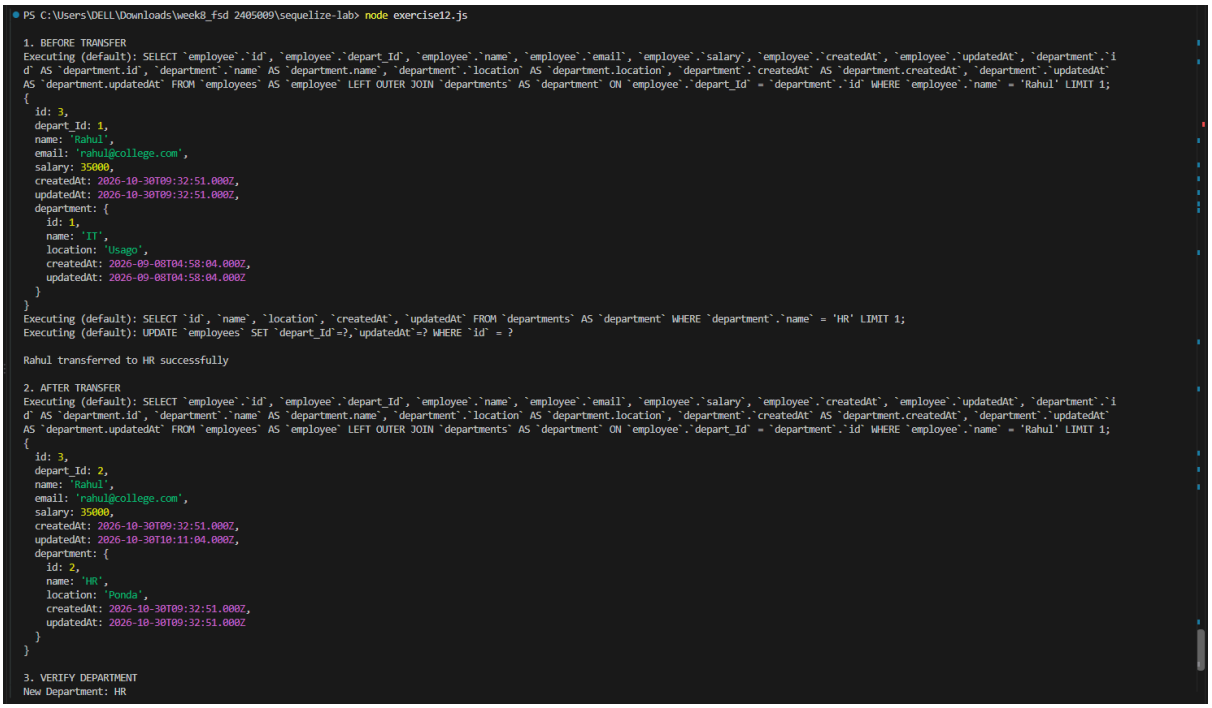

    // 5. Verify new department
    console.log('\n3. VERIFY DEPARTMENT');

    console.log('New Department:',
updatedEmployee.department.name);

}
```

```
transferEmployee();
```

Output



Exercise 13 – Many-to-Many Association

Objective: Create a Student–Course M:N relationship.

Student M : N Course
Student ——— Enrollment ——— Course

A student can take many courses, and a course can contain many students.

student.js



Exercise 14 – Create Course and Enrollment Models

course.js

sequelize-lab > models > JS course.js > ...

```
1  'use strict';
2  const { Model } = require('sequelize');
3
4  module.exports = (sequelize, DataTypes) => {
5    class Course extends Model {
6      static associate(models) {
7        Course.belongsToMany(models.Student, {
8          through: models.Enrollment,
9          foreignKey: 'courseId',
10         otherKey: 'studentId'
11       });
12     }
13   }
14
15   Course.init({
16     name: DataTypes.STRING,
17     credits: DataTypes.INTEGER
18   }, {
19     sequelize,
20     modelName: 'Course',
21   });
22
23   return Course;
24 };
```

```
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab>npx sequelize-cli model:generate
 --name Course --attributes name:string,credits:integer
```

[enrollment.js]

sequelize-lab > models > JS enrollment.js > ...

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class Enrollment extends Model {
    /**
     * Helper method for defining associations.
     * This method is not a part of Sequelize lifecycle.
     * The `models/index` file will call this method automatically.
     */
    static associate(models) {
      // define association here
    }
  }
  Enrollment.init({
    studentId: DataTypes.INTEGER,
    courseId: DataTypes.INTEGER,
    enrollmentDate: DataTypes.DATE,
    grade: DataTypes.STRING
  }, {
    sequelize,
    modelName: 'Enrollment',
  });
  return Enrollment;
};
```

```
C:\Users\TY BCA.DESKTOP-I01NH63\Desktop\week 8_09\sequelize-lab>npx sequelize-cli model:generate
 --name Enrollment --attributes studentId:integer,courseId:integer,enrollmentDate:date,grade:str
ing
```

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> npx sequelize-cli db:migrate

Sequelize CLI [Node: 24.14.0, CLI: 6.6.5, ORM: 6.37.8]

Loaded configuration file "config\config.json".
Using environment "development".
== 20261030102428-create-enrollment: migrating =======
== 20261030102428-create-enrollment: migrated (0.023s)

== 20261030104416-update-course-table: migrating =======
== 20261030104416-update-course-table: migrated (0.410s)
```

Exercise 15 – Define Basic M:N Association

Student model:

```
Student.belongsToMany(models.Course, {
    through: models.Enrollment,
    foreignKey: 'studentId',
    otherKey: 'courseId'
});
```

Course model:

```
static associate(models) {
  Course.belongsToMany(models.Student, {
    through: models.Enrollment,
    foreignKey: 'courseId',
    otherKey: 'studentId'
  });
}
```

through: Specifies the intermediate Enrollment table that connects Students and Courses.
foreignKey: Specifies the column in Enrollment that refers to the current model.
otherKey: Specifies the column in Enrollment that refers to the associated model.

Exercise 16 – Enroll Students in Courses

sequelize-lab > JS createcourses.js > ...

```
1  const models = require('./models');
2
3  async function addData(name, credits) {
4    const course = await models.Course.create({
5      name: name,
6      credits: credits
7    });
8
9    console.log(course.toJSON());
10 }
11
12 async function main() {
13   await addData('Database Management', 4);
14   await addData('Web Development', 4);
15   await addData('Data Structures', 4);
16   await addData('Computer Networks', 2);
17 }
18
19 main()
20   .catch(console.error)
21   .finally(() => models.sequelize.close());
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node createcourses.js
  id: 1,
  name: 'Database Management',
  credits: 4,
  updatedAt: 2026-10-30T10:59:09.590Z,
  createdAt: 2026-10-30T10:59:09.590Z
}
Executing (default): INSERT INTO `Courses` (`id`,`name`,`credits`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?);
{
  id: 2,
  name: 'Web Development',
  credits: 4,
  updatedAt: 2026-10-30T10:59:09.615Z,
  createdAt: 2026-10-30T10:59:09.615Z
}
Executing (default): INSERT INTO `Courses` (`id`,`name`,`credits`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?);
{
  id: 3,
  name: 'Data Structures',
  credits: 4,
  updatedAt: 2026-10-30T10:59:09.618Z,
  createdAt: 2026-10-30T10:59:09.618Z
}
Executing (default): INSERT INTO `Courses` (`id`,`name`,`credits`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?);
{
  id: 4,
  name: 'Computer Networks',
  credits: 2,
  updatedAt: 2026-10-30T10:59:09.621Z,
  createdAt: 2026-10-30T10:59:09.621Z
}
```

	id	name	credits	createdAt	updatedAt
Edit Copy Delete	1	Database Management	4	2026-10-30 10:59:09	2026-10-30 10:59:09
Edit Copy Delete	2	Web Development	4	2026-10-30 10:59:09	2026-10-30 10:59:09
Edit Copy Delete	3	Data Structures	4	2026-10-30 10:59:09	2026-10-30 10:59:09
Edit Copy Delete	4	Computer Networks	2	2026-10-30 10:59:09	2026-10-30 10:59:09

Create at least five students and enroll each student in multiple courses.

sequelize-lab > JS enrollstudents.js > ...

```
const models = require('./models');

async function main() {

  await models.Enrollment.bulkCreate([
    // Riya
    { studentId: 1, courseId: 1, enrollmentDate: new Date(), grade: 'A' },
    { studentId: 1, courseId: 2, enrollmentDate: new Date(), grade: 'A' },

    // Hasmani
    { studentId: 2, courseId: 1, enrollmentDate: new Date(), grade: 'B' },
    { studentId: 2, courseId: 3, enrollmentDate: new Date(), grade: 'A' },

    // Mansi
    { studentId: 3, courseId: 2, enrollmentDate: new Date(), grade: 'A' },
    { studentId: 3, courseId: 4, enrollmentDate: new Date(), grade: 'B' },

    // Pramila
    { studentId: 4, courseId: 1, enrollmentDate: new Date(), grade: 'B' },
    { studentId: 4, courseId: 3, enrollmentDate: new Date(), grade: 'A' },

    // Amit
    { studentId: 5, courseId: 2, enrollmentDate: new Date(), grade: 'A' },
    { studentId: 5, courseId: 4, enrollmentDate: new Date(), grade: 'B' }
  ]);

  console.log('Students enrolled in courses successfully!');
}

main()
  .catch(console.error)
  .finally(() => models.sequelize.close());
```

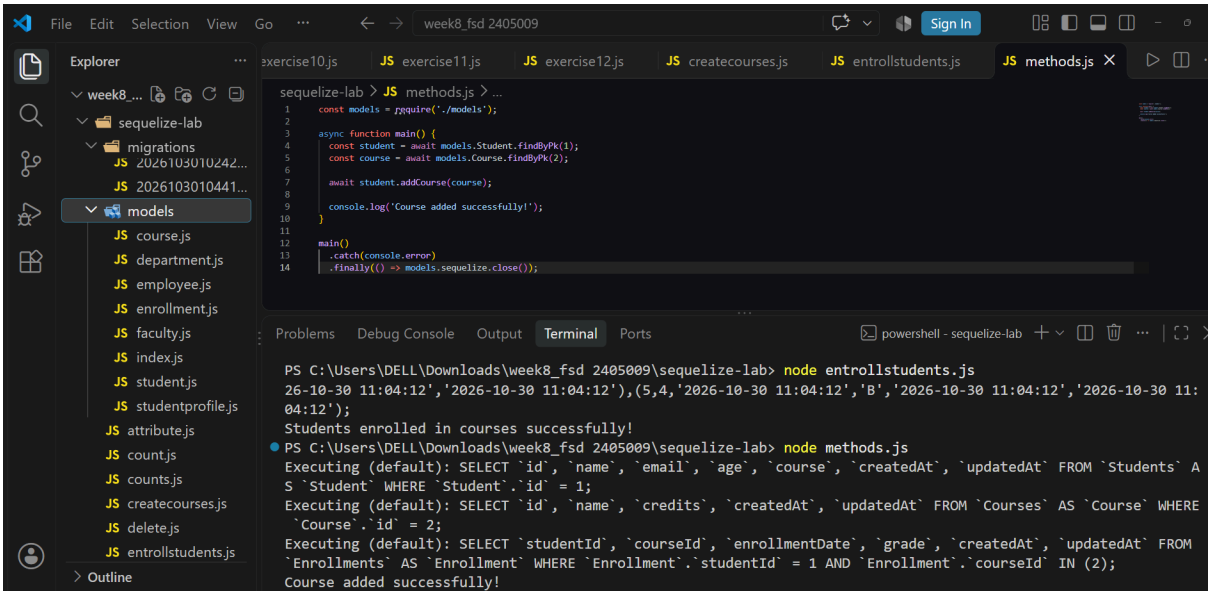
Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node enrollstudents.js
Executing (default): INSERT INTO `Enrollments` (`studentId`,`courseId`,`enrollmentDate`,`grade`,`createdAt`,`updatedAt`) VALUES (1,1,'2026-10
-30 11:04:12','A','2026-10-30 11:04:12','2026-10-30 11:04:12'),(1,2,'2026-10-30 11:04:12','A','2026-10-30 11:04:12','2026-10-30 11:04:12'),(2
,1,'2026-10-30 11:04:12','B','2026-10-30 11:04:12','2026-10-30 11:04:12'),(2,3,'2026-10-30 11:04:12','A','2026-10-30 11:04:12','2026-10-30 11
:04:12'),(3,2,'2026-10-30 11:04:12','A','2026-10-30 11:04:12','2026-10-30 11:04:12'),(3,4,'2026-10-30 11:04:12','B','2026-10-30 11:04:12','20
26-10-30 11:04:12'),(4,1,'2026-10-30 11:04:12','B','2026-10-30 11:04:12','2026-10-30 11:04:12'),(4,3,'2026-10-30 11:04:12','A','2026-10-30 11
:04:12','2026-10-30 11:04:12'),(5,2,'2026-10-30 11:04:12','A','2026-10-30 11:04:12','2026-10-30 11:04:12'),(5,4,'2026-10-30 11:04:12','B','20
26-10-30 11:04:12','2026-10-30 11:04:12');
Students enrolled in courses successfully!
```

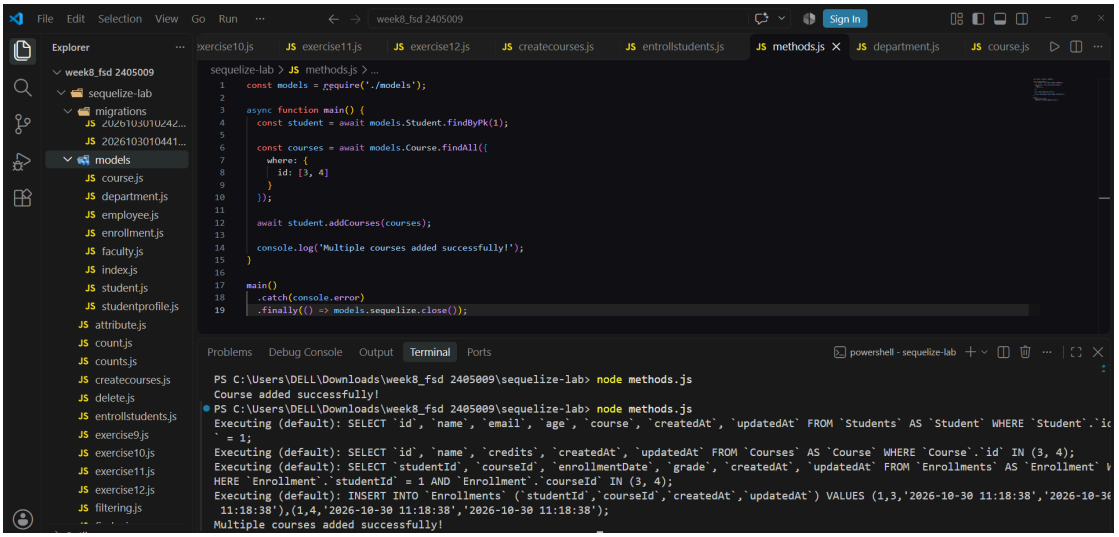
	id	studentId	courseId	enrollmentDate	grade	createdAt	updatedAt
Edit Copy Delete	1	1	1	2026-10-30 11:04:12	A	2026-10-30 11:04:12	2026-10-30 11:04:12
Edit Copy Delete	2	1	2	2026-10-30 11:04:12	A	2026-10-30 11:04:12	2026-10-30 11:04:12
Edit Copy Delete	3	2	1	2026-10-30 11:04:12	B	2026-10-30 11:04:12	2026-10-30 11:04:12
Edit Copy Delete	4	2	3	2026-10-30 11:04:12	A	2026-10-30 11:04:12	2026-10-30 11:04:12
Edit Copy Delete	5	3	2	2026-10-30 11:04:12	A	2026-10-30 11:04:12	2026-10-30 11:04:12
Edit Copy Delete	6	3	4	2026-10-30 11:04:12	B	2026-10-30 11:04:12	2026-10-30 11:04:12
Edit Copy Delete	7	4	1	2026-10-30 11:04:12	B	2026-10-30 11:04:12	2026-10-30 11:04:12
Edit Copy Delete	8	4	3	2026-10-30 11:04:12	A	2026-10-30 11:04:12	2026-10-30 11:04:12
Edit Copy Delete	9	5	2	2026-10-30 11:04:12	A	2026-10-30 11:04:12	2026-10-30 11:04:12
Edit Copy Delete	10	5	4	2026-10-30 11:04:12	B	2026-10-30 11:04:12	2026-10-30 11:04:12

Exercise 17 – M:N Association Methods

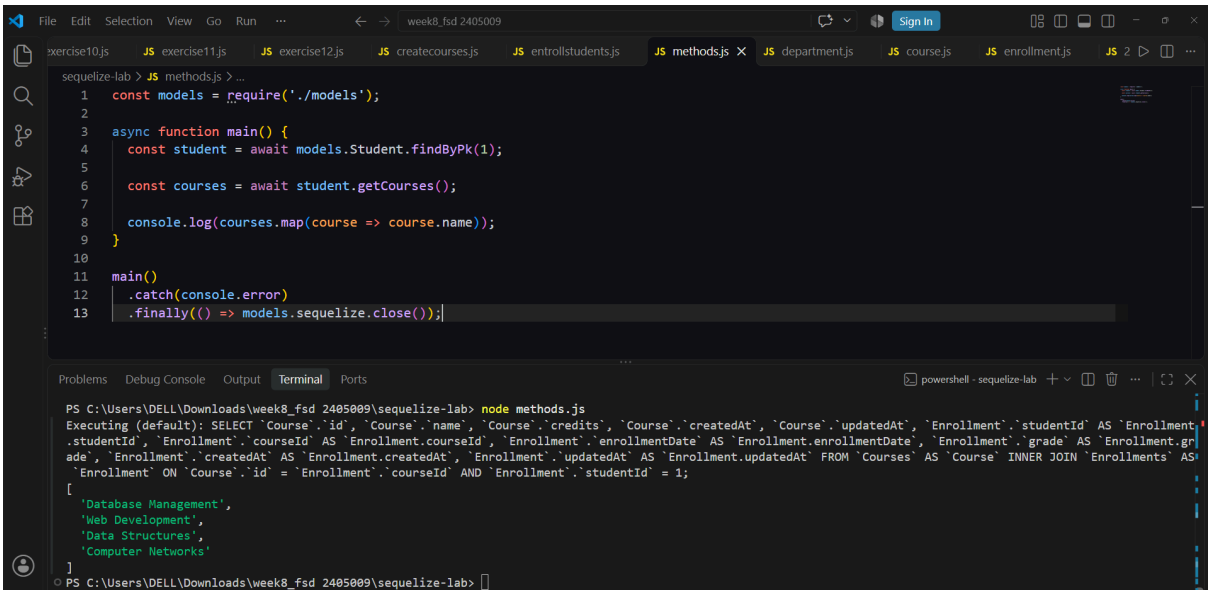
student.addCourse() - Adds one course to a student.



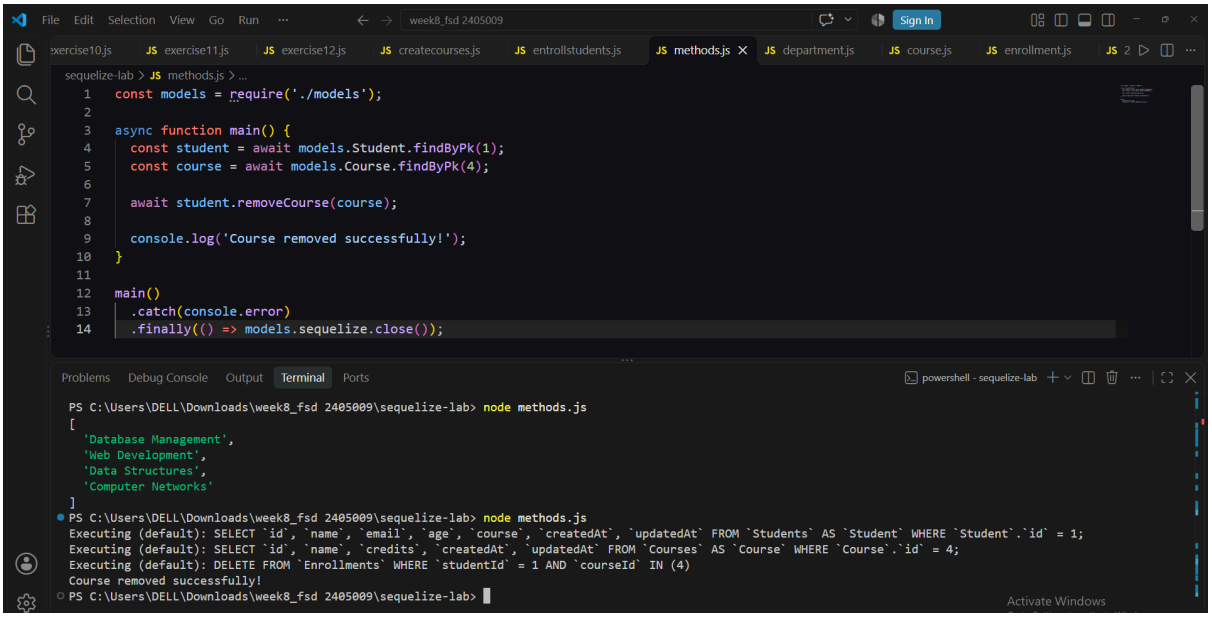
student.addCourses() - Adds multiple courses to one student at once.



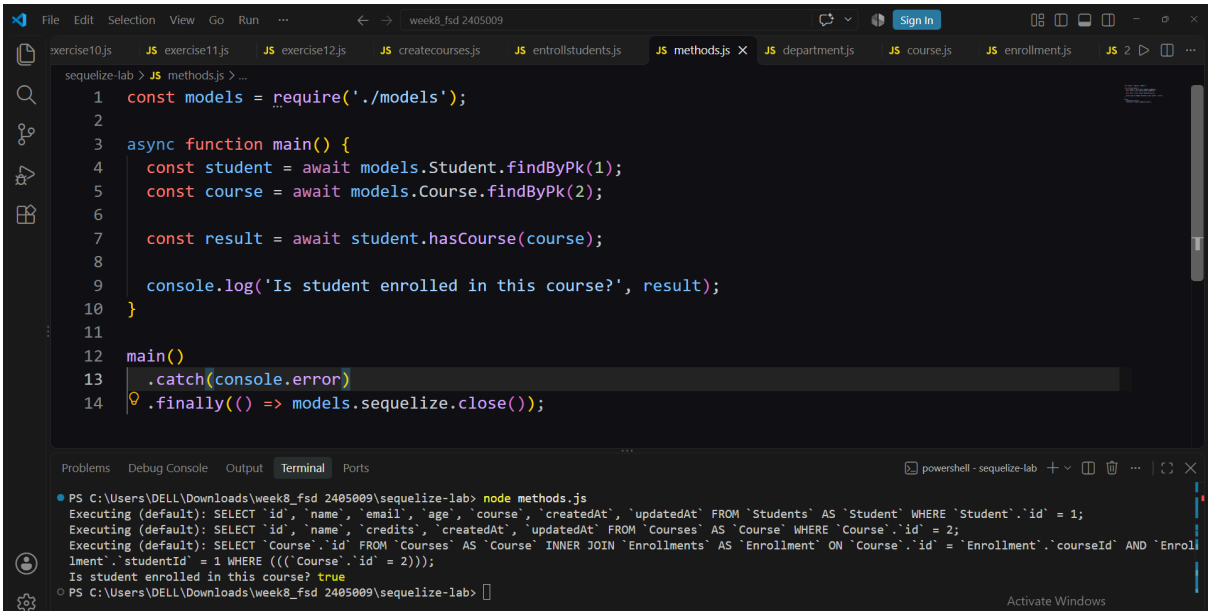
student.getCourses() - Retrieves all courses of a student.



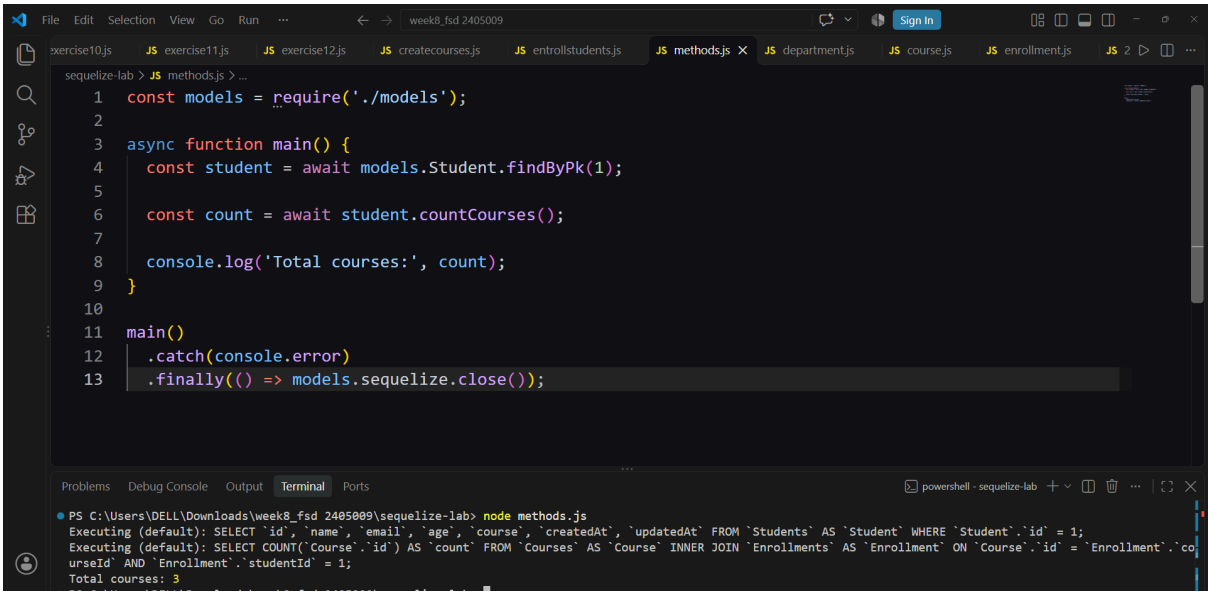
student.removeCourse() - Removes a course from a student.



student.hasCourse() - Checks whether a student is enrolled in a particular course.



student.countCourses() - Counts the total courses taken by a student.



Exercise 18 – Retrieve Student with Courses

1. Display one student's courses

```
sequelize-lab > JS methods.js > ...
1  const models = require('./models');
2
3  async function main() {
4    const student = await models.Student.findByPk(1, {
5      include: models.Course
6    });
7
8    console.log(student.toJSON());
9  }
10
11 main()
12   .catch(console.error)
13   .finally(() => models.sequelize.close());
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node methods.js
`Courses.name`, `Courses`.`credits` AS `Courses.credits`, `Courses`.`createdAt` AS `Courses.createdAt`, `Courses`.`updatedAt` AS `Courses.updatedAt`, `Courses->Enrollment`.`studentId` AS `Courses.Enrollm
`, `Courses->Enrollment`.`courseId` AS `Courses.Enrollment.courseId`, `Courses->Enrollment`.`enrollmentDate` AS `Courses.Enrollment.enrollmentDate`, `Courses->Enrollment`.`grade` AS `Courses.Enrollment.g
es->Enrollment`.`createdAt` AS `Courses.Enrollment.createdAt`, `Courses->Enrollment`.`updatedAt` AS `Courses.Enrollment.updatedAt` FROM `Students` AS `Student` LEFT OUTER JOIN ( `Enrollments` AS `Courses
 INNER JOIN `Courses` AS `Courses` ON `Courses`.`id` = `Courses->Enrollment`.`courseId`) ON `Student`.`id` = `Courses->Enrollment`.`studentId` WHERE `Student`.`id` = 1;
{
  id: 1,
  name: 'Riya',
  email: 'riya@gmail.com',
  age: 19,
  course: 'MCA',
  createdAt: 2026-10-27T06:05:49.000Z,
  updatedAt: 2026-10-29T19:02:41.000Z,
  Courses: [
    {
      id: 1,
      name: 'Database Management',
      credits: 4,
      createdAt: 2026-10-30T10:59:09.000Z,
      updatedAt: 2026-10-30T10:59:09.000Z,
      Enrollment: [Object]
    },
    {
      id: 2,
      name: 'Web Development',
      credits: 4,
      createdAt: 2026-10-30T10:59:09.000Z,
      updatedAt: 2026-10-30T10:59:09.000Z,
      Enrollment: [Object]
    },
    {
      id: 3,
      name: 'Data Structures',
      credits: 4,
      createdAt: 2026-10-30T10:59:09.000Z,
      updatedAt: 2026-10-30T10:59:09.000Z,
      Enrollment: [Object]
    }
  ]
}
```

2. Display all students with their courses

sequelize-lab > JS methods.js > ...

```
1  const models = require('./models');
2
3  async function main() {
4    const students = await models.Student.findAll({
5      include: models.Course
6    });
7
8    console.log(JSON.stringify(students, null, 2));
9  }
10
11 main()
12   .catch(console.error)
13   .finally(() => models.sequelize.close());
```

Output

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node methods.js
 `Courses.Enrollment.studentId`, `Courses->Enrollment`.`courseId` AS `Courses.Enrollment.courseId`, `Courses->Enrollment`.`enrollmentDate` AS `Courses.Enrollment.enrollmentDate`, `Courses->Enrollment`.
`grade` AS `Courses.Enrollment.grade`, `Courses->Enrollment`.`createdAt` AS `Courses.Enrollment.createdAt`, `Courses->Enrollment`.`updatedAt` AS `Courses.Enrollment.updatedAt` FROM `Students` AS `Stude
nt` LEFT OUTER JOIN ( `Enrollments` AS `Courses->Enrollment` INNER JOIN `Courses` AS `Courses` ON `Courses`.`id` = `Courses->Enrollment`.`courseId`) ON `Student`.`id` = `Courses->Enrollment`.`studentId
`;
[
  {
    "id": 1,
    "name": "Riya",
    "email": "riya@gmail.com",
    "age": 19,
    "course": "MCA",
    "createdAt": "2026-10-27T06:05:49.000Z",
    "updatedAt": "2026-10-29T19:02:41.000Z",
    "Courses": [
      {
        "id": 1,
        "name": "Database Management",
        "credits": 4,
        "createdAt": "2026-10-30T10:59:09.000Z",
        "updatedAt": "2026-10-30T10:59:09.000Z",
        "Enrollment": {
          "studentId": 1,
          "courseId": 1,
          "enrollmentDate": "2026-10-30T11:04:12.000Z",
          "grade": "A",
          "createdAt": "2026-10-30T11:04:12.000Z",
          "updatedAt": "2026-10-30T11:04:12.000Z"
        }
      },
```

```
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node methods.js
      {
        "id": 2,
        "name": "Web Development",
        "credits": 4,
        "createdAt": "2026-10-30T10:59:09.000Z",
        "updatedAt": "2026-10-30T10:59:09.000Z",
        "Enrollment": {
          "studentId": 1,
          "courseId": 2,
          "enrollmentDate": "2026-10-30T11:04:12.000Z",
          "grade": "A",
          "createdAt": "2026-10-30T11:04:12.000Z",
          "updatedAt": "2026-10-30T11:04:12.000Z"
        }
      },
      {
        "id": 3,
        "name": "Data Structures",
        "credits": 4,
        "createdAt": "2026-10-30T10:59:09.000Z",
        "updatedAt": "2026-10-30T10:59:09.000Z",
        "Enrollment": {
          "studentId": 1,
          "courseId": 3,
          "enrollmentDate": null,
          "grade": null,
          "createdAt": "2026-10-30T11:18:38.000Z",
          "updatedAt": "2026-10-30T11:18:38.000Z"
        }
      }
    ]
  },
```

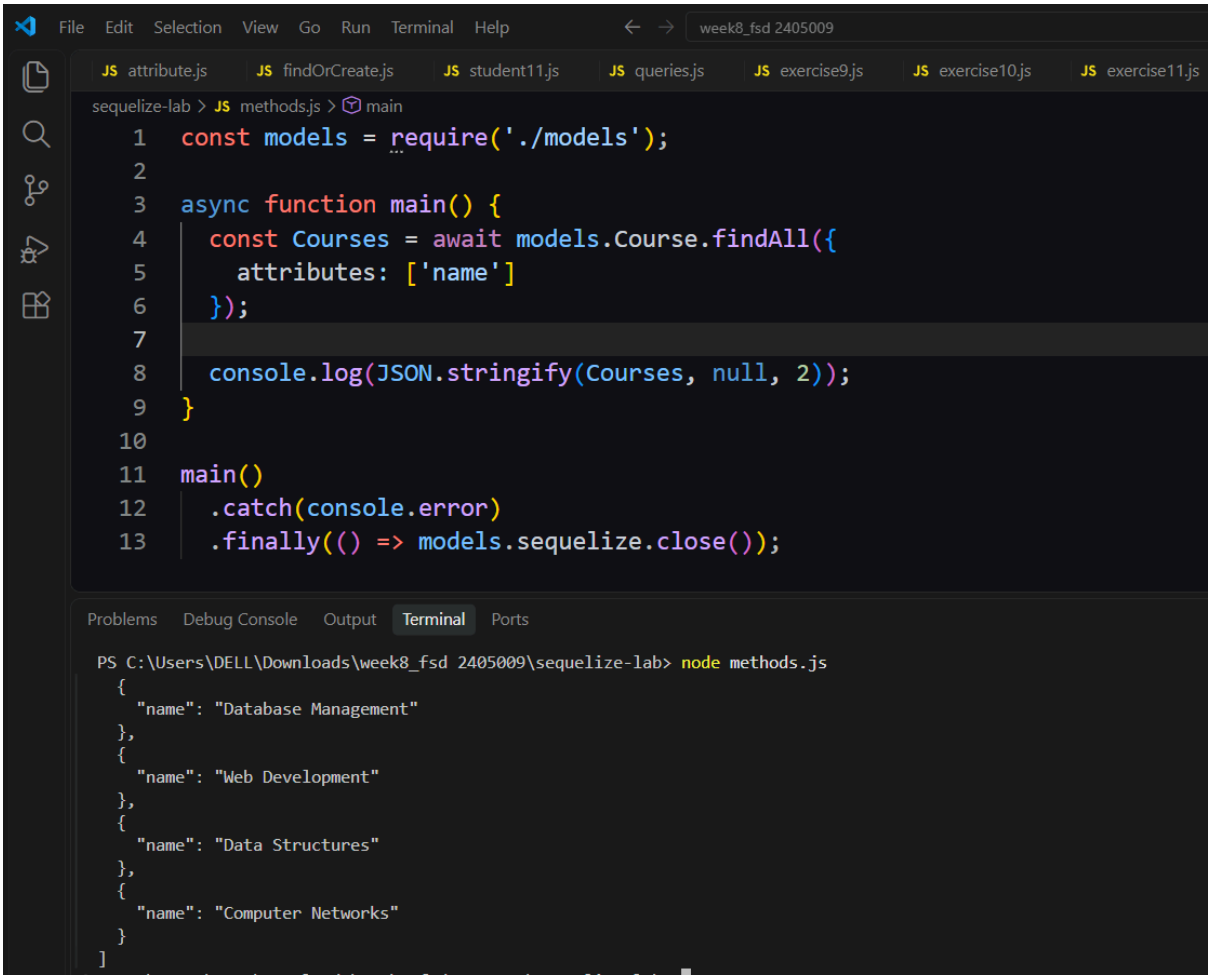
```
{
  "id": 2,
  "name": "Hasmani Bilung",
  "email": "asmanibilung08@gmail.com",
  "age": 20,
  "course": "BCA",
  "createdAt": "2026-10-29T19:02:14.000Z",
  "updatedAt": "2026-10-29T19:02:14.000Z",
  "Courses": [
    {
      "id": 1,
      "name": "Database Management",
      "credits": 4,
      "createdAt": "2026-10-30T10:59:09.000Z",
      "updatedAt": "2026-10-30T10:59:09.000Z",
      "Enrollment": {
        "studentId": 2,
        "courseId": 1,
        "enrollmentDate": "2026-10-30T11:04:12.000Z",
        "grade": "B",
        "createdAt": "2026-10-30T11:04:12.000Z",
        "updatedAt": "2026-10-30T11:04:12.000Z"
      }
    },
    {
      "id": 3,
      "name": "Data Structures",
      "credits": 4,
      "createdAt": "2026-10-30T10:59:09.000Z",
      "updatedAt": "2026-10-30T10:59:09.000Z",
      "Enrollment": {
        "studentId": 2,
        "courseId": 3,
        "enrollmentDate": "2026-10-30T11:04:12.000Z",
        "grade": "A",
        "createdAt": "2026-10-30T11:04:12.000Z",
        "updatedAt": "2026-10-30T11:04:12.000Z"
      }
    }
  ]
},
```

```
{
  "id": 3,
  "name": "Mansi Bhandari",
  "email": "mansi@gmail.com",
  "age": 20,
  "course": "BCA",
  "createdAt": "2026-10-29T20:08:27.000Z",
  "updatedAt": "2026-10-29T20:08:27.000Z",
  "Courses": [
    {
      "id": 2,
      "name": "Web Development",
      "credits": 4,
      "createdAt": "2026-10-30T10:59:09.000Z",
      "updatedAt": "2026-10-30T10:59:09.000Z",
      "Enrollment": {
        "studentId": 3,
        "courseId": 2,
        "enrollmentDate": "2026-10-30T11:04:12.000Z",
        "grade": "A",
        "createdAt": "2026-10-30T11:04:12.000Z",
        "updatedAt": "2026-10-30T11:04:12.000Z"
      }
    },
    {
      "id": 4,
      "name": "Computer Networks",
      "credits": 2,
      "createdAt": "2026-10-30T10:59:09.000Z",
      "updatedAt": "2026-10-30T10:59:09.000Z",
      "Enrollment": {
        "studentId": 3,
        "courseId": 4,
        "enrollmentDate": "2026-10-30T11:04:12.000Z",
        "grade": "B",
        "createdAt": "2026-10-30T11:04:12.000Z",
        "updatedAt": "2026-10-30T11:04:12.000Z"
      }
    }
  ]
},
```

```
{
  "id": 4,
  "name": "Pramila",
  "email": "pramila@gmail.com",
  "age": 23,
  "course": "BBA",
  "createdAt": "2026-10-29T20:08:59.000Z",
  "updatedAt": "2026-10-29T20:08:59.000Z",
  "Courses": [
    {
      "id": 1,
      "name": "Database Management",
      "credits": 4,
      "createdAt": "2026-10-30T10:59:09.000Z",
      "updatedAt": "2026-10-30T10:59:09.000Z",
      "Enrollment": {
        "studentId": 4,
        "courseId": 1,
        "enrollmentDate": "2026-10-30T11:04:12.000Z",
        "grade": "B",
        "createdAt": "2026-10-30T11:04:12.000Z",
        "updatedAt": "2026-10-30T11:04:12.000Z"
      }
    },
    {
      "id": 3,
      "name": "Data Structures",
      "credits": 4,
      "createdAt": "2026-10-30T10:59:09.000Z",
      "updatedAt": "2026-10-30T10:59:09.000Z",
      "Enrollment": {
        "studentId": 4,
        "courseId": 3,
        "enrollmentDate": "2026-10-30T11:04:12.000Z",
        "grade": "A",
        "createdAt": "2026-10-30T11:04:12.000Z",
        "updatedAt": "2026-10-30T11:04:12.000Z"
      }
    }
  ]
},
```

```
{
  "id": 5,
  "name": "Amit",
  "email": "amit@gmail.com",
  "age": 21,
  "course": "BCA",
  "createdAt": "2026-10-30T09:07:24.000Z",
  "updatedAt": "2026-10-30T09:07:24.000Z",
  "Courses": [
    {
      "id": 2,
      "name": "Web Development",
      "credits": 4,
      "createdAt": "2026-10-30T10:59:09.000Z",
      "updatedAt": "2026-10-30T10:59:09.000Z",
      "Enrollment": {
        "studentId": 5,
        "courseId": 2,
        "enrollmentDate": "2026-10-30T11:04:12.000Z",
        "grade": "A",
        "createdAt": "2026-10-30T11:04:12.000Z",
        "updatedAt": "2026-10-30T11:04:12.000Z"
      }
    },
    {
      "id": 4,
      "name": "Computer Networks",
      "credits": 2,
      "createdAt": "2026-10-30T10:59:09.000Z",
      "updatedAt": "2026-10-30T10:59:09.000Z",
      "Enrollment": {
        "studentId": 5,
        "courseId": 4,
        "enrollmentDate": "2026-10-30T11:04:12.000Z",
        "grade": "B",
        "createdAt": "2026-10-30T11:04:12.000Z",
        "updatedAt": "2026-10-30T11:04:12.000Z"
      }
    }
  ]
}
```


3. Display only course names



4. Count courses taken by each student

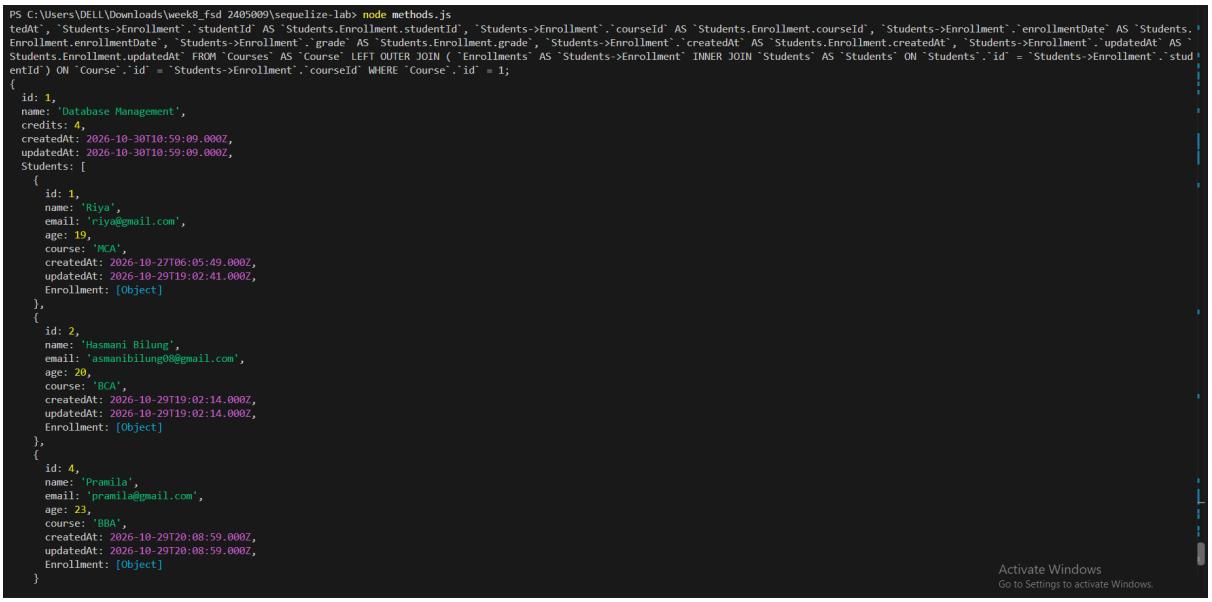


Exercise 19 – Retrieve Course with Students

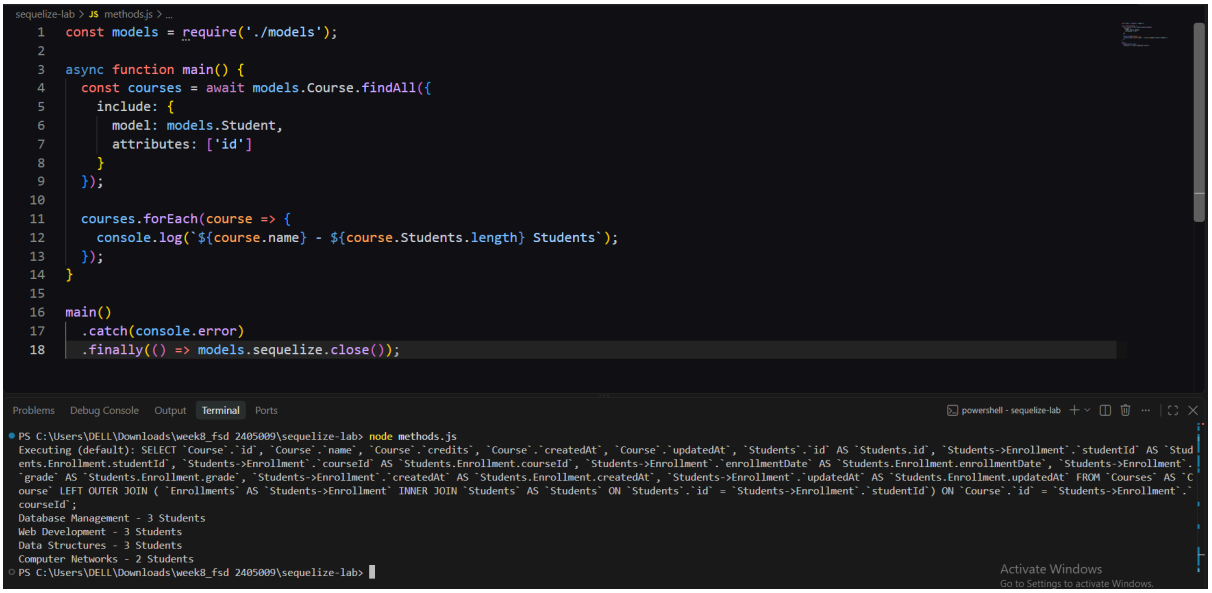
1. Display all students enrolled in a course

```
sequelize-lab > JS methods.js > ...
1   const models = require('./models');
2
3   async function main() {
4     const course = await models.Course.findByPk(1, {
5       include: models.Student
6     });
7
8     console.log(course.toJSON());
9   }
10
11  main()
12    .catch(console.error)
13    .finally(() => models.sequelize.close());
```

Output



2. Display the number of students in each course



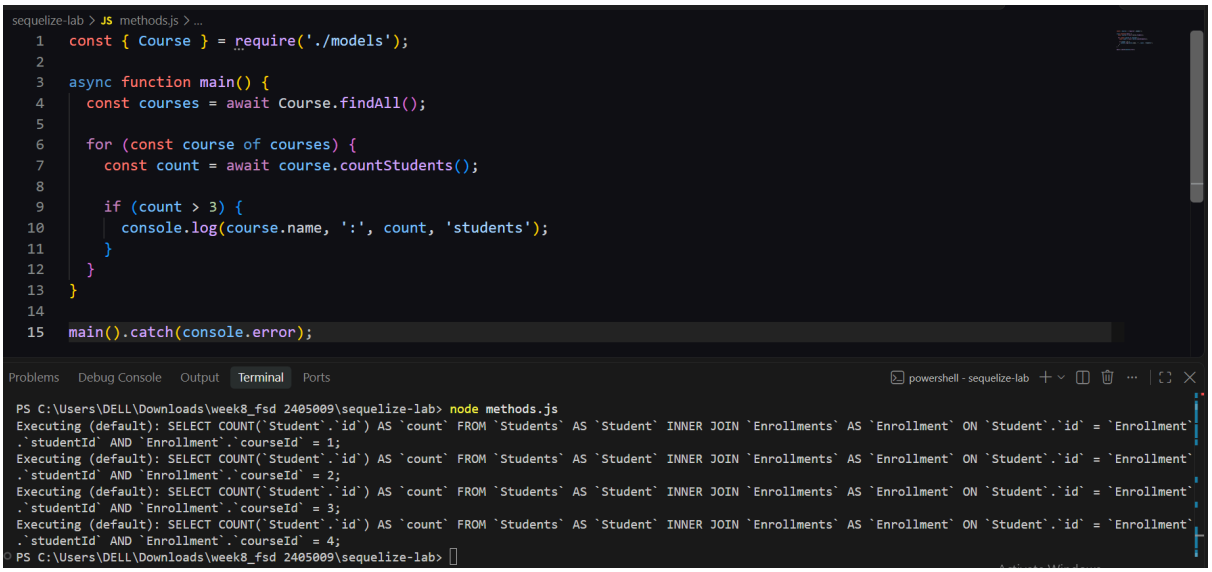
```
sequelize-lab > JS methods.js > ...
1  const models = require('./models');
2
3  async function main() {
4    const courses = await models.Course.findAll({
5      include: {
6        model: models.Student,
7        attributes: ['id']
8      }
9    });
10
11   courses.forEach(course => {
12     console.log(`${course.name} - ${course.Students.length} Students`);
13   });
14 }
15
16 main()
17   .catch(console.error)
18   .finally(() => models.sequelize.close());
```

```
Problems  Debug Console  Output  Terminal  Ports                powershell - sequelize-lab

PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node methods.js
Executing (default): SELECT `Course`.`id`, `Course`.`name`, `Course`.`credits`, `Course`.`createdAt`, `Course`.`updatedAt`, `Students`.`id` AS `Students.id`, `Students->Enrollment`.`studentId` AS `Students.Enrollment.studentId`, `Students->Enrollment`.`courseId` AS `Students.Enrollment.courseId`, `Students->Enrollment`.`enrollmentDate` AS `Students.Enrollment.enrollmentDate`, `Students->Enrollment`.`grade` AS `Students.Enrollment.grade`, `Students->Enrollment`.`createdAt` AS `Students.Enrollment.createdAt`, `Students->Enrollment`.`updatedAt` AS `Students.Enrollment.updatedAt` FROM `Courses` AS `Course` LEFT OUTER JOIN ( `Enrollments` AS `Students->Enrollment` INNER JOIN `Students` AS `Students` ON `Students`.`id` = `Students->Enrollment`.`studentId`) ON `Course`.`id` = `Students->Enrollment`.`courseId`;
Database Management - 3 Students
Web Development - 3 Students
Data Structures - 3 Students
Computer Networks - 2 Students
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab>
```

Activate Windows
Go to Settings to activate Windows.

3. Find courses with more than three students



```
sequelize-lab > JS methods.js > ...
1  const { Course } = require('./models');
2
3  async function main() {
4    const courses = await Course.findAll();
5
6    for (const course of courses) {
7      const count = await course.countStudents();
8
9      if (count > 3) {
10       console.log(course.name, ':', count, 'students');
11     }
12   }
13 }
14
15 main().catch(console.error);
```

```
Problems  Debug Console  Output  Terminal  Ports                powershell - sequelize-lab

PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab> node methods.js
Executing (default): SELECT COUNT(`Student`.`id`) AS `count` FROM `Students` AS `Student` INNER JOIN `Enrollments` AS `Enrollment` ON `Student`.`id` = `Enrollment`.`studentId` AND `Enrollment`.`courseId` = 1;
Executing (default): SELECT COUNT(`Student`.`id`) AS `count` FROM `Students` AS `Student` INNER JOIN `Enrollments` AS `Enrollment` ON `Student`.`id` = `Enrollment`.`studentId` AND `Enrollment`.`courseId` = 2;
Executing (default): SELECT COUNT(`Student`.`id`) AS `count` FROM `Students` AS `Student` INNER JOIN `Enrollments` AS `Enrollment` ON `Student`.`id` = `Enrollment`.`studentId` AND `Enrollment`.`courseId` = 3;
Executing (default): SELECT COUNT(`Student`.`id`) AS `count` FROM `Students` AS `Student` INNER JOIN `Enrollments` AS `Enrollment` ON `Student`.`id` = `Enrollment`.`studentId` AND `Enrollment`.`courseId` = 4;
PS C:\Users\DELL\Downloads\week8_fsd 2405009\sequelize-lab>
```

Week 11 – React.js Fundamentals

Exercise 11.1 – Installation and React Application Setup

Aim:
To install and configure React.js and understand the basic structure of a React application.

Tasks:
1. Install Node.js and npm.
2. Create a React application using Vite.
3. Understand the project structure.
4. Start the development server.
5. Modify the default application.
6. Display:
    - o Student name
    - o Roll number
    - o Programme
    - o College name

```
PS C:\Users\DELL\Downloads\2405009> node -v
v24.14.0
PS C:\Users\DELL\Downloads\2405009> npm -v
11.9.0
PS C:\Users\DELL\Downloads\2405009> npm create vite@latest react-week11-15
Need to install the following packages:
create-vite@9.3.3
Ok to proceed? (y) y

> npx
> cva react-week11-15


create-vite v9.3.3

|
◇  Select a framework:
|  React
|
◇  Select a variant:
|  JavaScript
|
◇  Which linter to use?
|  ESLint
|
◇  Install with npm and start now?
|  Yes
|
◇  Scaffolding project in C:\Users\DELL\Downloads\2405009\react-week11-15...
|
◇  Installing dependencies with npm...

added 143 packages, and audited 144 packages in 26s

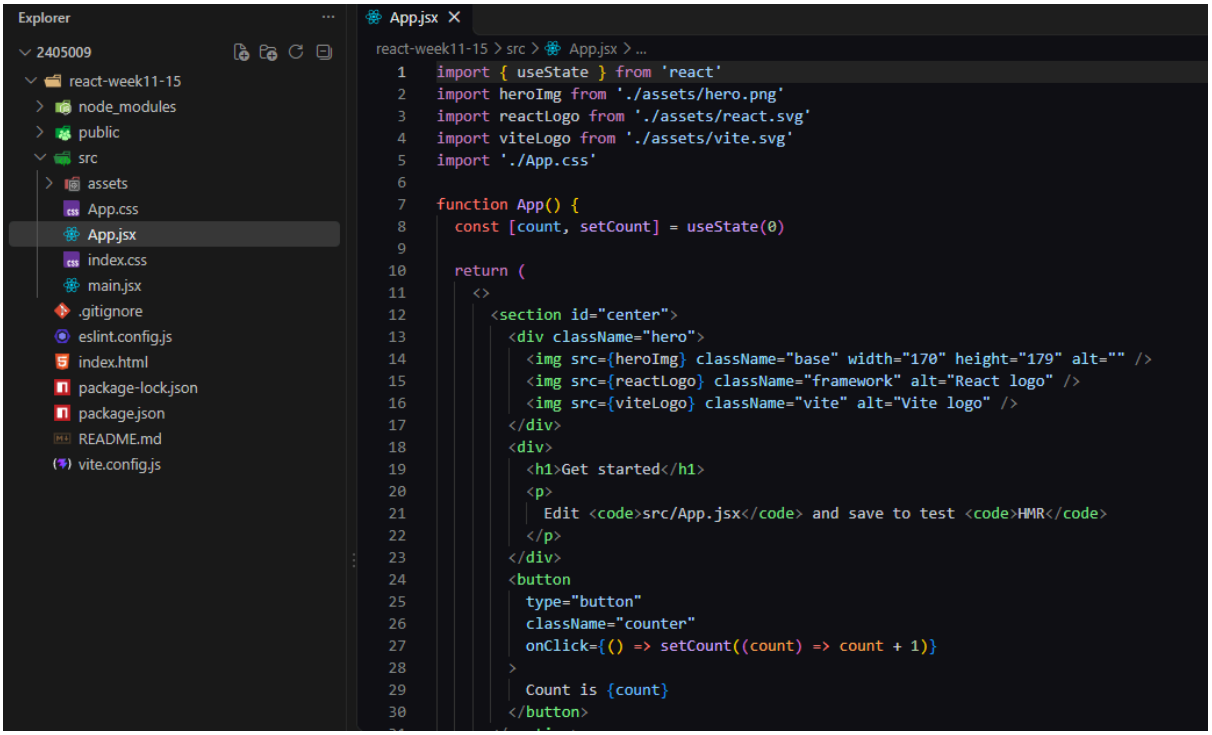
31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
◇  Starting dev server...

> react-week11-15@0.0.0 dev
> vite


  VITE v8.3.5  ready in 254 ms

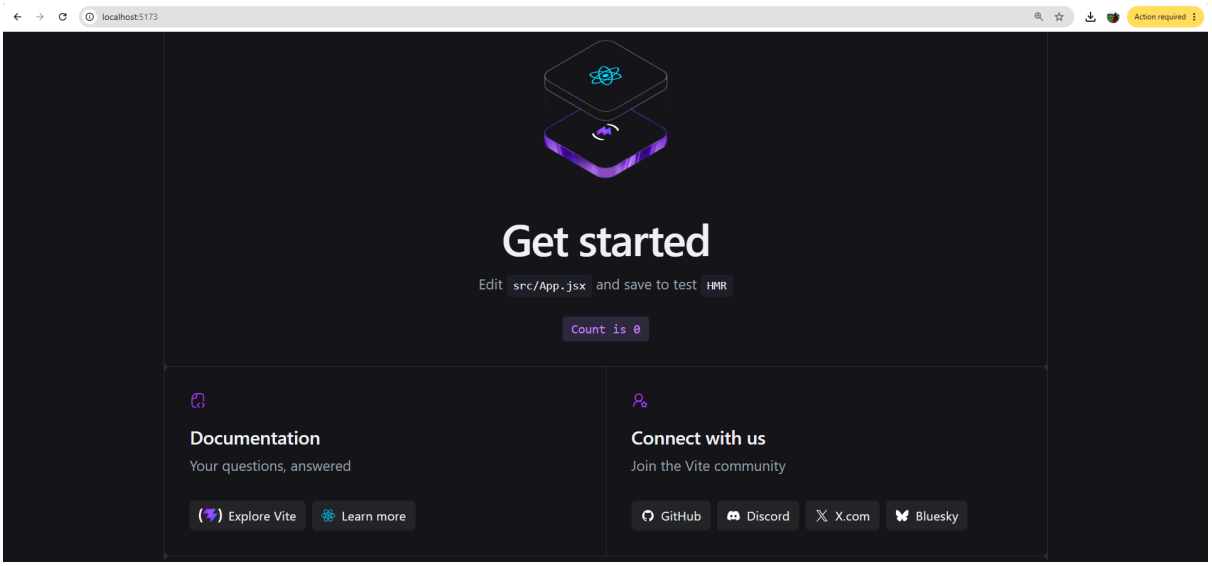
  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

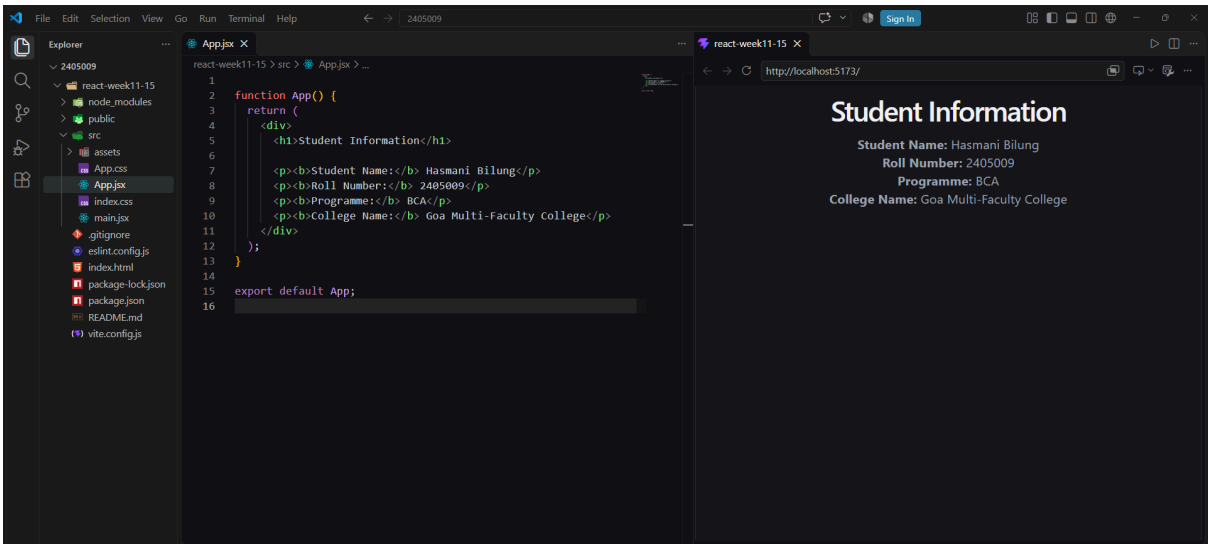

Explorer
2405009
react-week11-15
node_modules
public
src
assets
App.css
App.jsx
index.css
main.jsx
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
README.md
vite.config.js
App.jsx
react-week11-15 > src > App.jsx > ...
import { useState } from 'react'
import heroImg from './assets/hero.png'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import './App.css'
function App() {
const [count, setCount] = useState(0)
return (
<>
<section id="center">

<h1>Get started</h1>

Edit <code>src/App.jsx</code> and save to test <code>HMR</code>

<button
type="button"
className="counter"
onClick={() => setCount((count) => count + 1)}
>
Count is {count}
</button>


localhost:5173
Action required
Get started
Edit src/App.jsx and save to test HMR
Count is 0
Documentation
Your questions, answered
Explore Vite
Learn more
Connect with us
Join the Vite community
GitHub
Discord
X.com
Bluesky


File Edit Selection View Go Run Terminal Help
2405009
Sign In
Explorer
App.jsx
react-week11-15
http://localhost:5173/
function App() {
return (

<h1>Student Information</h1>
Student Name: Hasmani Bilung
Roll Number: 2405009
Programme: BCA
College Name: Goa Multi-Faculty College

);
}
export default App;
Student Information
Student Name: Hasmani Bilung
Roll Number: 2405009
Programme: BCA
College Name: Goa Multi-Faculty College

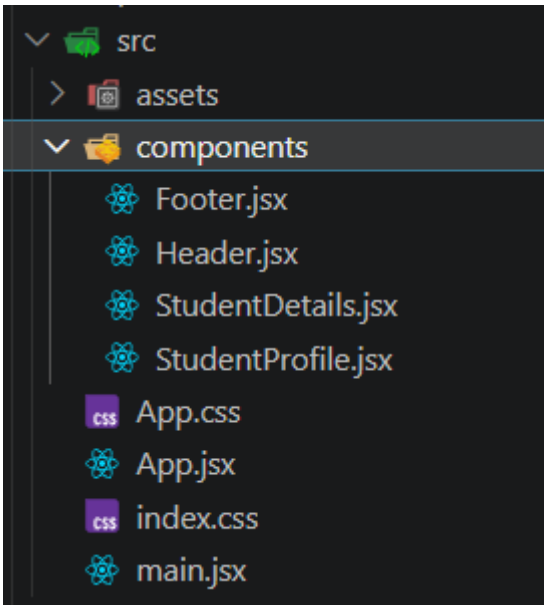
Exercise 11.2 – Creating Custom Components

Aim:
To create and use custom React components.

Problem Statement:
Create a student profile application using separate components.

Requirements:
Create the following components:

```
App
├── Header
├── StudentProfile
├── StudentDetails
└── Footer
```



Each component should display appropriate information.

Tasks:
1. Create Header.jsx.

react-week11-15 > src > components > Header.jsx > ...

```jsx

function Header() {
  return (
    <header>
      <h1>Goa Multi-Faculty College</h1>
      <h2>Student Profile Application</h2>
    </header>
  );
}

export default Header;
```

2. Create StudentProfile.jsx.

react-week11-15 > src > components > StudentProfile.jsx > ...

```jsx

function StudentProfile() {
  return (
    <div>
      <br/>
      <h2>Student Profile</h2>
      <h3>Hasmani Bilung</h3>
      <p>Programme: BCA</p>
    </div>
  );
}

export default StudentProfile;
```

3. Create StudentDetails.jsx.

react-week11-15 > src > components > StudentDetails.jsx > ...

```jsx

function StudentDetails() {
  return (
    <div>
      <br/>
      <h2>Student Details</h2>
      <p>Roll Number: 2405009</p>
      <p>College: Goa Multi-Faculty College</p>
      <p>Class: TYBCA</p>
      <br/>
    </div>
  );
}

export default StudentDetails;
```

4. Create Footer.jsx.

react-week11-15 > src > components > Footer.jsx > ...

```jsx

function Footer() {
  return (
    <footer>
      <p>© 2026 Goa Multi-Faculty College</p>
    </footer>
  );
}

export default Footer;
```

5. Import and use the components in App.jsx.

react-week11-15 > src > App.jsx > ...

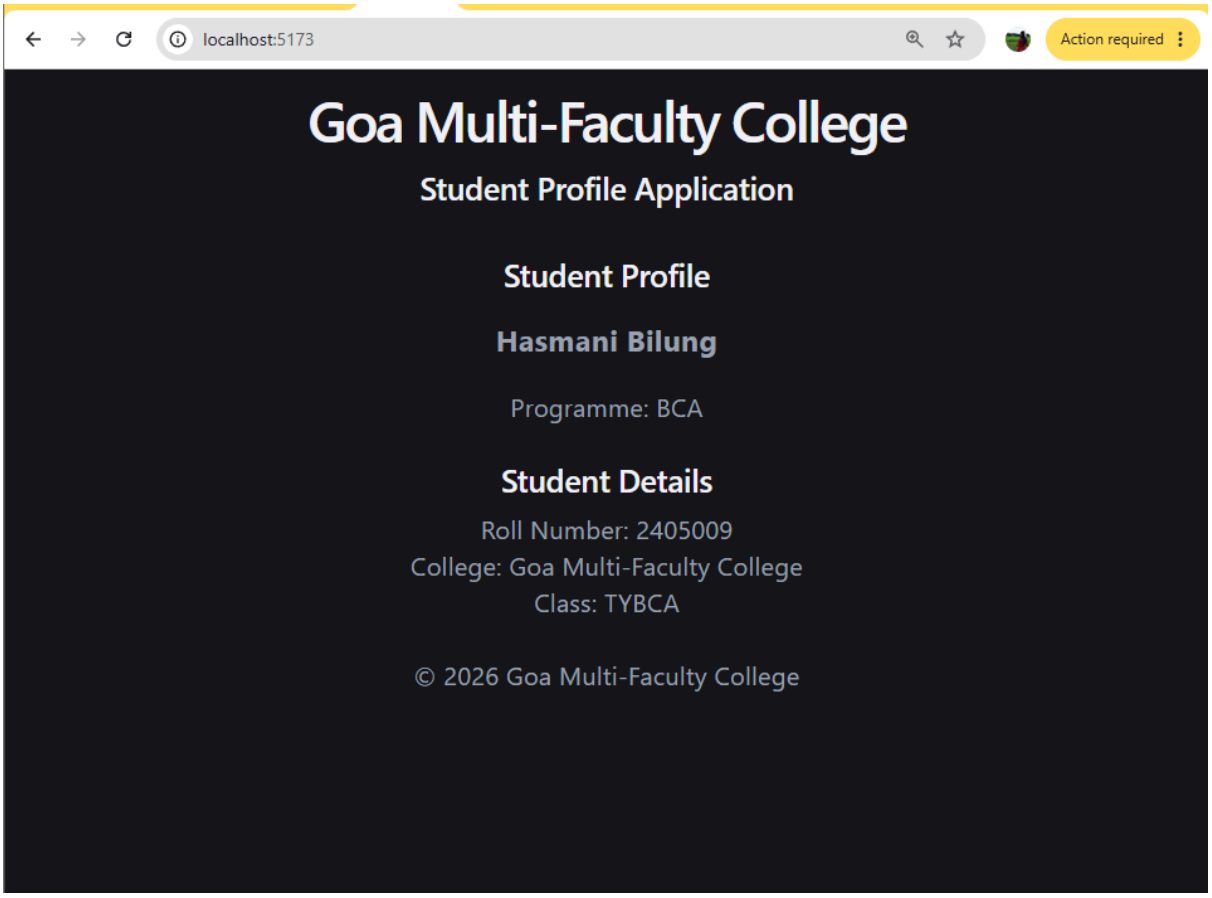
```jsx

import Header from "./components/Header";
import StudentProfile from "./components/StudentProfile";
import StudentDetails from "./components/StudentDetails";
import Footer from "./components/Footer";

function App() {
  return (
    <div>
      <Header />
      <StudentProfile />
      <StudentDetails />
      <Footer />
    </div>
  );
}

export default App;
```

Output



Exercise 11.3 – Passing Data Using Props

Aim:
To understand and implement props in React.

Problem Statement:
Create a StudentCard component that receives student information through props.

Requirements:
- Pass student data from App.jsx.
- Display the information using StudentCard.
- Create cards for at least three students.

App.jsx

```jsx
import StudentCard from "./components/StudentCard";
import "./components/StudentCard.css";
function App() {
  return (
    <div className="student-list">
      <h1 className="page-heading">Student Information</h1>
      <StudentCard
        name="Hasmani Bilung"
        rollNo={2405009}
        course="BCA"
        semester={5}
      />

      <StudentCard
        name="Shreya Gajinkar"
        rollNo={2405033}
        course="BCA"
        semester={5}
      />

      <StudentCard
        name="Mansi Bhandari"
        rollNo={2405040}
        course="BCA"
        semester={5}
      />
    </div>
  );
}
export default App;
```

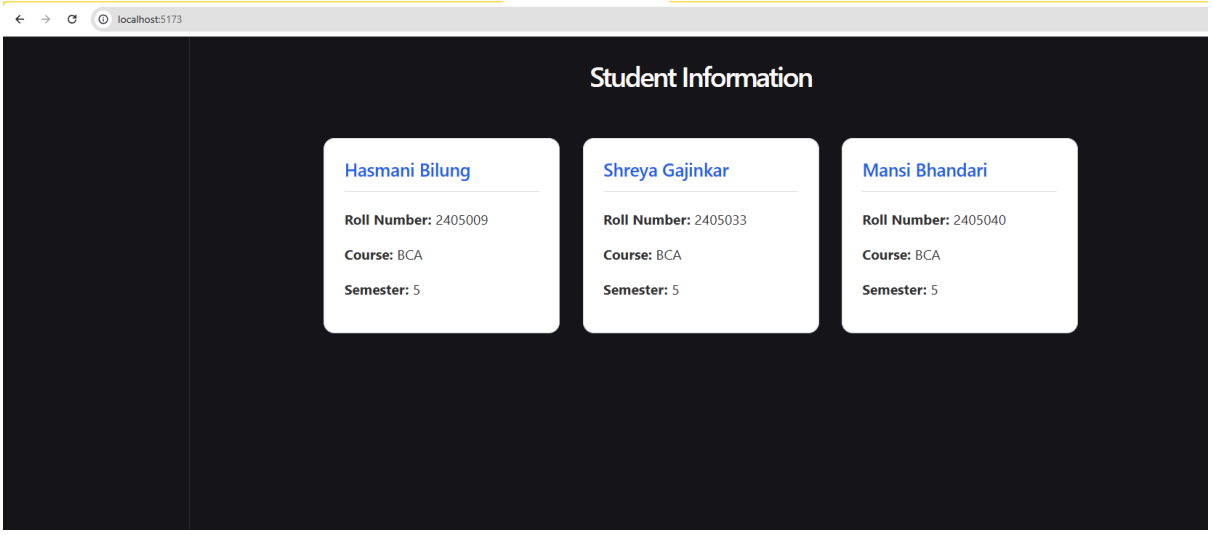
StudentCard.jsx

```jsx
import "./StudentCard.css";

function StudentCard(props) {
  return (
    <div className="student-card">
      <h2>{props.name}</h2>
      <p><strong>Roll Number:</strong> {props.rollNo}</p>
      <p><strong>Course:</strong> {props.course}</p>
      <p><strong>Semester:</strong> {props.semester}</p>
    </div>
  );
}

export default StudentCard;
```

Output



Exercise 11.4 – Managing State Using useState

Aim:
To understand state management in React.

Problem 1 – Counter
Create a counter application with:

- Increment
- Decrement
- Reset

Counter.jsx

react-week11-15 > src > components > Counter.jsx > Counter

```jsx
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1);
  };

  const decrement = () => {
    setCount(count - 1);
  };

  const reset = () => {
    setCount(0);
  };
  return (
    <div>
      <h1>Counter Application</h1>

      <h2>Count: {count}</h2>

      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}
export default Counter;
```

App.jsx

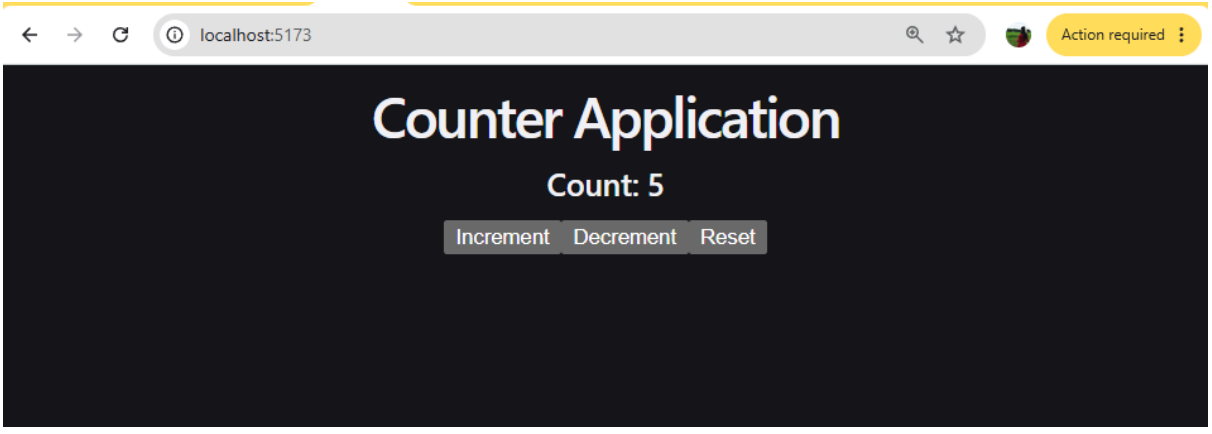
react-week11-15 > src > App.jsx > default

```jsx
import Counter from "./components/Counter.jsx";

function App() {
  return (
    <div>
      <Counter />
    </div>
  );
}

export default App;
```

Output



Problem 2 – Student Attendance

Create an attendance counter with:
Student Name

Classes Attended
[+] [-]

The number of classes attended should be managed using useState.

StudentAttendance.jsx

```jsx
import { useState } from "react";

function StudentAttendance() {
  const [classesAttended, setClassesAttended] = useState(0);

  const increment = () => {
    setClassesAttended(classesAttended + 1);
  };

  const decrement = () => {
    if (classesAttended > 0) {
      setClassesAttended(classesAttended - 1);
    }
  };

  return (
    <div>
      <h1>Student Attendance</h1>

      <h2>Student Name: Hasmani Bilung</h2>

      <h3>Classes Attended: {classesAttended}</h3>

      <button onClick={increment}>+</button>
      <button onClick={decrement}>-</button>
    </div>
  );
}

export default StudentAttendance;
```

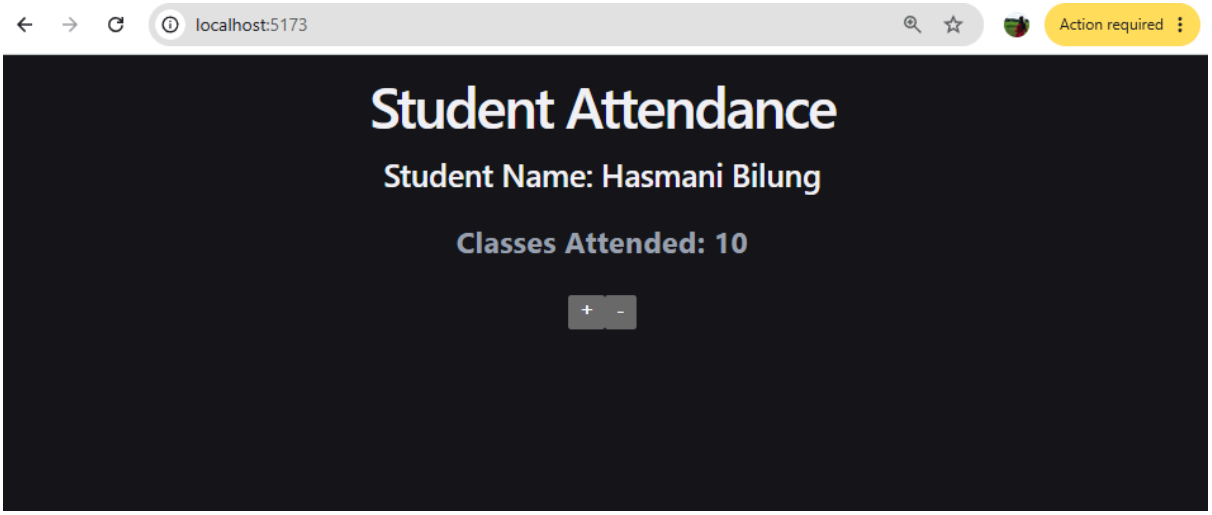
App.jsx

```jsx
import StudentAttendance from "./components/StudentAttendance.jsx";

function App() {
  return (
    <div>
      <StudentAttendance />
    </div>
  );
}

export default App;
```

Output



Exercise 11.5 – Form Handling Using State

Aim:
To create a controlled form using React state.

Create a Student Registration Form containing:

- Name
- Roll Number
- Email
- Course
- Semester

- Gender

On clicking Submit, display the entered information below the form.

Concepts:

- useState
- Controlled components
- onChange
- onSubmit
- Form handling

```
import { useState } from "react";

export default function StudentForm() {
  const [name, setName] = useState("");
  const [roll, setRoll] = useState("");
  const [email, setEmail] = useState("");
  const [course, setCourse] = useState("");
  const [semester, setSemester] = useState("");
  const [gender, setGender] = useState("");
  const [show, setShow] = useState(false);

  function submitForm(e) {
    e.preventDefault();
    setShow(true);
  }

  return (
    <div>
      <h1>Student Registration Form</h1>

      <form onSubmit={submitForm}>
        Name:
        <input
          type="text"
          value={name}
          onChange={e => setName(e.target.value)}
          required
        />
        <br /><br />

        Roll Number:
```

```
        <input
          type="text"
          value={roll}
          onChange={e => setRoll(e.target.value)}
          required
        />
        <br /><br />

        Email:
        <input
          type="email"
          value={email}
          onChange={e => setEmail(e.target.value)}
          required
        />
        <br /><br />

        Course:
        <select
          value={course}
          onChange={e => setCourse(e.target.value)}
          required
        >
          <option value="">Select Course</option>
          <option value="BCA">BCA</option>
          <option value="BBA">BBA</option>
          <option value="BCom">BCom</option>
          <option value="BA">BA</option>
        </select>
        <br /><br />

        Semester:
        <select
          value={semester}
          onChange={e => setSemester(e.target.value)}
          required
        >
          <option value="">Select Semester</option>
          <option value="1">Semester 1</option>
          <option value="2">Semester 2</option>
          <option value="3">Semester 3</option>
```

```
          <option value="4">Semester 4</option>
          <option value="5">Semester 5</option>
          <option value="6">Semester 6</option>
        </select>
        <br /><br />

        Gender:
        <select
          value={gender}
          onChange={e => setGender(e.target.value)}
          required
        >
          <option value="">Select Gender</option>
          <option value="Male">Male</option>
          <option value="Female">Female</option>
          <option value="Other">Other</option>
          <option value="Prefer not to say">Prefer not to
say</option>
        </select>
        <br /><br />

        <button type="submit">Submit</button>
      </form>

      {show && (
        <div>
          <h2>Student Details</h2>
          <p>Name: {name}</p>
          <p>Roll Number: {roll}</p>
          <p>Email: {email}</p>
          <p>Course: {course}</p>
          <p>Semester: {semester}</p>
          <p>Gender: {gender}</p>
        </div>
      )}
    </div>
  );
}
```

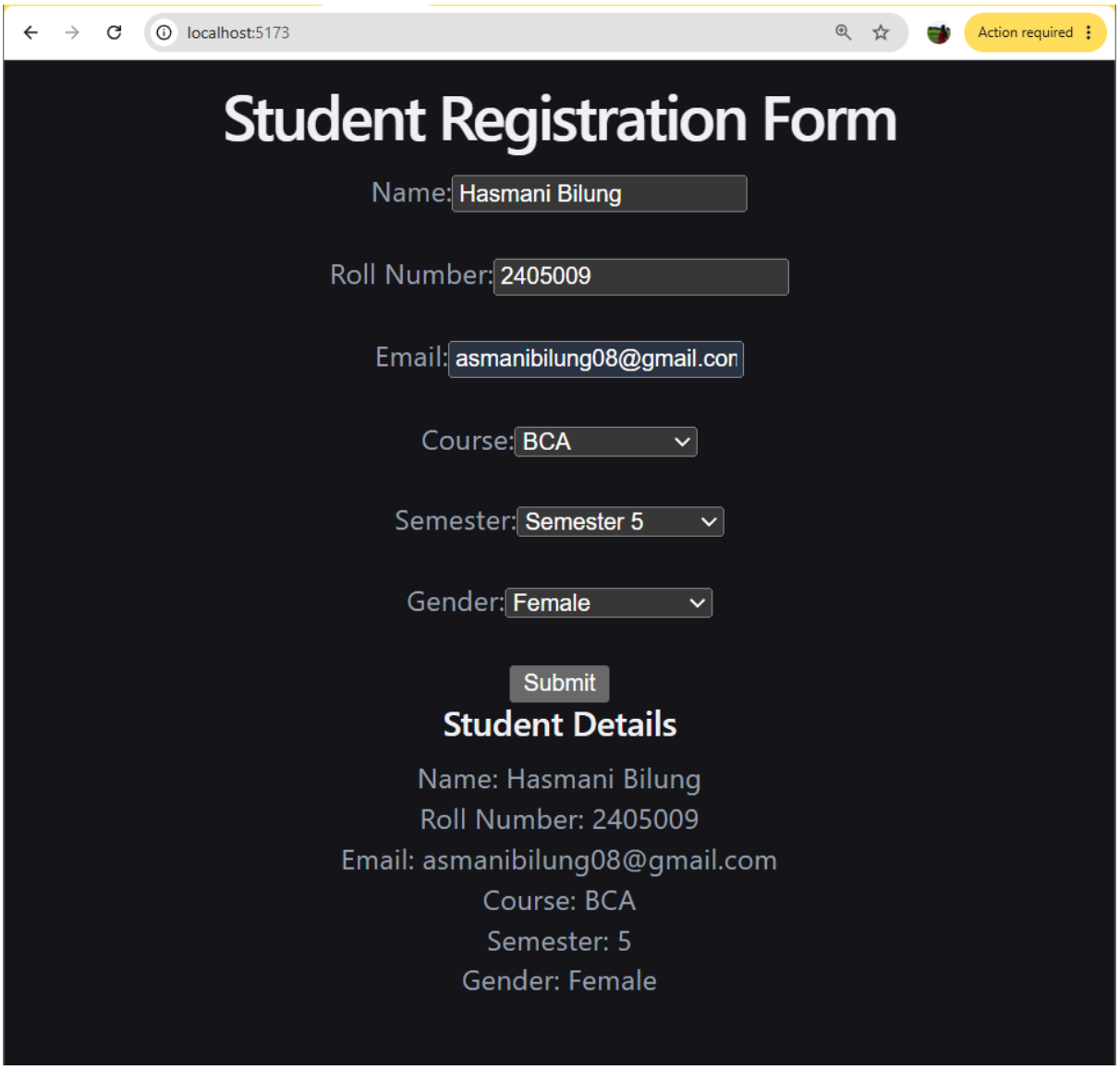
react-week11-15 > src > App.jsx > default

```jsx
import StudentRegistration from "./components/StudentRegistration.jsx";

function App() {
  return (
    <div>
      <StudentRegistration />
    </div>
  );
}

export default App;
```

Output



Week 12 – React Hooks

Exercise 12.1 – useState() – Dynamic To-Do List

Aim:
To use useState() for managing an array of objects.

Create a To-Do application with:

- Add task
- Display tasks
- Mark task as completed
- Delete task

```
import { useState } from "react";

function ToDoList() {

    // State for the text entered in the input box
    const [taskText, setTask] = useState("");

    // State for storing all tasks
    const [tasks, setTasks] = useState([
        {
            id: 1,
            text: "Complete React assignment",
            completed: false
        },
        {
            id: 2,
            text: "Study Node.js",
            completed: true
        },
        {
            id: 3,
            text: "Prepare Practical",
            completed: false
        }
    ]);

    // Add a new task
    const addTask = () => {

        // Do not add an empty task
```

```
        if (taskText.trim() === "") {
            return;
        }

        const newTask = {
            id: Date.now(),
            text: taskText,
            completed: false
        };

        // Add new task to existing tasks
        setTasks([...tasks, newTask]);

        // Clear input box
        setTask("");
    };

    // Mark task as completed or incomplete
    const toggleTask = (id) => {
        setTasks(
            tasks.map((item) =>
                item.id === id
                    ? { ...item, completed: !item.completed }
                    : item
            )
        );
    };

    // Delete a task
    const deleteTask = (id) => {
        setTasks(tasks.filter((item) => item.id !== id));
    };

    return (
        <div>
            <h1>To-Do List</h1>

            <input
                type="text"
                placeholder="Enter Task.."
                value={taskText}
                onChange={(e) => setTask(e.target.value)}
            />
```

```
            <br />

            <button onClick={addTask}>
                Add
            </button>

            {
                tasks.map((item) => (
                    <div key={item.id}>

                        <input
                            type="checkbox"
                            checked={item.completed}
                            onChange={() =>
toggleTask(item.id)}
                        />

                        <span style={{
                            textDecoration: item.completed
                                ? "line-through"
                                : "none"
                        }}>
                            {item.text}
                        </span>

                        <button onClick={() =>
deleteTask(item.id)}>
                            Delete
                        </button>

                    </div>
                ))
            }

        </div>
    );
}

export default ToDoList;
```

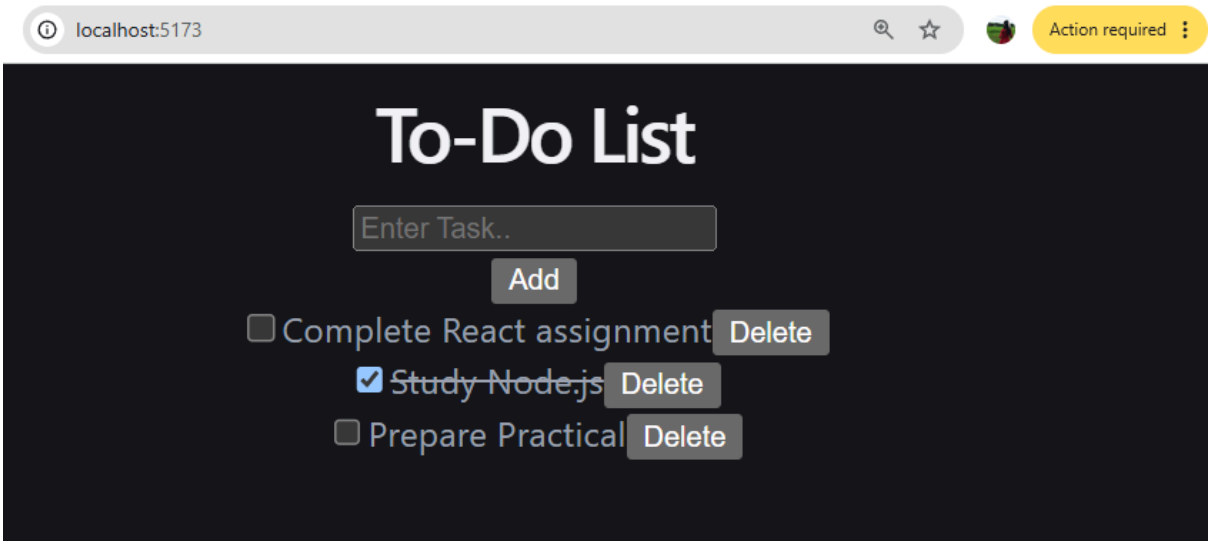
react-week11-15 > src > App.jsx > default

```jsx
import TodoList from "./components/TodoList.jsx";

function App() {
  return (
    <div>
      <TodoList />
    </div>
  );
}

export default App;
```

Output



Exercise 12.2 – useEffect() – API Data

Aim:
To understand side effects and API calls using useEffect().

Use a public API such as:
https://jsonplaceholder.typicode.com/users

Tasks:
1. Fetch users when the component loads.
2. Store the response using useState.
3. Display users in a table.
4. Display a loading message while data is being fetched.

5. Handle errors.

Expected concepts:
useEffect()
↓
API Request
↓
Response
↓
setState()
↓
Display Data

```
import { useState, useEffect } from "react";

function Users() {

    // State for storing users
    const [users, setUsers] = useState([]);

    // State for loading message
    const [loading, setLoading] = useState(true);

    // State for error message
    const [error, setError] = useState("");

    // Fetch users when component loads
    useEffect(() => {

        const fetchUsers = async () => {

            try {
                const response = await fetch(

"https://jsonplaceholder.typicode.com/users"
                );

                if (!response.ok) {
                    throw new Error("Failed to fetch users");
                }
```

```
                const data = await response.json();

                setUsers(data);

            } catch (err) {
                setError(err.message);

            } finally {
                setLoading(false);
            }
        };

        fetchUsers();

    }, []);

    return (
        <div>
            <h1>User Details</h1>

            {loading && <p>Loading users...</p>}

            {error && <p>{error}</p>}

            {!loading && !error && (
                <table border="1" cellPadding="10" style={{
margin: "auto" }}>
                    <thead>
                        <tr>
                            <th>ID</th>
                            <th>Name</th>
                            <th>Email</th>
                        </tr>
                    </thead>

                    <tbody>
                        {users.map((user) => (
                            <tr key={user.id}>
                                <td>{user.id}</td>
                                <td>{user.name}</td>
                                <td>{user.email}</td>
```

```
                            </tr>
                        ))}
                    </tbody>
                </table>
            )}

        </div>
    );
}

export default Users;
```

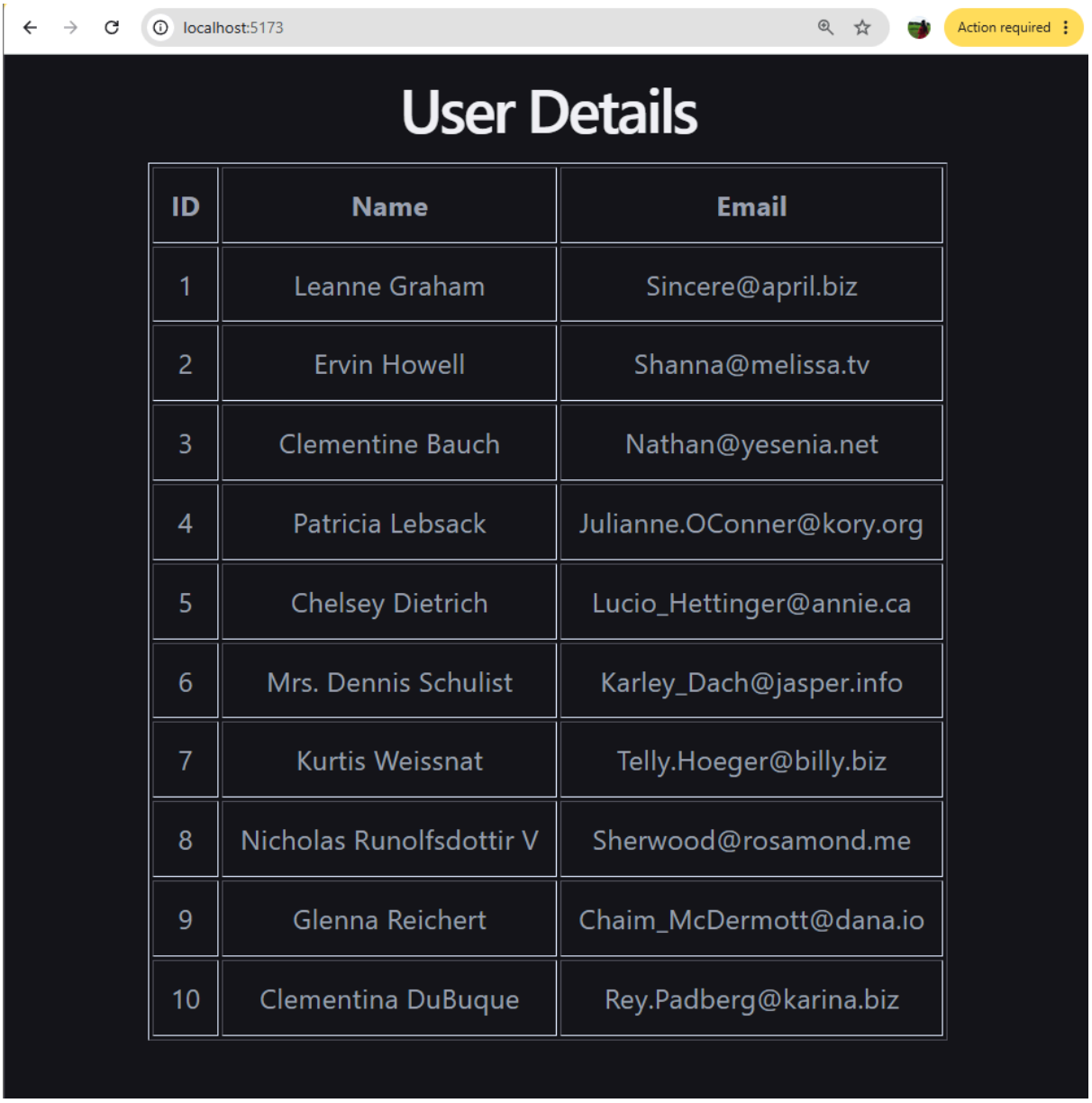
react-week11-15 > src > App.jsx > default

```jsx
import Users from "./components/users.jsx";

function App() {
    return (
        <div>
            <Users />
        </div>
    );
}

export default App;
```

Output



Exercise 12.3 – useReducer() – Student Management

Aim:
To manage complex state using useReducer().

Create a Student Management application supporting:

- Add student
- Delete student
- Update student
- Clear all students

```
import { useReducer, useState } from "react";

const initialStudents = [];
```

```
function studentReducer(state, action) {
    switch (action.type) {
        case "ADD_STUDENT":
            return [...state, action.payload];

        case "DELETE_STUDENT":
            return state.filter(s => s.id !== action.payload);

        case "UPDATE_STUDENT":
            return state.map(s =>
                s.id === action.payload.id ? action.payload : s
            );

        case "CLEAR_STUDENTS":
            return [];

        default:
            return state;
    }
}

function StudentManagement() {
    const [students, dispatch] = useReducer(studentReducer,
initialStudents);
    const [name, setName] = useState("");
    const [course, setCourse] = useState("");
    const [editId, setEditId] = useState(null);

    const handleSubmit = (e) => {
        e.preventDefault();

        if (!name.trim() || !course.trim()) return;

        const student = {
            id: editId ?? Date.now(),
            name,
            course
        };

        dispatch({
            type: editId === null ? "ADD_STUDENT" :
"UPDATE_STUDENT",
            payload: student
        });

        setName("");
```

```
        setCourse("");
        setEditId(null);
    };

    const handleEdit = (student) => {
        setName(student.name);
        setCourse(student.course);
        setEditId(student.id);
    };

    return (
        <div>
            <h1>Student Management</h1>

            <form onSubmit={handleSubmit}>
                <input
                    type="text"
                    placeholder="Enter student name"
                    value={name}
                    onChange={e => setName(e.target.value)}
                />

                <br /><br />

                <input
                    type="text"
                    placeholder="Enter course"
                    value={course}
                    onChange={e => setCourse(e.target.value)}
                />

                <br /><br />

                <button type="submit">
                    {editId === null ? "Add Student" : "Update
Student"}
                </button>
            </form>

            <br />

            <button onClick={() => {
                dispatch({ type: "CLEAR_STUDENTS" });
                setName("");
                setCourse("");
                setEditId(null);
```

```
            }}>
                Clear All Students
            </button>

            <h2>Student List</h2>

            {students.length === 0 ? (
                <p>No students available.</p>
            ) : (
                <table border="1" cellPadding="10" style={{ margin:
"auto" }}>
                    <thead>
                        <tr>
                            <th>ID</th>
                            <th>Name</th>
                            <th>Course</th>
                            <th>Actions</th>
                        </tr>
                    </thead>

                    <tbody>
                        {students.map(student => (
                            <tr key={student.id}>
                                <td>{student.id}</td>
                                <td>{student.name}</td>
                                <td>{student.course}</td>
                                <td>
                                    <button onClick={() =>
handleEdit(student)}>
                                        Edit
                                    </button>
                                    <button onClick={() =>
                                        dispatch({
                                            type: "DELETE_STUDENT",
                                            payload: student.id
                                        })
                                    }>
                                        Delete
                                    </button>
                                </td>
                            </tr>
                        ))}
                    </tbody>
                </table>
            )}
        </div>
```

```
    );
}

export default StudentManagement;
```

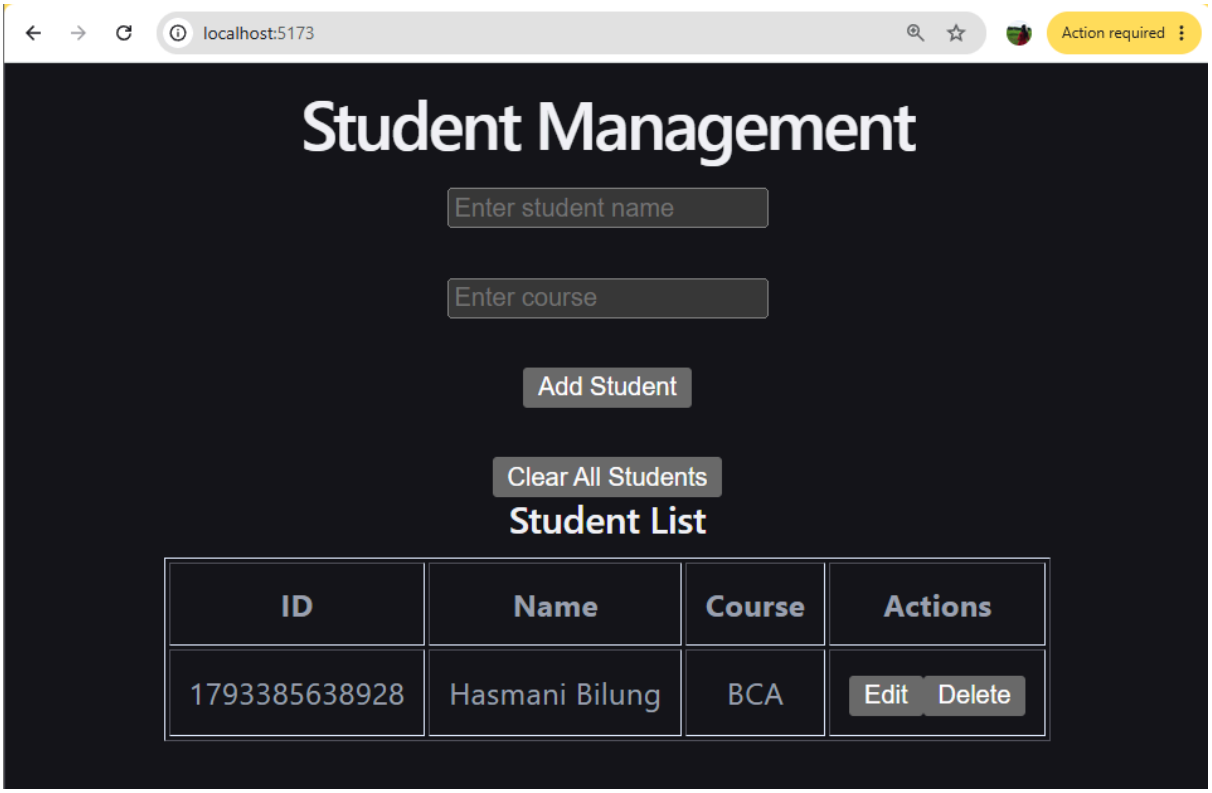
react-week11-15 > src > App.jsx > default

```jsx
import StudentManagement from "./components/StudentManagement.jsx";

function App() {
    return (
        <div>
            <StudentManagement />
        </div>
    );
}

export default App;
```

Output



Exercise 12.4 – useContext() – Theme Management

Aim:
To share data between components using Context API.

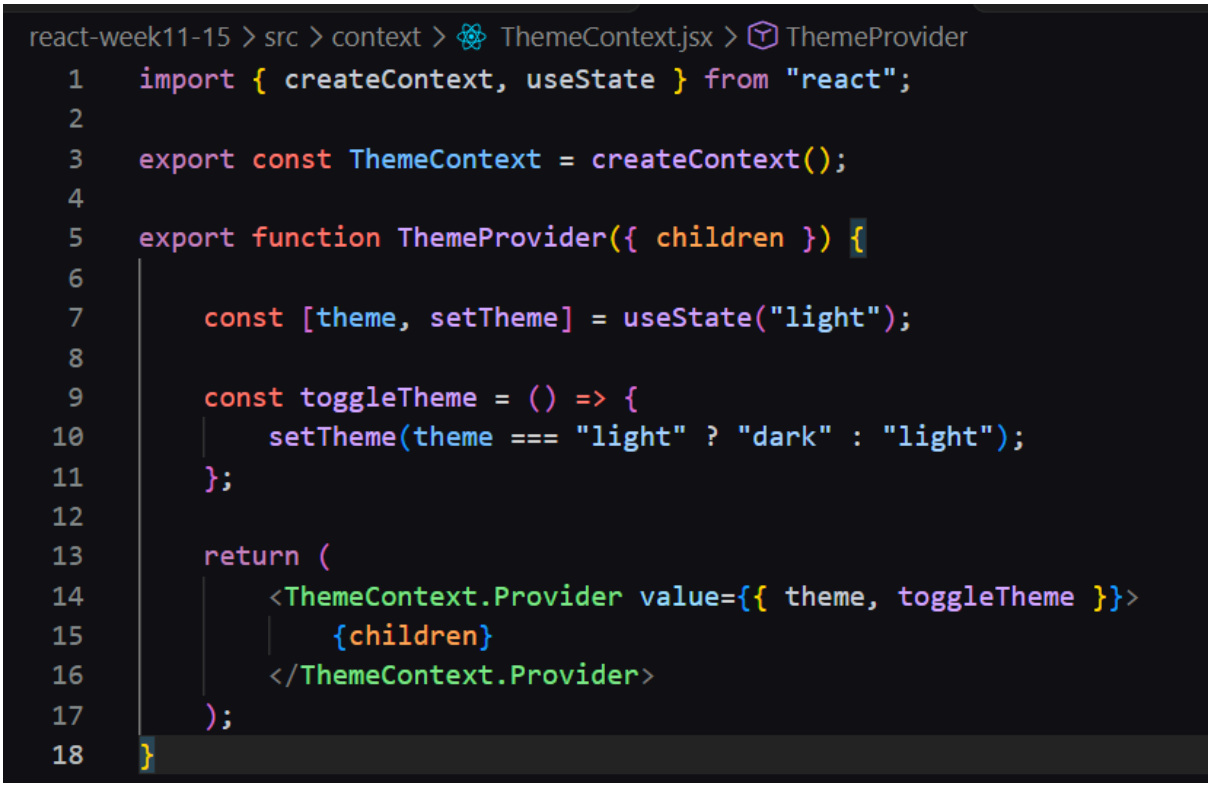
Create an application with:
ThemeContext

↓
App
├── Header
├── Content
└── Footer

Implement:

- Light theme
- Dark theme
- Theme toggle button

The theme should be accessible to multiple components without passing props manually.



react-week11-15 > src > components > Header.jsx > default

```jsx
import { useContext } from "react";
import { ThemeContext } from "../context/ThemeContext.jsx";

function Header() {

    const { theme, toggleTheme } = useContext(ThemeContext);

    return (
        <header style={{
            background: theme === "light" ? "⬜#ddd" : "⬛#222",
            color: theme === "light" ? "⬛#000" : "⬜#fff",
            padding: "20px",
            textAlign: "center"
        }}>
            <h2>My Website</h2>

            <button onClick={toggleTheme}>
                Change to {theme === "light" ? "Dark" : "Light"} Theme
            </button>
        </header>
    );
}

export default Header;
```

react-week11-15 > src > components > Content.jsx > default

```jsx
import { useContext } from "react";
import { ThemeContext } from "../context/ThemeContext.jsx";

function Content() {

    const { theme } = useContext(ThemeContext);

    return (
        <main style={{
            background: theme === "light" ? "⬜#fff" : "⬛#333",
            color: theme === "light" ? "⬛#000" : "⬜#fff",
            padding: "40px",
            textAlign: "center"
        }}>
            <h2>Welcome to My Website</h2>
            <p>This is the main content section.</p>
            <p>The current theme is {theme}.</p>
        </main>
    );
}

export default Content;
```

react-week11-15 > src > components > Footer.jsx > default

```jsx
import { useContext } from "react";
import { ThemeContext } from "../context/ThemeContext.jsx";

function Footer() {

    const { theme } = useContext(ThemeContext);

    return (
        <footer style={{
            background: theme === "light" ? "⬜#ddd" : "⬛#222",
            color: theme === "light" ? "⬛#000" : "⬜#fff",
            padding: "20px",
            textAlign: "center"
        }}>
            <p>Copyright 2026 - My Website</p>
        </footer>
    );
}

export default Footer;
```

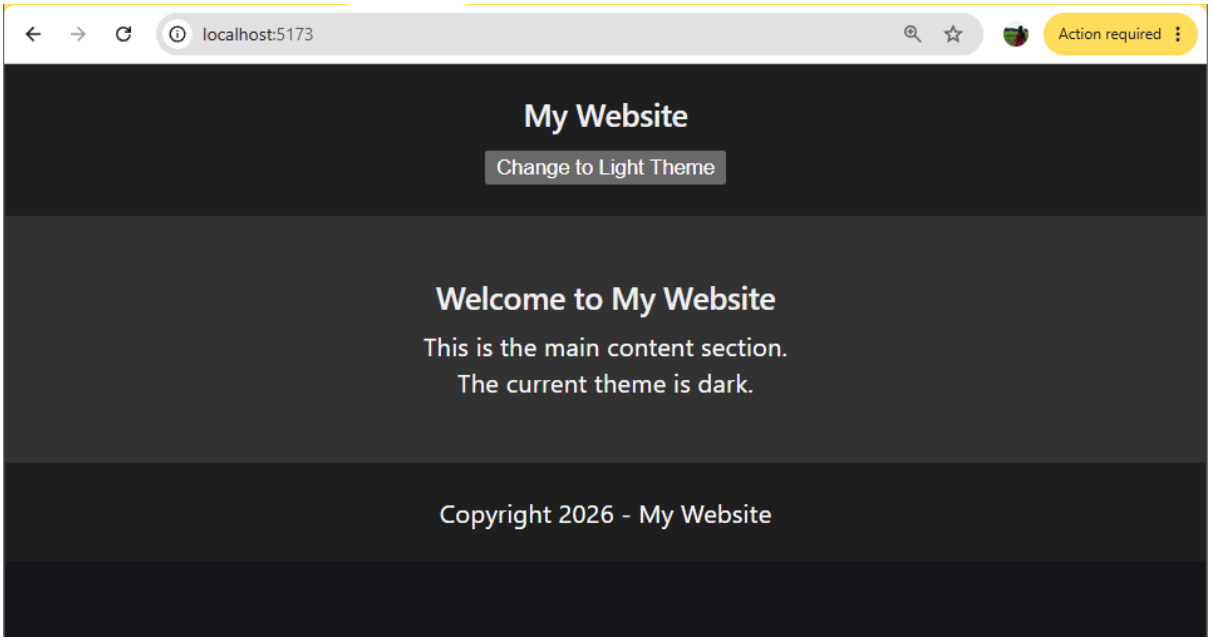
react-week11-15 > src > App.jsx > default

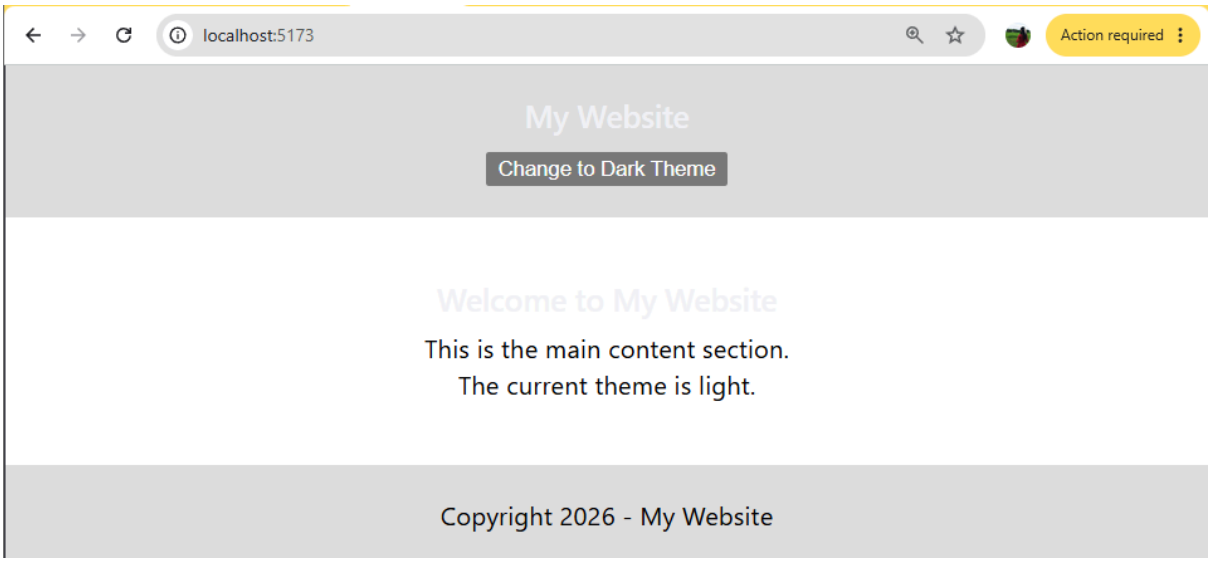
```jsx
import Header from "./components/Header.jsx";
import Content from "./components/Content.jsx";
import Footer from "./components/Footer.jsx";
import { ThemeProvider } from "./context/ThemeContext.jsx";

function App() {
    return (
        <ThemeProvider>
            <Header />
            <Content />
            <Footer />
        </ThemeProvider>
    );
}

export default App;
```

Output





Exercise 12.5 – useRef() – DOM and Form Handling

Aim:
To understand the use of useRef().

Create a form containing:
Name: [_____]
[Focus Name Field]

When the button is clicked, automatically focus the Name textbox.

react-week11-15 > src > App.jsx > App

```jsx
import { useRef } from "react";

export default function App() {

    const nameRef = useRef(null);

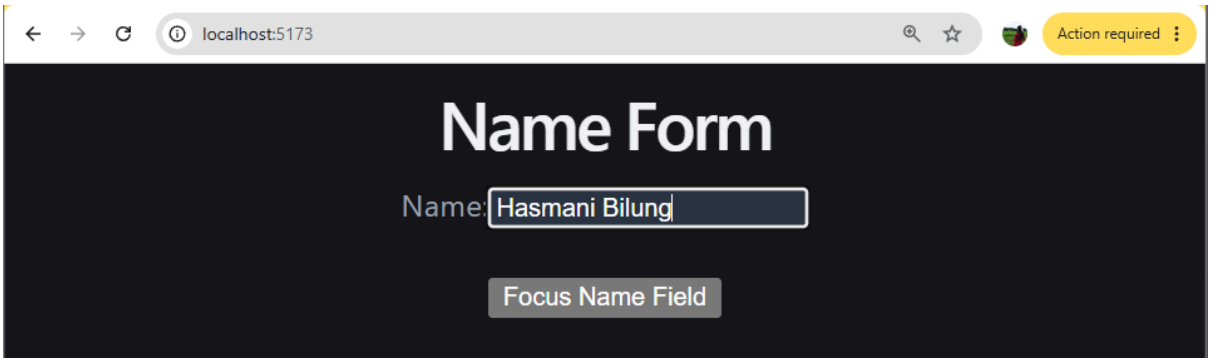
    return (
        <main>
            <h1>Name Form</h1>

            <label>
                Name:
                <input
                    ref={nameRef}
                    placeholder="Enter your name"
                />
            </label>

            <br /><br />

            <button onClick={() => nameRef.current?.focus()}>
                Focus Name Field
            </button>
        </main>
    );
}
```

Output



Exercise 12.6 – useMemo() and useCallback()

Aim:

To understand basic React performance optimization.

Create an application that displays a list of students and allows the user to search students.

Implement:

- useMemo() for an expensive filtering/calculation operation.
- useCallback() for a function passed to a child component.

react-week11-15 > src > App.jsx > App

```jsx
import { memo, useCallback, useMemo, useState } from "react";

const StudentList = memo(function StudentList({ students, onSelect }) {
    return (
        <ul>
            {students.map(s => (
                <li key={s.id}>
                    {s.name}
                    <button onClick={() => onSelect(s.name)}>Select</button>
                </li>
            ))}
        </ul>
    );
});

const allStudents = [
    { id: 1, name: "Hasmani" },
    { id: 2, name: "Shreya" },
    { id: 3, name: "Shreya" },
    { id: 4, name: "Pramila" }
];

export default function App() {
    const [query, setQuery] = useState("");
    const [selected, setSelected] = useState("");

    const filtered = useMemo(
        () => allStudents.filter(s =>
            s.name.toLowerCase().includes(query.toLowerCase())
        ),
        [query]
    );
```

```
    const selectStudent = useCallback(
        name => setSelected(name),
        []
    );

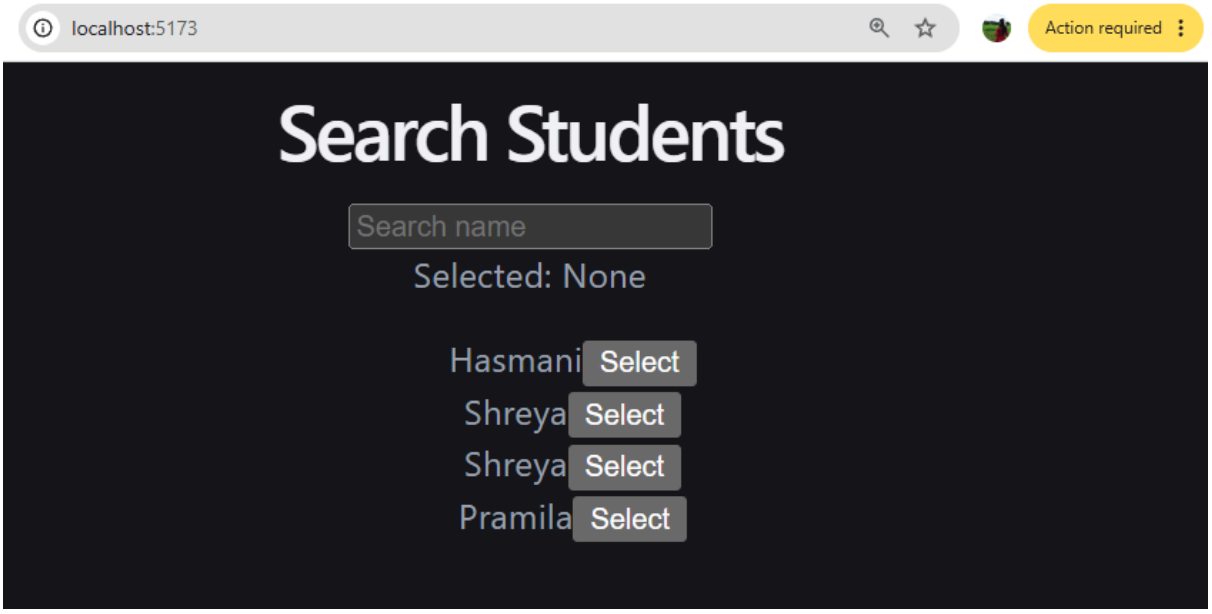
    return (
        <main>
            <h1>Search Students</h1>

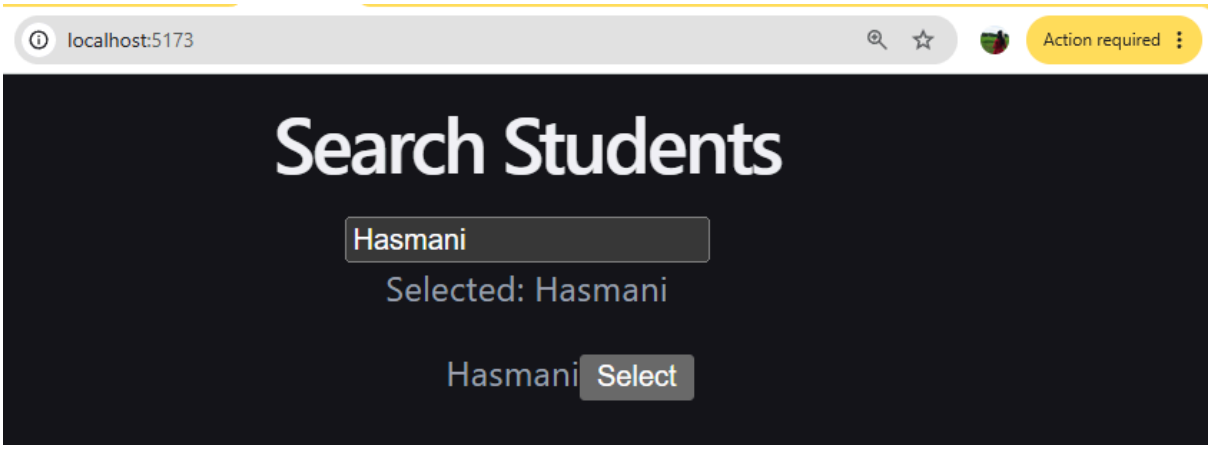
            <input
                value={query}
                onChange={e => setQuery(e.target.value)}
                placeholder="Search name"
            />

            <p>Selected: {selected || "None"}</p>

            <StudentList
                students={filtered}
                onSelect={selectStudent}
            />
        </main>
    );
}
```

Output




localhost:5173
Action required
Search Students
Hasmani
Selected: Hasmani
Hasmani
Select

Week 13 – Routing and Navigation

Exercise 13.1 – React Router Setup

Aim:
To implement routing in a React application.

Install React Router:
npm install react-router-dom

```
C:\Users\TYBCA\Downloads\ReactProject-2405009>npm install react-router-dom

added 7 packages in 8s

1 package is looking for funding
  run `npm fund` for details
```

Create the following pages:
Home
About
Students
Contact

Implement navigation using:
Home | About | Students | Contact

week13 > src > pages > Home.jsx > default

```jsx
function Home() {
    return (
        <div>
            <h1>Home Page</h1>
        </div>
    )
}

export default Home;
```

week13 > src > pages > Contact.jsx > default

```jsx
function Contact() {
    return (
        <div>
            <h1>Contact Page</h1>
        </div>
    )
}

export default Contact;
```

week13 > src > pages > About.jsx > default

```jsx
function About() {
    return (
        <div>
            <h1>About Page</h1>
        </div>
    )
}

export default About;
```

week13 > src > pages > Students.jsx > default

```jsx
function Students() {
    return (
        <div>
            <h1>Students Page</h1>
        </div>
    )
}

export default Students;
```

week13 > src > App.jsx > default

```jsx
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom'

import Home from './pages/Home'
import About from './pages/About'
import Students from './pages/Students'
import Contact from './pages/Contact'

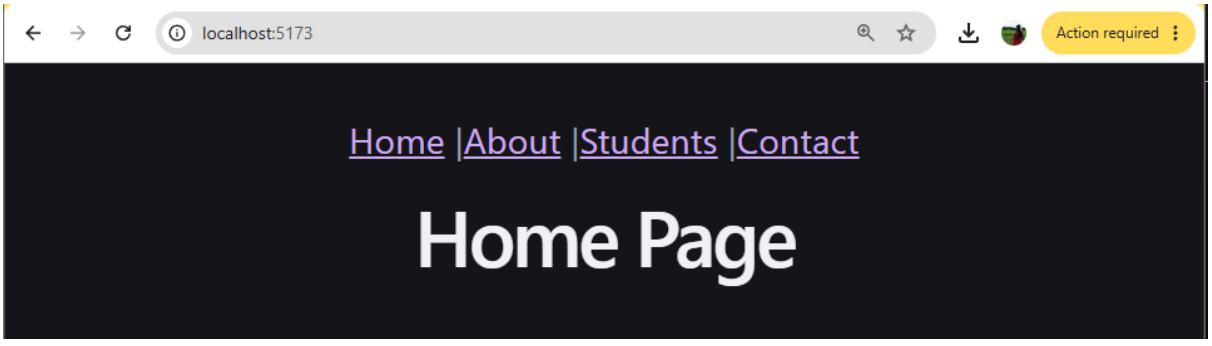
import './App.css'

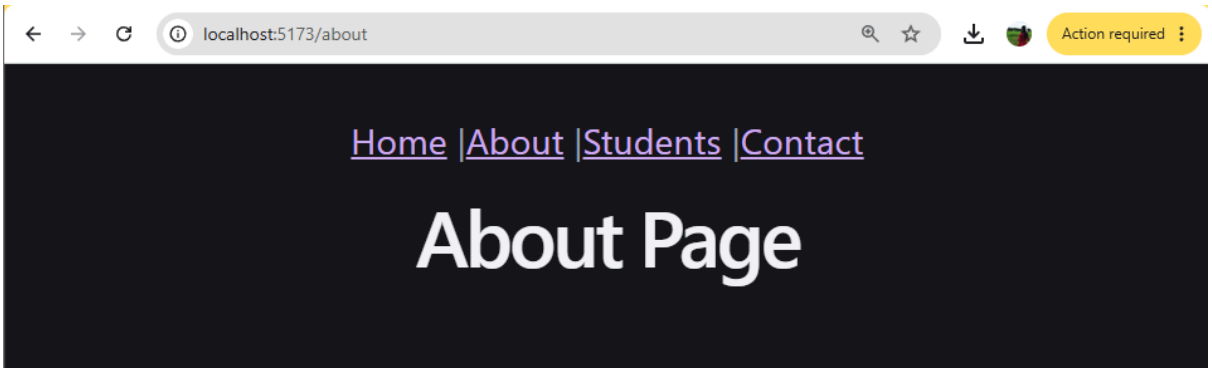
function App() {
    return (
        <BrowserRouter>
            <div>
                <nav>
                  <br/>
                    <Link to="/">Home</Link> |
                    <Link to="/about">About</Link> |
                    <Link to="/students">Students</Link> |
                    <Link to="/contact">Contact</Link>
                </nav>

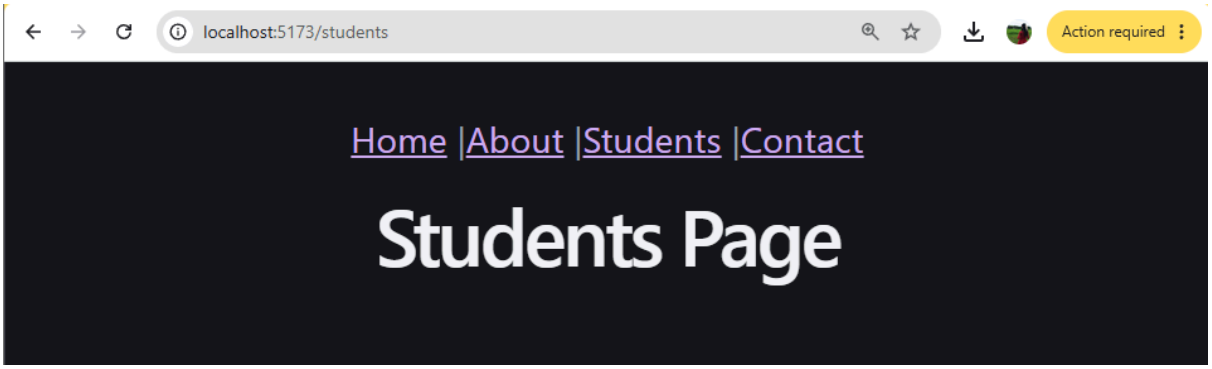
                <Routes>
                    <Route path="/" element={<Home />} />
                    <Route path="/about" element={<About />} />
                    <Route path="/students" element={<Students />} />
                    <Route path="/contact" element={<Contact />} />
                </Routes>
            </div>
        </BrowserRouter>
    )
}

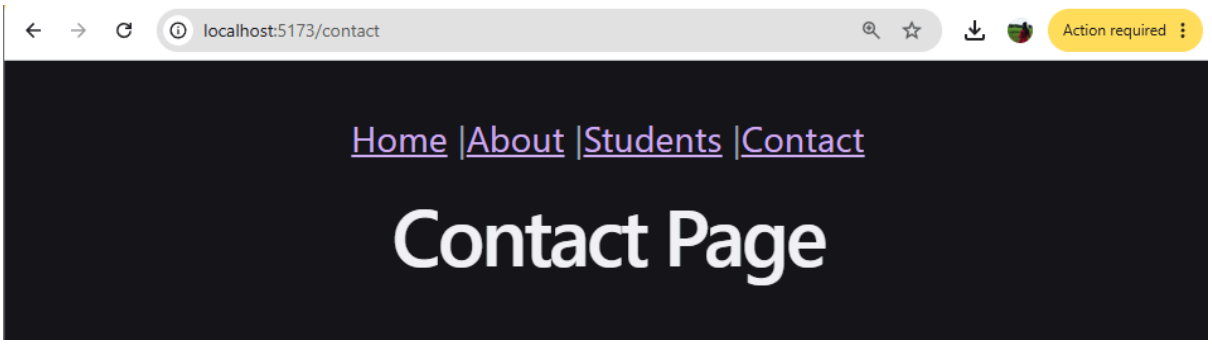
export default App;
```

Output









Exercise 13.2 – Dynamic Routing

Aim:
To implement dynamic routes.

Create a student list

When the user clicks a student, navigate to:
/students/101
/students/102
/students/103

Use route parameters to display the corresponding student.

Concepts:

- Routes
- Route
- Link
- useParams
- Dynamic routing

week13 > src > pages > Students.jsx > default

```jsx
import { Link } from 'react-router-dom'

function Students() {
    return (
        <div>
            <h1>Students</h1>

            <p><Link to="/students/101">101 - Hasmani</Link></p>
            <p><Link to="/students/102">102 - Shreya</Link></p>
            <p><Link to="/students/103">103 - Mansi</Link></p>
        </div>
    )
}

export default Students;
```

week13 > src > components > StudentDetails.jsx > default

```jsx
import { useParams } from 'react-router-dom'

function StudentDetails() {
    const { id } = useParams()

    const students = [
        { id: '101', name: 'Hasmani' },
        { id: '102', name: 'Shreya' },
        { id: '103', name: 'Mansi' }
    ]

    const student = students.find(s => s.id === id)

    return (
        <div>
            {student ? (
                <h1>Student {student.id}: {student.name}</h1>
            ) : (
                <h1>Student Not Found</h1>
            )}
        </div>
    )
}

export default StudentDetails;
```

week13 > src > App.jsx > default

```jsx
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom'

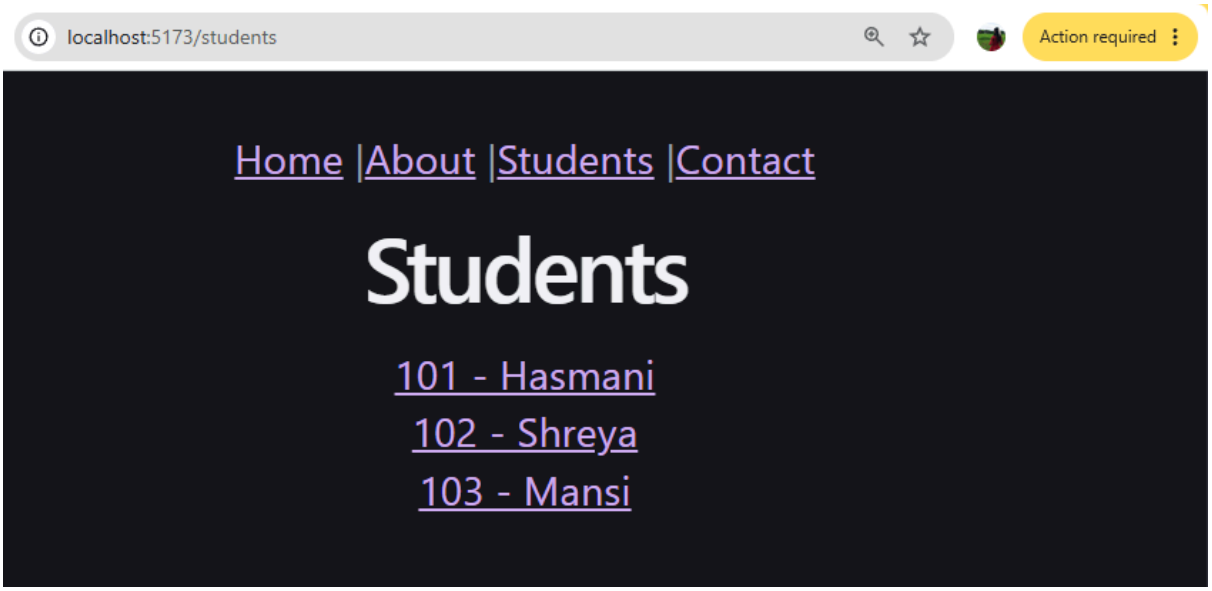
import Home from './pages/Home'
import About from './pages/About'
import Students from './pages/Students'
import Contact from './pages/Contact'
import StudentDetails from './components/StudentDetails'

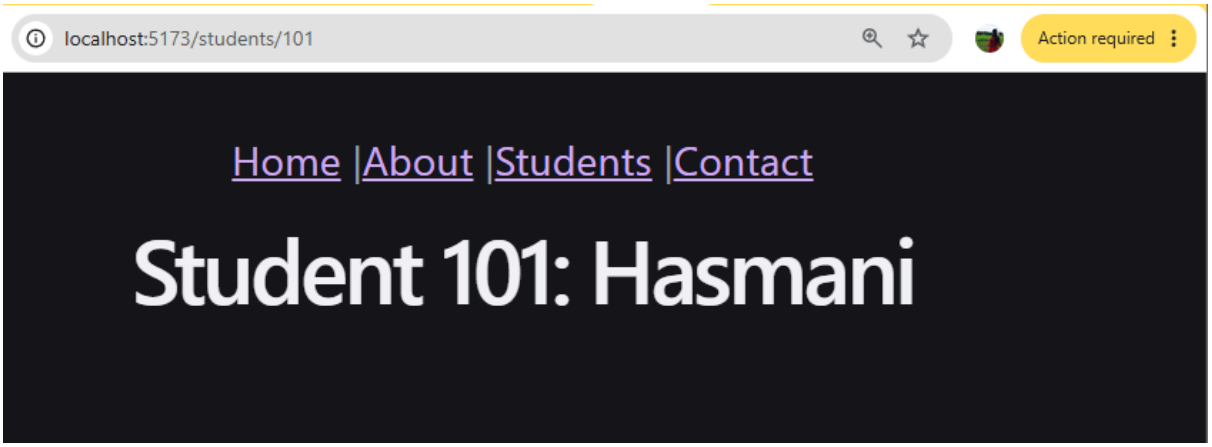
function App() {
    return (
        <BrowserRouter>
            <div>
                <nav>
                  <br/>
                    <Link to="/">Home</Link> |
                    <Link to="/about">About</Link> |
                    <Link to="/students">Students</Link> |
                    <Link to="/contact">Contact</Link>
                </nav>

                <Routes>
                    <Route path="/" element={<Home />} />
                    <Route path="/about" element={<About />} />
                    <Route (property) path: string Students />} />
                    <Route path="/contact" element={<Contact />} />
                    <Route path="/students/:id" element={<StudentDetails />} />
                </Routes>
            </div>
        </BrowserRouter>
    )
}

export default App;
```

Output





Exercise 13.3 – Nested Routes

Aim:
To implement nested routing.

Create:

/students
├── /list
├── /add
└── /details/:id

Application structure:

Student Management
├── Student List
├── Add Student
└── Student Details

week13 > src > pages > StudentManagement.jsx > ...

```jsx
import { Link, Outlet } from 'react-router-dom'

function StudentManagement() {
    return (
        <div>
            <h1>Student Management</h1>

            <nav>
                <Link to="list">Student List</Link> |
                <Link to="add">Add Student</Link> |
                <Link to="details/101">Student Details</Link>
            </nav>

            <hr />

            <Outlet />
        </div>
    )
}

export default StudentManagement;
```

week13 > src > pages > StudentList.jsx > ...

```jsx
function StudentList() {
    return (
        <div>
            <h2>Student List</h2>

            <ul>
                <li>101 - Hasmani</li>
                <li>102 - Shreya</li>
            </ul>
        </div>
    )
}

export default StudentList;
```

week13 > src > pages > AddStudent.jsx > ...

```jsx
function AddStudent() {
    return (
        <div>
            <h2>Add Student</h2>

            <form>
                <label>Student Name: </label>
                <input type="text" placeholder="Enter student name" />

                <br /><br />

                <label>Student ID: </label>
                <input type="text" placeholder="Enter student ID" />

                <br /><br />

                <button type="button">Add Student</button>
            </form>
        </div>
    )
}

export default AddStudent;
```

week13 > src > pages > StudentDetails.jsx > ...

```jsx
import { useParams } from 'react-router-dom'

function StudentDetails() {
    const { id } = useParams()

    const students = [
        { id: '101', name: 'Hasmani' },
        { id: '102', name: 'Shreya' }
    ]

    const student = students.find(s => s.id === id)

    return (
        <div>
            <h2>Student Details</h2>

            {student ? (
                <p>Student ID: {student.id} | Name: {student.name}</p>
            ) : (
                <p>Student Not Found</p>
            )}
        </div>
    )
}

export default StudentDetails;
```

week13 > src > App.jsx > ...

```jsx
import { BrowserRouter, Routes, Route } from 'react-router-dom'

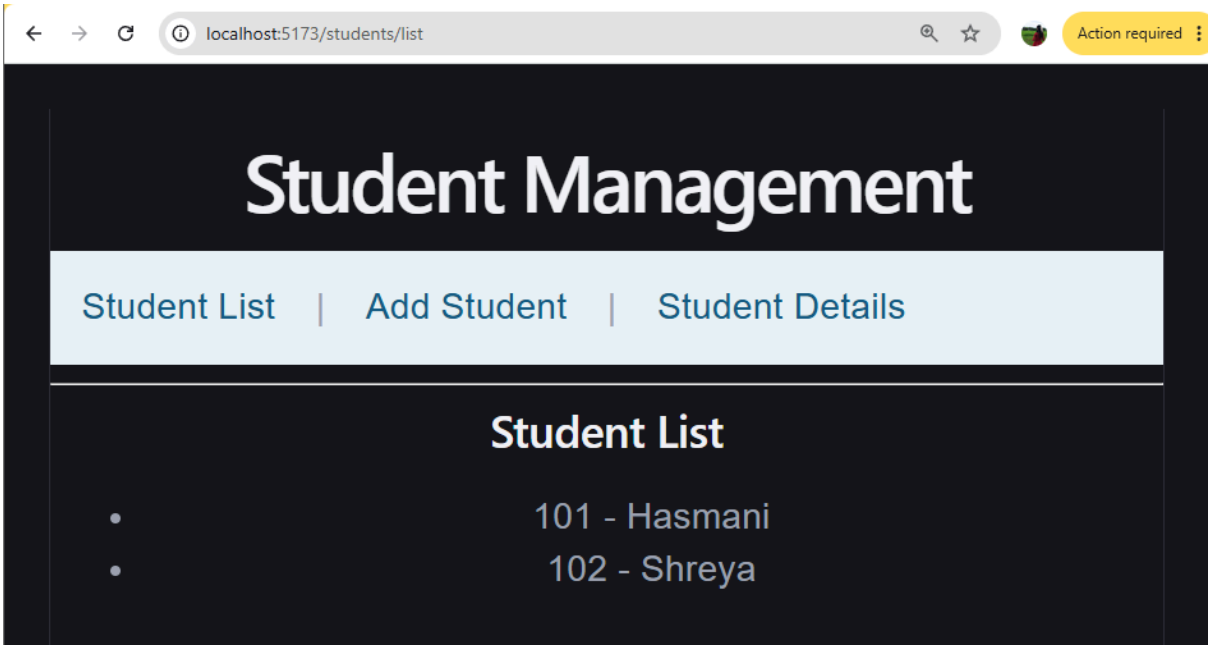
import StudentManagement from './pages/StudentManagement'
import StudentList from './pages/StudentList'
import AddStudent from './pages/AddStudent'
import StudentDetails from './pages/StudentDetails'

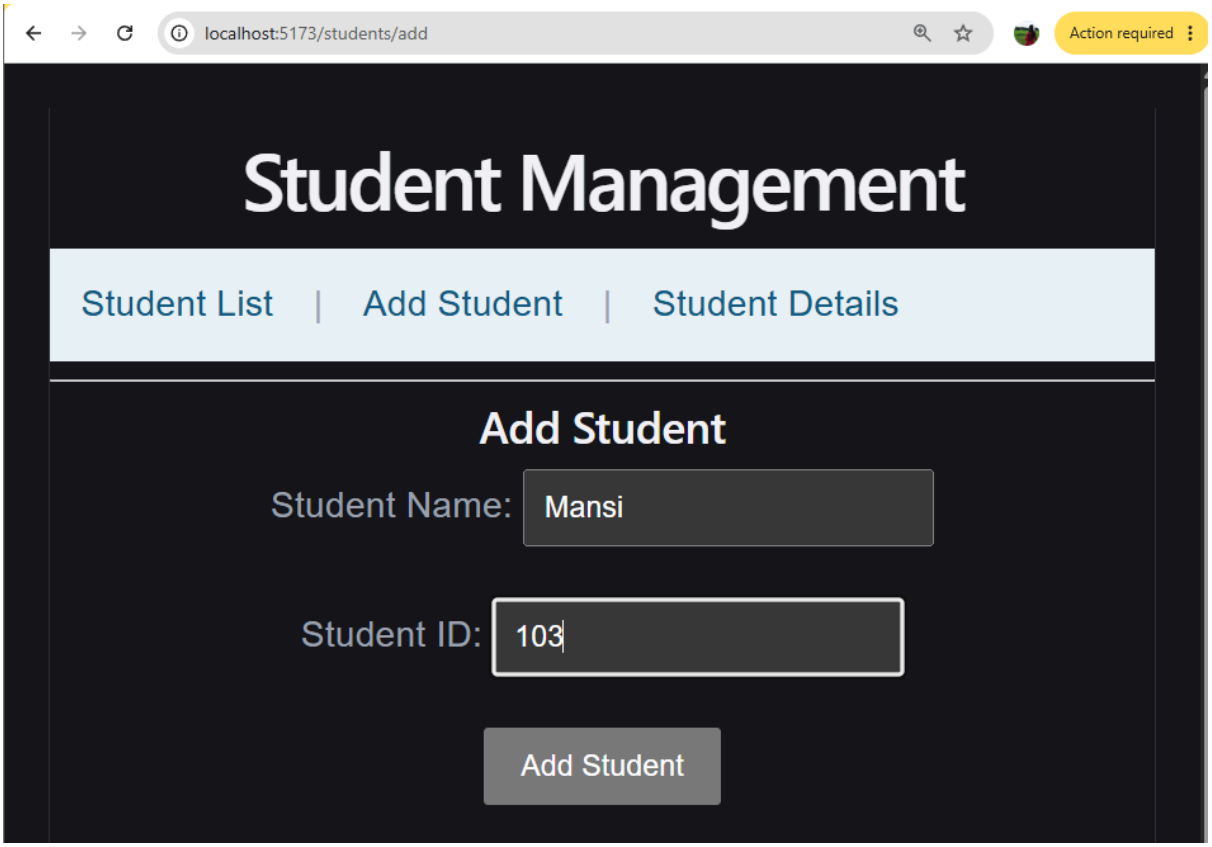
import './App.css'

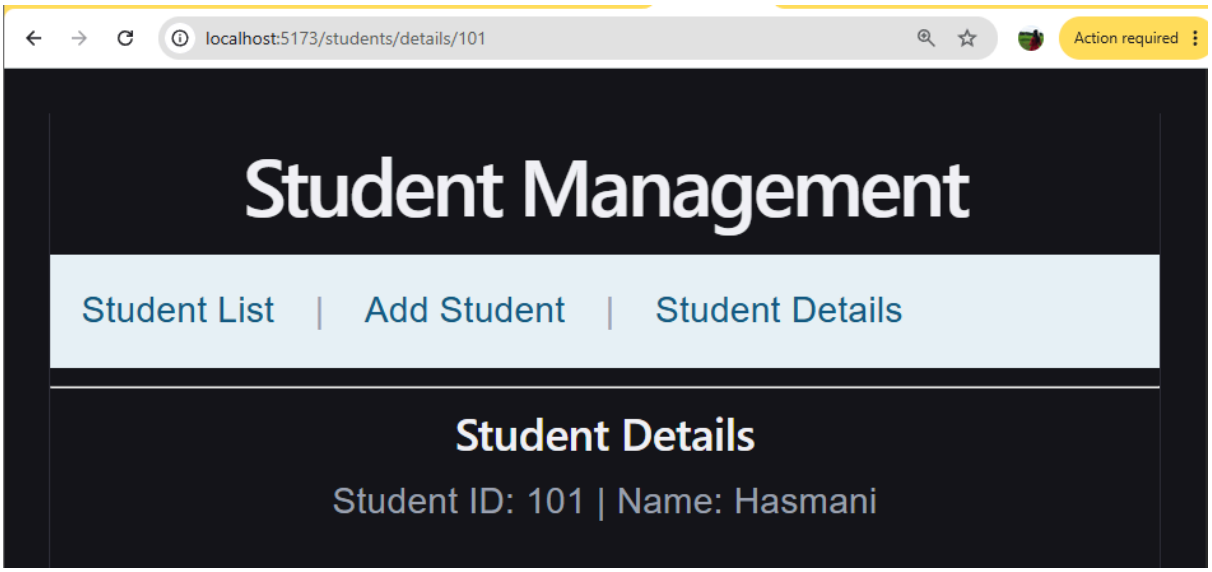
function App() {
    return (
        <BrowserRouter>
            <Routes>
                <Route path="/students" element={<StudentManagement />}>
                    <Route path="list" element={<StudentList />} />
                    <Route path="add" element={<AddStudent />} />
                    <Route path="details/:id" element={<StudentDetails />} />
                </Route>
            </Routes>
        </BrowserRouter>
    )
}

export default App;
```

Output







Exercise 13.4 – Protected Route Simulation

Aim:
To understand the concept of protected routes.

Create a simple login page.
Login
Username
Password
[Login]

After successful login, allow access to:
/dashboard
/students
/profile

If the user is not logged in, redirect them to /login.

week13 > src > components > ProtectedRoute.jsx > default

```jsx
import { Navigate, Outlet, Link, useNavigate } from 'react-router-dom'

function ProtectedRoute() {
    const isLoggedIn = localStorage.getItem('isLoggedIn')
    const navigate = useNavigate()

    function handleLogout() {
        localStorage.removeItem('isLoggedIn')
        navigate('/login')
    }

    if (isLoggedIn !== 'true') {
        return <Navigate to="/login" replace />
    }

    return (
        <div>
            <h2>Student Management System</h2>

            <nav>
                <Link to="/dashboard">Dashboard</Link>
                <Link to="/students">Students</Link>
                <Link to="/profile">Profile</Link>
                <button onClick={handleLogout}>Logout</button>
            </nav>

            <hr />

            <Outlet />
        </div>
    )
}

export default ProtectedRoute;
```

week13 > src > pages > Login.jsx > default

```jsx
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'

function Login() {
    const [username, setUsername] = useState('')
    const [password, setPassword] = useState('')
    const [error, setError] = useState('')

    const navigate = useNavigate()

    function handleLogin(e) {
        e.preventDefault()

        if (username === 'admin' && password === '1234') {
            localStorage.setItem('isLoggedIn', 'true')
            navigate('/dashboard')
        } else {
            setError('Invalid username or password')
        }
    }

    return (
        <div>
            <h1>Login</h1>

            <form onSubmit={handleLogin}>
                <label>Username: </label>
                <input
                    type="text"
                    value={username}
                    onChange={(e) => setUsername(e.target.value)}
                    placeholder="Enter username"
                />

                <br /><br />

                <label>Password: </label>
                <input
                    type="password"
                    value={password}
                    onChange={(e) => setPassword(e.target.value)}
                    placeholder="Enter password"
                />

                <br /><br />

                <button type="submit">Login</button>
            </form>

            <p>{error}</p>
        </div>
    )
}

export default Login;
```

week13 > src > pages > Dashboard.jsx > default

```jsx
function Dashboard() {
    return (
        <div>
            <h1>Dashboard</h1>
            <p>Welcome to the Dashboard!</p>
        </div>
    )
}

export default Dashboard;
```

week13 > src > pages > Student.jsx > default

```jsx
function Students() {
    return (
        <div>
            <h1>Students</h1>

            <ul>
                <li>101 - Hasmani</li>
                <li>102 - Shreya</li>
                <li>103 - Mansi</li>
            </ul>
        </div>
    )
}

export default Students;
```

week13 > src > pages > Profile.jsx > default

```jsx
function Profile() {
    return (
        <div>
            <h1>Profile</h1>
            <p>Welcome to your Profile!</p>
        </div>
    )
}

export default Profile;
```

week13 > src > App.jsx > default

```jsx
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'

import Login from './pages/Login'
import Dashboard from './pages/Dashboard'
import Students from './pages/Student'
import Profile from './pages/Profile'

import ProtectedRoute from './components/ProtectedRoute'

import './App.css'

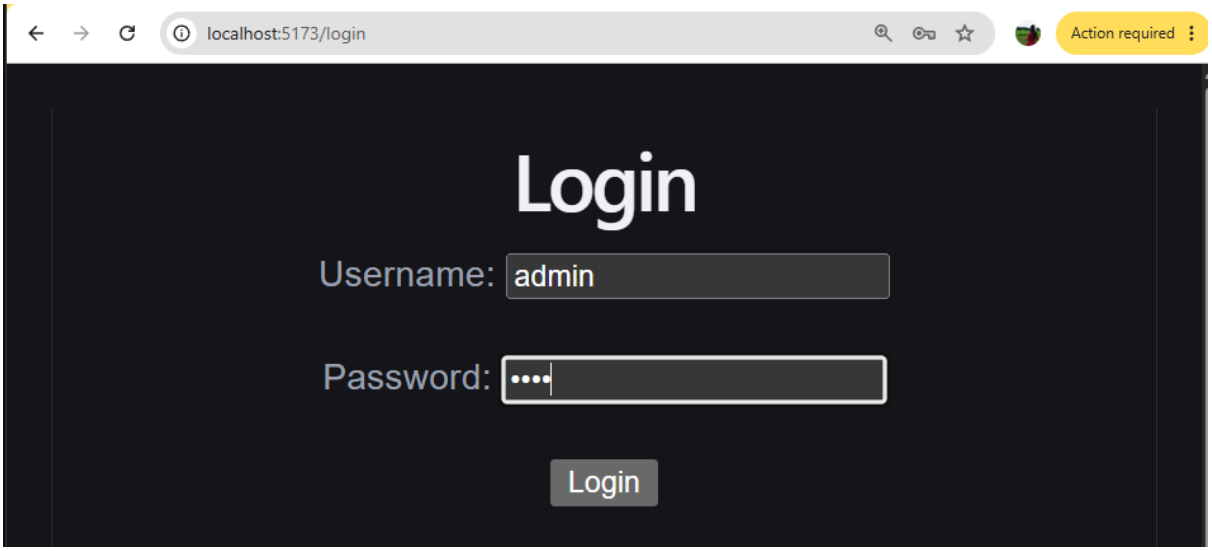
function App() {
    return (
        <BrowserRouter>
            <Routes>
                <Route path="/login" element={<Login />} />

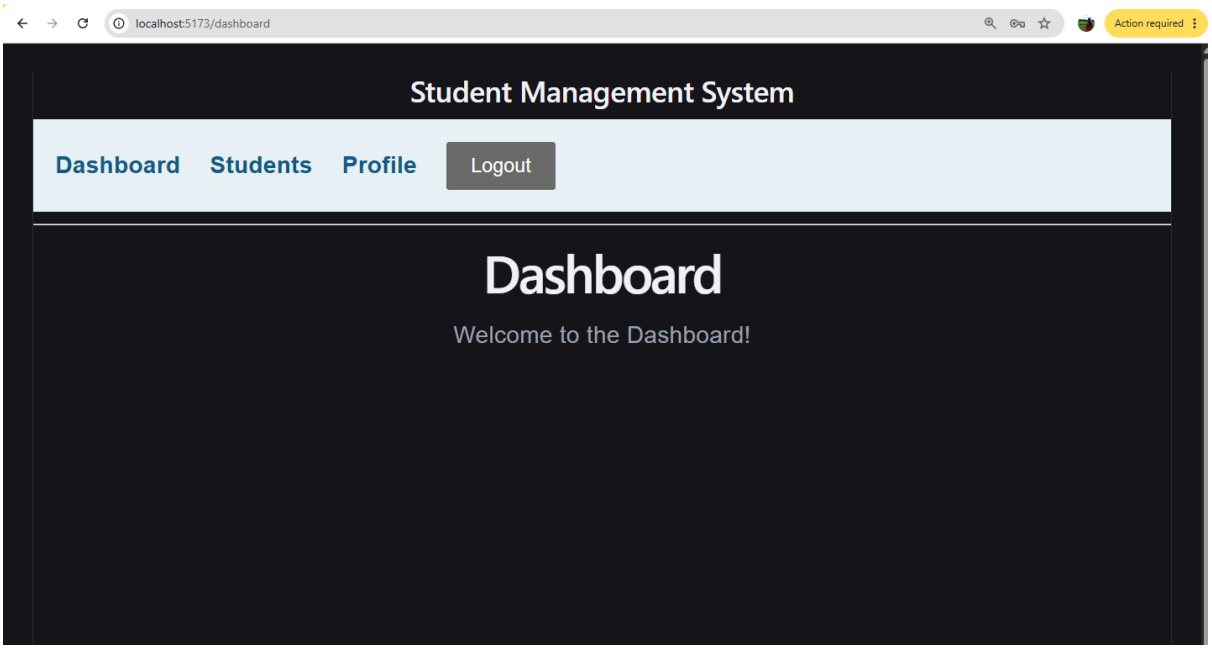
                <Route element={<ProtectedRoute />}>
                    <Route path="/dashboard" element={<Dashboard />} />
                    <Route path="/students" element={<Students />} />
                    <Route path="/profile" element={<Profile />} />
                </Route>

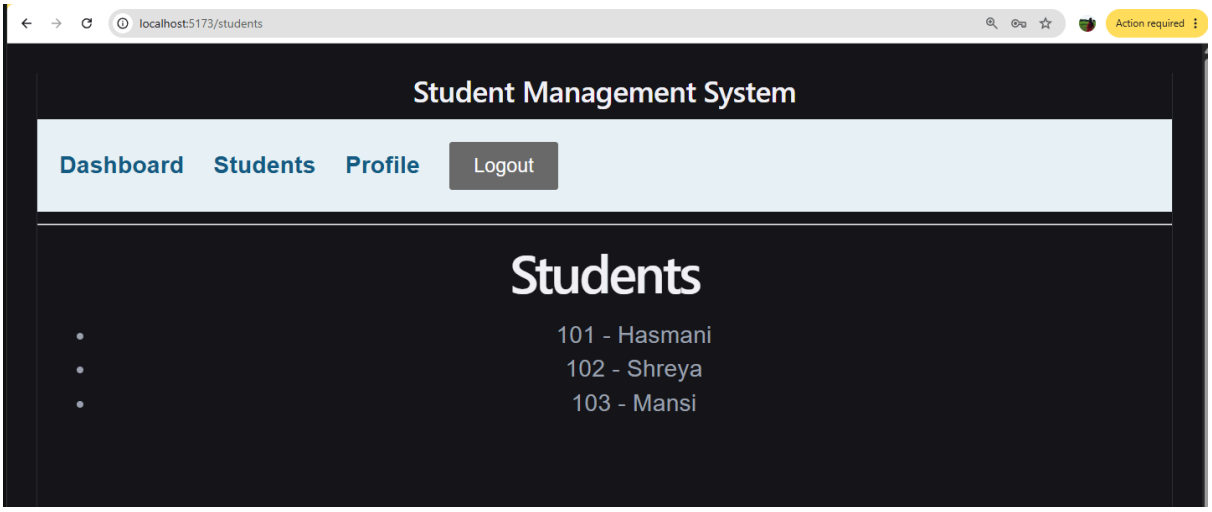
                <Route path="/" element={<Navigate to="/login" />} />
            </Routes>
        </BrowserRouter>
    )
}

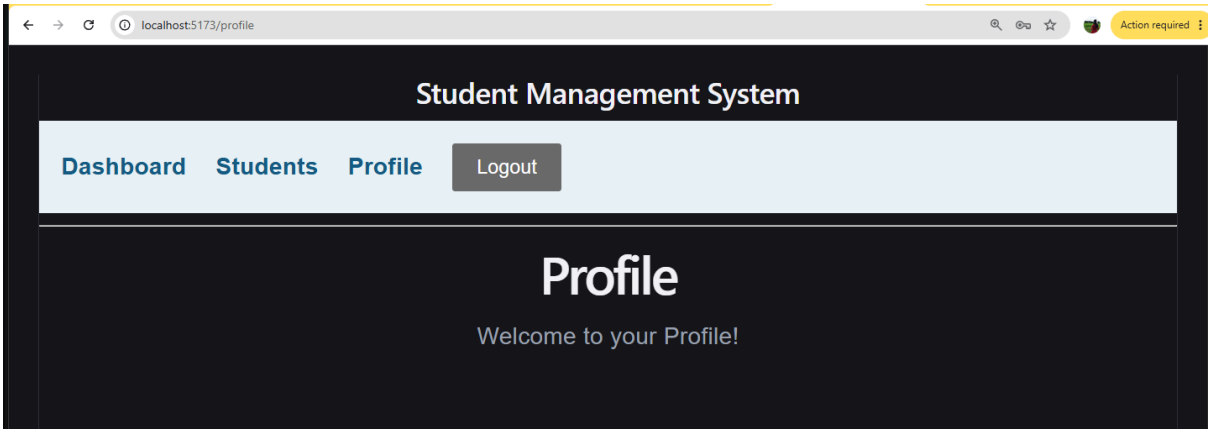
export default App;
```

Output




localhost:5173/dashboard
Action required
Student Management System
Dashboard
Students
Profile
Logout
Dashboard
Welcome to the Dashboard!


localhost:5173/students
Action required
Student Management System
Dashboard
Students
Profile
Logout
Students
101 - Hasmani
102 - Shreya
103 - Mansi


localhost:5173/profile
Action required
Student Management System
Dashboard
Students
Profile
Logout
Profile
Welcome to your Profile!

Week 14 – React.js + Node.js Integration

Exercise 14.1 – Creating a Node.js REST API

Aim:
To create a backend API using Node.js and Express.

Create a Node.js project:
mkdir react-node-api
cd react-node-api
npm init -y
npm install express cors

```
PS C:\Users\DELL\Downloads\ReactProject> mkdir react-node-api


    Directory: C:\Users\DELL\Downloads\ReactProject


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        31-10-2026  03:07 AM                react-node-api


PS C:\Users\DELL\Downloads\ReactProject> cd react-node-api
PS C:\Users\DELL\Downloads\ReactProject\react-node-api> npm init -y
Wrote to C:\Users\DELL\Downloads\ReactProject\react-node-api\package.json:

{
  "name": "react-node-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}


PS C:\Users\DELL\Downloads\ReactProject\react-node-api> npm install express cors

added 67 packages, and audited 68 packages in 3s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Create:
server.js

react-node-api > JS server.js > students

```
1   const express = require('express')
2   const cors = require('cors')
3
4   const app = express()
5
6   app.use(cors())
7   app.use(express.json())
8
9   const students = [
10      {
11          id: 1,
12          name: "Hasmani",
13          course: "BCA"
14      },
15      {
16          id: 2,
17          name: "Shreya",
18          course: "BCA"
19      }
20  ]
21
22  app.get('/api/students', (req, res) => {
23      res.json(students)
24  })
25
26  app.listen(3000, () => {
27      console.log('Server running on http://localhost:3000')
28  })
```

Implement:
GET /api/students



Exercise 14.2 – Connecting React to Node.js

Aim:
To retrieve data from a Node.js backend in React.

Architecture:
React.js
|
| HTTP Request
↓
Express.js
|
↓
Node.js

Tasks:

1. Start the Express server.

react-node-api > JS server.js > ...

```
1   const express = require('express')
2   const cors = require('cors')
3
4   const app = express()
5
6   app.use(cors())
7   app.use(express.json())
8
9   const students = [
10      {
11          id: 1,
12          name: "Hasmani Bilung",
13          course: "BCA"
14      },
15      {
16          id: 2,
17          name: "Shreya Gajinkar",
18          course: "BCA"
19      }
20  ]
21
22  app.get('/api/students', (req, res) => {
23      res.json(students)
24  })
25
26  app.listen(5000, () => {
27      console.log('Server running on http://localhost:5000')
28  })
```



2. Create a React frontend.

```
PS C:\Users\DELL\Downloads\ReactProject> npm create vite@latest react-frontend -- --template react

> npx
> cva react-frontend --template react


create-vite v9.3.3

│
◇  Which linter to use?
│  ESLint
│
◇  Install with npm and start now?
│  Yes
│
◇  Scaffolding project in C:\Users\DELL\Downloads\ReactProject\react-frontend...
│
◇  Installing dependencies with npm...

added 143 packages, and audited 144 packages in 21s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
│
◇  Starting dev server...

> react-frontend@0.0.0 dev
> vite


  VITE v8.3.5  ready in 282 ms

  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

3. Use fetch() from React.
4. Call: GET http://localhost:5000/api/students
5. Store the response using useState().
6. Use useEffect() to make the API call.

react-frontend > src > App.jsx > default

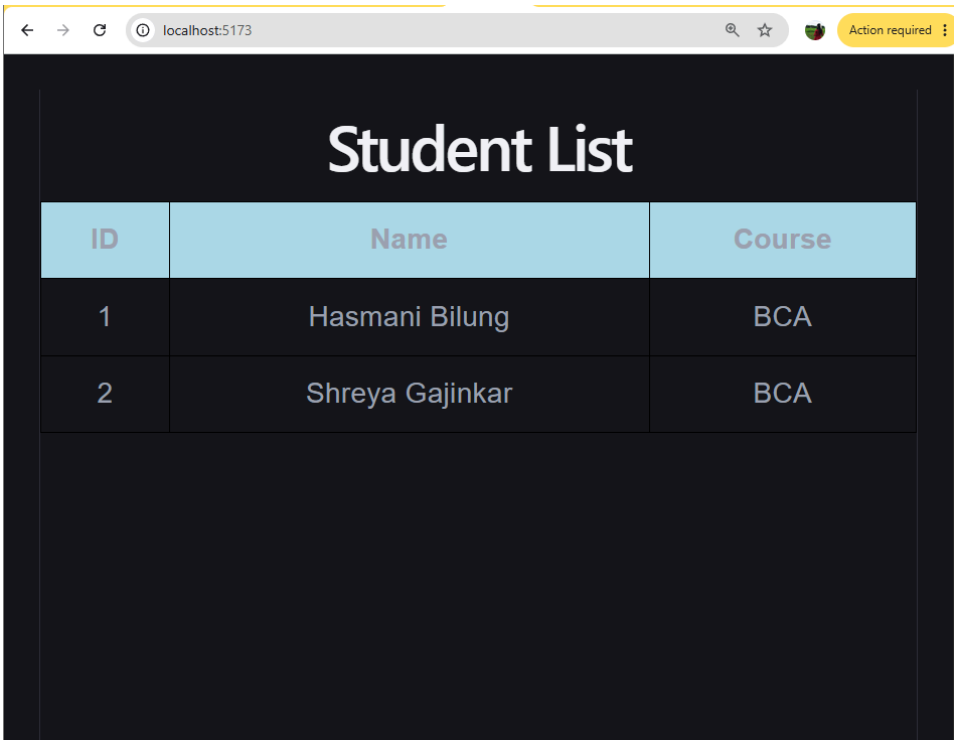
```jsx
import { useState, useEffect } from 'react'
import './App.css'

function App() {
    const [students, setStudents] = useState([])
    const [loading, setLoading] = useState(true)

    useEffect(() => {
        fetch('http://localhost:5000/api/students')
            .then(response => response.json())
            .then(data => {
                setStudents(data)
                setLoading(false)
            })
            .catch(error => {
                console.log('Error fetching students:', error)
                setLoading(false)
            })
    }, [])
    return (
        <div>
            <h1>Student List</h1>
            {loading ? (
                <p>Loading students...</p>
            ) : (
                <table>
                    <thead>
                        <tr>
                            <th>ID</th>
                            <th>Name</th>
                            <th>Course</th>
                        </tr>
                    </thead>
                    <tbody>
                        {students.map(student => (
                            <tr key={student.id}>
                                <td>{student.id}</td>
                                <td>{student.name}</td>
                                <td>{student.course}</td>
                            </tr>
                        ))}
                    </tbody>
                </table>
            )}
        </div>
    )
}
export default App;
```

7. Display the students in a table.





Exercise 14.3 – CORS Configuration

Aim:
To understand and configure Cross-Origin Resource Sharing.

Install:
npm install cors

Configure Express:
const cors = require(“cors”);
app.use(cors());
Students should understand why CORS is required when:
React
localhost:5173
↓
↓ API Request
↓
Express
localhost:5000

```
PS C:\Users\DELL\Downloads\ReactProject\react-node-api> npm install cors

up to date, audited 68 packages in 2s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

react-node-api > JS server.js > students

```
const express = require('express')
const cors = require('cors')

const app = express()

app.use(cors())

app.use(express.json())

const students = [
    {
        id: 1,
        name: "Hasmani",
        course: "BCA"
    },
    {
        id: 2,
        name: "Shreya",
        course: "BCA"
    }
]

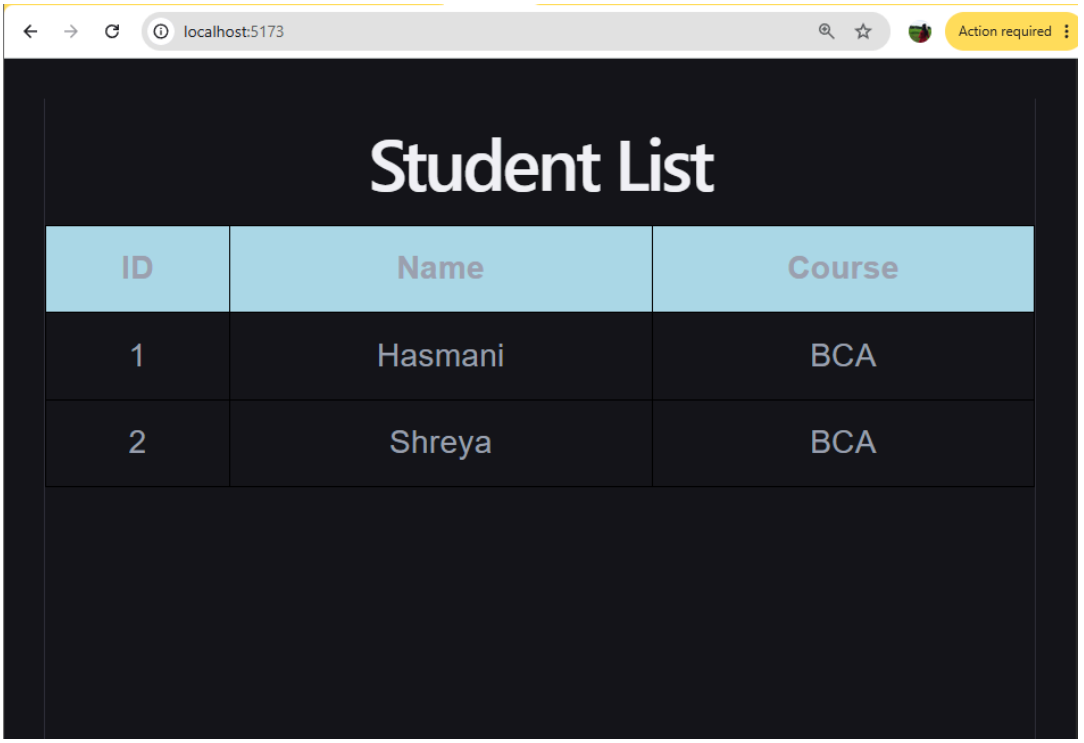
app.get('/api/students', (req, res) => {
    res.json(students)
})

app.listen(5000, () => {
    console.log('Server running on http://localhost:5000')
})
```

Output







Exercise 14.4 – React + Node.js POST Request

Aim:
To send data from React to Node.js.

Create a React student registration form:
Name
Roll No
Course
Email
[Add Student]

When the form is submitted:
React Form
↓
POST /api/students
↓
Express
↓
Store Student

The Node.js server should receive:

{
“name”:”Rahul”,
“rollNo”: 101,
“course”: “BCA”,
“email”: “rahul@example.com”;
}

Return:

{
“message”: “Student added successfully”

react-node-api > JS server.js > ...

```
const express = require('express')
const cors = require('cors')

const app = express()

app.use(cors())
app.use(express.json())

let students = [
    { id: 1, name: "Rahul", course: "BCA" },
    { id: 2, name: "Priya", course: "BCA" }
]

// GET request - Display all students
app.get('/api/students', (req, res) => {
    res.json(students)
})

// POST request - Add a new student
app.post('/api/students', (req, res) => {
    const student = req.body

    students.push(student)

    res.json({
        message: "Student added successfully"
    })
})

app.listen(5000, () => {
    console.log('Server running on http://localhost:5000')
})
```

react-frontend > src > App.jsx > App

```jsx
import { useState } from 'react'
import './App.css'

function App() {
    const [name, setName] = useState('')
    const [rollNo, setRollNo] = useState('')
    const [course, setCourse] = useState('')
    const [email, setEmail] = useState('')
    const [message, setMessage] = useState('')

    async function handleSubmit(e) {
        e.preventDefault()

        const student = {
            name: name,
            rollNo: Number(rollNo),
            course: course,
            email: email
        }

        try {
            const response = await fetch('http://localhost:5000/api/students', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                },
                body: JSON.stringify(student)
            })

            const data = await response.json()

            setMessage(data.message)

            setName('')
            setRollNo('')
            setCourse('')
            setEmail('')
        } catch (error) {
            setMessage('Error adding student')
            console.log(error)
        }
    }
```

```
    return (
        <div className="container">
            <h1>Student Registration</h1>

            <form onSubmit={handleSubmit}>
                <label>Name:</label>
                <input
                    type="text"
                    value={name}
                    onChange={(e) => setName(e.target.value)}
                    required
                />


                <label>Roll No:</label>
                <input
                    type="number"
                    value={rollNo}
                    onChange={(e) => setRollNo(e.target.value)}
                    required
                />

                <label>Course:</label>
                <input
                    type="text"
                    value={course}
                    onChange={(e) => setCourse(e.target.value)}
                    required
                />

                <label>Email:</label>
                <input
                    type="email"
                    value={email}
                    onChange={(e) => setEmail(e.target.value)}
                    required
                />

                <button type="submit">Add Student</button>
            </form>

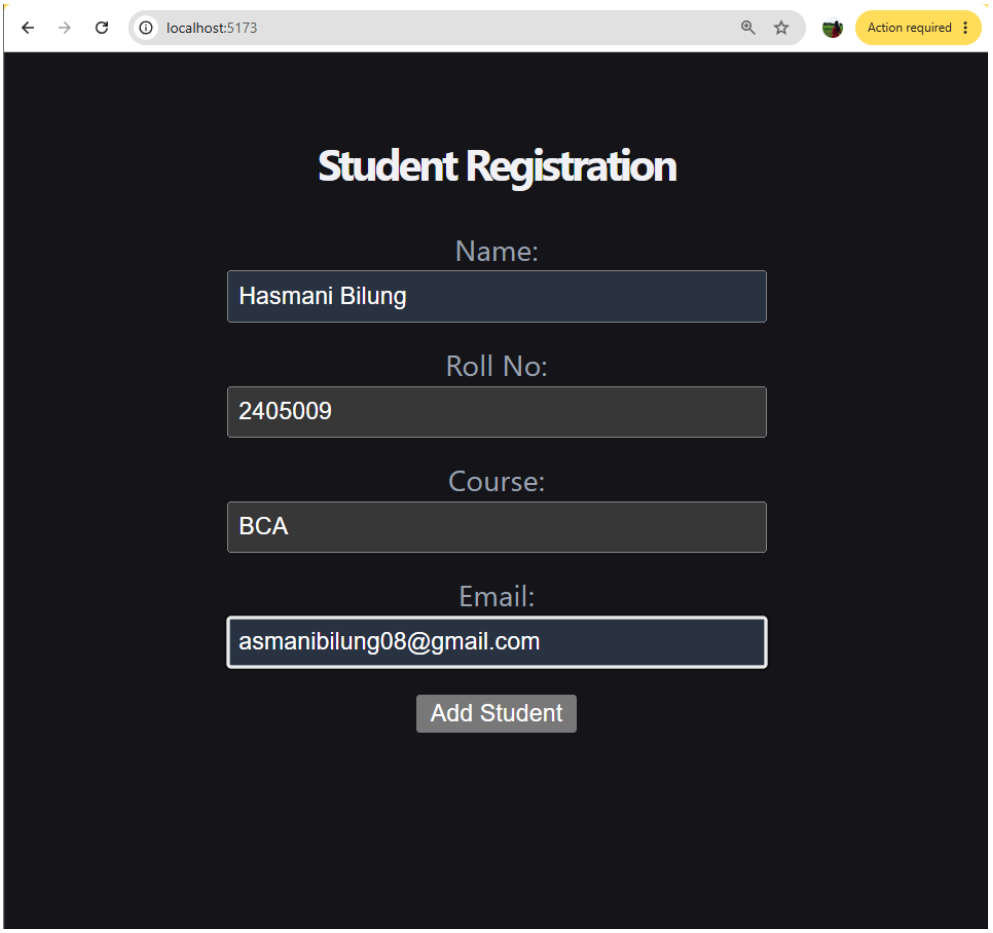
            <p>{message}</p>
        </div>
    )
}

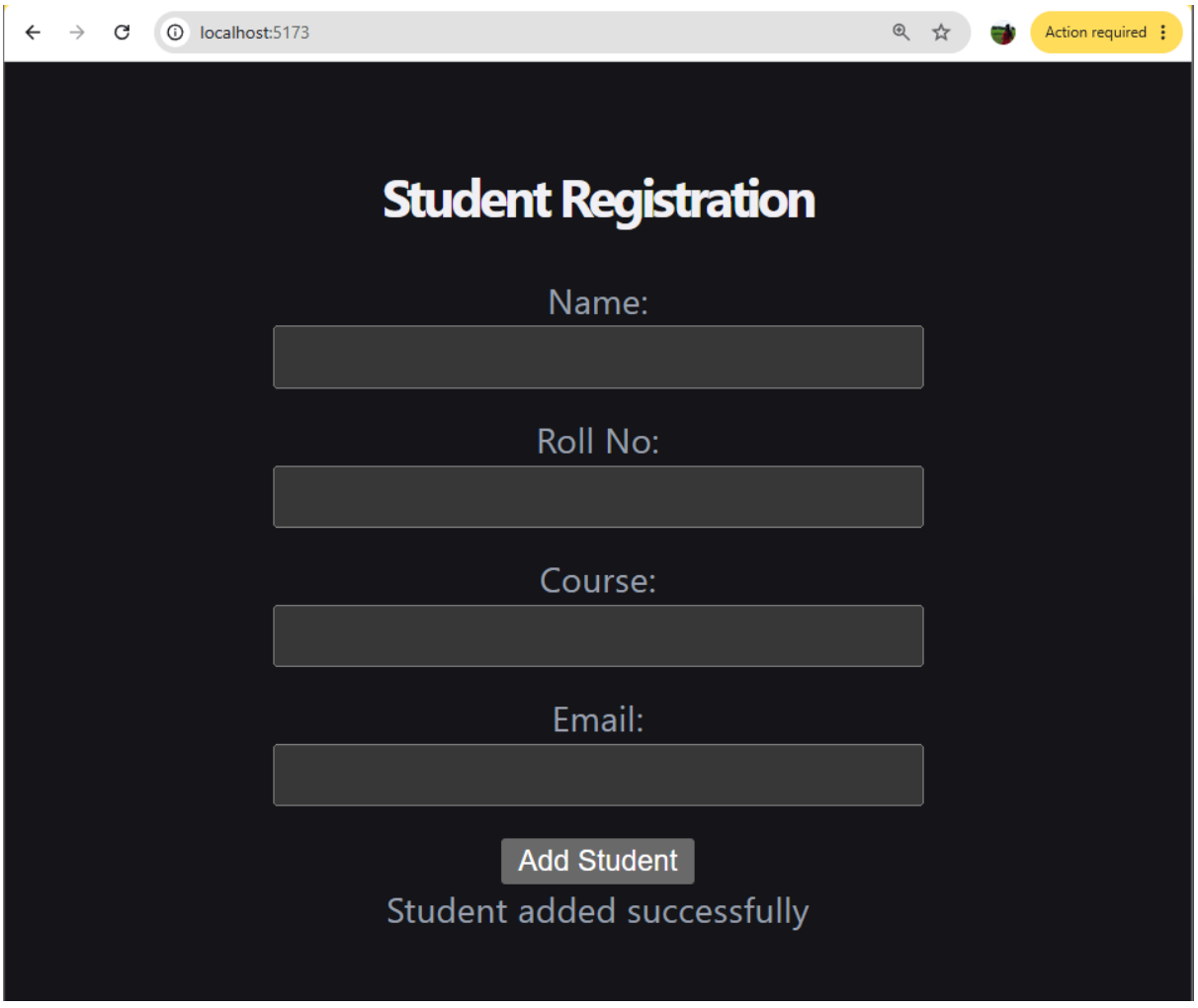
export default App;
```

Output






localhost:5173
Action required
Student Registration
Name:
Hasmani Bilung
Roll No:
2405009
Course:
BCA
Email:
asmanibilung08@gmail.com
Add Student


localhost:5173
Action required
Student Registration
Name:
Roll No:
Course:
Email:
Add Student
Student added successfully

Exercise 14.5 – Complete CRUD

Title
Student Management System using React.js and Node.js

Objective
To develop a full-stack CRUD application using:

- React.js
- React Hooks
- React Router
- Node.js
- Express.js
- REST API

Features
1. View Students
API: GET /api/students

2. Add Student
API: POST /api/students

3. Edit Student
Click Edit.
Load the existing student information into the form.

API: PUT /api/students/:id

4. Delete Student
Click Delete.

API: DELETE /api/students/:id

5. Student Details
Use React Router:
/students/:id

Fetch: GET /api/students/:id

Display complete student information.

server.js (backend)

```
const express = require('express')
const cors = require('cors')

const app = express()

app.use(cors())
app.use(express.json())

let students = [
    {
        id: 1,
        name: "Hasmani",
        rollNo: 101,
        email: "asmani@gmail.com",
        course: "BCA",
        semester: 5
    },
    {
        id: 2,
        name: "Shreya",
        rollNo: 102,
        email: "shreya@gmail.com",
        course: "BCA",
        semester: 5
    }
]

// GET - Display all students
app.get('/api/students', (req, res) => {
    res.json(students)
})

// GET - Display one student by ID
app.get('/api/students/:id', (req, res) => {
    const student = students.find(s => s.id === Number(req.params.id))

    if (!student) {
        return res.status(404).json({ message: "Student not found" })
    }

    res.json(student)
})

// POST - Add a new student
app.post('/api/students', (req, res) => {
    const newStudent = {
        id: students.length > 0
```

```
            ? Math.max(...students.map(s => s.id)) + 1
            : 1,
        name: req.body.name,
        rollNo: Number(req.body.rollNo),
        email: req.body.email,
        course: req.body.course,
        semester: Number(req.body.semester)
    }

    students.push(newStudent)

    res.status(201).json({
        message: "Student added successfully",
        student: newStudent
    })
})

// PUT - Update student information
app.put('/api/students/:id', (req, res) => {
    const id = Number(req.params.id)

    const index = students.findIndex(s => s.id === id)

    if (index === -1) {
        return res.status(404).json({ message: "Student not found" })
    }

    students[index] = {
        id: id,
        name: req.body.name,
        rollNo: Number(req.body.rollNo),
        email: req.body.email,
        course: req.body.course,
        semester: Number(req.body.semester)
    }

    res.json({
        message: "Student updated successfully",
        student: students[index]
    })
})

// DELETE - Delete a student
app.delete('/api/students/:id', (req, res) => {
    const id = Number(req.params.id)

    const index = students.findIndex(s => s.id === id)
```

```
    if (index === -1) {
        return res.status(404).json({ message: "Student not found" })
    }

    students.splice(index, 1)

    res.json({ message: "Student deleted successfully" })
})

app.listen(5000, () => {
    console.log('Server running on http://localhost:5000')
})
```

pages/StudentForm.jsx

```
import { useState, useEffect } from 'react'
import { useNavigate, useParams } from 'react-router-dom'

function StudentForm() {
    const [name, setName] = useState('')
    const [rollNo, setRollNo] = useState('')
    const [email, setEmail] = useState('')
    const [course, setCourse] = useState('')
    const [semester, setSemester] = useState('')

    const { id } = useParams()
    const navigate = useNavigate()

    // Load existing student details for editing
    useEffect(() => {
        if (id) {
            fetch(`http://localhost:5000/api/students/${id}`)
                .then(response => response.json())
                .then(student => {
                    setName(student.name)
                    setRollNo(student.rollNo)
                    setEmail(student.email)
                    setCourse(student.course)
                    setSemester(student.semester)
                })
                .catch(error => console.log(error))
        }
    }, [id])

    async function handleSubmit(e) {
        e.preventDefault()
```

```
        const student = {
            name,
            rollNo,
            email,
            course,
            semester
        }

        const url = id
            ? `http://localhost:5000/api/students/${id}`
            : 'http://localhost:5000/api/students'

        const method = id ? 'PUT' : 'POST'

        try {
            const response = await fetch(url, {
                method: method,
                headers: {
                    'Content-Type': 'application/json'
                },
                body: JSON.stringify(student)
            })

            const data = await response.json()

            if (!response.ok) {
                alert(data.message)
                return
            }

            alert(data.message)
            navigate('/')
        } catch (error) {
            console.log(error)
            alert('Something went wrong')
        }
    }

    return (
        <div>
            <h2>{id ? 'Edit Student' : 'Add Student'}</h2>

            <form onSubmit={handleSubmit}>
                <label>Name:</label>
                <input
                    type="text"
```

```
                    value={name}
                    onChange={e => setName(e.target.value)}
                    required
                />

                <label>Roll Number:</label>
                <input
                    type="number"
                    value={rollNo}
                    onChange={e => setRollNo(e.target.value)}
                    required
                />

                <label>Email:</label>
                <input
                    type="email"
                    value={email}
                    onChange={e => setEmail(e.target.value)}
                    required
                />

                <label>Course:</label>
                <input
                    type="text"
                    value={course}
                    onChange={e => setCourse(e.target.value)}
                    required
                />

                <label>Semester:</label>
                <input
                    type="number"
                    value={semester}
                    onChange={e => setSemester(e.target.value)}
                    required
                />

                <button type="submit">Save Student</button>
            </form>
        </div>
    )
}

export default StudentForm;
```

pages/StudentList.jsx

```
import { useState, useEffect } from 'react'
import { Link } from 'react-router-dom'

function StudentList() {
    const [students, setStudents] = useState([])

    useEffect(() => {
        fetch('http://localhost:5000/api/students')
            .then(response => response.json())
            .then(data => setStudents(data))
            .catch(error => console.log(error))
    }, [])

    async function deleteStudent(id) {
        const confirmDelete = window.confirm(
            'Are you sure you want to delete this student?'
        )

        if (!confirmDelete) return

        try {
            const response = await fetch(
                `http://localhost:5000/api/students/${id}`,
                { method: 'DELETE' }
            )

            if (response.ok) {
                setStudents(students.filter(s => s.id !== id))
            }
        } catch (error) {
            console.log(error)
        }
    }

    return (
        <div>
            <h2>Student List</h2>

            <Link to="/students/add">Add Student</Link>

            <table>
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Roll No</th>
```

```
                        <th>Course</th>
                        <th>Actions</th>
                    </tr>
                </thead>

                <tbody>
                    {students.map(student => (
                        <tr key={student.id}>
                            <td>{student.id}</td>
                            <td>{student.name}</td>
                            <td>{student.rollNo}</td>
                            <td>{student.course}</td>
                            <td>
                                <Link
to={`/students/edit/${student.id}`}>
                                    Edit
                                </Link>
                                {' | '}
                                <button onClick={() =>
deleteStudent(student.id)}>
                                    Delete
                                </button>
                                {' | '}
                                <Link to={`/students/${student.id}`}>
                                    Details
                                </Link>
                            </td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    )
}

export default StudentList;
```

pages/StudentDetails.jsx

react-frontend > src > pages > StudentDetails.jsx > default

```jsx
import { useState, useEffect } from 'react'
import { useParams, Link } from 'react-router-dom'

function StudentDetails() {
    const { id } = useParams()
    const [student, setStudent] = useState(null)

    useEffect(() => {
        fetch(`http://localhost:5000/api/students/${id}`)
            .then(response => {
                if (!response.ok) {
                    throw new Error('Student not found')
                }
                return response.json()
            })
            .then(data => setStudent(data))
            .catch(error => console.log(error))
    }, [id])

    if (!student) {
        return <p>Loading student details...</p>
    }

    return (
        <div>
            <h2>Student Details</h2>

            <p>ID: {student.id}</p>
            <p>Name: {student.name}</p>
            <p>Roll Number: {student.rollNo}</p>
            <p>Email: {student.email}</p>
            <p>Course: {student.course}</p>
            <p>Semester: {student.semester}</p>

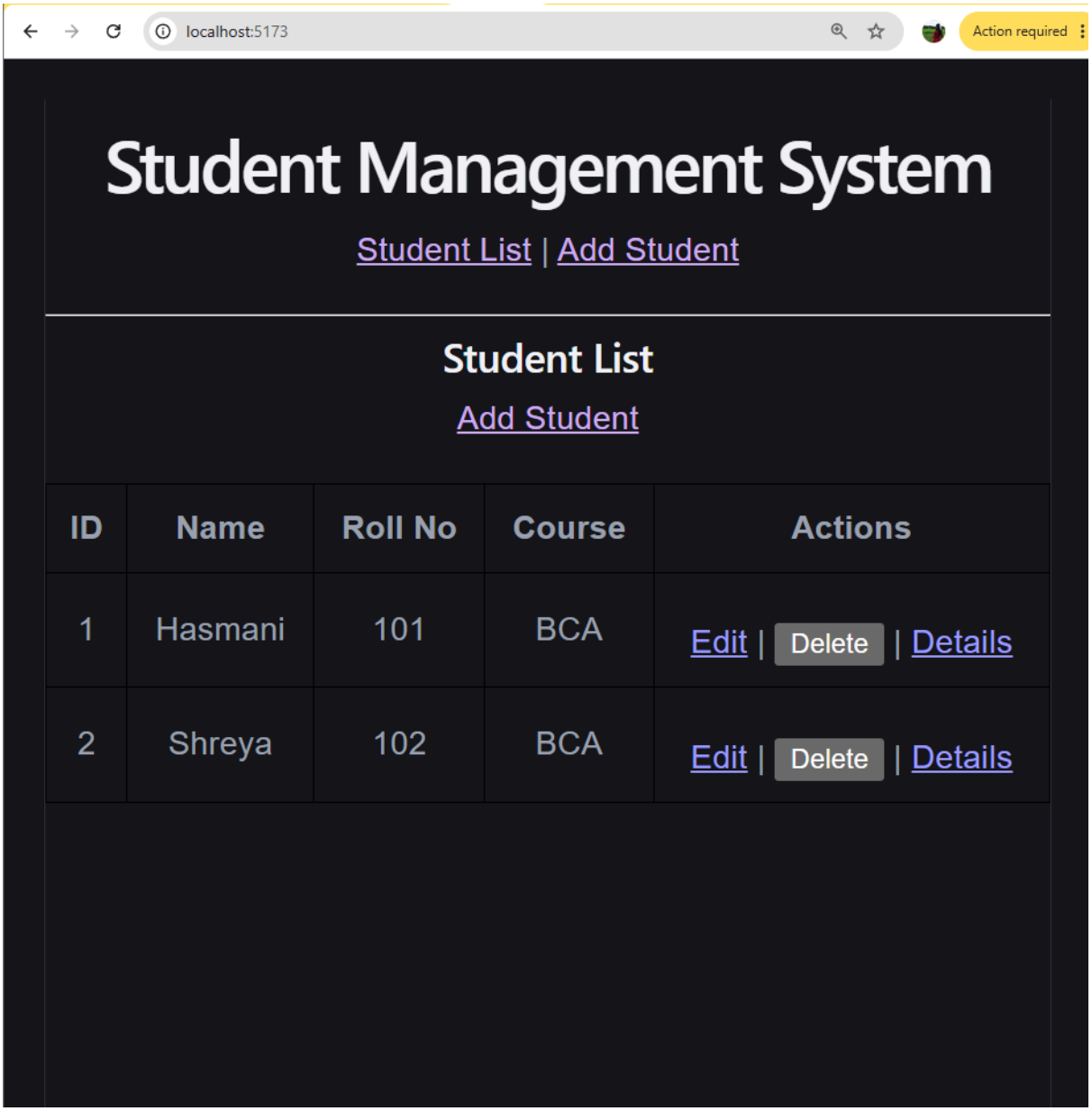
            <Link to="/">Back to Student List</Link>
        </div>
    )
}

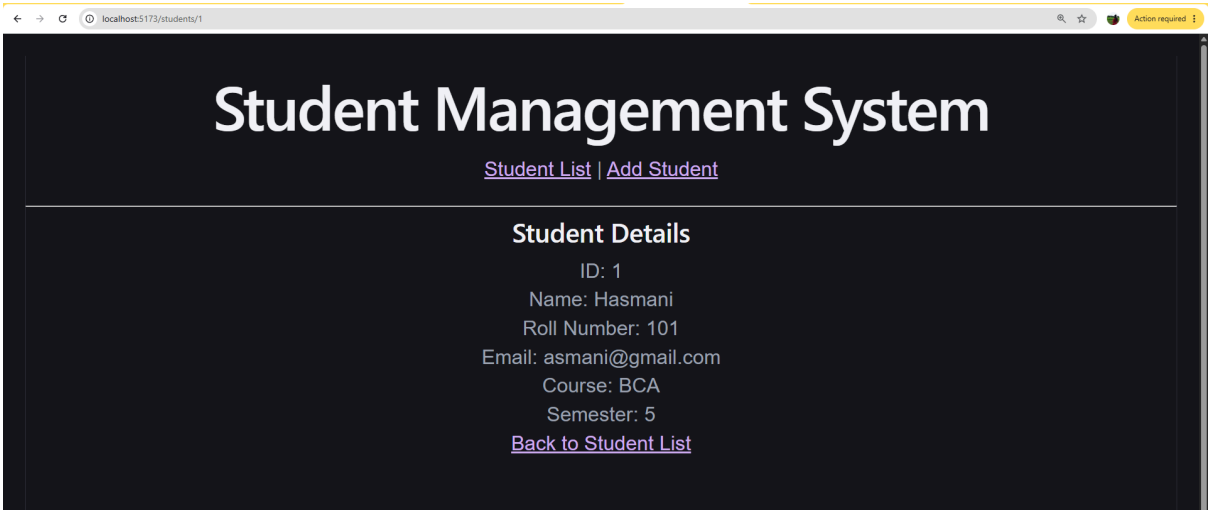
export default StudentDetails;
```

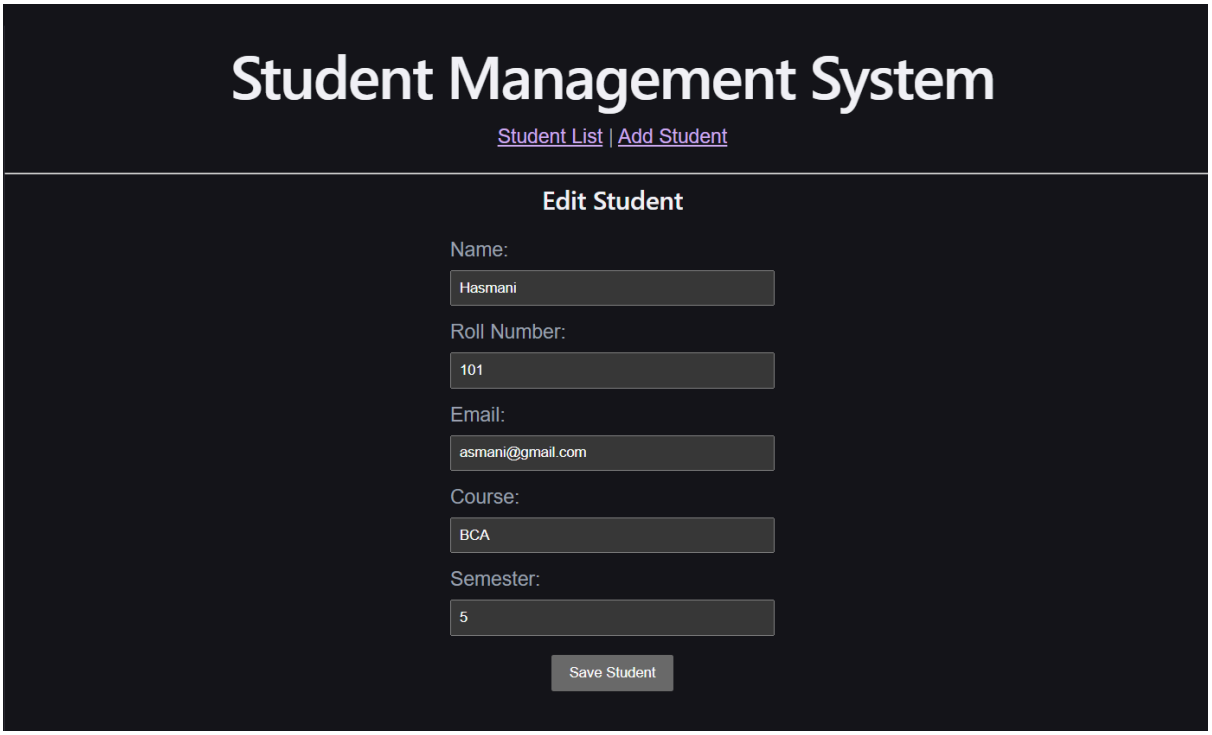
Output

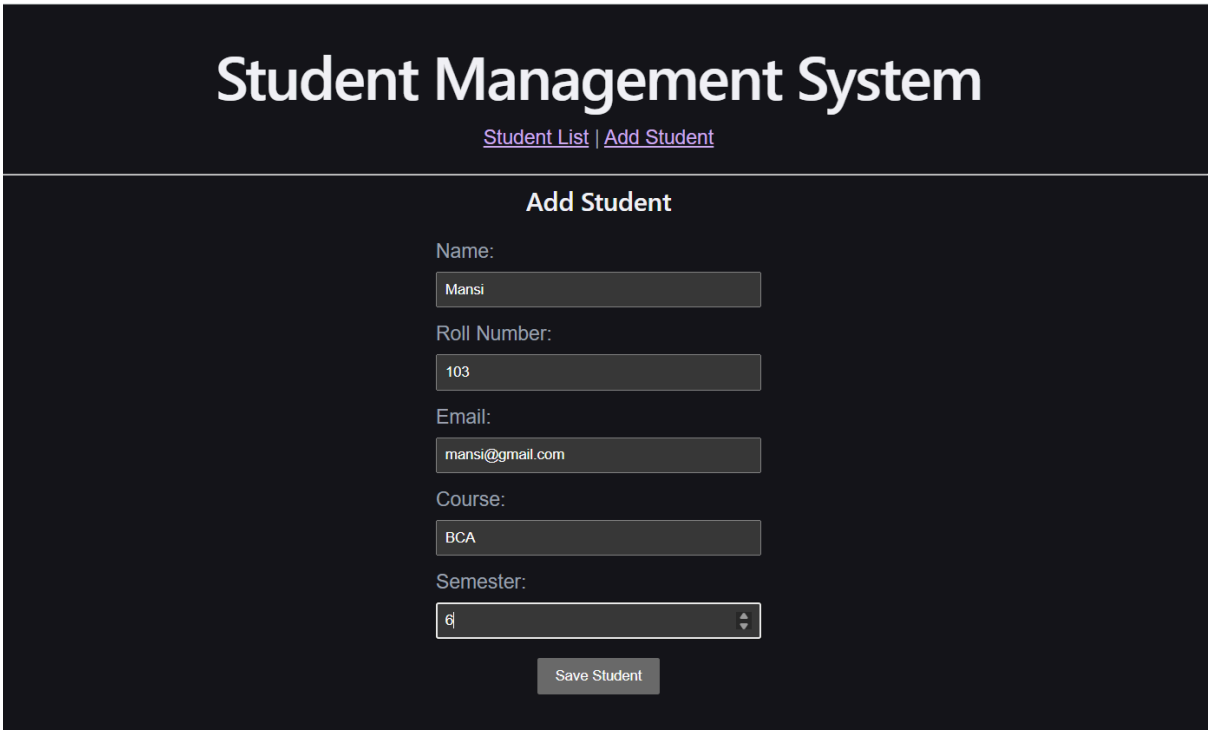


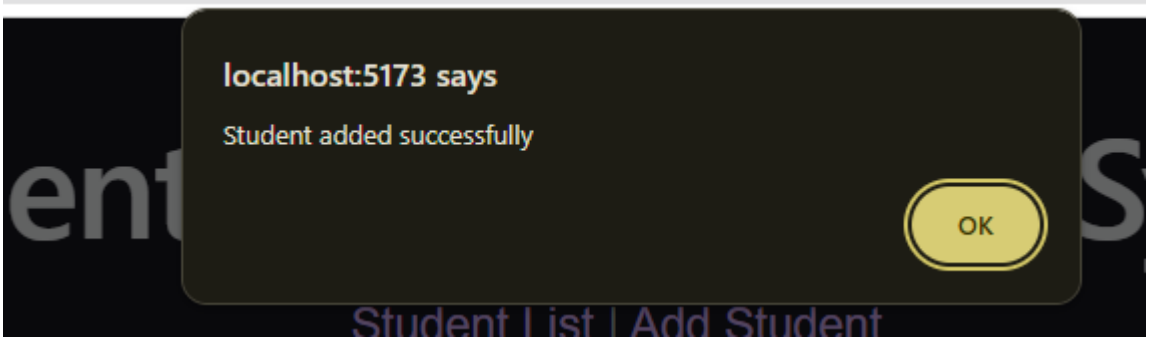


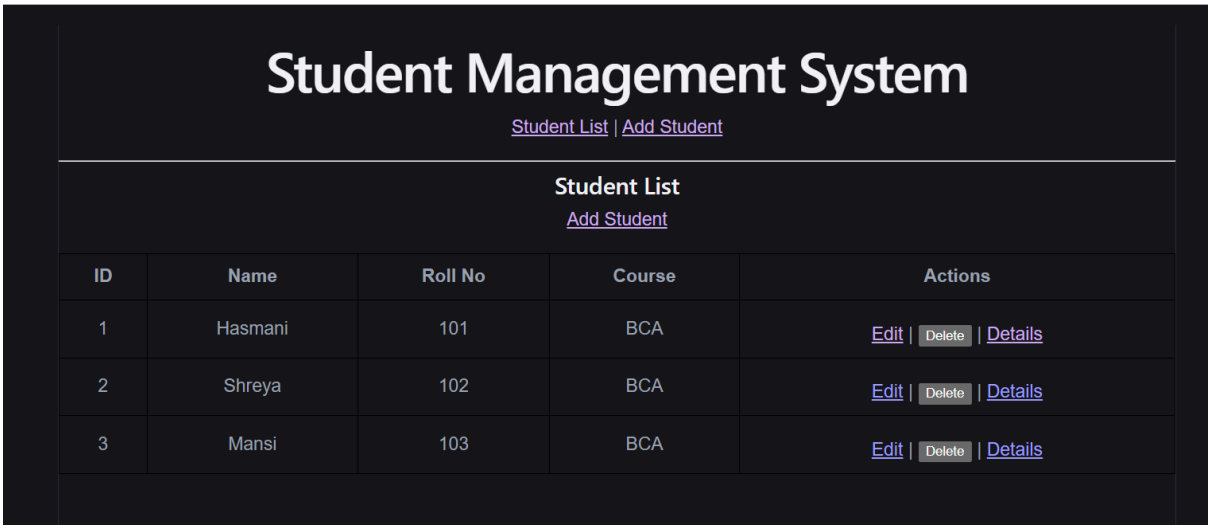

localhost:5173
Action required
Student Management System
Student List | Add Student
Student List
Add Student
ID
Name
Roll No
Course
Actions
1
Hasmani
101
BCA
Edit | Delete | Details
2
Shreya
102
BCA
Edit | Delete | Details


Student Management System
Student List | Add Student
Student Details
ID: 1
Name: Hasmani
Roll Number: 101
Email: asmani@gmail.com
Course: BCA
Semester: 5
Back to Student List


Student Management System
Student List | Add Student
Edit Student
Name:
Hasmani
Roll Number:
101
Email:
asmani@gmail.com
Course:
BCA
Semester:
5
Save Student


Student Management System
Student List | Add Student
Add Student
Name:
Mansi
Roll Number:
103
Email:
mansi@gmail.com
Course:
BCA
Semester:
6
Save Student


localhost:5173 says
Student added successfully
OK
Student List | Add Student


Student Management System
Student List | Add Student
Student List
Add Student
ID
Name
Roll No
Course
Actions
1
Hasmani
101
BCA
Edit | Delete | Details
2
Shreya
102
BCA
Edit | Delete | Details
3
Mansi
103
BCA
Edit | Delete | Details

WEEK 15- Node js Integration with React Frontend

Student Registration Application

Create the Node.js backend

mkdir student-app
cd student-app

mkdir backend
cd backend

npm init -y
npm install express cors

```
C:\Users\TYBCA\Documents\ReactProject-2405009>mkdir student-app

C:\Users\TYBCA\Documents\ReactProject-2405009>cd student-app

C:\Users\TYBCA\Documents\ReactProject-2405009\student-app>mkdir backend

C:\Users\TYBCA\Documents\ReactProject-2405009\student-app>cd backend
```

```
C:\Users\TYBCA\Documents\ReactProject-2405009\student-app\backend>npm init -y
Wrote to C:\Users\TYBCA\Documents\ReactProject-2405009\student-app\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}



C:\Users\TYBCA\Documents\ReactProject-2405009\student-app\backend>npm install express cors

added 67 packages, and audited 68 packages in 3s

29 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Create server.js

student-app > backend > JS server.js > ...

```
const express = require("express");
const cors = require("cors");

const app = express();
const PORT = 5000;

app.use(cors());
app.use(express.json());

let students = [
    {id: 1, name: 'Hasmani', course:'BCA'},
    {id: 2, name: 'Shreya', course:'BCA'},
    {id: 3, name: 'mansi', course:'BCA'},
]

//test route
app.get('/api/health', (req,res) => {
    res.json({message: 'Node.js server is running!'});
});

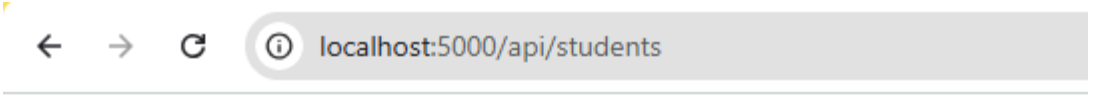
//GET: Retrieve all students
app.get('/api/students',(req,res) => {
    res.json(students);
});

app.listen(PORT,() => {
    console.log("server is running");
});
```

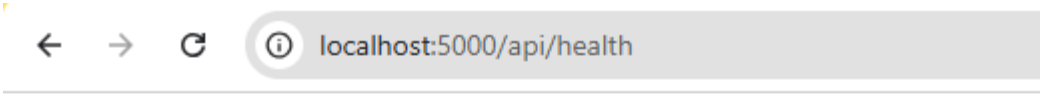
Run the backend

node server.js

```
C:\Users\TYBCA\Documents\ReactProject-2405009\student-app\backend>node server.js
server is running
```









Create the React frontend

npm create vite@latest frontend -- --template react

cd frontend

npm install

npm run dev

```
PS C:\Users\DELL\Downloads\ReactProject\week13> npm create vite@latest frontend -- --template react

> week13@0.0.0 npx
> cva frontend --template react


create-vite v9.3.3

◇  Which linter to use?
│  ESLint
│
◇  Install with npm and start now?
│  Yes
│
◇  Scaffolding project in C:\Users\DELL\Downloads\ReactProject\week13\frontend...
│
◇  Installing dependencies with npm...

added 143 packages, and audited 144 packages in 29s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
│
◇  Starting dev server...

> frontend@0.0.0 dev
> vite


  VITE v8.3.5  ready in 252 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

```
PS C:\Users\DELL\Downloads\ReactProject\week13> cd frontend
PS C:\Users\DELL\Downloads\ReactProject\week13\frontend> npm run dev

> frontend@0.0.0 dev
> vite


  VITE v8.3.5  ready in 171 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Connect React to Node.js

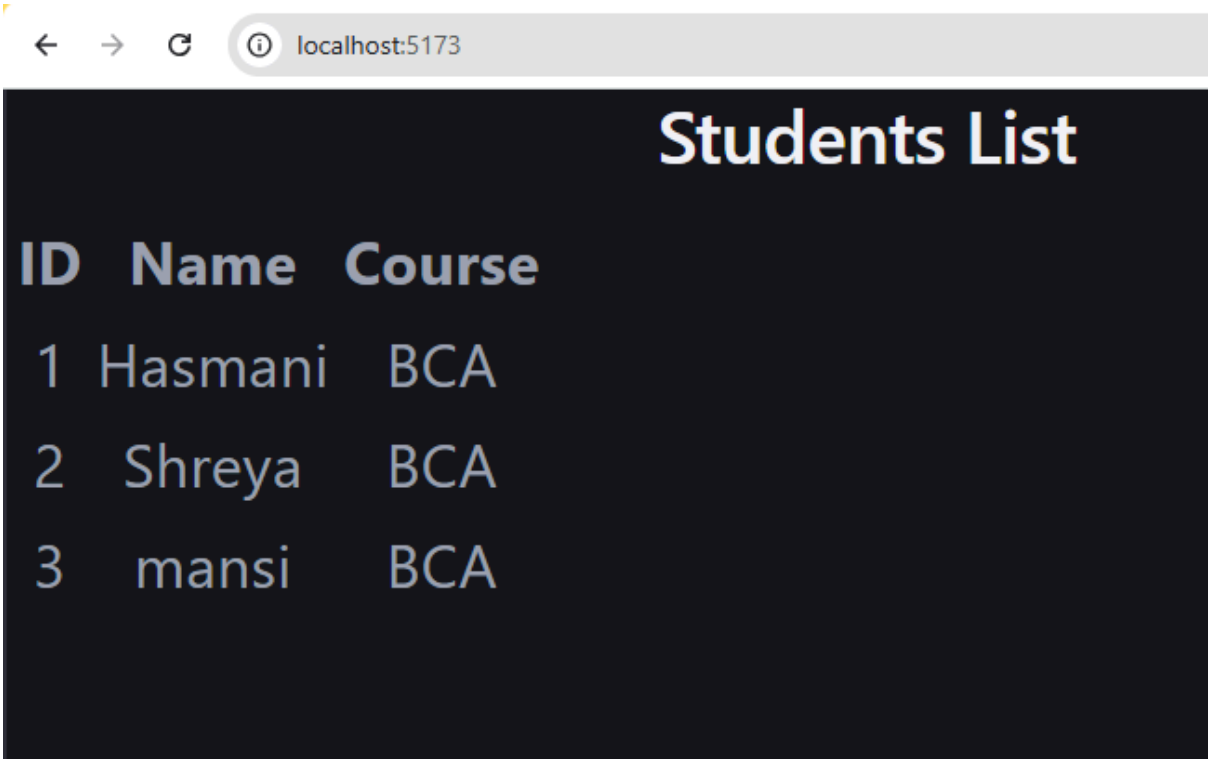
frontend > src > App.jsx > App

```jsx
import { useEffect, useState } from "react";
import "./App.css";

const API_URL = "http://localhost:5000/api/students";

function App() {
  const [students, setStudents] = useState([]);
  async function fetchStudents() {
    try {
      const response = await fetch(API_URL);

      if (!response.ok) {
        throw new Error("Could not load students");
      }
      const data = await response.json();
      setStudents(data);
    } catch (error) {
      console.log(error.message);
    }
  }
  useEffect(() => {
    fetchStudents();
  }, []);
  return (
    <div className="container">
      <h2> Students List</h2>
      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Course</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student) => (
            <tr key={student.id}>
              <td>{student.id}</td>
              <td>{student.name}</td>
              <td>{student.course}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
export default App;
```


localhost:5173
Students List
ID Name Course
1 Hasmani BCA
2 Shreya BCA
3 mansi BCA